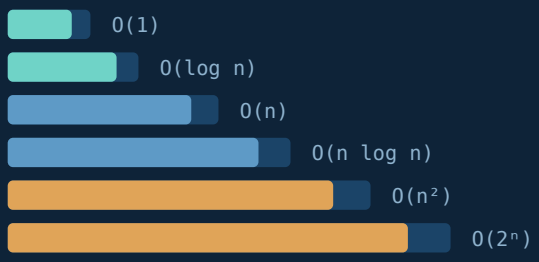
FROM ABSOLUTE ZERO TO INTERVIEW-READY

Arrays & Linked Lists

A complete beginner's course in C++
Forty chapters, from what a variable is
to the hardest interview questions asked.



Written for the beginner who wants to understand, not memorise

First edition · 2026

Arrays & Linked Lists

A complete beginner's course in C++
Forty chapters, from what a variable is
to the hardest interview questions asked.

You do not need to be clever to be good at this. You need to see what is happening in memory, one small step at a time. Every concept in this book is drawn before it is coded, traced before it is trusted, and practised before it is assumed.

Written for the beginner who wants to understand, not memorise

First edition · 2026

How to Read This Book

You are holding a course, not a reference. It was written for one specific person: somebody who can already write a `for` loop in some language, who has heard the words "data structure" and felt a small chill, and who wants to genuinely *understand* arrays and linked lists rather than memorise twenty solutions and hope the interviewer picks one of them.

If that is you, read on. If you are already comfortable with pointers and Big O, skip to Part III and start with the patterns.

The promise

By the last page you will be able to look at an array or linked list problem you have never seen before, and know **how to start**. Not because you have seen it, but because you will recognise its shape. That is the whole game.

Every single idea in this book is introduced in the same order:

1. **A picture first.** You will see the memory before you see the code.
2. **An everyday analogy.** Something from the physical world that behaves the same way.
3. **A tiny example.** Three or four elements, never twenty.
4. **A hand trace.** We walk through it step by step, writing down the value of every variable at every moment.
5. **Then, and only then, the C++ code**, explained line by line.
6. **The common mistakes** — because knowing the five ways to get it wrong is more useful than knowing the one way to get it right.
7. **An exercise**, with the answer in Chapter 38.

The rules I have set myself

> **KEY INSIGHT | The four promises**
>
> - **No unexplained words.** Every technical term is defined in plain English the first time it appears, and again in the glossary in Chapter 40.
> - **No skipped steps.** If a line of code depends on an idea, that idea comes first — even if it means an extra page.
> - **No cleverness before clarity.** You always see the obvious, slow solution before the fast, surprising one. The fast one is only impressive once you know what it is beating.
> - **Nothing you cannot run.** Every program in this book compiles with `g++ -std=c++17` and produces exactly the output shown.

How the book is laid out

Part	What it covers	Read it when
I	Data structures, memory, and Big O	First. Everything rests on it.
II	Arrays, `std::vector`, every basic operation	Straight after Part I
III	The ten array patterns	Once Part II feels comfortable
IV	Twenty-five array problems, graded	Alongside Part III
V	Pointers, the heap, `new` and `delete`	Before any linked list
VI	Linked lists built from nothing	After Part V, never before
VII	Linked list patterns and sixteen interview problems	After Part VI
VIII	Complexity comparison, and a method for unseen problems	Any time after Part VII
IX	Practice roadmap, answer key, mock interview, 30-day plan	Continuously

The symbols in the margin

Coloured boxes appear throughout. They always mean the same thing:

DEFINITION
A word is being given a precise meaning. These are worth re-reading.

INTUITION
The *why* behind something, usually with an analogy. If a formal explanation did not land, the intuition box is where to look.

COMMON PITFALL
A mistake that beginners make, with the reason it happens. I have made every single one of these myself.

PERFORMANCE NOTE
A note about speed or memory. Usually an explanation of why the obvious thing is slower than it looks.

INTERVIEW ANGLE
What an interviewer is actually listening for. These matter more than you think — most candidates fail on communication, not on code.

EXERCISES
Something to do before reading on. Full answers are in Chapter 38.

How to actually study this

Reading a DSA book is almost useless. Here is what works:

1. **Read with a pen.** When a trace table appears, cover the answer column and fill it in yourself first.
2. **Type the code, do not copy it.** Typing forces you to read every character. Copying teaches your clipboard, not you.
3. **Break the code on purpose.** Change a `<` to a `<=`. Run it. Predict the bug before you see it. This single habit is worth fifty solved problems.
4. **Re-draw the diagrams from memory.** If you can draw the four pointer changes of a linked list insertion on a blank page, you know it. If you cannot, you have only recognised it.
5. **Do not move on when you are confused.** Confusion is information: it is telling you a prerequisite is missing. Go back one section.

> **NOTE | A word about speed**
> If a chapter takes you three days, that is normal and it is fine. This material is not difficult because you are slow; it is difficult because it is the first time anybody has asked you to hold a picture of memory in your head while reading code. That skill is built, not born. Chapter 40 contains a thirty-day plan if you would like a schedule to follow.

Let us begin.

Contents at a Glance

How to Read This Book iii

Part I — Before Anything Else 1

1. What a Data Structure Really Is 2

2. Memory, Variables and Addresses 9

3. Big O From Absolute Zero 18

Part II — Arrays From the Ground Up 32

4. The Array 33

5. Static Arrays, Dynamic Arrays and std::vector 45

6. The Core Array Operations 54

7. Two-Dimensional Arrays 70

Part III — The Array Patterns 81

8. Pattern 1: Linear Traversal 82

9. Pattern 2: Two Pointers 92

10. Pattern 3: Sliding Window 102

11. Pattern 4: Prefix Sums 110

12. Pattern 5: Binary Search 119

13. Pattern 6: In-Place Modification 129

14. Pattern 7: Frequency Counting 137

15. Pattern 8: Sorting-Based Approaches 146

16. Pattern 9: Kadane's Algorithm 155

17. Pattern 10: Matrix Patterns 164

Part IV — The Array Problem Set 173

18. Foundational Array Problems 174

19. Intermediate Array Problems 182

20. Advanced Array Problems 193

Part V — Pointers and Memory 201

21. Pointers, References and Dereferencing 202

22. The Stack, the Heap, new and delete 213

Part VI — Linked Lists From Scratch 222

23. What a Linked List Is, and Why It Exists 223

24. Building a Linked List From Scratch 231

25. Insertion 242

26. Deletion 252

27. Doubly and Circular Linked Lists 264

28. Mastering Pointer Manipulation: Reversal 274

Part VII — Linked List Patterns and Problems 286

29. The Linked List Patterns 287

30. Essential Linked List Problems I 301

31. Essential Linked List Problems II 309

32. Advanced Linked List Problems 321

Part VIII — Thinking Like a Problem Solver 332

33. Arrays versus Linked Lists: The Complete Picture 333

34. How to Solve Any DSA Problem 340

35. C++ Implementation Standards 349

Part IX — Practice, Revision, Mastery 357

36. The Five-Level Practice Roadmap 358

37. Quizzes and Exercises 365

38. The Complete Answer Key 373

39. The Mock Interview 395

40. Reference: Plan, Checklist, Glossary 401

Contents

How to Read This Book iii

Part I — Before Anything Else 1

1. What a Data Structure Really Is . . . 2

Start with a drawer . . . 2

What "data" means here . . . 3

Why arrangement changes speed at all . . . 3

What we mean by an "operation" . . . 5

A first look at the cost of a wrong choice . . . 6

The map of this book . . . 6

Why arrays and linked lists first . . . 7

What you should take away . . . 8

2. Memory, Variables and Addresses . . . 9

The smallest thing there is . . . 9

Memory is a numbered street . . . 10

What a variable actually is . . . 11

Seeing it for yourself . . . 12

Two variables, side by side? Not necessarily . . . 13

Address arithmetic: the pay-off . . . 14

Where memory comes from: a first look . . . 15

Common misunderstandings . . . 16

What you should take away . . . 17

3. Big O From Absolute Zero . . . 18

The question Big O answers . . . 18

Counting steps, slowly . . . 19

The two rules that look like cheating . . . 20

The definition, in English and then in symbols . . . 21

The six complexities you must know on sight . . . 21

Why nested loops give O(n^2) . . . 23

Why halving gives O(log n) . . . 24

Best, average and worst case . . . 25

Space complexity . . . 26

Time–space trade-offs . . . 27

A recipe for reading any C++ function . . . 28

What you should take away . . . 30

Part II — Arrays From the Ground Up **32**

4. The Array . . . 33

The picture that never changes . . . 33

Why indexing starts at zero . . . 34

What arr[3] actually compiles into . . . 35

Declaring arrays: the forms you will meet . . . 36

Traversal: visiting every element . . . 37

Searching: the first real algorithm . . . 38

Going out of bounds . . . 40

The cost table for arrays . . . 42

Common beginner mistakes . . . 42

What you should take away . . . 43

5. Static Arrays, Dynamic Arrays and std::vector . . . 45

The problem with a fixed size . . . 45

Meeting std::vector . . . 46

Size versus capacity . . . 47

What happens when the block is full . . . 48

Amortised O(1), explained properly . . . 49

reserve: skipping the copying entirely . . . 50

The full cost table . . . 51

Iterator invalidation . . . 51

Raw array or vector? . . . 52

What you should take away . . . 52

6. The Core Array Operations . . . 54

How to read this chapter . . . 54

1. Access and update . . . 54

2. Traversal . . . 55

3. Searching . . . 55

4. Insertion . . . 56

5. Deletion . . . 58

6. Reversing an array . . . 60

7. Minimum and maximum . . . 61

8. Sum and average . . . 63

9. Counting occurrences . . . 64

10. Removing duplicates from a sorted array . . . 64

11. Moving zeroes to the end . . . 66

The complete cost table . . . 68

7. Two-Dimensional Arrays . . . 70

A grid is a line, folded . . . 70

The address formula, extended . . . 71
Declaring grids . . . 72
Traversing a grid . . . 73
Rows, columns and diagonals . . . 74
Transposing a square matrix in place . . . 76
Passing grids to functions . . . 77
Jagged grids . . . 78
Cost summary . . . 79

Part III — The Array Patterns **81**

8. Pattern 1: Linear Traversal . . . 82
When to use it . . . 82
The idea: running state . . . 83
Problem 1: Largest element . . . 83
Problem 2: Second-largest element . . . 84
Problem 3: Is the array sorted? . . . 86
Problem 4: Maximum consecutive ones . . . 87
Turning two passes into one . . . 88
Cost summary . . . 90

9. Pattern 2: Two Pointers . . . 92
The two shapes . . . 92
Shape 1: Converging pointers . . . 93
Shape 2: Same-direction pointers . . . 98
Why two moving pointers is still O(n) . . . 100
Cost summary . . . 100

10. Pattern 3: Sliding Window . . . 102

The waste the pattern removes . . . 102

Kind 1: Fixed-size window . . . 103

Kind 2: Variable-size window . . . 105

Why the variable window is O(n) . . . 107

Problem: longest sub-string without repeating characters . . . 107

When the pattern does not apply . . . 108

Cost summary . . . 109

11. Pattern 4: Prefix Sums . . . 110

The problem it solves . . . 110

Building the array . . . 111

The range-sum formula . . . 112

Where prefix sums beat sliding windows . . . 113

Prefix counts, not just sums . . . 115

Two-dimensional prefix sums . . . 116

Choosing between the three range techniques . . . 117

12. Pattern 5: Binary Search . . . 119

The guessing game . . . 119

The code, line by line . . . 120

Why it cannot miss an element . . . 122

Variant: first and last occurrence . . . 123

Binary search on the answer . . . 125

The standard library . . . 126

Cost summary . . . 127

13. Pattern 6: In-Place Modification . . . 129

What “in place” means . . . 129

Technique 1: The write pointer . . . 130

Technique 2: The reversal trick for rotation . . . 130
Technique 3: Using indices as storage . . . 133
When in-place is impossible . . . 135

14. Pattern 7: Frequency Counting . . . 137

The waste the pattern removes . . . 137
Two tools . . . 138
Building and using a frequency table . . . 139
The complement trick: Two Sum, unsorted . . . 140
Counting with a plain array: anagrams . . . 142
Sets: when you only need “have I seen it?” . . . 143
What a hash map really costs . . . 143
Cost summary . . . 144

15. Pattern 8: Sorting-Based Approaches . . . 146

What sorting buys you . . . 146
Using std::sort . . . 147
Sorting structures without losing the association . . . 148
When sorting is the right move . . . 149
When sorting is the wrong move . . . 151
Choosing between the three approaches . . . 152
What std::sort actually is . . . 153

16. Pattern 9: Kadane’s Algorithm . . . 155

The problem . . . 155
From brute force to the insight . . . 156
The code . . . 157
Recovering the sub-array itself . . . 159

The variation: maximum product sub-array . . . 160

The family of problems this shape solves . . . 161

17. Pattern 10: Matrix Patterns . . . 164

Rotating a matrix by 90° . . . 164

Spiral traversal . . . 166

Set matrix zeroes: the trap . . . 167

Searching a sorted matrix: the staircase walk . . . 169

Flood fill and the four-neighbour walk . . . 170

Cost summary . . . 171

Part IV — The Array Problem Set 173

18. Foundational Array Problems . . . 174

How these are presented . . . 174

1. Largest element . . . 175

2. Second-largest element . . . 175

3. Check whether an array is sorted . . . 175

4. Reverse an array . . . 176

5. Remove duplicates from a sorted array . . . 176

6. Move zeroes to the end . . . 176

7. Rotate an array by k . . . 177

8. Union of two sorted arrays . . . 177

9. Missing number . . . 177

10. Maximum consecutive ones . . . 179

11. Single number . . . 179

The whole set, at a glance . . . 180

19. Intermediate Array Problems . . . 182

1. Two Sum 182

2. Best Time to Buy and Sell Stock 183

3. Maximum sub-array sum 183

4. Majority element 183

5. Sort an array of 0s, 1s and 2s 184

6. Merge two sorted arrays in place 186

7. Find the duplicate number 187

8. Product of array except self 189

9. Sub-array sum equals K 190

10. Longest sub-array with a given sum 190

The whole set, at a glance 191

20. Advanced Array Problems 193

1. Search in a rotated sorted array 193

2. First and last position of an element 195

3. Trapping Rain Water 196

4. Maximum product sub-array 199

Where to go from here 200

Part V — Pointers and Memory 201

21. Pointers, References and Dereferencing 202

The one-sentence definition 202

The two operators 203

Dereferencing changes the original 204

nullptr: a pointer to nothing 205

Pointers to structs, and the arrow 206

References: an alias, not a pointer 207

Passing to functions: three ways . . . 209

Pointer arithmetic, and the link back to arrays . . . 210

The three pointer bugs . . . 211

What you should take away . . . 211

22. The Stack, the Heap, new and delete . . . 213

Two regions . . . 213

What new actually does . . . 214

Memory leaks . . . 215

The other two heap bugs . . . 217

Ownership . . . 218

Smart pointers, and why this book uses raw ones . . . 219

Cost summary . . . 220

Part VI — Linked Lists From Scratch 222

23. What a Linked List Is, and Why It Exists . . . 223

The array's weakness, restated . . . 223

The node . . . 224

The list . . . 225

Why contiguity is not needed . . . 226

Reaching the i-th element . . . 226

The complete comparison . . . 227

The three variants . . . 228

What is coming . . . 229

24. Building a Linked List From Scratch . . . 231

The Node definition . . . 231

Building the smallest possible list . . . 234

Traversing: the loop you will write a hundred times . . . 236

Counting, searching, updating . . . 237

Building from an array, and freeing everything . . . 239

Cost summary . . . 240

25. Insertion . . . 242

The rule that makes every insertion correct . . . 242

1. Insert at the front . . . 243

2. Insert at the back . . . 245

3. Insert at a position . . . 247

4. Insert after a given node . . . 249

All four, compared . . . 250

The symptom of getting the order wrong . . . 250

26. Deletion . . . 252

The two rules . . . 252

1. Delete the first node . . . 253

2. Delete the last node . . . 254

3. Delete at a position . . . 256

4. Delete by value . . . 258

Clearing the whole list . . . 261

The three deletion bugs . . . 261

Cost summary . . . 262

27. Doubly and Circular Linked Lists . . . 264

The doubly linked node . . . 264

Insertion in a doubly linked list . . . 265

Deletion: the reason doubly linked lists exist . . . 267

Circular linked lists . . . 269

Choosing a variant 272

28. Mastering Pointer Manipulation: Reversal 274

The problem 274

Why this is harder than reversing an array 275

The code 275

The complete trace 276

Why nextNode must be saved first 279

What each wrong ordering actually produces 279

The complete program 281

Recursive reversal 282

Reversing part of a list 284

Part VII — Linked List Patterns and Problems 286

29. The Linked List Patterns 287

Pattern 1: The dummy node 287

Pattern 2: Fast and slow pointers 290

Pattern 3: The fixed gap 292

Pattern 4: Floyd's cycle detection 293

Pattern 5: Merge and split 297

Pattern 6: Recursion on lists 299

The pattern map 300

30. Essential Linked List Problems I 301

1. Reverse Linked List 301

2. Middle of the Linked List 302

3. Linked List Cycle 302

4. Linked List Cycle II 303

5. Merge Two Sorted Lists . . . 305

6. Remove Nth Node From End . . . 306

The six, at a glance . . . 307

31. Essential Linked List Problems II . . . 309

1. Palindrome Linked List . . . 309

2. Intersection of Two Linked Lists . . . 312

3. Add Two Numbers . . . 313

4. Remove Duplicates from a Sorted List . . . 315

5. Delete a Node, given only that node . . . 317

6. Reorder List . . . 318

The six, at a glance . . . 320

32. Advanced Linked List Problems . . . 321

1. Reverse Nodes in k-Group . . . 321

2. Rotate List . . . 324

3. Sort List . . . 326

4. Copy List with Random Pointer . . . 328

The four, at a glance . . . 330

Part VIII — Thinking Like a Problem Solver 332

33. Arrays versus Linked Lists: The Complete Picture . . . 333

The master table . . . 333

The single idea behind all of it . . . 334

Why array indexing is O(1): the exact reason . . . 335

The claim people get wrong . . . 336

What the tables hide: real hardware . . . 337

Choosing, for a stated workload . . . 338

The questions to ask before choosing . . . 338

34. How to Solve Any DSA Problem . . . 340

The method . . . 340

The bottleneck-to-pattern table . . . 342

The method applied, end to end . . . 342

Using the method under time pressure . . . 347

When the method does not produce an answer . . . 347

35. C++ Implementation Standards . . . 349

1. Signatures say what the function does . . . 349

2. Use std::vector, and know what is expensive . . . 350

3. One consistent Node definition . . . 351

4. Memory rules . . . 351

5. Naming . . . 352

6. Say whether the function modifies its input . . . 353

7. Handle edge cases explicitly, or show that they fall out . . . 353

8. Avoid cleverness before clarity . . . 354

9. Make every example compilable . . . 354

10. The pre-submission checklist . . . 355

Part IX — Practice, Revision, Mastery 357

36. The Five-Level Practice Roadmap . . . 358

How to use this . . . 358

Level 1 — Absolute beginner . . . 359

Level 2 — Basic operations . . . 359

Level 3 — Pattern recognition . . . 360

Level 4 — Intermediate interview problems . . . 361

Level 5 — Advanced interview problems 362

The problems worth revisiting 363

37. Quizzes and Exercises 365

How to use this chapter 365

Set A — Multiple choice 365

Set B — Trace the code 366

Set C — Find the bug 368

Set D — Write the code 370

Set E — Complexity analysis 371

38. The Complete Answer Key 373

Chapter 37, Set A — Multiple choice 373

Chapter 37, Set B — Trace the code 374

Chapter 37, Set C — Find the bug 375

Chapter 37, Set D — Write the code 376

Chapter 37, Set E — Complexity analysis 377

Part I — Chapters 1 to 3 377

Part II — Chapters 4 to 7 378

Part III — Chapters 8 to 17 380

Part IV — Chapters 18 to 20 385

Part V — Chapters 21 and 22 387

Part VI — Chapters 23 to 28 388

Part VII — Chapters 29 to 32 391

Part VIII and IX — Chapters 33 to 36 392

39. The Mock Interview 395

What is actually being assessed 395

Round 1 — Warm-up (15 minutes) 396

Round 2 — The standard question (25 minutes) 397

Round 3 — The harder question (35 minutes) 398

Round 4 — The follow-up round (20 minutes) 399

The five behaviours that cost offers 400

How to run a mock on your own 400

40. Reference: Plan, Checklist, Glossary 401

The thirty-day plan 401

Pattern summary sheet 403

Complexity summary sheet 404

The revision checklist 405

Glossary 405

Where to go next 407

PART I

Before Anything Else

Three ideas hold up everything that follows: what a data structure is, what a computer's memory actually looks like, and how we measure whether an idea is fast or slow.

01 What a Data Structure Really Is

02 Memory, Variables and Addresses

03 Big O From Absolute Zero

CHAPTER

01

What a Data Structure Really Is

Why the arrangement of your data decides the speed of your program

FOUNDATION

Before arrays, before pointers, before a single line of C++, there is one idea to settle: a data structure is a decision about arrangement, and that decision — made once, early — fixes what your program can do quickly for the rest of its life. This chapter builds that idea from a kitchen drawer up to a formal definition, and shows you the two arrangements this book is about.

WHAT YOU WILL LEARN

- Say in your own words what a data structure is and why anybody needs one
- Explain why *where* data sits in memory changes how fast you can use it
- Describe the two fundamental arrangements: one block, or many linked pieces
- Read the shape of a problem and guess which arrangement suits it
- Understand what "operation" means and why we count operations at all

Start with a drawer

Imagine two kitchen drawers.

The first drawer is a mess. Forks, knives, spoons, a bottle opener and three batteries, all thrown in together. The second drawer has a cutlery tray: forks in one slot, knives in the next, spoons in the third.

Both drawers hold the same objects. The same forks, the same knives. Nothing has been added and nothing removed. Yet one of them is obviously better, and if I ask you *why*, you will say something like "because in the second one you can **find** things".

That is the whole subject. You have just explained data structures.

DEFINITION | Data structure
A **data structure** is a way of arranging data in a computer's memory, chosen so that the operations you care about are fast.
Two parts matter in that sentence. *Arranging* — the data is laid out in some specific shape. *The operations you care about* — different jobs want different shapes. There is no best drawer, only a drawer that is best for what you do most.

Notice what the definition does **not** say. It does not say a data structure makes your data smaller, or more correct, or prettier. It changes exactly one thing: **how much work it takes to do something with the data.**

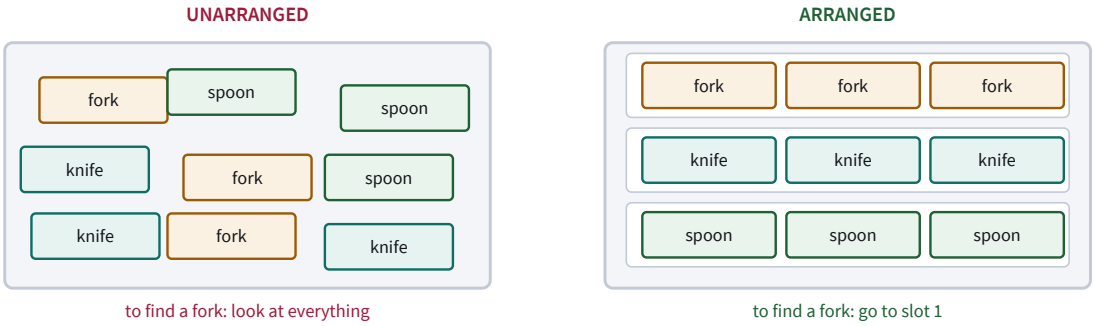


Figure 1.1 Two drawers, identical contents. The arrangement is the only difference, and the arrangement is everything: finding a fork in the left drawer means searching the whole drawer; in the right one you go straight to the slot.

What “data” means here

Let us be careful, because “data” is one of those words that sounds obvious and then turns out to be vague.

> **DEFINITION | Data**
> **Data** is any collection of values your program is holding on to. A list of exam marks. The names of everybody in a class. The pixels of a photograph. The moves played so far in a chess game.
> In this book, data will almost always be a group of numbers, because numbers keep the diagrams short. Everything you learn applies identically to names, records, or anything else.

So the subject of this book is: *you have a group of values; how should you lay them out so your program can work with them quickly?*

Why arrangement changes speed at all

Here is the part that trips up beginners, so we will go slowly.

A computer’s memory is not magic. It is a very long row of numbered boxes. Each box holds a small amount of information, and each box has a number — its **address** — the way houses on a street have numbers. (Chapter 2 does this properly; for now the picture is enough.)

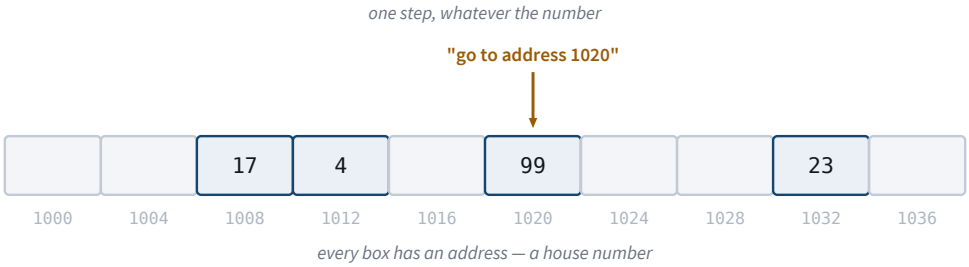


Figure 1.2 Memory as a street of numbered houses. The processor can jump to any house instantly if it knows the number. What it cannot do is know a number it was never told.

The processor is very good at one thing: *"go to address 4,000 and tell me what is there."* That takes the same tiny amount of time no matter which address you name. Address 8 and address 8,000,000 cost the same.

But the processor is only good at that when it **knows the address**. And whether it knows the address depends entirely on how you arranged the data.

Two arrangements are possible, and they lead to two completely different worlds:

> **KEY INSIGHT | The two arrangements, and the whole book**
> **Arrangement 1 — one solid block.** Put all the values side by side in memory, with no gaps. Now, if you know where the block starts, you can *calculate* the address of value number 7 with a tiny piece of arithmetic. No searching. This is an **array**, and it is Parts II–IV.
> **Arrangement 2 — scattered pieces, each holding a signpost.** Put each value wherever there is room, and have each piece store the address of the next piece. Now you cannot calculate anything; you must follow the signposts one by one. But you can add a new piece anywhere without disturbing anything else. This is a **linked list**, and it is Parts V–VII.
> Everything else in this book — every strength, every weakness, every interview question — comes from these two sentences.

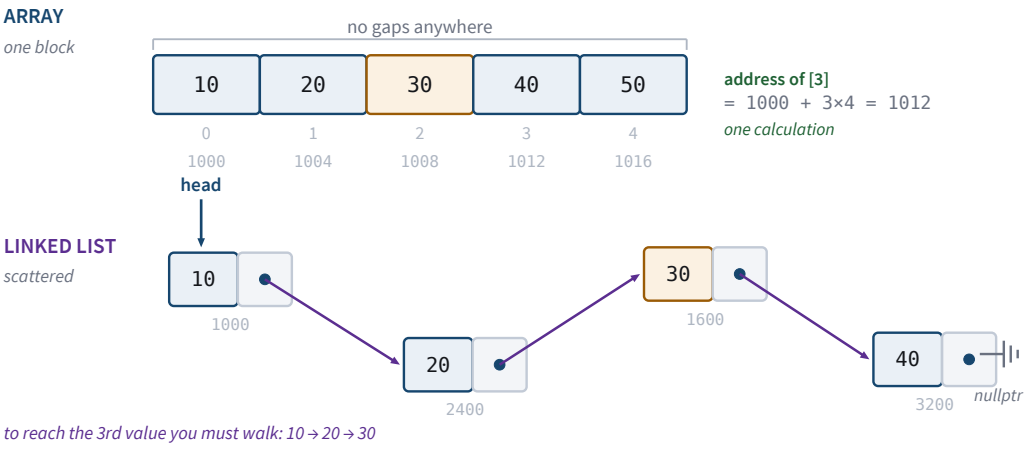


Figure 1.3 The two arrangements side by side. Top: an array, one unbroken block, so element 3's address is pure arithmetic. Bottom: a linked list, scattered across memory, where the only way to reach element 3 is to walk from the front.

Look at the two pictures and ask yourself two questions.

"How do I get the 3rd element?" In the array you compute an address and jump — one step. In the linked list you start at the front and follow three signposts — three steps. With a million elements it would be one step versus a million steps. The array wins enormously.

"How do I insert a new value at the front?" In the array, every existing value has to shuffle one place to the right to make room — with a million elements that is a million moves. In the linked list you make one new piece and point it at the old front — one step. The linked list wins enormously.

Neither structure is better. They are better at *different questions*. Learning which question you are being asked is the skill.

What we mean by an "operation"

I keep saying "fast" and "steps". Let us make that precise enough to be useful without being precise enough to be painful.

> **DEFINITION | Operation**
> An **operation** is one thing you ask a data structure to do: *get the element at position 5, insert the value 42 at the front, find out whether 99 is in here, delete the last element.*
> When we say a data structure is fast, we always mean: *fast at some particular operation.* Never fast in general.

The operations we will care about, over and over, are these six:

Operation	What it means	Everyday version
Access	Get the element at a known position	"Give me the 7th book on the shelf"
Search	Find whether a value is present, and where	"Is there a copy of *Dune* here?"
Insert	Add a new value at some position	"Squeeze this book in at position 3"
Delete	Remove the value at some position	"Take out the 3rd book and close the gap"
Update	Change the value at a known position	"Replace the 7th book with another one"
Traverse	Visit every element once, in order	"Read the title of every book"

For every structure in this book we will fill in a table: what does each of these six cost? Those tables are the entire practical content of data structures. Everything else is explanation.

INTUITION | **Why we count steps instead of timing with a stopwatch**
A stopwatch measures your laptop on a Tuesday. If you time the same program on a faster machine, the number changes, but the *shape* of the answer does not: if one method needed a million steps and another needed three, that stays true on every machine ever built.
So we count steps, not seconds. Chapter 3 turns this into a proper notation.

A first look at the cost of a wrong choice

Let us make this concrete with a small story, because abstract claims are unconvincing.

Suppose you are storing the names of students arriving at an exam hall, and students arrive one at a time. You decide to keep the list **sorted alphabetically at all times**, so you insert each new name into its correct position.

With an array, every insertion in the middle means shifting everybody after it. If 10,000 students arrive, the total work is roughly $10{,}000 \times 10{,}000 / 2 = 50$ million small moves. On a modern machine that is fast enough that you would not notice — perhaps a tenth of a second.

Now make it a million students. The work is not a hundred times larger; it is ten thousand times larger — about 500 billion moves. Your program, which seemed fine in testing, now takes several minutes. Nothing broke. No bug was introduced. The arrangement simply did not scale.

KEY INSIGHT | **The real lesson**
Choosing the wrong data structure does not usually produce a *wrong* program. It produces a program that is correct and unusably slow — and that is much harder to notice, because it works perfectly on your small test data.

This is why the subject matters, and it is why interviewers ask about it. They are not testing whether you can write a loop. They are testing whether you can predict what will happen when the input gets large.

The map of this book

Here is where we are going, so you always know why you are reading something.

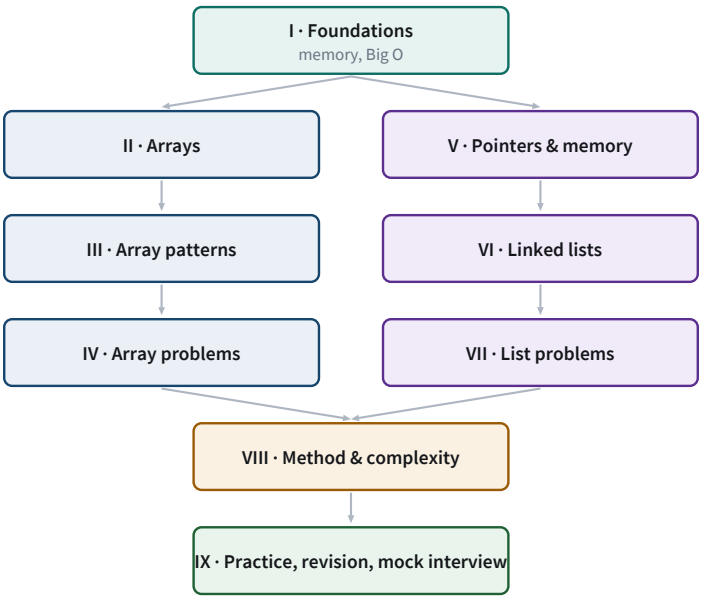


Figure 1.4 The road through this book. Each box depends on the ones before it, which is why the order matters: pointers before linked lists, memory before arrays, Big O before everything.

Notice that Part V — pointers and memory — sits *between* arrays and linked lists. That placement is deliberate. Most people who struggle with linked lists are not struggling with linked lists at all; they are struggling with pointers, and nobody ever taught them pointers properly. We will not make that mistake.

Why arrays and linked lists first

There are dozens of data structures: stacks, queues, trees, graphs, hash tables, heaps, tries. Why spend an entire book on two of them?

Because the other dozens are **built out of these two**.

- A **stack** and a **queue** are an array or a linked list with a rule about which end you are allowed to touch.
- A **hash table** is an array of buckets, where each bucket is usually a linked list.
- A **graph** is most often stored as an array of linked lists.
- A **tree** is a linked list where each node has two signposts instead of one.
- A **heap** is a tree pretending to be an array.

Learn these two properly and the rest of the subject is variations. Learn them badly and every later structure will feel like magic.

> **INTERVIEW ANGLE | Why interviewers love these two**
> Roughly a third of all coding-interview questions are array questions, and linked list questions are the classic test of whether you can manipulate pointers without losing your place. More importantly: these two structures are where an interviewer can see you *think*, because the brute-force solution is always obvious and the good solution never is.

What you should take away

CHAPTER RECAP | **Chapter 1 in six lines**

- A data structure is an **arrangement** of data chosen to make certain operations fast.
- Memory is a long row of numbered boxes; the processor can jump to any box instantly **if it knows the number**.
- An **array** is one solid block, so positions can be *calculated*.
- A **linked list** is scattered pieces joined by signposts, so positions must be *walked to*.
- Arrays are fast at reaching a position; linked lists are fast at adding and removing pieces.
- There is no best structure, only a structure that fits the operations you perform most.

EXERCISES | **Before you go on**

1. A music app stores your playlist. You mostly press “skip to track 47” and almost never reorder tracks. Which arrangement fits better, and why?
2. A text editor stores the characters of a document. You mostly type in the middle. Which arrangement fits better, and why?
3. In your own words, and without using the word “fast”, explain why putting values side by side in memory lets you jump straight to the 7th one.
4. Name one operation an array is bad at, and say *why* it is bad at it in terms of memory, not in terms of Big O (which you have not met yet).
5. The chapter claimed “a hash table is an array of buckets, where each bucket is usually a linked list”. Without knowing anything about hash tables, guess why both structures are needed instead of just one.

Answers are in Chapter 38. Do not skip them — the answers explain reasoning the chapter did not have room for.

CHAPTER

02

Memory, Variables and Addresses

What is really happening when you write int x = 42;

FOUNDATION PREREQS: Chapter 1

Everything in this book — array indexing, pointer arithmetic, why insertion shifts elements, why a linked list node can live anywhere — is a consequence of how memory works. This chapter builds that picture from a single bit up to a block of integers, using nothing but diagrams and three tiny C++ programs you can run yourself.

WHAT YOU WILL LEARN

- Explain what a bit, a byte and an address are, in plain words
- Say exactly what the computer does when it reads the line `int x = 42;`
- Use `sizeof` and `&` to see the size and address of any variable
- Read a memory diagram and work out the address of any element in a block
- Explain why values stored side by side can be reached by calculation
- Describe the difference between a variable's *name*, its *address*, and its *value*

The smallest thing there is

A computer stores everything — numbers, letters, pictures, this sentence — as switches that are either off or on. One switch is called a **bit**, and its value is written as 0 or 1.

One bit is not very useful. You can say yes or no with it and nothing more. So bits are grouped into eights.

DEFINITION | Bit and byte
A **bit** is one switch: 0 or 1.
A **byte** is a group of 8 bits. Because each of the 8 bits can be 0 or 1, one byte has 2^8=256 possible patterns, so a byte can represent any whole number from 0 to 255.
The byte is the unit the computer actually works in. When we talk about memory, we count bytes.

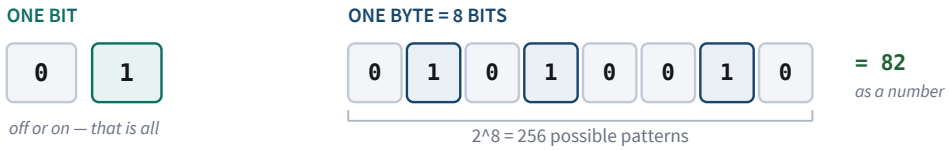


Figure 2.1 One bit is a switch. Eight bits make a byte, and a byte has 256 possible patterns — enough for a small number or a single character of English text.

256 is not a lot. You cannot store the number 1,000,000 in one byte. So for bigger numbers the computer uses **several bytes together**. In C++, the type `int` normally uses **4 bytes** — that is 32 bits, which gives $2^{32} \approx 4.3$ billion patterns, enough to hold any whole number from about −2 billion to +2 billion.

C++ type	Typical size	What it can hold
`bool`	1 byte	`true` or `false`
`char`	1 byte	One character, e.g. `'A'`
`int`	4 bytes	Whole numbers, about ±2 billion
`long long`	8 bytes	Whole numbers, about $\pm 9 \times 10^{18}$
`float`	4 bytes	Decimal numbers, ~7 digits of precision
`double`	8 bytes	Decimal numbers, ~15 digits of precision
a pointer	8 bytes	An address (on a 64-bit machine)

> **NOTE | "Typical" size**
> The C++ standard does not force these sizes; it only sets minimums. On every machine you are likely to use in 2026, the table above is correct. If you ever need certainty, `sizeof(int)` tells you the truth on *your* machine, and we use it in the first program below.

Memory is a numbered street

Now picture the computer's memory — its RAM — as an extremely long street of houses, where **each house holds exactly one byte**, and the houses are numbered $0, 1, 2, 3, \ldots$ with no gaps.

> **DEFINITION | Address**
> The number of a byte in memory is called its **address**. It is just a position: "the 1,000th byte". Addresses start at 0 and go up by 1 for each byte.
> The crucial property: **if the processor knows an address, it can read that byte immediately** — it does not have to walk past the other bytes to get there. Address 4 and address 4,000,000,000 take the same amount of time.

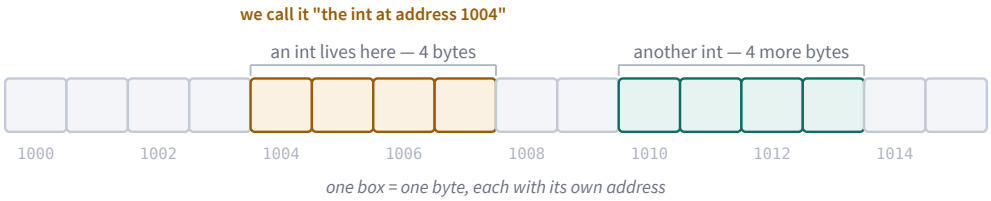


Figure 2.2 Memory, byte by byte. Each small box is one byte with its own address. A four-byte int occupies four consecutive boxes, and we name the whole group by the address of its first byte.

Look carefully at the picture. An `int` is 4 bytes, so it occupies **four houses in a row**, say addresses 1000, 1001, 1002 and 1003. When we say "the int lives at address 1000" we mean "it starts at 1000 and continues for `sizeof(int)` bytes".

That phrase — *starts here and continues for this many bytes* — is the single most important idea in this chapter. Hold on to it.

What a variable actually is

Now we can answer the question properly.

The simplest C++ program that has anything in memory

```
1  #include <iostream>
2
3  int main() {
4      int x = 42;
5      std::cout << x << "\n";
6      return 0;
7  }
```

Output:

```
42
```

What happened when the computer ran line 4?

1. The compiler decided *"this function needs 4 bytes of space for a variable called `x`"*, and set aside four consecutive bytes somewhere. Say those bytes turn out to be at addresses 1000–1003.
2. The pattern of bits meaning 42 was written into those four bytes.
3. The name `x` was recorded — **in the compiler, not in memory** — as "the `int` that lives at address 1000".

That third point surprises people, so let us say it loudly:

KEY INSIGHT | The name is not stored anywhere
The word `x` does not exist while your program runs. It exists only while your program is being *compiled*. The compiler translates every use of `x` into "the 4 bytes at address 1000" and then throws the name away.
A variable is nothing but **a name the compiler gives to an address**, plus a type that says how many bytes to read there and how to interpret them.




```
int x = 42;
```

Figure 2.3 What `int x = 42;` actually creates. The name lives in the compiler, the value lives in memory, and the address is the bridge between them. Three different things that beginners often merge into one.

Three separate things, and you must be able to tell them apart:

Thing	Example	Where it lives
The **name**	`x`	In the compiler only
The **address**	1000	A number; you can print it
The **value**	42	In the bytes at that address

Seeing it for yourself

C++ gives you two operators that let you look at all of this directly.

- `sizeof(thing)` — how many **bytes** does this type or variable occupy?
- `&variable` — the **address** of this variable. Read `&` out loud as "address of".

Looking at memory directly

```cpp
#include <iostream>

int main() {
    int x = 42;
    char c = 'A';
    double d = 3.14;

    std::cout << "sizeof(int)    = " << sizeof(int)    << " bytes\n";
    std::cout << "sizeof(char)   = " << sizeof(char)   << " bytes\n";
    std::cout << "sizeof(double) = " << sizeof(double) << " bytes\n\n";

    std::cout << "value of x   = " << x  << "\n";
```

```
13      std::cout << "address of x = " << &x << "\n";
14      std::cout << "address of c = " << (void*)&c << "\n";
15      std::cout << "address of d = " << &d << "\n";
16      return 0;
17  }
```

A possible output (your addresses will be different — that is fine and expected):

```
sizeof(int)    = 4 bytes
sizeof(char)   = 1 bytes
sizeof(double) = 8 bytes

value of x   = 42
address of x = 0x7ffd4b2a3c5c
address of c = 0x7ffd4b2a3c5b
address of d = 0x7ffd4b2a3c50
```

Line by line, the parts that matter:

- **Line 8** — `sizeof(int)` gives `4`. This is not a guess; it is your machine answering.
- **Line 13** — `&x` is the address of `x`. It prints as something like `0x7ffd4b2a3c5c`. The `0x` prefix means the number is written in **hexadecimal** (base 16) because addresses are long and hex is shorter. You never need to do hex arithmetic in this book; just read it as "some big number".
- **Line 14** — `(void*)&c`. Without the `(void*)`, `std::cout` would see a `char*` and try to print the *text* starting at that address instead of the address itself. Casting to `void*` says "treat this as a plain address". This is a C++ quirk, not a concept — do not let it worry you.

> **COMMON PITFALL | Addresses change on every run**
> If you run that program twice you will get different addresses. Modern operating systems deliberately move programs around in memory each time they start, for security. The *relationships* between addresses stay the same; the absolute numbers do not. Never write code that depends on a specific address.

Two variables, side by side? Not necessarily

Here is where beginners get an incorrect mental picture. If you write:

```
int a = 10;
int b = 20;
int c = 30;
```

you might assume `a`, `b` and `c` sit next to each other in memory in that order. **They might. They might not.** The compiler is free to arrange separate variables however it likes — reordered, with gaps between them for alignment, or kept entirely in processor registers and never written to memory at all.

This matters enormously, because it is exactly the problem an array solves.

> **KEY INSIGHT | Why arrays exist, in one sentence**
> Three separate `int` variables give you no guarantee about where they are. An array of three `int` s gives you a **promise**: these values are in one block, in order, with no gaps. That promise is the entire reason arrays are fast, and it is the subject of Chapter 4.

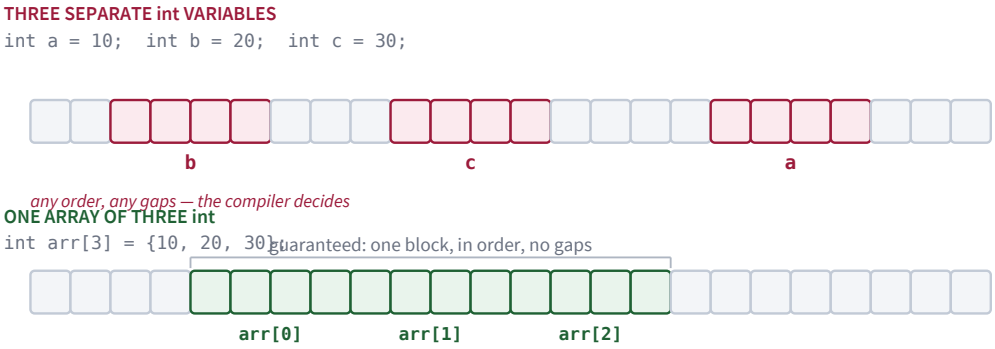


Figure 2.4 Three separate variables versus an array of three. On the left the compiler may place them anywhere, in any order. On the right the language guarantees one contiguous block — and a guarantee is something you can calculate with.

Address arithmetic: the pay-off

Because an array is a guaranteed block, you can compute where any element is without looking at memory at all. Here is the rule, and it is worth memorising because it explains half of this book:

> **DEFINITION | The address formula**
> If an array starts at address B and each element takes S bytes, then element number i (counting from 0) lives at address
>
> $$\text{address}(i) = B + i \times S$$
>
> Nothing is searched. It is one multiplication and one addition — a fixed, tiny amount of work no matter how large i is or how long the array is.

Let us check it with a real program.

Seeing the address formula with your own eyes

```cpp
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    std::cout << "element size = " << sizeof(arr[0]) << " bytes\n\n";
    for (int i = 0; i < 5; ++i) {
        std::cout << "arr[" << i << "] = " << arr[i]
                  << "   at address " << &arr[i] << "\n";
    }
    return 0;
}
```

A possible output:

```
element size = 4 bytes

arr[0] = 10   at address 0x7ffe8c3b1a40
arr[1] = 20   at address 0x7ffe8c3b1a44
arr[2] = 30   at address 0x7ffe8c3b1a48
arr[3] = 40   at address 0x7ffe8c3b1a4c
arr[4] = 50   at address 0x7ffe8c3b1a50
```

Look at the last two hexadecimal digits: `40`, `44`, `48`, `4c`, `50`. In hexadecimal those go up by 4 each time (`4c` is 76 in ordinary numbers, and `50` is 80). Exactly four bytes apart, exactly as the formula promises, exactly `sizeof(int)`.

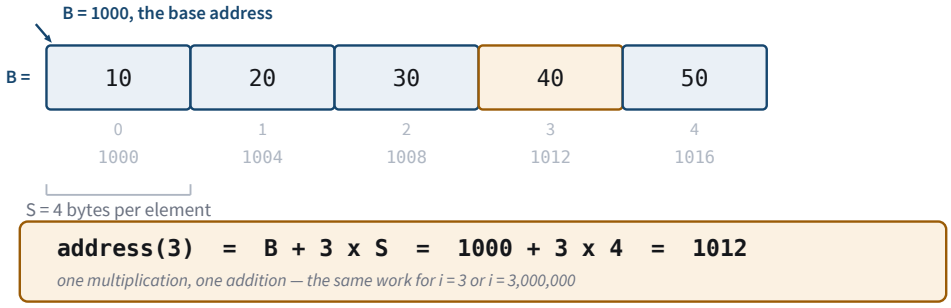


Figure 2.5 The address formula in action. Element 3 of this array is found by one multiplication and one addition — never by looking at elements 0, 1 and 2 first.

> **INTUITION | An analogy for the formula**
> A hotel corridor where every room is the same width. If room 0 is at the start of the corridor and each room is 4 metres wide, you know room 37 is 148 metres along without walking past rooms 1 to 36.
> Now imagine a corridor where rooms are different sizes and scattered across several buildings. To find room 37 you have to ask at room 0, which tells you where room 1 is, and so on. That is a linked list, and that is why it is slower to reach a position — and faster to add a room.

Where memory comes from: a first look

Not all memory is the same. There are two places your data can live, and the difference becomes critical in Part VI. For now, a one-paragraph preview.

DEFINITION | The stack and the heap
The **stack** is memory the compiler manages for you automatically. Local variables like `int x = 42;` live here. It is fast, and it is cleaned up for you the moment the function ends. The catch: the size must be known when the program is compiled, and it is small — typically 1–8 MB in total.
The **heap** is memory you ask for while the program is running, with `new` (and give back with `delete`). It can be large and its size can be decided at run time. The catch: **you** are responsible for giving it back, and if you forget, the memory is lost until the program exits.
Every linked list node you will ever create lives on the heap. Chapter 22 covers this properly.

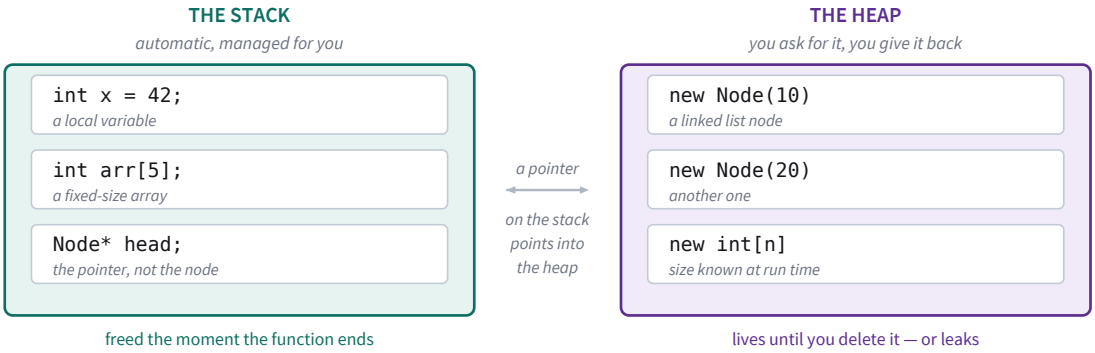


Figure 2.6 The two regions. Local variables are automatic and short-lived; anything created with `new` lives until you say otherwise. Every node of every linked list in this book is a heap object.

Common misunderstandings

COMMON PITFALL | Four things that are not true
"A variable is a box." It is closer to a *label stuck on a stretch of bytes*. The distinction matters as soon as two labels refer to the same bytes, which is what references and pointers do.
"`int` is always 4 bytes." Overwhelmingly common, not guaranteed. Use `sizeof` when it matters.
"Address 1000 is slower to reach than address 8." No. Reading any single address costs the same. (Reading many *nearby* addresses is faster than many *scattered* ones, for cache reasons — but that is a refinement, not a contradiction. Chapter 33 revisits it.)
"Declaring a variable sets it to 0." In C++, a local variable you do not initialise contains **whatever bytes happened to be there already**. Reading it is undefined behaviour and a classic source of bugs that appear and disappear at random. Always initialise.

The uninitialised-variable trap

```
#include <iostream>

int main() {
    int good = 0;        // explicitly initialised - always 0
    int bad;             // uninitialised - contains leftover bytes
```

```
    std::cout << good << "\n";
    std::cout << bad  << "\n";   // could print 0, 32767, -1094795586,
        anything
    return 0;
}
```

The dangerous part is that `bad` very often prints `0` on a simple test, which convinces you the code is fine. Then it prints garbage the day you demonstrate it to somebody.

What you should take away

CHAPTER RECAP | **Chapter 2 in seven lines**

- A **bit** is one switch; a **byte** is 8 bits; memory is a long row of numbered bytes.
- An **address** is the number of a byte. The processor reaches any address in the same tiny time.
- A multi-byte value like an `int` occupies **consecutive** bytes, and we name it by the address of its first byte.
- A **variable** is a name the compiler gives to an address, plus a type that says how many bytes and how to read them.
- `sizeof` tells you the byte count; `&` tells you the address.
- Separate variables have **no guaranteed arrangement**; an array does, and that guarantee is what makes indexing fast.
- address$(i) = B + i \times S$ — the formula behind every array access in this book.

EXERCISES | **Before you go on**

1. How many distinct patterns can 2 bytes represent? Show your working.
2. An array of `double` starts at address 2000. At what address does element 6 begin? Which formula did you use, and what was S?
3. Explain, to somebody who has never programmed, the difference between a variable's *name*, its *address*, and its *value*.
4. Write a program that prints `&x` and `&y` for two `int` locals. Run it three times. Do the addresses change between runs? Does the *difference* between them change?
5. Why can the compiler put three separate `int` variables anywhere it likes, but not three elements of an `int` array?
6. A student says "my linked list is slow because its nodes are at high memory addresses". Using this chapter, explain why that reasoning is wrong, and suggest the real reason linked lists are slower to traverse.

CHAPTER

03

Big O From Absolute Zero

How to say "this gets slow when the input grows" in one symbol

FOUNDATION PREREQS: Chapter 2

Big O is not mathematics for its own sake; it is a vocabulary for a single practical question — what happens to my program when the input gets ten times bigger? This chapter builds the notation from counting lines of code, explains every rule (including the two that look like cheating), and ends with a recipe you can apply to any C++ function on sight.

WHAT YOU WILL LEARN

- Count the operations a piece of C++ code performs, as a function of input size
- Explain why we drop constants and keep only the fastest-growing term
- Recognise $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ and $O(2^n)$ on sight
- Explain why nested loops usually give $O(n^2)$ and why halving gives $O(\log n)$
- Distinguish best case, average case and worst case, and know which one we quote
- Measure space complexity and tell auxiliary space from total space
- Read the complexity of unfamiliar code using a four-step recipe

The question Big O answers

You have written two programs that both do the job. One is 40 lines, one is 8 lines. Which should you use?

You cannot answer by timing them, because the answer would depend on your laptop, your compiler, how warm your processor is, and what else is running. Run the same test tomorrow and you get a different number.

So we ask a better question — one whose answer is the same on every machine that has ever existed:

KEY INSIGHT | The only question that matters
If I make the input ten times bigger, how much more work does this do?
If the answer is "ten times more work", the program scales well. If it is "a hundred times more work", it scales badly. If it is "no extra work at all", the program is wonderful. Big O is just a compact way of writing that answer.

Counting steps, slowly

Let us count. Here is a function that adds up an array of n numbers.

Summing an array

```cpp
int sum(const std::vector<int>& v) {
    int total = 0;                      // 1 step
    for (int i = 0; i < v.size(); ++i) { //  the loop runs n times
        total += v[i];                  // 1 step, n times  -> n steps
    }
    return total;                       // 1 step
}
```

Count the steps for an array of n numbers:

- `total = 0` happens once. That is **1** step.
- The body `total += v[i]` happens once per element. That is n steps.
- `return total` happens once. That is **1** step.
- The loop bookkeeping (`i < v.size()` and `++i`) is roughly 2 more steps per iteration, so $\mathbf{2}n$.

Total: $3n+2$ steps.

Now do the same for a function that returns the first element:

Getting the first element

```cpp
int first(const std::vector<int>& v) {
    return v[0];                        // 1 step, always
}
```

Total: 1 step. Not 1 step per element — **one step, full stop**, whether the array has 5 elements or 5 billion.

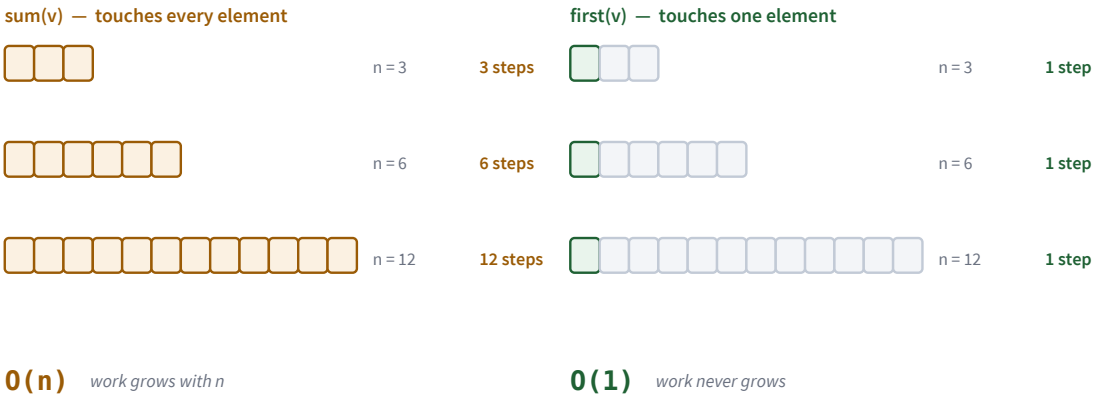


Figure 3.1 Two functions, counted. The first does work proportional to the size of the input; the second does a fixed amount of work. That difference — proportional versus fixed — is the only thing Big O records.

The two rules that look like cheating

We said `sum` takes $3n+2$ steps. Big O throws away almost all of that and writes simply $O(n)$. Two rules do the throwing:

> **DEFINITION | Rule 1 — drop the constants**
> $3n$ becomes n. $100n$ also becomes n. We do not care about the multiplier.

> **DEFINITION | Rule 2 — keep only the fastest-growing term**
> $n^2+500n+9000$ becomes n^2. Everything except the biggest term is discarded.

These look like vandalism. They are not. Here is why each is justified.

Why dropping constants is fair. The constant depends on things we are not trying to measure: your processor, your compiler's optimiser, whether the data is in cache. A $3n$ algorithm on a slow machine and a $30n$ algorithm on a fast machine can take exactly the same wall-clock time. What *cannot* change is that both of them do ten times more work when n gets ten times bigger. That shared, machine-independent fact is what we are recording.

Why dropping smaller terms is fair. Because for large n they stop mattering — not approximately, but overwhelmingly.

n	n^2	$500n$	9000	$500n+9000$ as % of total
10	100	5,000	9,000	99.3%
100	10,000	50,000	9,000	85.5%
1,000	1,000,000	500,000	9,000	33.7%
10,000	100,000,000	5,000,000	9,000	4.8%
1,000,000	10^{12}	5×10^{8}	9,000	0.05%

At $n=10$ the n^2 term is a rounding error. At $n=1{,}000{,}000$ it is essentially the whole cost. Big O describes what happens **as n grows**, and in that limit only the fastest-growing term survives.

> **COMMON PITFALL | What this means in practice**
> Big O deliberately ignores small inputs. An $O(n^2)$ algorithm can easily beat an $O(n\log n)$ one for $n=20$ — this is exactly why real sorting libraries switch to insertion sort for tiny arrays.
> So: Big O is a statement about **scaling**, not a promise about any particular input. Use it to choose between designs; use a real measurement to tune a specific case.

The definition, in English and then in symbols

DEFINITION | Big O
We say an algorithm is $O(g(n))$ if, **once** n **gets large enough**, its step count is **at most** some fixed multiple of $g(n)$.
Formally: $f(n)=O(g(n))$ if there exist constants $c>0$ and n_0 such that $f(n)\le c\cdot g(n)$ for all $n\ge n_0$.
In English: *"beyond some point, g is an upper boundary for how fast this grows, if we are allowed to scale g up by a constant."*

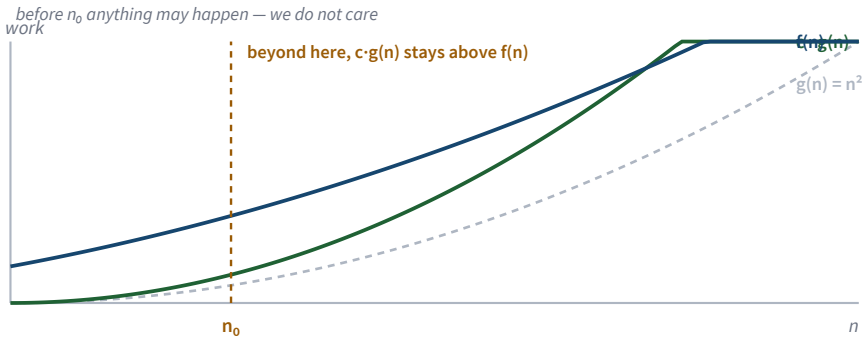


Figure 3.2 What the definition is saying. We may stretch $g(n)$ vertically by any constant c we like, and we may ignore everything to the left of some threshold n_0. All that must be true is that beyond that point the stretched g stays above f.

You will rarely use the formal definition. It is here so that when somebody says "but Big O is only an upper bound", you know what they mean: $O(n)$ code is *also* technically $O(n^2)$, in the same way that somebody 170 cm tall is "at most 3 metres tall". True, but unhelpful. By convention we always quote the **tightest** bound we can.

The six complexities you must know on sight

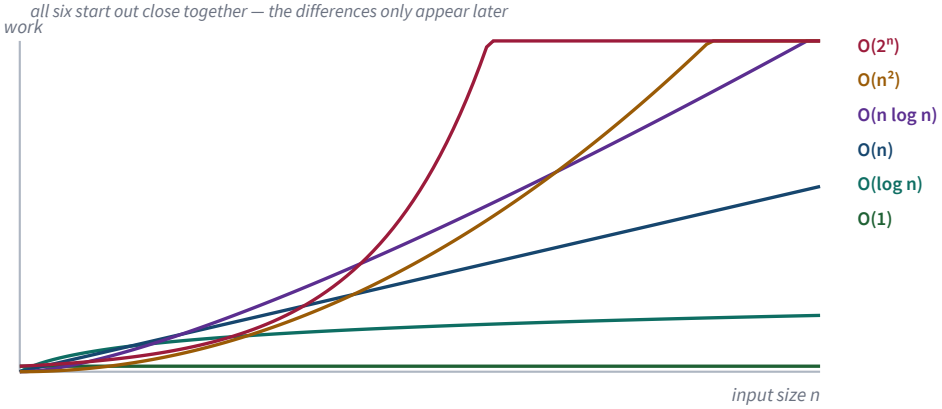


Figure 3.3 The growth hierarchy. Each curve eventually and permanently overtakes everything below it. The only question that matters for a given algorithm is which band it lives in.

Here they are, from best to worst, each with the mental image that makes it stick.

$O(1)$ — constant time

The work does not depend on n at all.

```
int x = v[500];      // one address calculation, one read
v.push_back(9);      // amortised O(1) - see Chapter 5
```

Image: looking up a word when you already know the page number.

$O(\log n)$ — logarithmic time

Each step throws away **half** of what is left.

Image: the guessing game. "I am thinking of a number from 1 to 1000." If you always guess the middle, you need at most 10 questions, because $2^{10}=1024>1000$.

> **DEFINITION | What $\log n$ means here**
> $\log_2 n$ is the answer to: *"how many times can I halve n before I reach 1?"* For $n=1000$ it is about 10. For $n=1{,}000{,}000$ it is about 20. For $n=1{,}000{,}000{,}000$ it is about 30.
> In computer science $\log$ always means $\log_2$ unless stated otherwise, and because constants are dropped, the base does not actually matter.

That table is the whole reason logarithmic algorithms feel like magic: a billion-element array needs only about 30 steps.

$O(n)$ — linear time

You touch every element a fixed number of times.

```
for (int x : v) total += x;     // one pass
```

Image: reading every page of a book once.

$O(n \log n)$ — linearithmic time

You do a logarithmic amount of work for each of n elements, or you split the problem in half repeatedly and do linear work at each level.

Image: sorting. This is the best any comparison-based sort can do, and `std::sort` achieves it.

$O(n^2)$ — quadratic time

For each element, you look at every element.

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        if (v[i] + v[j] == target) { /* ... */ }
```

Image: everybody in a room shaking hands with everybody.

$O(2^n)$ — exponential time

Every extra element **doubles** the work.

Image: trying every possible subset. At $n=50$ there are $2^{50} \approx 10^{15}$ subsets — about 11 days of work if you can check a billion per second. At $n=60$ it is 30 years.

What these mean in real seconds

Assume a machine doing 100 million simple operations per second — roughly right for ordinary C++.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	instant	instant	instant	instant	instant	instant
1,000	instant	instant	instant	instant	0.01 s	—
100,000	instant	instant	0.001 s	0.017 s	100 s	—
1,000,000	instant	instant	0.01 s	0.2 s	2.8 hours	—
10,000,000	instant	instant	0.1 s	2.3 s	11.6 days	—

Table 3.1 What each complexity costs at realistic input sizes. The dashes are not "slow"; they are "will not finish".

> **INTERVIEW ANGLE | The number to memorise**
> Interviewers expect you to know that **about 10^8 simple operations per second** is the right order of magnitude. From that single number you can reason out loud: *"n is 10^5, so $O(n^2)$ is 10^{10} operations, which is about 100 seconds — too slow. I need at least $O(n \log n)$."* Saying that sentence in an interview is worth more than knowing a clever trick.

Why nested loops give $O(n^2)$

This is asked constantly, so let us see it rather than assert it.

The classic quadratic pattern

```
for (int i = 0; i < n; ++i) {        // outer: runs n times
    for (int j = 0; j < n; ++j) {    // inner: runs n times, for EACH i
        doSomething(v[i], v[j]);     // so this runs n x n times
    }
}
```

The inner loop does not run n times in total. It runs n times **for every single value of** `i`, and `i` takes n values. So the body executes $n \times n = n^2$ times.

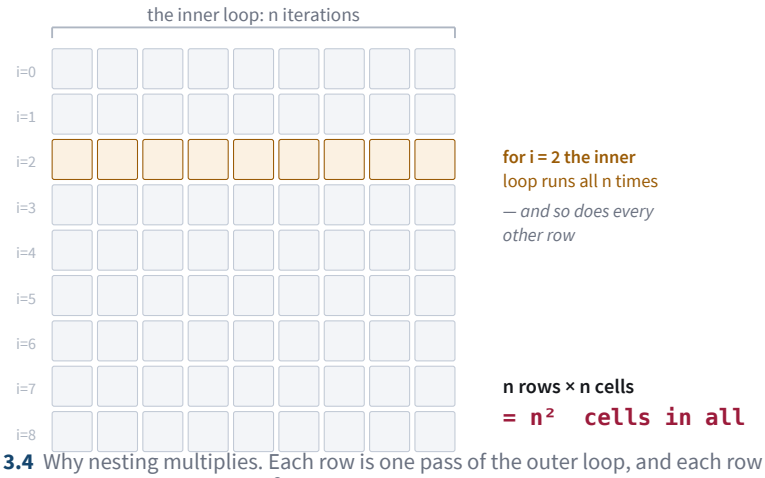


Figure 3.4 Why nesting multiplies. Each row is one pass of the outer loop, and each row contains a full inner loop. Count the cells: n rows of n cells is n^2.

A common variation starts the inner loop at `i + 1`:

The triangular variation - still quadratic

```
for (int i = 0; i < n; ++i)
    for (int j = i + 1; j < n; ++j)
        doSomething(v[i], v[j]);
```

Now the inner loop runs $n-1$ times, then $n-2$, then $n-3$, down to 0. Adding those up:

$$(n-1)+(n-2)+\cdots+1+0=n(n-1)/2=n^2-n/2$$

Drop the constant $1/2$ and the smaller term $-n$, and you are left with n^2. **It is still $O(n^2)$** — it does half the work of the full version, but halving is a constant factor, and constants are dropped.

COMMON PITFALL | Nested loops are not automatically $O(n^2)$
Look at what the loops actually count, not at how deeply they nest.

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < 10; ++j)      // fixed 10, not n
        work();
```

This is $10n$ steps, which is $O(n)$. The inner loop does not depend on n.
Conversely, a *single* loop can be quadratic if its body is linear:

```
for (int i = 0; i < n; ++i)
    v.insert(v.begin(), i);           // each insert shifts n elements: O(
        n)
```

One loop, $O(n^2)$ total. **Always ask what the body costs.**

Why halving gives $O(\log n)$

Suppose you have n items and each step throws away half.

After this many steps	Items remaining
0	n
1	$n/2$
2	$n/4$
3	$n/8$
k	$n/2^k$

You stop when one item is left, so you want the k where $n/2^k=1$, that is $2^k=n$, that is $k=\log_2 n$.



Figure 3.5 Binary search on 16 elements. Four halvings reach a single element, and $\log_2 16=4$. Doubling the array to 32 elements adds exactly one more step, not 16 more.

> **INTUITION | Why logarithms feel so powerful**
> Going from 1,000 to 1,000,000 elements multiplies a linear algorithm's work by 1,000. It multiplies a logarithmic algorithm's work by **two** (from about 10 steps to about 20). This is why sorted data is worth so much, and why Chapter 12 spends a whole chapter on binary search.

Best, average and worst case

The same algorithm can do different amounts of work on different inputs of the same size. Searching an array for a value:

Linear search

```cpp
int find(const std::vector<int>& v, int target) {
    for (int i = 0; i < (int)v.size(); ++i)
        if (v[i] == target) return i;   // might happen immediately
    return -1;                          // or we check everything
}
```

- **Best case:** the target is at index 0. One comparison. $O(1)$.
- **Worst case:** the target is at the end, or absent. n comparisons. $O(n)$.

- **Average case:** if the target is present and equally likely to be anywhere, you check $n/2$ elements on average. Drop the constant: $O(n)$.

> **KEY INSIGHT | Which one do we quote?**
> **The worst case, almost always.** Three reasons:
>
> 1. It is a *guarantee*. "Never slower than this" is a useful promise; "usually fast" is not, because somebody will hand you the unusual input.
> 2. The worst case is often the common case in disguise — searching for something that is not there is the normal outcome of a search.
> 3. Average case requires assuming a distribution of inputs, and real inputs are rarely random.
>
> When this book writes a bare complexity with no qualifier, it means the worst case. Where the average differs importantly (quicksort, hash tables) it is stated explicitly.

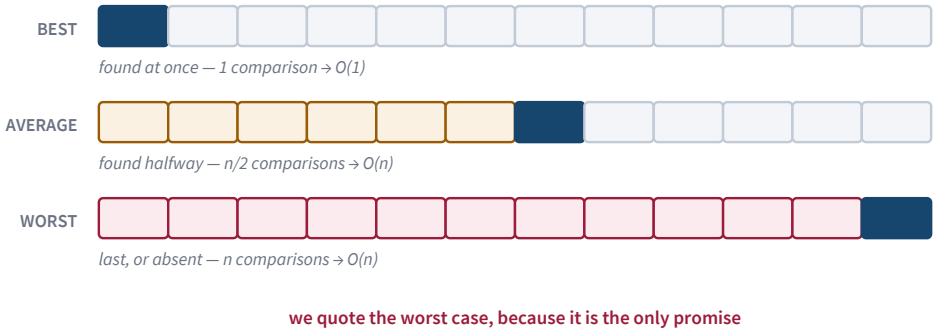


Figure 3.6 The three cases for linear search. The quoted complexity, $O(n)$, is the guarantee — the line nobody can push you above.

Space complexity

Everything so far has been about *time*. The same notation measures *memory*.

> **DEFINITION | Space complexity**
> **Space complexity** is how much memory an algorithm uses, as a function of the input size n, using the same $O(\cdot)$ notation.
> **Auxiliary space** is the memory an algorithm needs *in addition to* the input. **Total space** includes the input itself.
> When a problem says "solve this in $O(1)$ space", it always means **auxiliary** space — you are allowed to have the input; you are not allowed to make a copy of it.

Three examples, all operating on an array of n integers:

O(1) auxiliary space - a few variables, whatever n is

```cpp
int maxOf(const std::vector<int>& v) {
    int best = v[0];            // 1 int
    for (int x : v)             // 1 more int for the loop
        if (x > best) best = x;
    return best;
```

```
}
```

Two extra integers. Not $2n$ — two. $O(1)$ auxiliary space.

O(n) auxiliary space - a second array as big as the input

```cpp
std::vector<int> doubled(const std::vector<int>& v) {
    std::vector<int> out;
    out.reserve(v.size());      // n new ints
    for (int x : v) out.push_back(x * 2);
    return out;
}
```

The new vector grows with n. $O(n)$ auxiliary space.

O(n) space you did not ask for - recursion

```cpp
int sumTo(int n) {
    if (n == 0) return 0;
    return n + sumTo(n - 1);    // n nested calls, each one a stack frame
}
```

This looks like it uses no memory. It uses $O(n)$, because each of the n pending calls keeps a frame on the call stack. For large n it crashes with a stack overflow. **Recursion depth is space.** Remember this when you meet the recursive linked list reversal in Chapter 28.

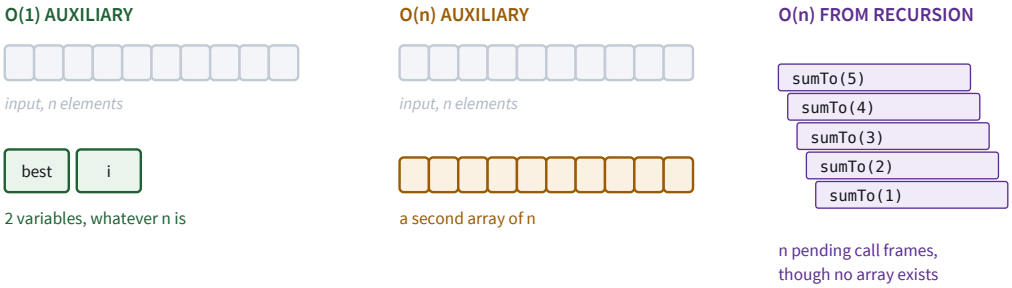


Figure 3.7 Three shapes of space usage. A handful of variables is $O(1)$; a second array is $O(n)$; and a chain of n pending recursive calls is also $O(n)$, even though no array appears anywhere in the code.

Time–space trade-offs

Very often you can buy time with memory, or memory with time.

Problem	Low memory	Low time
Does this array contain duplicates?	Sort it, then scan: $O(n \log n)$ time, $O(1)$ extra	Hash set: $O(n)$ time, $O(n)$ extra
Sum of elements from i to j, asked many times	Loop each time: $O(n)$ per query, $O(1)$ extra	Prefix sums: $O(1)$ per query, $O(n)$ extra

Problem	Low memory	Low time
Nth Fibonacci number	Recompute: exponential time, $O(n)$ stack	Memoise: $O(n)$ time, $O(n)$ space

KEY INSIGHT | The trade-off is the real skill
Beginners look for "the fast solution". Experienced engineers ask "what am I willing to spend?". In an interview, saying *"I can do this in $O(n)$ time if I may use $O(n)$ extra space, or $O(n \log n)$ time with no extra space — which would you prefer?"* is a strong answer by itself.

A recipe for reading any C++ function

Here is a procedure that works. Apply it mechanically until it becomes instinct.

CHECKLIST | How to find the complexity of code you have never seen

1. **Decide what n is.** The length of the array? The value of a number? The number of nodes? Write it down — most mistakes happen here.
2. **Find the loops and the recursion.** Straight-line code with no loops is $O(1)$, however long it is.
3. **For each loop, ask two questions:** *how many times does it run?* and *what does one iteration cost?* Multiply them. A loop with `i++` from 0 to n runs n times; a loop with `i *= 2` runs $\log n$ times; a loop whose bound depends on an outer loop needs a sum, as in the triangular example above.
4. **Watch for hidden costs in the body** — the table below lists the usual traps, every one of which looks like a single cheap statement.
5. **Add the parts, then simplify.** Sequential blocks add ($O(n)+O(n)=O(n)$); nested blocks multiply ($O(n)\times O(\log n)=O(n \log n)$). Then drop constants and lower-order terms.

Statement	Real cost	Why
`v.insert(v.begin(), x)`	$O(n)$	every element shifts right
`v.erase(v.begin())`	$O(n)$	every element shifts left
`s.substr(a, b)`	$O(b)$	it copies the characters
`s1 + s2` (strings)	$O(\|s_1\|+\|s_2\|)$	a new string is built
`std::find(v.begin(), v.end(), x)`	$O(n)$	a linear scan
`f(std::vector<int> v)`	$O(n)$ per call	the vector is copied on entry
`m[key]` for `std::map`	$O(\log n)$	a balanced tree walk
`m[key]` for `std::unordered_map`	$O(1)$ average	a hash lookup

Table 3.2 Innocent-looking C++ that is not free. Every one of these is a single statement that hides a loop.

Let us run the recipe on an unfamiliar function.

Apply the recipe to this

```cpp
bool hasPairSum(std::vector<int> v, int target) {
    std::sort(v.begin(), v.end());
    for (int i = 0; i < (int)v.size(); ++i) {
        int need = target - v[i];
        if (std::binary_search(v.begin(), v.end(), need)) return true;
    }
    return false;
}
```

1. n is `v.size()`.
2. There is one loop, plus a sort, plus a call inside the loop.
3. The parameter is taken **by value**, so the vector is copied on entry: $O(n)$ time and $O(n)$ space. (A reference would avoid this, but then we could not sort it — a real design decision, not a mistake.)
4. `std::sort` is $O(n \log n)$.
5. The loop runs n times; each iteration calls `binary_search`, which is $O(\log n)$. So the loop is $O(n \log n)$.
6. Total: $O(n) + O(n \log n) + O(n \log n) = O(n \log n)$ time, $O(n)$ auxiliary space (that copy).

EXERCISES | Complexity drills
State the time and auxiliary space complexity of each, and say what n is.

```cpp
// (a)
int f(std::vector<int>& v) {
    int s = 0;
    for (int i = 0; i < (int)v.size(); ++i) s += v[i];
    for (int i = 0; i < (int)v.size(); ++i) s -= v[i];
    return s;
}

// (b)
void g(int n) {
    for (int i = 1; i < n; i *= 2)
        for (int j = 0; j < n; ++j)
            std::cout << i << j;
}

// (c)
int h(std::vector<int> v) {        // note: by value
    std::vector<int> copy = v;
    return copy.size();
}

// (d)
void k(int n) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i; ++j)
            for (int l = 0; l < j; ++l)
                work();
}

// (e)
int m(std::vector<int>& v) {
    std::vector<int> out;
    for (int x : v) out.insert(out.begin(), x);
    return out[0];
}
```

1. An algorithm takes $n^2/1000+50n$ steps. What is its Big O? For which values of n is the $50n$ term actually the larger one? What does that tell you about trusting Big O for small inputs?
2. You need to process $n=10^6$ items in under one second. Using the 10^8-operations-per-second rule, which complexities are acceptable?

What you should take away

CHAPTER RECAP | **Chapter 3 in eight lines**

- Big O answers one question: **how does the work grow as the input grows?**
- Drop constants, keep the fastest-growing term — both are justified by what happens for large n.
- The hierarchy: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- Nested loops **multiply**; sequential blocks **add**.
- Halving each step gives $O(\log n)$ — about 20 steps for a million items.
- We quote the **worst case** unless we say otherwise, because it is a guarantee.
- **Auxiliary space** excludes the input. Recursion depth counts as space.
- About 10^8 operations per second is the number to reason with out loud.

PART II

Arrays From the Ground Up

An array is a row of boxes in a straight line. That single sentence explains every strength it has and every weakness it will ever show you. We take it apart completely.

04 The Array

05 Static Arrays, Dynamic Arrays and std::vector

06 The Core Array Operations

07 Two-Dimensional Arrays

CHAPTER

04

The Array

A row of boxes in a straight line, and everything that follows from it

FOUNDATION PREREQS: Chapters 2 and 3

An array is the simplest data structure there is, and the one every other structure borrows from. This chapter takes it completely apart: how it sits in memory, why indexing starts at zero (there is a real reason), what the compiler does when you write `arr[3]`, how to walk an array safely, and what actually happens when you walk off the end of one.

WHAT YOU WILL LEARN

- Declare, initialise and use a C-style array in C++
- Explain what `arr[3]` compiles into, in terms of addresses
- Give the real reason indexing starts at 0, not 1
- Traverse an array three different ways and say when each is appropriate
- Explain what an out-of-bounds access does, and why it is worse than a crash
- State the cost of access, update, search and traversal, and justify each

The picture that never changes

Here is an array. Memorise this picture; we will draw it two hundred more times.

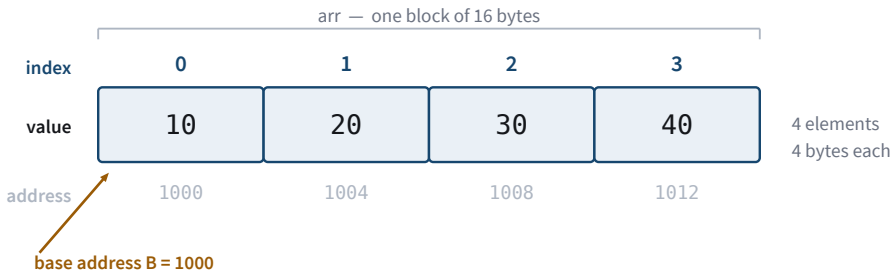


Figure 4.1 The anatomy of an array. Four things to keep separate: the **index** (a position label), the **value** (what is stored there), the **address** (where in memory it lives), and the **base** (where the whole block starts).

That picture, as C++

```cpp
int arr[4] = {10, 20, 30, 40};
```

Read the declaration from the inside out:

- `arr` is the name.
- `[4]` means **four elements**, decided now and fixed forever.
- `int` means each element is an `int`, so each takes 4 bytes.
- `= {10, 20, 30, 40}` fills them in, in order.

The compiler reserves 4×4=16 consecutive bytes and remembers that `arr` names the first of them.

> **DEFINITION | Array**
> An **array** is a fixed number of elements of the *same type*, stored in one unbroken block of memory, in order.
> Three words carry all the weight:
> **Fixed** — the length is decided when the array is created and never changes.
> **Same type** — so every element is the same number of bytes, which is what makes the address formula work.
> **Unbroken** — no gaps, nothing else in between. This is the guarantee Chapter 2 said separate variables do not give you.

Why indexing starts at zero

Beginners find this arbitrary and annoying. It is neither. Here is the real reason.

Recall the address formula from Chapter 2:

$$\text{address}(i) = B + i \times S$$

where B is the base address and S is the size of one element. Now ask: *what is the address of the first element?* It is B itself — the block starts there. For the formula to give B, we need $i \times S = 0$, so $i = 0$.

> **KEY INSIGHT | The index is not a count, it is a distance**
> `arr[i]` does not mean "the i-th element". It means **"the element i steps from the start"**. `arr[0]` is "zero steps from the start" — the first element. `arr[3]` is "three steps from the start" — the fourth element.
> Once you read `[i]` as an *offset* rather than a *count*, zero-based indexing stops being a quirk and becomes the only sensible choice.

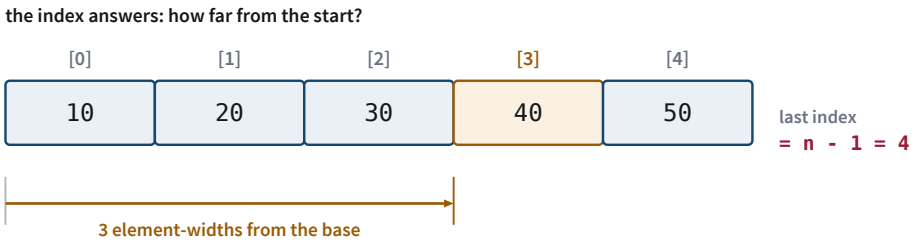


Figure 4.2 Index as distance. The index is the number of element-widths you move from the base, which is why the first element is at offset 0 and why the last of n elements is at $n-1$.

This immediately gives you the rule you will use in every loop you ever write:

> **DEFINITION | The last index**
> An array of n elements has valid indices $0, 1, 2, \ldots, n-1$.
> The last valid index is $n-1$, **not** n. Index n is one past the end and is not yours to touch.

If you remember nothing else from this chapter, remember `n - 1`. Off-by-one errors at the end of arrays are, by a wide margin, the most common bug in all of programming.

What `arr[3]` actually compiles into

When you write `arr[3]`, the compiler does not search. It emits roughly:

1. Take the base address B (it knows this).
2. Multiply 3 by `sizeof(int)`, which is 4, giving 12.
3. Add: $B+12$.
4. Read the 4 bytes there.

One multiply, one add, one read. **The same work regardless of the index and regardless of the array's length.**

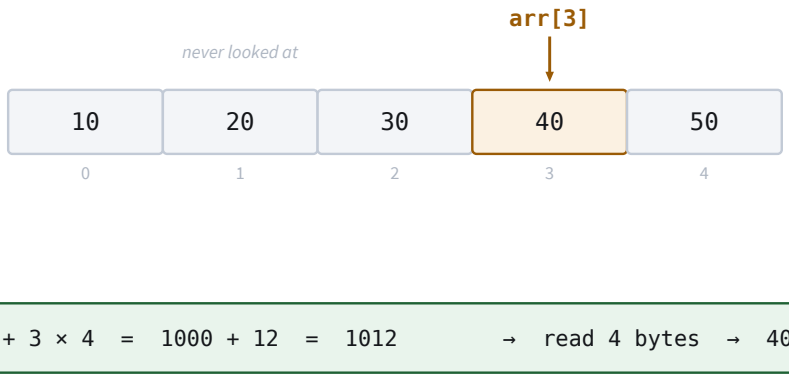


Figure 4.3 Accessing element 3. No element is examined on the way; the address is computed and read directly. This is why array access is $O(1)$.

> **DEFINITION | Random access**
> **Random access** means you can reach any element in the same, fixed amount of time, in any order — "random" here means "arbitrary", not "unpredictable".
> Arrays have random access. Linked lists do not, and that single difference drives half the decisions in this book.

Access and update are both O(1)

```cpp
#include <iostream>

int main() {
    int arr[4] = {10, 20, 30, 40};

    std::cout << "arr[0] = " << arr[0] << "\n";   // access
    std::cout << "arr[3] = " << arr[3] << "\n";

    arr[2] = 99;                                  // update, also O(1)
    std::cout << "after arr[2] = 99: " << arr[2] << "\n";
    return 0;
}
```

```
arr[0] = 10
arr[3] = 40
after arr[2] = 99: 99
```

Updating is the same operation as accessing, with the read replaced by a write. Same address calculation, same cost: $O(1)$.

Declaring arrays: the forms you will meet

Five ways to make an array, and what each gives you

```cpp
#include <iostream>

int main() {
    int a[5] = {1, 2, 3, 4, 5};   // all five given
    int b[5] = {1, 2};            // rest filled with 0  -> 1 2 0 0 0
    int c[5] = {};                // all zeros           -> 0 0 0 0 0
    int d[]  = {7, 8, 9};         // size deduced as 3
    int e[3];                     // UNINITIALISED - contains garbage

    std::cout << b[4] << " " << c[0] << " " << sizeof(d) / sizeof(d[0])
        << "\n";
    (void)a; (void)e;
    return 0;
}
```

```
0 0 3
```

Three things to take from that program:

- `int b[5] = {1, 2};` **zero-fills the rest.** Partial initialisation is complete initialisation in C++; the elements you leave out become 0.
- `int e[3];` **does not.** It contains whatever was in those bytes before. This is the single most common source of "it worked on my machine" bugs.
- `sizeof(d) / sizeof(d[0])` gives the number of elements: total bytes divided by bytes per element. It works here, but see the warning below.

COMMON PITFALL | `sizeof` stops working the moment you pass the array to a function

```
void f(int arr[]) {
    // sizeof(arr) is the size of a POINTER (8), not of the array
    std::cout << sizeof(arr) << "\n";   // prints 8, always
}
```

When you pass a C-style array to a function, it **decays to a pointer** — only the address survives, the length is thrown away. That is why every C-style array function must take the length as a second parameter:

```
void f(int arr[], int n) { /* now we know how far we may go */ }
```

This is one of the main reasons to prefer `std::vector`, which is Chapter 5.

Traversal: visiting every element

Traversal means visiting each element exactly once, in order. It is the most common thing anybody ever does with an array.

Three ways to traverse, all O(n)

```
#include <iostream>
#include <vector>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int n = 5;

    // 1. Index loop - use when you need to know WHERE you are
    for (int i = 0; i < n; ++i)
        std::cout << "index " << i << " holds " << arr[i] << "\n";

    // 2. Range-based for - use when you only need the VALUES
    for (int value : arr)
        std::cout << value << " ";
    std::cout << "\n";

    // 3. Range-based for by reference - use when you want to MODIFY
    for (int& value : arr)
        value *= 2;
    for (int value : arr)
        std::cout << value << " ";
    std::cout << "\n";
    return 0;
}
```

```
index 0 holds 10
index 1 holds 20
index 2 holds 30
index 3 holds 40
index 4 holds 50
10 20 30 40 50
20 40 60 80 100
```

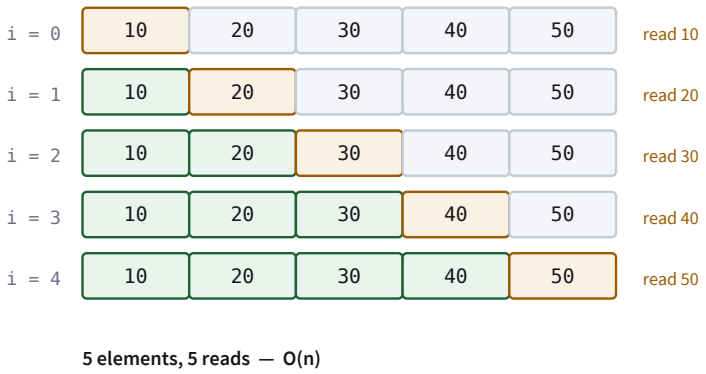


Figure 4.4 A traversal in progress. Each step reads one element and moves on. Five elements, five reads, and the cost grows exactly in step with the length — the definition of $O(n)$.

The third loop deserves a closer look, because the `&` is doing real work.

> **KEY INSIGHT** | `int value` **copies;** `int& value` **is the element itself**
> In `for (int value : arr)`, each iteration makes a **copy** of the element. Changing `value` changes the copy, and the array is untouched.
> In `for (int& value : arr)`, `value` is a **reference** — another name for the actual element. Changing it changes the array.
> Forgetting the `&` and wondering why your modifications vanished is a rite of passage. Chapter 21 explains references properly; for now, the rule is: `&` **if you want to change it, no** `&` **if you only want to read it.**

Complexity of traversal: you do $O(1)$ work n times, so $O(n)$ time and $O(1)$ auxiliary space. Nothing was copied, nothing was allocated.

Searching: the first real algorithm

Suppose you want to know whether 30 is in the array, and where.

Linear search - check them one by one

```cpp
#include <iostream>
#include <vector>

// Returns the index of target, or -1 if it is not present.
int linearSearch(const std::vector<int>& v, int target) {
    for (int i = 0; i < (int)v.size(); ++i) {
        if (v[i] == target)
            return i;              // found it - stop immediately
    }
```

```
    return -1;                      // checked everything, not here
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};
    std::cout << linearSearch(v, 30) << "\n";
    std::cout << linearSearch(v, 99) << "\n";
    return 0;
}
```

```
2
-1
```

Three details that matter more than they look:

- `const std::vector<int>&` — the `&` avoids copying the whole vector on every call ($O(n)$ saved), and the `const` promises we will not modify it. This is the standard way to pass a container you only read.
- `return -1` — a conventional "not found" marker. It works because -1 is never a valid index. Returning `0` would be a bug: 0 is a perfectly good index.
- `(int)v.size()` — `size()` returns an *unsigned* type. Comparing a signed `int i` against it produces a compiler warning, and worse, if `v` is empty, `v.size() - 1` wraps around to a huge positive number instead of -1. The cast keeps the comparison honest. See the pitfall below.

COMMON PITFALL | The unsigned trap — a bug that looks impossible

```
std::vector<int> v;                     // empty, size() == 0
for (int i = 0; i < v.size() - 1; ++i)   // DISASTER
    std::cout << v[i];
```

`v.size()` is 0, and it is unsigned. `0 - 1` for an unsigned type is not -1; it wraps to 18,446,744,073,709,551,615. The loop runs essentially forever, reading memory it does not own.
Fixes: cast to `int` (`(int)v.size() - 1`), or restructure as `i + 1 < v.size()`. The second is better because it never subtracts at all.

Cost of linear search: best case $O(1)$ (first element), worst case $O(n)$ (last element or absent). We quote $O(n)$.

KEY INSIGHT | Why searching is slow even though access is fast
Accessing `arr[i]` is $O(1)$ — but only when you already **know** i.
Searching is the problem of *finding* i, and with unsorted data there is no shortcut: the value you want could be anywhere, so you must be prepared to look everywhere.
If the array is **sorted**, everything changes: binary search gets you to $O(\log n)$. That is Chapter 12, and it is one of the highest-value chapters in this book.

Going out of bounds

Here is the most important warning in Part II.

What C++ does when you walk off the end

```cpp
#include <iostream>

int main() {
    int arr[3] = {10, 20, 30};
    std::cout << arr[0] << " " << arr[1] << " " << arr[2] << "\n";
    std::cout << arr[5] << "\n";    // OUT OF BOUNDS - no error, no crash
    return 0;
}
```

A possible output:

```
10 20 30
32764
```

Read that again. The program **did not crash**. It did not print an error. It printed a number — whatever happened to be in the 4 bytes 20 bytes past the start of the array. Those bytes belong to something else entirely.



Figure 4.5 Reading past the end. There is no fence at the end of an array — only more memory, belonging to other variables. `arr[5]` reads it cheerfully and tells you nothing is wrong.

WATCH OUT | Undefined behaviour
C++ does **not** check array bounds. Reading or writing outside an array is *undefined behaviour*: the standard permits literally anything to happen.
In practice, one of these:

- You read a meaningless value, and your program silently produces wrong answers.
- You **write** over another variable, and a completely unrelated part of the program misbehaves — often minutes later, in code you never touched.
- You hit memory the operating system protects, and the program crashes.
- Nothing visible happens at all, until the day it does.

The crash is the *lucky* case, because at least you find out. Silent corruption is the one that ruins a week.

Why does C++ not check? Because the check would cost time on every single access, and C++'s design principle is that you do not pay for what you did not ask for. The language assumes you know what you are doing. The consequence is that *you must* know what you are doing.

Three defences, in order of how much you should rely on them:

1. **Write the loop bound correctly.** `i < n`, never `i <= n`. This is the real defence.
2. **Use** `std::vector::at()` when you want a check. `v.at(5)` throws an exception instead of reading rubbish. It is slightly slower, which is why `v[5]` exists too.
3. **Turn on your compiler's sanitiser while developing:** `g++ -std=c++17 -fsanitize=address,undefined -g prog.cpp`. This catches out-of-bounds access at run time and prints exactly which line did it. Use it. It will find bugs you did not know you had.

at() checks, [] does not

```cpp
#include <iostream>
#include <stdexcept>
#include <vector>

int main() {
    std::vector<int> v = {10, 20, 30};
    try {
        std::cout << v.at(5) << "\n";       // throws
    } catch (const std::out_of_range& e) {
        std::cout << "caught: index was out of range\n";
    }
    return 0;
}
```

```
caught: index was out of range
```

The cost table for arrays

Now we can fill in the table promised in Chapter 1.

Operation	Cost	Why
Access `arr[i]`	$O(1)$	one address calculation
Update `arr[i] = x`	$O(1)$	the same calculation, then a write
Search (unsorted)	$O(n)$	the value could be anywhere
Search (sorted)	$O(\log n)$	binary search — Chapter 12
Traverse	$O(n)$	n elements, $O(1)$ each
Insert at the end	—	impossible: the size is fixed
Insert in the middle	$O(n)$	everything after it must shift right
Delete from the middle	$O(n)$	everything after it must shift left

Table 4.1 Cost of the six basic operations on an array of n elements. Every entry in this table is explained by one fact: the elements are contiguous and the length is fixed.

Two of those rows need explaining, and they get a full treatment in Chapter 6. The short version:

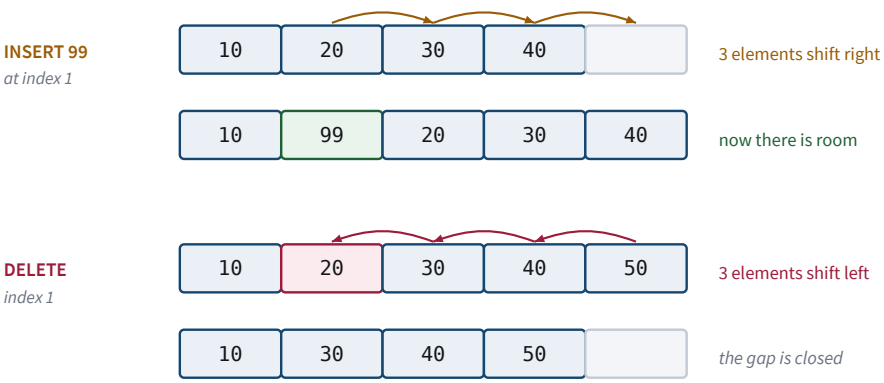


Figure 4.6 Why insertion and deletion cost $O(n)$. There is no spare room between elements, so making a gap means moving everything to the right of it. Closing a gap means moving everything back.

Notice that the last three rows are the array's weakness, and they are exactly the operations a linked list is good at. That is not a coincidence — it is the trade-off that the two structures exist to offer you.

Common beginner mistakes

COMMON PITFALL | The five that catch everyone

`i <= n` in the loop condition. The last valid index is `n - 1`. Writing `<=` reads one element past the end, every single time.

Assuming `int arr[n]` works with a variable `n`. Standard C++ requires the size to be a compile-time constant. Some compilers allow variable-length arrays as an extension; it is not portable. Use `std::vector` when the size is decided at run time.

Comparing arrays with `==`. `a == b` compares the two *addresses*, not the contents, and is almost always `false`. To compare contents, compare element by element (or use `std::vector`, where `==` does the right thing).

Expecting an array to remember its length inside a function. It decays to a pointer; pass the length too.

Forgetting `&` in a range-based `for` when you intend to modify. The loop runs, the values change, and the array is unchanged.

What you should take away

CHAPTER RECAP | Chapter 4 in seven lines

- An array is a **fixed number** of **same-typed** elements in **one unbroken block**.
- `arr[i]` means "i element-widths from the start", which is why indexing begins at 0 and the last index is $n-1$.
- Access and update are $O(1)$: one multiply, one add, one memory operation.
- Traversal is $O(n)$ and uses $O(1)$ extra space.
- Searching unsorted data is $O(n)$; sorting changes everything (Chapter 12).
- C++ does **not** check bounds. Out-of-bounds access is undefined behaviour, and silent corruption is worse than a crash.
- Insertion and deletion in the middle are $O(n)$, because contiguity must be restored by shifting.

EXERCISES | Practise

1. For `int arr[6]`, what is the last valid index? What is `sizeof(arr)` if an `int` is 4 bytes?
2. An array of `double` has base address 3000. What is the address of `arr[5]`?
3. Write a loop that prints an array **backwards**. What is the starting value of your index, and what is the loop condition? (This is where `<=` bugs live — be careful.)
4. Explain to a friend, in two sentences and without the word "offset", why `arr[0]` is the first element.
5. Compile and run the out-of-bounds program above with `-fsanitize=address,undefined`. What does it tell you that the plain compilation did not?
6. Why can an array not grow? Answer in terms of memory, not in terms of C++ rules.
7. The function below has exactly one bug. Find it by reading, then compile it with `-fsanitize=address` and run it to confirm you were right.

```
int sumAll(int arr[], int n) {
    int total = 0;
    for (int i = 0; i <= n; ++i)
        total += arr[i];
    return total;
}
```

CHAPTER

05

Static Arrays, Dynamic Arrays and std::vector

How a fixed-size block pretends to grow, and what that costs

FOUNDATION PREREQS: Chapter 4

A raw array cannot grow — the memory after it belongs to somebody else. `std::vector` gets around this by moving the whole block when it runs out of room, and the way it does that produces the most frequently misunderstood phrase in data structures: amortised constant time. This chapter explains size versus capacity, why vectors double instead of growing by one, and exactly what `push_back` costs.

WHAT YOU WILL LEARN

- Explain why a static array cannot grow, in terms of memory
- Use `std::vector` for its common operations and know the cost of each
- Tell `size()` from `capacity()` and predict both after a sequence of pushes
- Explain amortised $O(1)$ and why `push_back` deserves that description
- Explain why doubling is used instead of adding a fixed amount
- Avoid the two classic vector bugs: iterator invalidation and `[]` past the end

The problem with a fixed size

In Chapter 4 we wrote `int arr[4]`. Suppose we now want a fifth element.

We cannot simply use `arr[4]`. The bytes immediately after the array are not ours — they belong to another variable, or to nothing in particular, and writing there is the undefined behaviour we were warned about.

KEY INSIGHT | Why an array cannot grow
Growing would mean claiming the memory immediately after the block. But nothing reserved that memory for you, and something else is probably already using it. There is no way to "extend" a block in place. The only option is to **find a bigger empty block somewhere else and copy everything into it** — and that is precisely what a dynamic array does.

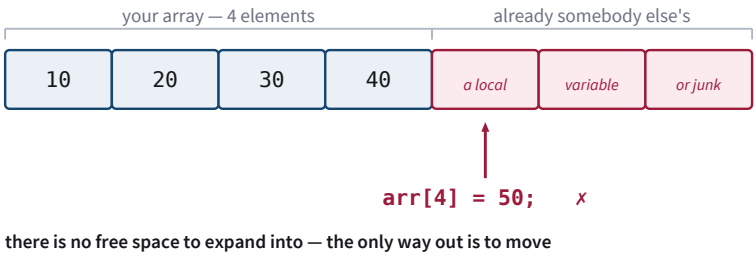


Figure 5.1 Why growing in place is impossible. The bytes after an array belong to whatever the compiler put there. To get more room you must move house, not knock through a wall.

DEFINITION | Static array vs dynamic array

A **static array** has a size fixed when it is created. `int arr[4];` is one. It cannot grow or shrink, ever.

A **dynamic array** can change size while the program runs. It works by owning a block, tracking how much of it is used, and — when the block fills up — allocating a bigger one and copying everything across.

In C++, the dynamic array is `std::vector`. In Java it is `ArrayList`, in Python it is `list`, in C# it is `List<T>`. They are the same idea everywhere.

Meeting std::vector

The vector operations you will use constantly

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;              // empty
    std::vector<int> w = {1, 2, 3};  // from a list
    std::vector<int> z(5, 7);        // five copies of 7

    v.push_back(10);                 // add to the end
    v.push_back(20);
    v.push_back(30);

    std::cout << "v has " << v.size() << " elements\n";
    std::cout << "first = " << v.front() << ", last = " << v.back() <<
        "\n";
    std::cout << "v[1]  = " << v[1] << "\n";

    v.pop_back();                    // remove the last one
    std::cout << "after pop_back, size = " << v.size() << "\n";
    std::cout << "is w empty? " << (w.empty() ? "yes" : "no") << "\n";
    std::cout << "z[4] = " << z[4] << "\n";
    return 0;
}
```

```
v has 3 elements
first = 10, last = 30
v[1]  = 20
after pop_back, size = 2
is w empty? no
z[4] = 7
```

The crucial point: `v[1]` **on a vector is exactly as fast as** `arr[1]` **on a raw array.** A vector is not a linked list in disguise. Internally it holds a pointer to one contiguous block, so indexing is still $B+i\times S$ — one multiply, one add. You lose nothing by using it.

Size versus capacity

Here is the idea that makes everything else make sense.

> **DEFINITION | size and capacity**
> `size()` is how many elements you have actually put in.
> `capacity()` is how many the current block could hold before a bigger one is needed.
> Always `size() <= capacity()`. The difference is empty space the vector is holding in reserve so that the next few `push_back` calls are cheap.

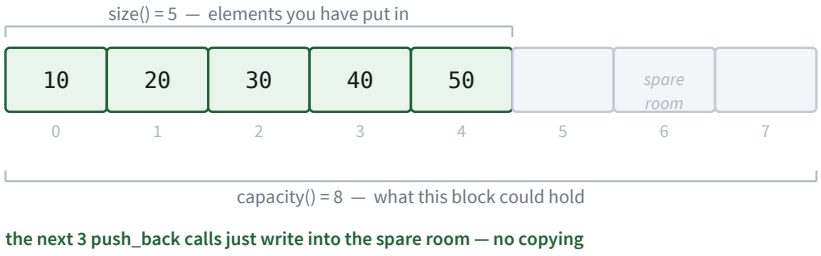


Figure 5.2 Size and capacity. The vector owns the whole block; `size` is the filled part, `capacity` is the whole thing. The spare room at the end is what makes most `push_back` calls free of copying.

Let us watch it happen.

Watching capacity grow

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::cout << "size  capacity\n";
    std::cout << v.size() << "     " << v.capacity() << "\n";
    for (int i = 1; i <= 9; ++i) {
        v.push_back(i);
        std::cout << v.size() << "     " << v.capacity() << "\n";
    }
    return 0;
```

```
13  }
```

On g++ (libstdc++) this prints:

```
size  capacity
0     0
1     1
2     2
3     4
4     4
5     8
6     8
7     8
8     8
9     16
```

Look at the capacity column: 0, 1, 2, 4, 4, 8, 8, 8, 8, 16. It **doubles**, and only at the moments where the old block was full.

> **NOTE | Capacity growth is not standardised**
> The C++ standard requires `push_back` to be amortised $O(1)$ but does not dictate the growth factor. libstdc++ (g++) doubles; MSVC grows by about 1.5×. Your numbers may differ in detail — the *shape* will not.

What happens when the block is full

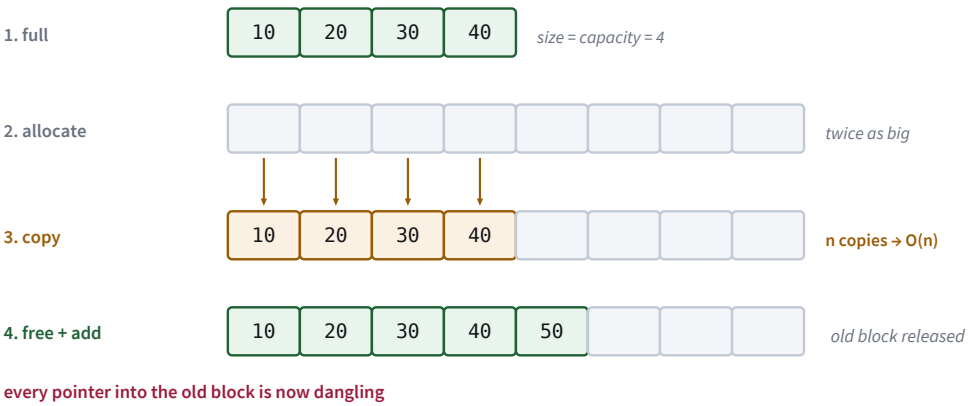


Figure 5.3 A reallocation, step by step. The vector asks for a block twice as large, copies every existing element across, destroys the old block, and only then stores the new value. That one push costs $O(n)$; the pushes either side of it cost $O(1)$.

When `size() == capacity()` and you call `push_back`:

1. Allocate a new block, usually twice as large.
2. Copy (or move) all n existing elements into it.
3. Free the old block.
4. Put the new element at the end.

Step 2 is $O(n)$. So **some** `push_back` calls are expensive. The question is how expensive they are *on average*, and the answer is the pleasant surprise of this chapter.

Amortised O(1), explained properly

DEFINITION | Amortised complexity
Amortised cost is the average cost per operation over a long sequence, when occasional expensive operations are paid for by many cheap ones.
It is *not* the same as "average case". Average case assumes random input; amortised makes no assumption about input at all — it is an arithmetic fact about the total work of a sequence.

Here is the arithmetic. Push n elements into an empty vector that doubles. Reallocations happen at sizes $1, 2, 4, 8, \ldots$ up to n, and the copying work at each is equal to the size at that moment:

$$1+2+4+8+\cdots+n/2+n<2n$$

That sum is a geometric series, and it is **less than $2n$** no matter how large n is. Add the n cheap writes themselves and the total work for n pushes is under $3n$ — which is $O(n)$ for the whole sequence, and therefore $O(1)$ **per push on average**.

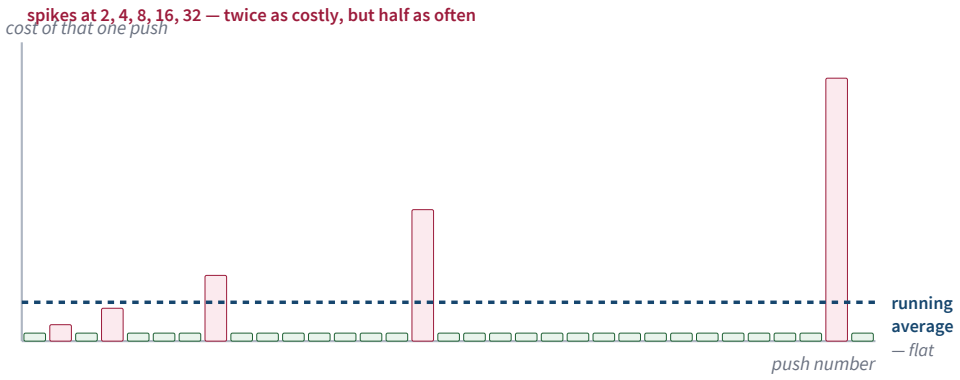


Figure 5.4 The cost of each individual `push_back`. The spikes are the reallocations, and they get rarer exactly as fast as they get more expensive. The running average (the flat line) never rises.

INTUITION | Why doubling, and not "grow by 10"?
Suppose the vector grew by a fixed 10 slots instead of doubling. Reallocations would happen at sizes 10, 20, 30, 40, …, and the copying work would be

$$10+20+30+\cdots+n=n(n+10)/20=O(n^2)$$

Pushing a million elements would do about 5×10^{10} copies instead of about two million. The whole structure would be unusable.
Doubling works because the gaps between expensive operations grow at the same rate as the expense. Each reallocation costs twice as much as the last one, but you get twice as long before the next one. Those two effects cancel exactly, and that cancellation is what "amortised $O(1)$" means.

INTERVIEW ANGLE | **Say this, not that**
Weak: "`push_back` is $O(1)$." **Weak:** "`push_back` is $O(n)$ because it sometimes copies."
Strong: *"`push_back` is amortised $O(1)$. Most calls just write into spare capacity. When the block is full it reallocates and copies, which is $O(n)$ for that one call — but because capacity doubles, that happens only $\log n$ times, and the total copying over n pushes is under $2n$. So the average is constant, though any individual call may not be."*
That paragraph is a genuinely common interview question, and most candidates answer it with one of the two weak versions.

reserve: skipping the copying entirely

If you know roughly how many elements you will add, you can ask for the capacity in advance.

reserve avoids every reallocation

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> slow;
    std::vector<int> fast;
    fast.reserve(1000);              // one allocation, right now

    for (int i = 0; i < 1000; ++i) slow.push_back(i);
    for (int i = 0; i < 1000; ++i) fast.push_back(i);

    std::cout << "slow: size " << slow.size()
              << ", capacity " << slow.capacity() << "\n";
    std::cout << "fast: size " << fast.size()
              << ", capacity " << fast.capacity() << "\n";
    return 0;
}
```

```
slow: size 1000, capacity 1024
fast: size 1000, capacity 1000
```

`slow` reallocated about 10 times and copied roughly 2,000 elements along the way. `fast` allocated once and copied nothing. Both are $O(n)$ overall, but the constant factor differs by a lot, and `reserve` is free to write.

COMMON PITFALL | `reserve` **is not** `resize`
`v.reserve(100)` sets capacity to at least 100 and leaves `size()` **unchanged** — the vector is still empty, and `v[0]` is undefined behaviour.
`v.resize(100)` sets `size()` to 100 and value-initialises the new elements to 0, so `v[0]` is valid and equal to 0.
Using `reserve` and then `v[i] = x` is a genuine bug that often appears to work, because the memory exists — it is just not officially yours yet.

The full cost table

Operation	Cost	What happens
`v[i]`, `v.at(i)`	$O(1)$	address arithmetic
`v.front()`, `v.back()`	$O(1)$	the ends are known
`v.size()`, `v.empty()`	$O(1)$	a stored counter
`v.push_back(x)`	$O(1)$ amortised	write; occasionally reallocate
`v.pop_back()`	$O(1)$	decrement the counter
`v.insert(v.begin(), x)`	$O(n)$	every element shifts right
`v.insert(pos, x)`	$O(n-i)$, so $O(n)$	elements after `pos` shift
`v.erase(v.begin())`	$O(n)$	every element shifts left
`v.erase(pos)`	$O(n-i)$, so $O(n)$	elements after `pos` shift
`v.clear()`	$O(n)$	destroys elements; capacity kept
`std::find(...)`	$O(n)$	a linear scan
`std::sort(...)`	$O(n \log n)$	introsort

Table 5.1 `std::vector` operations and what each really costs. Note that the two $O(n)$ rows are the same weakness a raw array has, for the same reason: contiguity must be preserved.

PERFORMANCE NOTE | The single most common vector performance bug

```
// Removing all even numbers, the slow way
for (int i = 0; i < (int)v.size(); ) {
    if (v[i] % 2 == 0) v.erase(v.begin() + i);   // O(n) EACH TIME
    else ++i;
}
```

Each `erase` shifts everything after it, so removing $n/2$ elements costs $O(n^2)$. On a vector of 100,000 this takes seconds.
The fix is the **erase–remove idiom**, which does it in one pass:

```
v.erase(std::remove_if(v.begin(), v.end(),
                       [](int x) { return x % 2 == 0; }),
        v.end());
```

`std::remove_if` shuffles the keepers to the front in $O(n)$ and returns where the kept part ends; the single `erase` then chops the tail. Total: $O(n)$.
You will meet this idea again in Chapter 13 as the *in-place write-pointer* pattern — it is the same trick, written by hand.

Iterator invalidation

WATCH OUT | The bug that is hard to see
When a vector reallocates, the old block is freed. Any pointer, reference or iterator into the old block now points at memory that is no longer yours.

```
std::vector<int> v = {1, 2, 3};
int& first = v[0];        // a reference into the current block
v.push_back(4);           // MAY reallocate, freeing that block
std::cout << first;       // undefined behaviour
```

This code usually prints `1` and looks fine, because the freed memory has not been reused yet. It will fail eventually, in production, under load.
Rule: after any operation that can change a vector's capacity (`push_back`, `insert`, `resize`, `reserve`), treat every previously obtained pointer, reference and iterator into it as invalid.

Raw array or vector?

	C-style array	`std::vector`
Size fixed at compile time	yes	no
Can grow	no	yes
Knows its own length	no (decays to a pointer)	yes, `size()`
Bounds-checked option	no	yes, `at()`
Copyable with `=`	no	yes
Comparable with `==`	no (compares addresses)	yes (compares contents)
Works with `std::sort` etc.	yes, with pointers	yes, naturally
Indexing speed	*O*(1)	*O*(1), identical
Memory overhead	none	3 pointers (24 bytes) plus spare capacity

Table 5.2 When to use which. In modern C++ the default should be `std::vector`, and the burden of proof is on the raw array.

Use `std::vector` unless you have a specific reason not to. The indexing cost is identical, you get the length for free, and you stop losing afternoons to bugs the type system could have caught.

Everything this book teaches about arrays applies to both. Where the code says `std::vector<int>& v`, you can mentally substitute `int arr[], int n` and nothing conceptual changes.

What you should take away

CHAPTER RECAP | Chapter 5 in seven lines

- A static array cannot grow because the memory after it is not yours.
- A dynamic array grows by **allocating a bigger block and copying** — never in place.
- `size()` is what you have used; `capacity()` is what you could use before the next move.
- `push_back` is **amortised** $O(1)$: occasional $O(n)$ copies, paid for by the many cheap pushes between them.
- Doubling is essential; growing by a fixed amount would make the total $O(n^2)$.
- `insert` and `erase` away from the end are $O(n)$, exactly as for a raw array.
- Reallocation invalidates every iterator, pointer and reference into the vector.

EXERCISES | Practise

1. A vector doubles from capacity 1. After 100 `push_back` calls, how many reallocations have happened, and what is the capacity?
2. Write out the total copying work for pushing 16 elements into a doubling vector. Confirm it is less than 2×16.
3. Explain, to somebody who has not read this chapter, why `push_back` is called amortised $O(1)$ rather than just $O(1)$.
4. What does snippet **(A)** below print, and why is its last line a bug?
5. Rewrite snippet **(B)** to be $O(n)$ instead of $O(n^2)$. (Hint: what are you actually trying to produce? There is a one-word answer and a one-line implementation.)
6. Suppose a vector grew by a factor of 1.5 instead of 2. Is `push_back` still amortised $O(1)$? Justify your answer with the series.
7. Why does `clear()` leave the capacity unchanged? When would that be helpful, and when would it waste memory?

(A) reserve is not resize

```
std::vector<int> v;
v.reserve(5);
std::cout << v.size() << "\n";
std::cout << v[0] << "\n";
```

(B) quadratic, and it does not need to be

```
std::vector<int> out;
for (int x : in) out.insert(out.begin(), x);
```

CHAPTER

06

The Core Array Operations

Twelve operations, each traced by hand before it is coded

CORE PREREQS: Chapters 4 and 5

This is the workshop chapter. Twelve operations — insert, delete, reverse, min and max, sum, count, remove duplicates, move zeroes and the rest — each one drawn, traced step by step, coded in C++, costed, and followed by the mistakes people actually make. Everything in Parts III and IV is built from these.

WHAT YOU WILL LEARN

- Implement every basic array operation correctly, including the edge cases
- Explain what happens *internally* during an insertion and a deletion
- Trace an algorithm by hand using a table of variable values
- Reverse an array in place with two pointers
- Remove duplicates from a sorted array in one pass, in place
- Move all zeroes to the end while preserving the order of the rest
- State and justify the cost of each operation

How to read this chapter

Every operation below follows the same six steps:

1. **What it must do** — stated precisely, including the awkward cases.
2. **The picture** — before and after.
3. **The trace** — a table showing every variable at every step.
4. **The code** — complete, compilable C++.
5. **The cost** — time and auxiliary space, with the reason.
6. **The mistakes** — what goes wrong and why.

Do the traces yourself with a pen before reading the code. The trace is where understanding happens; the code is just a transcript of it.

1. Access and update

Covered in Chapter 4, repeated here for completeness.

Access and update are the same operation

```
int x = v[3];        // read  the 4 bytes at B + 3*S
v[3] = 99;           // write the 4 bytes at B + 3*S
```

Both are $O(1)$ time, $O(1)$ space. The only hazard is the index being out of range, which C++ will not tell you about.

2. Traversal

Visit every element once

```
#include <iostream>
#include <vector>

void printAll(const std::vector<int>& v) {
    for (int i = 0; i < (int)v.size(); ++i)
        std::cout << v[i] << (i + 1 < (int)v.size() ? ", " : "\n");
}

int main() {
    std::vector<int> v = {10, 20, 30, 40};
    printAll(v);
    return 0;
}
```

```
10, 20, 30, 40
```

$O(n)$ time, $O(1)$ space. Note the condition `i + 1 < (int)v.size()` — it prints the separator for every element except the last, without ever subtracting from an unsigned value.

3. Searching

Linear search — unsorted data

Covered in Chapter 4. $O(n)$ worst case. Works on any array.

A first taste of binary search — sorted data

If the array is sorted, you can do far better by discarding half the candidates at every step.

Binary search (full treatment in Chapter 12)

```
#include <iostream>
#include <vector>

int binarySearch(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;      // never overflows
        if (v[mid] == target) return mid;
        if (v[mid] < target) lo = mid + 1; // answer is to the right
```

```
        else                hi = mid - 1; // answer is to the left
    }
    return -1;
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50, 60, 70};
    std::cout << binarySearch(v, 50) << "\n";
    std::cout << binarySearch(v, 35) << "\n";
    return 0;
}
```

```
4
-1
```

$O(\log n)$ time. The array **must** be sorted — on unsorted data it returns confident nonsense. Chapter 12 explains every line, including why `mid` is written that way.

4. Insertion

DEFINITION | **The insertion problem**
Given an array with n used slots (and room for at least one more), insert value x at index p, so that everything previously at index p and beyond moves one place to the right.

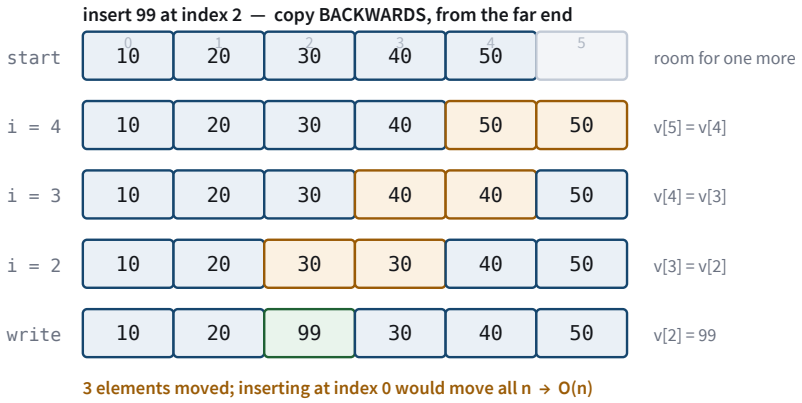


Figure 6.1 Inserting 99 at index 2. The elements from index 2 to the end must each move one place right, and they must be moved **from the back**, or each one overwrites the next.

The direction of the shifting loop is the whole difficulty.

KEY INSIGHT | **Why we shift from the right, not the left**
Suppose we tried to make room by going left to right, starting at index 2:
`v[3] = v[2]` — fine, 30 is now in two places. `v[4] = v[3]` — but `v[3]` is no longer 40, we just overwrote it with 30.
Everything after the insertion point becomes a copy of the same value. The fix is to copy **backwards**, starting from the last element, so that every slot is read before anything overwrites it.

The trace

Insert `99` at index 2 into `{10, 20, 30, 40, 50}` (which has room for 6).

Step	i	Action	Array afterwards
start	—	—	`10 20 30 40 50 _`
1	4	`v[5] = v[4]`	`10 20 30 40 50 50`
2	3	`v[4] = v[3]`	`10 20 30 40 40 50`
3	2	`v[3] = v[2]`	`10 20 30 30 40 50`
4	—	`v[2] = 99`	`10 20 99 30 40 50`

Table 6.1 Shifting right, from the back. Each row moves one element and nothing is ever overwritten before it is read.

Insertion into a vector, written by hand

```cpp
#include <iostream>
#include <vector>

// Insert x at index p. Everything from p onwards shifts right.
void insertAt(std::vector<int>& v, int p, int x) {
    if (p < 0 || p > (int)v.size()) return;    // p == size() means
        "append"
    v.push_back(0);                            // make one more slot
    for (int i = (int)v.size() - 2; i >= p; --i)
        v[i + 1] = v[i];                       // copy BACKWARDS
    v[p] = x;
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};
    insertAt(v, 2, 99);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    insertAt(v, 0, 7);                         // at the front: worst case
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
10 20 99 30 40 50
7 10 20 99 30 40 50
```

Cost. The loop runs from the end down to `p`, so it performs $n - p$ moves.

- Inserting at the **end** ($p = n$): 0 moves, $O(1)$.
- Inserting at the **front** ($p = 0$): n moves, $O(n)$.
- Worst case: $O(n)$.

KEY INSIGHT | The deep reason insertion is expensive
It is not that copying is slow. It is that **contiguity is a promise the array must keep**. The elements have to stay side by side with no gaps, so making room anywhere except the end forces everything after it to move.
A linked list makes no such promise, which is exactly why it can insert in $O(1)$ — and exactly why it cannot index in $O(1)$. You cannot have both.

In real code you would write `v.insert(v.begin() + p, x)`, which does precisely this and is also $O(n)$. We wrote it by hand so you can see that the library is not hiding anything clever.

5. Deletion

DEFINITION | The deletion problem
Remove the element at index p, closing the gap so the remaining elements are still contiguous, and reduce the size by one.

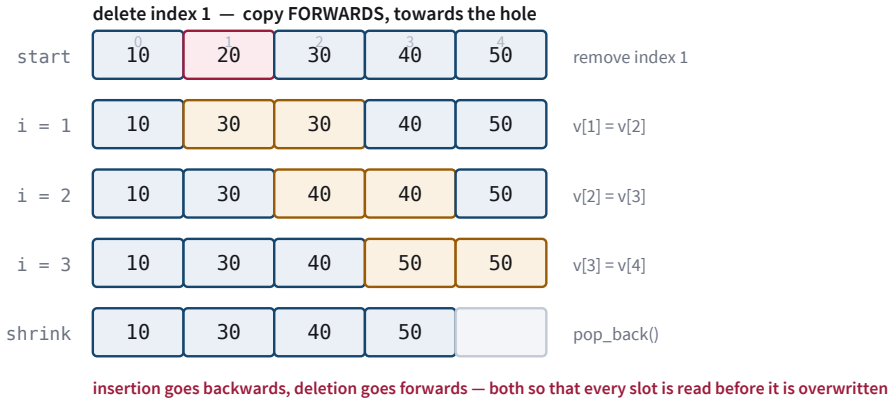


Figure 6.2 Deleting index 1. Everything after the hole moves one place left, and this time the loop must run **forwards** — the opposite of insertion, for the same reason.

KEY INSIGHT | Why deletion shifts forwards
We are moving things *left*, into a slot we have already finished with. Going left to right, `v[1] = v[2]` reads `v[2]` before anything has touched it, then `v[2] = v[3]`, and so on. Each read happens before the corresponding write.
Insertion moves right and must go backwards; deletion moves left and must go forwards. Both rules come from the same principle: **read a slot before you overwrite it**.

The trace

Delete index 1 from `{10, 20, 30, 40, 50}`.

Step	i	Action	Array afterwards
start	—	—	`10 20 30 40 50`
1	1	`v[1] = v[2]`	`10 30 30 40 50`

Step	i	Action	Array afterwards
2	2	`v[2] = v[3]`	`10 30 40 40 50`
3	3	`v[3] = v[4]`	`10 30 40 50 50`
4	—	`pop_back()`	`10 30 40 50`

Deletion from a vector, written by hand

```cpp
#include <iostream>
#include <vector>

void deleteAt(std::vector<int>& v, int p) {
    if (p < 0 || p >= (int)v.size()) return;   // nothing to delete
    for (int i = p; i + 1 < (int)v.size(); ++i)
        v[i] = v[i + 1];                       // copy FORWARDS
    v.pop_back();                              // drop the duplicated tail
}

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};
    deleteAt(v, 1);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    deleteAt(v, (int)v.size() - 1);            // last element: cheapest
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
10 30 40 50
10 30 40
```

Cost. $n-p-1$ moves. Deleting the last element is $O(1)$; deleting the first is $O(n)$. Worst case $O(n)$, $O(1)$ space.

COMMON PITFALL | Deleting while looping forwards

```cpp
for (int i = 0; i < (int)v.size(); ++i)
    if (v[i] == 0) deleteAt(v, i);      // BUG
```

After deleting index `i`, a new element has slid into position `i` — and then `++i` skips it. Given `{0, 0, 1}` this removes only one zero.
Two fixes: do not increment when you delete (`if (...) deleteAt(v, i); else ++i;`), or loop **backwards**, where removals never disturb the indices you have yet to visit. The backwards loop is usually cleaner. Better still, use the write-pointer pattern from operation 11 and do the whole job in $O(n)$.

6. Reversing an array

DEFINITION | The reversal problem
Turn `{10, 20, 30, 40, 50}` into `{50, 40, 30, 20, 10}`, using no second array.

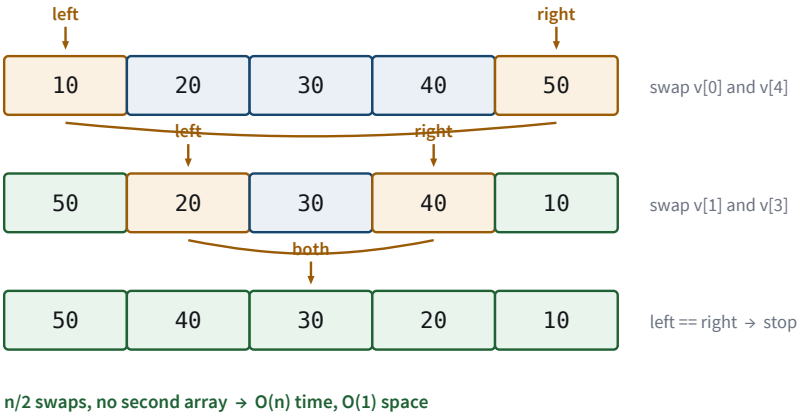


Figure 6.3 Reversing with two pointers. One starts at each end; they swap and step towards each other, and stop when they meet. Only $n/2$ swaps are needed because each swap fixes two positions.

This is your first **two-pointer** algorithm, and the pattern it introduces is the subject of Chapter 9.

The trace

Step	`left`	`right`	Action	Array afterwards
start	0	4	—	`10 20 30 40 50`
1	0	4	swap	`50 20 30 40 10`
2	1	3	swap	`50 40 30 20 10`
3	2	2	`left == right`, stop	`50 40 30 20 10`

Reversing in place with two pointers

```cpp
#include <iostream>
#include <vector>

void reverseArray(std::vector<int>& v) {
    int left = 0, right = (int)v.size() - 1;
    while (left < right) {
        std::swap(v[left], v[right]);
        ++left;
        --right;
    }
}

int main() {
    std::vector<int> odd = {10, 20, 30, 40, 50};
    reverseArray(odd);
    for (int x : odd) std::cout << x << " ";
```

```
    std::cout << "\n";

    std::vector<int> even = {1, 2, 3, 4};
    reverseArray(even);
    for (int x : even) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> one = {42};
    reverseArray(one);
    std::cout << one[0] << "\n";
    return 0;
}
```

```
50 40 30 20 10
4 3 2 1
42
```

Cost. $\lfloor n/2 \rfloor$ swaps: $O(n)$ time, $O(1)$ auxiliary space.

COMMON PITFALL | Three ways to get reversal wrong
`while (left <= right)` — when they are equal they point at the same middle element, and swapping it with itself is harmless but pointless. Worse, if you also `++left; --right;` the loop still terminates, so the bug is invisible here. It becomes a real bug in the palindrome check of Chapter 9, where `<=` makes you compare an element with itself and accept strings you should reject.
Looping all the way across — `for (i = 0; i < n; ++i) swap(v[i], v[n-1-i]);` swaps every pair **twice**, returning the original array. A silent and very confusing bug.
Forgetting odd lengths — with an odd count the middle element stays put, and it should. Always test with both an odd and an even length, and with an empty array.

7. Minimum and maximum

One pass finds both

```
#include <iostream>
#include <stdexcept>
#include <vector>

// Returns {min, max}. Throws if the array is empty - there is no answer.
std::pair<int, int> minMax(const std::vector<int>& v) {
    if (v.empty()) throw std::invalid_argument("empty array has no min or
        max");
    int lo = v[0], hi = v[0];           // start from a REAL element
    for (int i = 1; i < (int)v.size(); ++i) {
        if (v[i] < lo) lo = v[i];
        if (v[i] > hi) hi = v[i];
    }
    return {lo, hi};
}

int main() {
    std::vector<int> v = {7, -3, 12, 0, 5};
```

```
    auto [lo, hi] = minMax(v);
    std::cout << "min = " << lo << ", max = " << hi << "\n";

    std::vector<int> neg = {-9, -2, -40};
    auto [lo2, hi2] = minMax(neg);
    std::cout << "min = " << lo2 << ", max = " << hi2 << "\n";
    return 0;
}
```

```
min = -3, max = 12
min = -40, max = -2
```

Two details carry the whole operation.

`lo` and `hi` both start at `v[0]`. Not at 0, not at some number you hope is big enough — at a value that is genuinely in the array. That guarantees the answer is always an element that really exists.

One loop finds both. You do not need a pass for the minimum and another for the maximum. Two independent `if` statements in the same pass cost one extra comparison per element and save a whole traversal.

Trace it on `{7, -3, 12, 0, 5}` to see that nothing is missed:

`i`	`v[i]`	`v[i] < lo`?	`lo`	`v[i] > hi`?	`hi`
start	—	—	7	—	7
1	−3	yes	−3	no	7
2	12	no	−3	yes	12
3	0	no	−3	no	12
4	5	no	−3	no	12

Table 6.2 Tracing min and max in a single pass. Each row checks the current element against both running answers.

> **KEY INSIGHT | Trace, do not stare**
> When code and expectation disagree, reading the code again almost never finds the bug — you re-read what you *meant*, not what you wrote.
> Build the table instead. A trace cannot deceive you, because it only records what each line actually does. Every worked example in this book has one for exactly this reason, and you should write one whenever you are stuck for more than five minutes.

COMMON PITFALL | Never initialise `lo` to 0

```
int lo = 0;                        // WRONG
for (int x : v) if (x < lo) lo = x;
```

On `{3, 5, 9}` this returns 0 — a value that is not in the array. The same bug with `int hi = 0` breaks on all-negative input.
Initialise from `v[0]`, or from `INT_MAX` / `INT_MIN`, and handle the empty array explicitly. Starting from a real element is safest, because it makes the empty case impossible to forget.

$O(n)$ time, $O(1)$ space, in exactly one pass.

8. Sum and average

Sum, average, and the overflow trap

```
#include <iostream>
#include <vector>

long long sumOf(const std::vector<int>& v) {
    long long total = 0;             // long long, not int - see below
    for (int x : v) total += x;
    return total;
}

double averageOf(const std::vector<int>& v) {
    if (v.empty()) return 0.0;
    return (double)sumOf(v) / v.size();   // cast BEFORE dividing
}

int main() {
    std::vector<int> v = {10, 20, 30, 41};
    std::cout << "sum = " << sumOf(v) << "\n";
    std::cout << "avg = " << averageOf(v) << "\n";

    std::vector<int> big = {2000000000, 2000000000};
    std::cout << "big sum = " << sumOf(big) << "\n";
    return 0;
}
```

```
sum = 101
avg = 25.25
big sum = 4000000000
```

Two traps, both of which silently produce wrong numbers:

COMMON PITFALL | Overflow and integer division

Overflow. `int` holds up to about 2.1 billion. Two elements of 2 billion each overflow an `int` sum, and signed overflow is undefined behaviour — you may get a negative number, or anything. Using `long long` for the accumulator costs nothing and removes the problem.

Integer division. `sum / v.size()` with both operands integral discards the fraction: 101/4 becomes 25, not 25.25. Cast to `double` **before** dividing. `(double)(sum / n)` is still wrong — it divides first, then converts the already truncated result.

$O(n)$ time, $O(1)$ space.

9. Counting occurrences

How many times does a value appear?

```cpp
#include <iostream>
#include <vector>

int countOf(const std::vector<int>& v, int target) {
    int count = 0;
    for (int x : v)
        if (x == target) ++count;
    return count;
}

int main() {
    std::vector<int> v = {1, 3, 7, 3, 3, 9};
    std::cout << countOf(v, 3) << "\n";
    std::cout << countOf(v, 5) << "\n";
    return 0;
}
```

```
3
0
```

$O(n)$ time, $O(1)$ space.

PERFORMANCE NOTE | Counting many values at once

If you need the count of *every* value, do not call `countOf` once per value — that is $O(n)$ per call and $O(n \cdot k)$ overall. Build a frequency table in one pass instead:

```cpp
std::unordered_map<int, int> freq;
for (int x : v) ++freq[x];          // one pass, O(n) total
```

This is the **frequency counting** pattern, and it is Chapter 14.

10. Removing duplicates from a sorted array

DEFINITION | **The problem**
Given a **sorted** array, remove the repeats so each value appears once, keeping the order, using no extra array. Return the new length.

The sortedness is the gift: **duplicates must be adjacent**. So you never have to remember what you have seen — you only compare with the previous kept value.

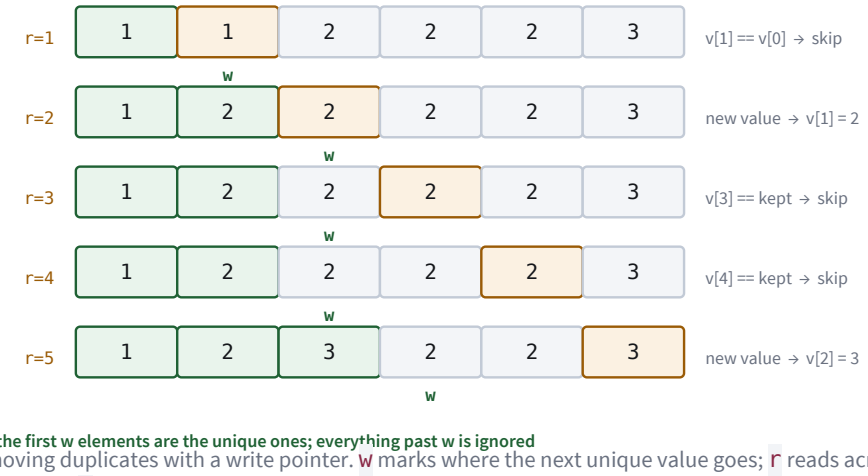


Figure 6.4 Removing duplicates with a write pointer. `w` marks where the next unique value goes; `r` reads across. They start together and `w` falls behind by exactly the number of duplicates skipped.

The trace

Input `{1, 1, 2, 2, 2, 3}`.

r	v[r]	v[w-1]	New?	Action	Array so far	w
start	—	—	—	keep `v[0]`	`1 . . . . .`	1
1	1	1	no	skip	`1 . . . . .`	1
2	2	1	yes	`v[1] = 2`	`1 2 . . . .`	2
3	2	2	no	skip	`1 2 . . . .`	2
4	2	2	no	skip	`1 2 . . . .`	2
5	3	2	yes	`v[2] = 3`	`1 2 3 . . .`	3

Table 6.3 Trace of the two-pointer deduplication. `w` only advances when a genuinely new value is found.

Result: the first `w = 3` elements are `{1, 2, 3}`.

Remove duplicates from a sorted array, in place

```cpp
#include <iostream>
#include <vector>

// Returns the number of unique elements, which occupy v[0 .. k-1].
int removeDuplicates(std::vector<int>& v) {
    if (v.empty()) return 0;
    int w = 1;                                  // next write position
    for (int r = 1; r < (int)v.size(); ++r) {
        if (v[r] != v[w - 1]) {                 // different from last kept
```

```
            v[w] = v[r];
            ++w;
        }
    }
    return w;
}

int main() {
    std::vector<int> v = {1, 1, 2, 2, 2, 3};
    int k = removeDuplicates(v);
    std::cout << "unique count = " << k << "\n";
    for (int i = 0; i < k; ++i) std::cout << v[i] << " ";
    std::cout << "\n";

    std::vector<int> all = {5, 5, 5, 5};
    std::cout << "all same -> " << removeDuplicates(all) << "\n";

    std::vector<int> empty;
    std::cout << "empty -> " << removeDuplicates(empty) << "\n";
    return 0;
}
```

```
unique count = 3
1 2 3
all same -> 1
empty -> 0
```

Cost. One pass: $O(n)$ time, $O(1)$ auxiliary space. Compare that with the obvious approach of calling `erase` for each duplicate, which is $O(n^2)$.

> **COMMON PITFALL | This only works on sorted input**
> On `{1, 2, 1}` the algorithm keeps all three, because the two 1s are not adjacent. If the input is unsorted you must either sort first ($O(n \log n)$) or use a hash set ($O(n)$ time, $O(n)$ space). That is a time–space trade-off exactly like the ones in Chapter 3.

11. Moving zeroes to the end

> **DEFINITION | The problem**
> Move every 0 to the end of the array while keeping the **relative order** of the non-zero elements, in place. `{0, 1, 0, 3, 12}` becomes `{1, 3, 12, 0, 0}`.

This is the same write-pointer idea as deduplication, and it is the cleanest example of the **in-place modification** pattern of Chapter 13.

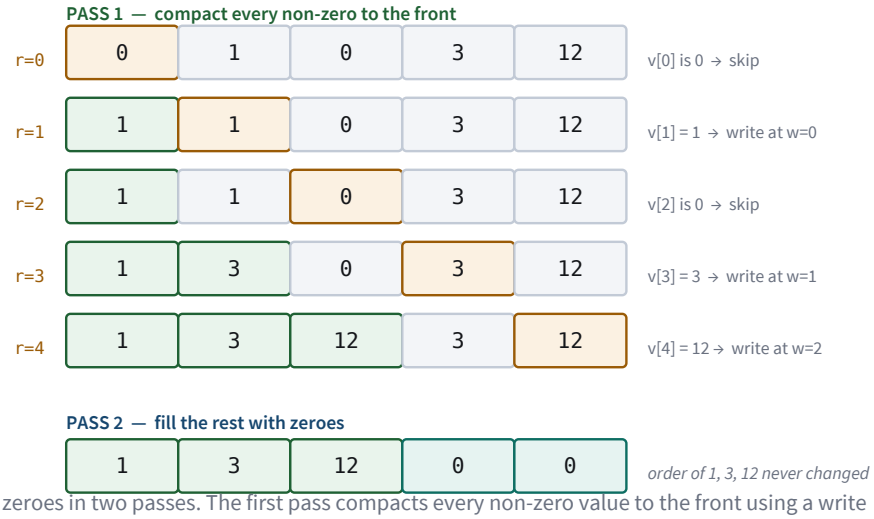


Figure 6.5 Moving zeroes in two passes. The first pass compacts every non-zero value to the front using a write pointer; the second fills the remainder with zeroes. The order of the non-zero values is never disturbed.

The trace

Input `{0, 1, 0, 3, 12}`.

r	v[r]	Zero?	Action	Array	w
start	—	—	—	`0 1 0 3 12`	0
0	0	yes	skip	`0 1 0 3 12`	0
1	1	no	`v[0] = 1`	`1 1 0 3 12`	1
2	0	yes	skip	`1 1 0 3 12`	1
3	3	no	`v[1] = 3`	`1 3 0 3 12`	2
4	12	no	`v[2] = 12`	`1 3 12 3 12`	3
fill	—	—	`v[3] = v[4] = 0`	`1 3 12 0 0`	3

Move zeroes to the end, order preserved

```cpp
#include <iostream>
#include <vector>

void moveZeroes(std::vector<int>& v) {
    int w = 0;                                  // next write position
    for (int r = 0; r < (int)v.size(); ++r)     // pass 1: compact
        non-zeroes
        if (v[r] != 0)
            v[w++] = v[r];
    while (w < (int)v.size())                   // pass 2: pad with zeroes
        v[w++] = 0;
}

int main() {
    std::vector<int> v = {0, 1, 0, 3, 12};
    moveZeroes(v);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

```

```
    std::vector<int> none = {1, 2, 3};
    moveZeroes(none);
    for (int x : none) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> allz = {0, 0, 0};
    moveZeroes(allz);
    for (int x : allz) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
1 3 12 0 0
1 2 3
0 0 0
```

Cost. Two passes over the array, so $2n$ operations, which is $O(n)$ time and $O(1)$ space.

KEY INSIGHT | A one-pass variation, and why it is not obviously better
You can do it in a single pass by swapping instead of copying:

```
int w = 0;
for (int r = 0; r < (int)v.size(); ++r)
    if (v[r] != 0) std::swap(v[w++], v[r]);
```

This is still $O(n)$ — it does not change the complexity, only the constant. It performs *more* writes than the two-pass version when zeroes are rare (a swap is three assignments), and fewer when zeroes are common.
Mention both in an interview. Knowing that two correct $O(n)$ solutions can differ in real cost, and being able to say *why*, is what separates a good answer from a complete one.

The complete cost table

Operation	Time	Auxiliary space	Why
Access, update	$O(1)$	$O(1)$	address arithmetic
Traverse	$O(n)$	$O(1)$	n elements
Linear search	$O(n)$	$O(1)$	could be anywhere
Binary search (sorted)	$O(\log n)$	$O(1)$	halves each step
Insert at end	$O(1)$ amortised	$O(1)$	spare capacity
Insert at front/middle	$O(n)$	$O(1)$	shift right
Delete at end	$O(1)$	$O(1)$	drop the last
Delete at front/middle	$O(n)$	$O(1)$	shift left
Reverse	$O(n)$	$O(1)$	$n/2$ swaps
Min / max	$O(n)$	$O(1)$	one pass

Operation	Time	Auxiliary space	Why
Sum / average	$O(n)$	$O(1)$	one pass
Count occurrences	$O(n)$	$O(1)$	one pass
Remove duplicates (sorted)	$O(n)$	$O(1)$	write pointer
Move zeroes	$O(n)$	$O(1)$	write pointer

Table 6.4 Every operation in this chapter. The pattern is clear: anything that touches one known position is $O(1)$; anything that must preserve contiguity across a change is $O(n)$.

CHAPTER RECAP | Chapter 6 in six lines

- Insertion shifts **right** and must copy **backwards**; deletion shifts **left** and must copy **forwards**. Both rules come from "read before you overwrite".
- Anything at a known index is $O(1)$; anything that disturbs contiguity is $O(n)$.
- Two pointers moving towards each other reverse an array in $O(1)$ space.
- A **write pointer** trailing a **read pointer** compacts an array in one pass — it solves deduplication, zero-moving, filtering, and much of Chapter 13.
- Initialise min/max from a real element, accumulate sums in `long long`, and cast before dividing.
- Deleting inside a forward loop skips elements. Loop backwards, or use a write pointer.

EXERCISES | Practise

1. Insert `5` at index 0 of `{1, 2, 3}` by hand, writing out every assignment in order. How many moves were there? Now insert at index 3 instead. How many?
2. Why must insertion copy backwards? Show what `{1, 2, 3, 4}` becomes if you copy forwards instead.
3. Write `deleteValue(v, x)` that removes the **first** occurrence of `x` and returns whether it found one. What should it do if `x` appears twice?
4. Modify `removeDuplicates` to keep **at most two** copies of each value. (Hint: compare against `v[w - 2]`, and think about what `w` must start at.)
5. Write `moveZeroes` so the zeroes go to the **front** instead, with the non-zero order preserved. Which pointer moves, and in which direction?
6. `countOf` is $O(n)$ per call. You must answer 1,000 count queries on a fixed array of 100,000 elements. What is the total cost as written, and how would you make it fast? What does your fix cost in space?
7. Combine operations 10 and 11: given a sorted array, remove the duplicates **and** move any zeroes to the end, in $O(n)$ with $O(1)$ space. In what order must you do the two steps, and why?
8. The two lines below are meant to reverse an array. Find the bug by tracing `{1, 2, 3, 4}`, not by staring at it.

```
for (int i = 0; i < (int)v.size(); ++i)
    std::swap(v[i], v[v.size() - 1 - i]);
```

CHAPTER

07

Two-Dimensional Arrays

A grid is a one-dimensional array wearing a disguise

CORE PREREQS: Chapters 4 and 5

Grids appear everywhere: images, game boards, spreadsheets, distance tables, the dynamic-programming tables of later courses. A 2-D array looks like a new structure but is not — it is a flat block with an index formula on top. Once you see that, the address arithmetic, the traversal order that runs four times faster, and the awkwardness of passing grids to functions all make sense at once.

WHAT YOU WILL LEARN

- Declare and use both `int g[R][C]` and `std::vector<std::vector<int>>`
- Explain row-major layout and compute the address of `g[r][c]`
- Traverse a grid by rows and by columns, and say which is faster and why
- Sum rows, columns and diagonals correctly
- Transpose a square matrix in place
- Choose between a fixed grid, a vector of vectors, and a flattened 1-D array

A grid is a line, folded

Here is a 3×4 grid of numbers:

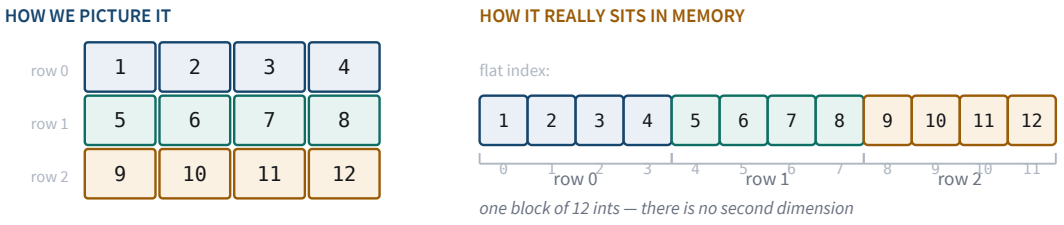


Figure 7.1 A grid as we picture it, and the same grid as it actually sits in memory: one unbroken line of 12 integers, row after row. The two dimensions are a convenience the compiler provides; the hardware only has one.

The picture on the left is how we think. The picture on the right is what exists. There is no second dimension in memory — there is one block of 12 integers, with row 0 first,

then row 1, then row 2.

> **DEFINITION | Row-major order**
> C and C++ store 2-D arrays in **row-major order**: the entire first row, then the entire second row, and so on. Elements of the same *row* are neighbours in memory; elements of the same *column* are far apart.
> (Fortran, MATLAB and Julia use *column-major* order, which is why code translated between the two families often becomes mysteriously slow.)

The address formula, extended

In Chapter 2 we had address(i)=B+i×S. For a grid with C columns, the formula gains one term:

> **DEFINITION | The 2-D address formula**
> For `g[r][c]` in a grid with C columns, element size S, base address B:
> $$\text{address}(r,c) = B + (r \times C + c) \times S$$
> Read it as: *skip r whole rows (each C elements long), then move c more elements.*
> The number inside the brackets, $r \times C + c$, is called the **flat index**: the position this element would have in a plain 1-D array.

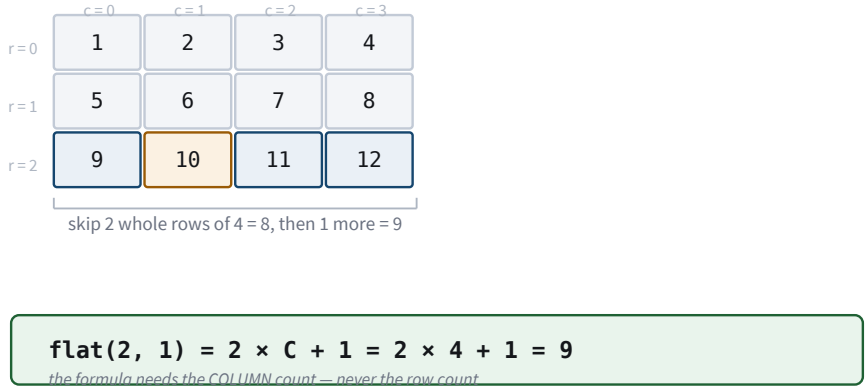


Figure 7.2 Finding `g[2][1]` . Skip two whole rows of four, then move one more — flat index 2×4+1=9. One multiplication, one addition, exactly as in one dimension.

Notice what the formula needs: **the number of columns**. It does not need the number of rows. That asymmetry explains a C++ rule that otherwise looks arbitrary — see "Passing grids to functions" below.

Proving that a grid is really a line

```
#include <iostream>

int main() {
    int g[3][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12}};

    // Walk the grid as if it were one long array of 12 ints.
```

```
    int* flat = &g[0][0];
    for (int i = 0; i < 12; ++i)
        std::cout << flat[i] << " ";
    std::cout << "\n";

    // The flat index of g[2][1] is 2*4 + 1 = 9
    std::cout << "g[2][1]  = " << g[2][1] << "\n";
    std::cout << "flat[9]  = " << flat[9] << "\n";
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9 10 11 12
g[2][1]  = 10
flat[9]  = 10
```

The grid printed in one straight line, in row order, and `g[2][1]` and `flat[9]` are the same memory. The second dimension is notation.

Declaring grids

Three ways to make a grid

```
#include <iostream>
#include <vector>

int main() {
    // 1. Fixed size, known at compile time. Fastest, least flexible.
    int fixed[2][3] = {{1, 2, 3},
                       {4, 5, 6}};

    // 2. Vector of vectors. Size decided at run time; rows may differ.
    int rows = 2, cols = 3;
    std::vector<std::vector<int>> grid(rows, std::vector<int>(cols, 0));
    grid[1][2] = 9;

    // 3. One flat vector plus the index formula. Fastest of the dynamic
    ↪ options.
    std::vector<int> flat(rows * cols, 0);
    flat[1 * cols + 2] = 9;

    std::cout << fixed[1][2] << " " << grid[1][2] << " "
              << flat[1 * cols + 2] << "\n";
    std::cout << "grid rows = " << grid.size()
              << ", cols = " << grid[0].size() << "\n";
    return 0;
}
```

```
6 9 9
grid rows = 2, cols = 3
```

Read `std::vector<std::vector<int>> grid(rows, std::vector<int>(cols, 0))` from the inside: `std::vector<int>(cols, 0)` is one row — `cols` zeroes. The outer constructor

then makes `rows` copies of it.

> **KEY INSIGHT | Which of the three should you use?**
> `std::vector<std::vector<int>>` for almost everything. It is readable, it carries its own dimensions, `grid[r][c]` works naturally, and rows can have different lengths if you need that.
> **A flat `std::vector<int>` with `flat[r * cols + c]`** when performance genuinely matters. A vector of vectors is R separate allocations scattered across memory; a flat vector is one block, which the processor's cache handles far better. The cost is that you write the index formula yourself.
> `int g[R][C]` only when both dimensions are compile-time constants and the grid is small. It cannot be resized, it does not know its own size inside a function, and it is easy to overflow the stack with a large one — a `double g[1000][1000]` is 8 MB and will simply crash most programs.

Traversing a grid

Row-wise and column-wise traversal

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<std::vector<int>> g = {{1, 2, 3},
                                       {4, 5, 6}};
    int R = (int)g.size(), C = (int)g[0].size();

    std::cout << "row-major:    ";
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c)
            std::cout << g[r][c] << " ";
    std::cout << "\n";

    std::cout << "column-major: ";
    for (int c = 0; c < C; ++c)
        for (int r = 0; r < R; ++r)
            std::cout << g[r][c] << " ";
    std::cout << "\n";
    return 0;
}
```

```
row-major:    1 2 3 4 5 6
column-major: 1 4 2 5 3 6
```

Both loops visit all $R \times C$ elements, so both are $O(R \cdot C)$. But they do not take the same amount of time on a real machine, and the difference can be large.

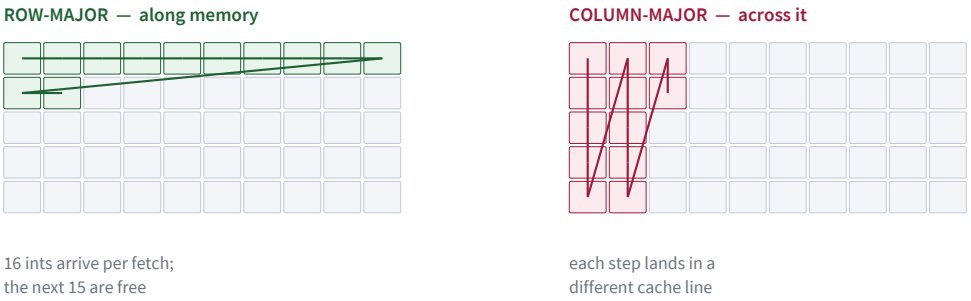


Figure 7.3 Why the loop order matters. Row-major traversal walks along memory; column-major jumps a whole row at every step. The processor fetches memory in blocks, so the first order gets several useful elements per fetch and the second often gets one.

PERFORMANCE NOTE | The four-times-faster loop

Processors do not read one integer at a time. They read a **cache line** — typically 64 bytes, which is 16 `int` s — and keep it in fast memory nearby.

Row-major traversal reads `g[r][0]` , then `g[r][1]` , and so on: after the first fetch, the next 15 elements are already there. Roughly one fetch per 16 elements.

Column-major traversal reads `g[0][c]` , then `g[1][c]` , which is *C* elements further along. If *C* is large, every single access is a new fetch, and 15 of the 16 integers brought in are thrown away.

On a 2000×2000 grid, summing row-wise is commonly **three to six times faster** than summing column-wise — identical Big O, identical number of operations, very different wall-clock time.

Rule: make the last index the innermost loop. It costs nothing to get right and it is free performance.

This is the first time Big O has failed to tell you the whole story, and it is worth noticing. Big O is about *scaling*; it deliberately ignores constant factors, and here the constant factor is 4. Both facts are true at once: the loops have the same complexity, and one is four times faster.

Rows, columns and diagonals

Summing a row, a column, and both diagonals

```
#include <iostream>
#include <vector>

using Grid = std::vector<std::vector<int>>;

int rowSum(const Grid& g, int r) {
    int s = 0;
    for (int c = 0; c < (int)g[r].size(); ++c) s += g[r][c];
    return s;
}

```

```cpp
int colSum(const Grid& g, int c) {
    int s = 0;
    for (int r = 0; r < (int)g.size(); ++r) s += g[r][c];
    return s;
}

// Main diagonal: r == c. Anti-diagonal: r + c == n - 1.
int mainDiagonal(const Grid& g) {
    int s = 0;
    for (int i = 0; i < (int)g.size(); ++i) s += g[i][i];
    return s;
}

int antiDiagonal(const Grid& g) {
    int n = (int)g.size(), s = 0;
    for (int i = 0; i < n; ++i) s += g[i][n - 1 - i];
    return s;
}

int main() {
    Grid g = {{1, 2, 3},
              {4, 5, 6},
              {7, 8, 9}};
    std::cout << "row 1       = " << rowSum(g, 1)      << "\n";
    std::cout << "col 2       = " << colSum(g, 2)      << "\n";
    std::cout << "main diag   = " << mainDiagonal(g)   << "\n";
    std::cout << "anti diag   = " << antiDiagonal(g)   << "\n";
    return 0;
}
```

```
row 1       = 15
col 2       = 18
main diag   = 15
anti diag   = 15
```

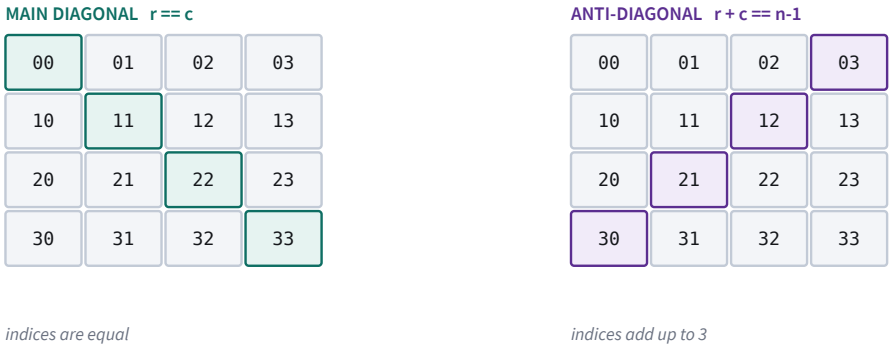


Figure 7.4 The two diagonals. On the main diagonal the row and column indices are equal; on the anti-diagonal they add up to $n-1$. Both facts are worth memorising — they appear in a surprising number of problems.

Transposing a square matrix in place

DEFINITION | Transpose
The **transpose** of a matrix swaps rows with columns: element (r, c) moves to (c, r). The main diagonal does not move, because (i, i) maps to itself.

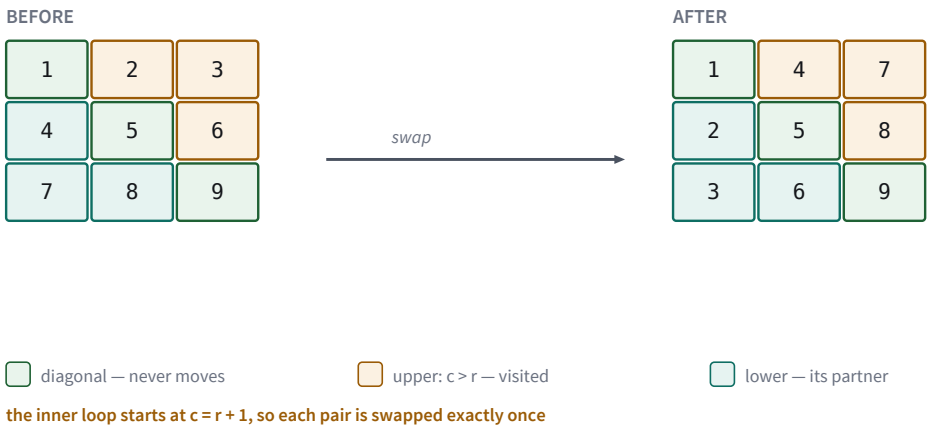


Figure 7.5 Transposing in place. Only the elements **above** the diagonal are visited; each is swapped with its mirror below. Visiting all of them instead would swap every pair twice and leave the matrix unchanged.

In-place transpose: the loop bounds are the whole trick

```cpp
#include <iostream>
#include <vector>

void transpose(std::vector<std::vector<int>>& g) {
    int n = (int)g.size();
    for (int r = 0; r < n; ++r)
        for (int c = r + 1; c < n; ++c)      // c starts at r+1, not 0
            std::swap(g[r][c], g[c][r]);
}

int main() {
    std::vector<std::vector<int>> g = {{1, 2, 3},
                                       {4, 5, 6},
                                       {7, 8, 9}};
    transpose(g);
    for (const auto& row : g) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }
    return 0;
}
```

```
1 4 7
2 5 8
3 6 9
```

COMMON PITFALL | `c = 0` transposes twice, which is the same as not at all

```
for (int r = 0; r < n; ++r)
    for (int c = 0; c < n; ++c)              // BUG
        std::swap(g[r][c], g[c][r]);
```

When r=0, c=1 this swaps (0,1) with (1,0). Later, when r=1, c=0, it swaps them straight back. Every pair is swapped twice and the matrix is unchanged — a bug that produces no error and no visible effect, which makes it unusually annoying.
Starting the inner loop at `r + 1` visits each pair exactly once. The same "upper triangle only" bound appears whenever you process unordered pairs, and it is why the triangular loop of Chapter 3 is worth recognising.

Transposing is the first half of rotating a matrix by 90°, which is Chapter 17.

Passing grids to functions

This is where fixed 2-D arrays become genuinely unpleasant.

A fixed grid loses its row count, but not its column count

```
#include <iostream>

// The number of COLUMNS must be given; the number of rows need not be.
void printGrid(int g[][4], int rows) {
    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < 4; ++c) std::cout << g[r][c] << " ";
        std::cout << "\n";
    }
}

int main() {
    int g[2][4] = {{1, 2, 3, 4},
                   {5, 6, 7, 8}};
    printGrid(g, 2);
    return 0;
}
```

```
1 2 3 4
5 6 7 8
```

Why must the column count be in the type? Because of the address formula: $B+(r\times C+c)\times S$ needs C. Without it the compiler cannot work out where row r begins. The row count never appears in the formula, so it can be passed as an ordinary argument.

The practical consequence is that `printGrid` works **only** on grids with exactly 4 columns. A 5-column grid needs a different function.

KEY INSIGHT | Which is why you should use vectors

```
void printGrid(const std::vector<std::vector<int>>& g) {
    for (const auto& row : g) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }
}
```

One function, any dimensions, no size arguments, no decay, and the `const&` means nothing is copied. Unless you are writing code for a microcontroller, this is the right default.

Jagged grids

Nothing requires the rows to be the same length.

Rows of different lengths

```
#include <iostream>
#include <vector>

int main() {
    std::vector<std::vector<int>> tri = {{1},
                                         {1, 1},
                                         {1, 2, 1},
                                         {1, 3, 3, 1}};
    for (const auto& row : tri) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }
    std::cout << "row 3 has " << tri[3].size() << " elements\n";
    return 0;
}
```

```
1
1 1
1 2 1
1 3 3 1
row 3 has 4 elements
```

That is Pascal's triangle, and it is a **jagged** array — a vector of vectors where each row has its own length. A fixed `int g[4][4]` cannot do this without wasting space and inventing padding values.

COMMON PITFALL | Always ask a row for its own size

```
std::vector<std::vector<int>> tri = {{1}, {1, 1}, {1, 2, 1}};
int C = (int)tri[0].size();                 // 1
for (int r = 0; r < (int)tri.size(); ++r)
    for (int c = 0; c < C; ++c)             // BUG on jagged data
        std::cout << tri[r][c];
```

Measuring the width once, from row 0, is a habit from fixed grids. On a jagged grid it either skips elements (as here) or reads out of bounds. Write `c < (int)tri[r].size()` and the problem disappears.
Also beware `g[0].size()` on an **empty** grid — `g[0]` on a vector with no rows is undefined behaviour. Check `g.empty()` first.

Cost summary

Operation	Cost	Note
`g[r][c]` access or update	$O(1)$	one flat-index calculation
Traverse the whole grid	$O(R \cdot C)$	every cell once
Sum one row	$O(C)$	contiguous — cache-friendly
Sum one column	$O(R)$	strided — cache-unfriendly
Sum a diagonal	$O(\min(R, C))$	one cell per row
Transpose (square, in place)	$O(n^2)$ time, $O(1)$ space	upper triangle only
Search unsorted	$O(R \cdot C)$	could be anywhere

Table 7.1 Operations on an $R \times C$ grid. Notice that every cost is just the 1-D cost with n replaced by the number of cells visited.

CHAPTER RECAP | Chapter 7 in six lines

- A 2-D array is a 1-D block plus the formula flat$= r \times C + c$.
- C++ is **row-major**: elements of a row are neighbours; elements of a column are not.
- Make the last index the innermost loop. Same Big O, several times faster.
- `std::vector<std::vector<int>>` is the sensible default; a flat vector is faster; `int g[R][C]` is for compile-time constants only.
- A fixed 2-D array passed to a function must declare its **column** count, because the address formula needs it.
- Transpose the upper triangle only (`c = r + 1`), or you swap every pair twice and change nothing.

EXERCISES | **Practise**

1. For a 5×8 grid of `int` starting at address 4000, what is the address of `g[3][6]`? What is its flat index?
2. A grid is stored flat in a `std::vector<int>` with `cols = 7`. Given flat index 30, what are r and c? (You need integer division and remainder.)
3. Write a function that returns the sum of the border cells of a grid (the first and last rows, and the first and last columns), counting each corner only once.
4. Why does the transpose loop start at `c = r + 1`? What exactly happens with `c = r`? With `c = 0`?
5. Write a function that checks whether a square grid is **symmetric**, that is, `g[r][c] == g[c][r]` for all r, c. How many comparisons does the efficient version need?
6. A colleague says "column-major traversal is $O(n^2)$ and row-major is $O(n)$". They are wrong. What is the correct statement about the difference?
7. Write Pascal's triangle with n rows as a jagged vector of vectors. Each row starts and ends with 1, and every other entry is the sum of the two above it.

PART III

The Array Patterns

Almost every array problem you will ever be asked is one of ten shapes wearing a costume. Learn to see the shape and the problem stops being a puzzle and becomes a recognition task.

08 Pattern 1: Linear Traversal
09 Pattern 2: Two Pointers
10 Pattern 3: Sliding Window
11 Pattern 4: Prefix Sums
12 Pattern 5: Binary Search
13 Pattern 6: In-Place Modification
14 Pattern 7: Frequency Counting
15 Pattern 8: Sorting-Based Approaches
16 Pattern 9: Kadane's Algorithm
17 Pattern 10: Matrix Patterns

CHAPTER

08

Pattern 1: Linear Traversal

One pass, carrying everything you need to know with you

CORE PREREQS: Chapter 6

The humblest pattern and the one you will use most. The skill is not writing the loop — you can already do that — it is working out what to carry as you go, so that a problem which looks like it needs two or three passes needs only one. This chapter builds that skill on five problems and ends with the question to ask yourself whenever you are tempted to loop twice.

WHAT YOU WILL LEARN

- Recognise when a problem needs nothing more than one pass
- Design the *running state*: the few variables that must travel with the loop
- Convert an obvious two-pass solution into a one-pass one
- Handle the awkward cases: empty input, all-equal input, one element
- Solve largest, second-largest, is-sorted, max consecutive ones, and
- single-pass counting
- Know when an early exit is valuable and when it changes nothing

When to use it

DEFINITION | The linear traversal pattern
Visit each element once, in order, maintaining a small amount of **running state** — a few variables that summarise everything seen so far.
Use it when the answer depends on **every** element, and when what you need to remember about the past is small and fixed.

How to recognise it in a problem statement:

Words in the problem	What they mean
“find the maximum / minimum / sum”	one pass, one or two variables
“count how many …”	one pass, a counter
“check whether all / any …”	one pass, possibly with an early exit
“the longest run of …”	one pass, a current-run and a best-run variable

Words in the problem	What they mean
“the first element that …”	one pass with an early return

If the answer depends on **pairs** of elements, this pattern is not enough and you want Chapter 9 or Chapter 14 instead.

The idea: running state

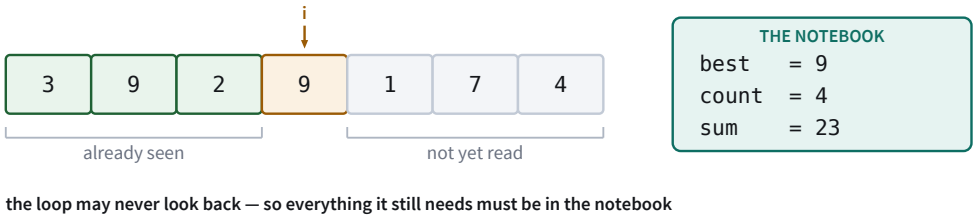


Figure 8.1 Running state. The loop moves left to right; the small box of variables beside it holds everything the algorithm still needs to know. When the loop ends, the answer is in that box.

Here is the mental model. You are walking along the array with a notebook that has room for only two or three lines. At each element you may update the notebook, but you can never go back and look at the array again.

What must be written in the notebook? That question *is* the algorithm. Answer it and the code writes itself.

Problem 1: Largest element

Notebook: one line — the largest so far.

Largest element

```cpp
#include <iostream>
#include <stdexcept>
#include <vector>

int largest(const std::vector<int>& v) {
    if (v.empty()) throw std::invalid_argument("no largest in an empty
        array");
    int best = v[0];                // start from a real element
    for (int i = 1; i < (int)v.size(); ++i)
        if (v[i] > best) best = v[i];
    return best;
}

int main() {
    std::cout << largest({3, 9, 2, 9, 1}) << "\n";
```

```
    std::cout << largest({-5, -2, -8})    << "\n";
    std::cout << largest({42})            << "\n";
    return 0;
}
```

```
9
-2
42
```

$O(n)$ time, $O(1)$ space. The all-negative case is the one that catches people who start `best` at 0.

Problem 2: Second-largest element

Now the notebook needs two lines, and the updates get interesting.

> **DEFINITION | The problem, stated precisely**
> Return the second-largest **distinct** value. In `{5, 5, 4}` the answer is 4, not 5. If there is no second distinct value — as in `{7, 7, 7}` or `{7}` — report that.
> Beginners skip this paragraph and lose the problem on it. *"Second largest"* is ambiguous until you decide what duplicates mean, and in an interview you should ask.

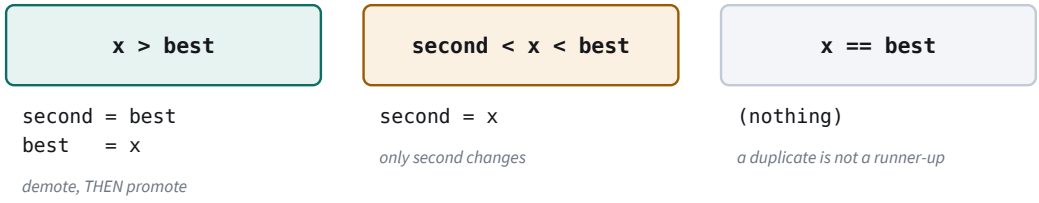
The obvious approach, and why we improve on it

Sort descending and take the first element different from the front: correct, and $O(n \log n)$. Or find the maximum, then scan again for the largest value below it: correct, $O(n)$, two passes.

Both work. One pass is better, and the thinking that gets you there is the point.

The one-pass idea

Carry two values: `best` and `second`. For each element x there are exactly three cases, and getting their **order** right is the whole problem.




```
best = x;  second = best;    ✗   both end up equal to x
second = best;  best = x;    ✓   the old champion survives
```

Figure 8.2 The three cases for the second-largest. A new champion pushes the old champion down into second place — which is why `second = best` must happen *before* `best = x`.

The trace

Input `{2, 9, 4, 9, 7}`. `LOW` stands for "nothing yet".

`x`	Case	`best` before	`second` before	`best` after	`second` after
2	new best	LOW	LOW	2	LOW
9	new best	2	LOW	9	2
4	middle	9	2	9	4
9	equal to best	9	4	9	4
7	middle	9	4	9	7

Table 8.1 Tracing second-largest. Watch row 4: a value equal to `best` must change nothing, which is what makes the answer *distinct*.

Second largest in one pass

```cpp
#include <climits>
#include <iostream>
#include <vector>

// Returns the second-largest DISTINCT value, or INT_MIN if there is none.
int secondLargest(const std::vector<int>& v) {
    long long best = LLONG_MIN, second = LLONG_MIN;
    for (int x : v) {
        if (x > best) {
            second = best;          // the old champion is demoted FIRST
            best = x;
        } else if (x < best && x > second) {
            second = x;             // not a champion, but better than second
        }
        // x == best: a duplicate of the champion changes nothing
    }
    return second == LLONG_MIN ? INT_MIN : (int)second;
}

int main() {
    std::cout << secondLargest({2, 9, 4, 9, 7}) << "\n";
    std::cout << secondLargest({5, 5, 4})       << "\n";
    std::cout << (secondLargest({7, 7, 7}) == INT_MIN ? "none" : "?") << "\n";
    std::cout << (secondLargest({7}) == INT_MIN ? "none" : "?")       << "\n";
    return 0;
}
```

```
7
4
none
none
```

COMMON PITFALL | The three bugs in this problem
Updating in the wrong order. Writing `best = x;` before `second = best;` loses the old champion — `second` just becomes `x` as well. Demote first, then promote.
Using `INT_MIN` as the sentinel for an `int`. If the array genuinely contains `INT_MIN`, the "none" answer and a real answer become indistinguishable. Here the accumulators are `long long`, so `LLONG_MIN` is a value no `int` input can ever be.
Forgetting `x < best` in the `else if`. Without it, a duplicate of the champion satisfies `x > second` and wrongly becomes the second largest, so `{5, 5, 4}` returns 5.

Problem 3: Is the array sorted?

Notebook: nothing at all — you only need the current element and the one before it.

Is it sorted (non-decreasing)?

```cpp
#include <iostream>
#include <vector>

bool isSorted(const std::vector<int>& v) {
    for (int i = 1; i < (int)v.size(); ++i)
        if (v[i] < v[i - 1]) return false;   // one violation is enough
    return true;                             // empty and 1-element are
        sorted
}

int main() {
    std::cout << isSorted({1, 2, 2, 5}) << "\n";
    std::cout << isSorted({1, 3, 2})    << "\n";
    std::cout << isSorted({})           << "\n";
    std::cout << isSorted({9})          << "\n";
    return 0;
}
```

```
1
0
1
1
```

Two design points worth noticing:

The loop starts at `i = 1`. Comparing `v[i]` with `v[i-1]` at `i = 0` would read `v[-1]`. Starting at 1 makes the out-of-bounds access impossible rather than merely unlikely.

An empty array is sorted, and so is a one-element array. Both are "vacuously true" — there is no pair out of order because there is no pair. Returning `true` is correct, and the loop does it automatically by never running. Edge cases that fall out of the structure are better than edge cases you special-case.

KEY INSIGHT | **Early exit changes the best case, not the complexity**
`return false` on the first violation means `{9, 1, 1, 1, ... }` finishes in one comparison. Lovely — but the **worst case** (a genuinely sorted array) still examines everything, so it is still $O(n)$.
Early exits are worth having. They are not worth claiming a better complexity for, and claiming one in an interview is a clear tell.

Problem 4: Maximum consecutive ones

DEFINITION | **The problem**
Given an array of 0s and 1s, find the length of the longest unbroken run of 1s.
`{1, 1, 0, 1, 1, 1}` gives 3.

Notebook: two lines — the current run, and the best run seen.

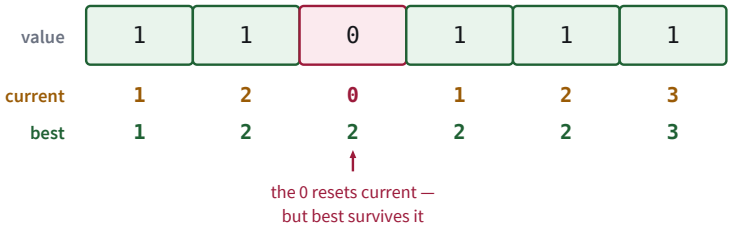


Figure 8.3 The current-run / best-run pair. The current run grows on a 1 and resets to zero on a 0; `best` only ever increases, so it survives the resets and holds the answer at the end.

The trace

x	current	best
start	0	0
1	1	1
1	2	2
0	0	2
1	1	2
1	2	2
1	3	3

Longest run of 1s

```
#include <algorithm>
#include <iostream>
```

```cpp
#include <vector>

int maxConsecutiveOnes(const std::vector<int>& v) {
    int current = 0, best = 0;
    for (int x : v) {
        if (x == 1) {
            ++current;
            best = std::max(best, current);   // update INSIDE the run
        } else {
            current = 0;                      // the run is broken
        }
    }
    return best;
}

int main() {
    std::cout << maxConsecutiveOnes({1, 1, 0, 1, 1, 1}) << "\n";
    std::cout << maxConsecutiveOnes({0, 0, 0})          << "\n";
    std::cout << maxConsecutiveOnes({1, 1, 1})          << "\n";
    std::cout << maxConsecutiveOnes({})                 << "\n";
    return 0;
}
```

```
3
0
3
0
```

COMMON PITFALL | Updating `best` only when the run ends

```cpp
for (int x : v) {
    if (x == 1) ++current;
    else { best = std::max(best, current); current = 0; }
}
return best;                                  // BUG
```

On `{1, 1, 1}` the run never ends, so `best` is never updated and the function returns 0. You can fix it with one more `best = std::max(best, current);` after the loop — but updating inside the run, as in the correct version, means there is nothing to remember and nothing to forget. **Prefer the version with no after-the-loop correction**; code that needs a patch at the end usually has a cleaner shape waiting to be found.

This "current versus best" shape is the seed of the sliding window (Chapter 10) and of Kadane's algorithm (Chapter 16). You have just met the idea that carries both.

Turning two passes into one

Here is the general move, and it is worth internalising.

Suppose you want the number of elements above the average. The obvious solution is two passes: one to compute the average, one to count. That is fine — it is still $O(n)$.

But sometimes the second pass can be folded in, and the question that reveals it is:

KEY INSIGHT | The one-pass question
"At the moment I reach element i, do I already know everything I need in order to do the work for element i?"

- **Yes** → fold the passes together.
- **No** → you genuinely need the earlier pass. Two passes is $2n$ steps, which is still $O(n)$, and clarity beats cleverness.

For counting-above-average the answer is **no**: you cannot know the average until you have seen the last element. Two passes it is, and that is the right answer.
For sum-and-maximum the answer is **yes**: both are running totals, so one pass does both.

One pass computing four things at once

```cpp
#include <iostream>
#include <vector>

struct Stats {
    long long sum = 0;
    int count = 0;
    int lo = 0, hi = 0;
};

Stats summarise(const std::vector<int>& v) {
    Stats s;
    if (v.empty()) return s;
    s.lo = s.hi = v[0];
    for (int x : v) {
        s.sum += x;
        ++s.count;
        if (x < s.lo) s.lo = x;
        if (x > s.hi) s.hi = x;
    }
    return s;
}

int main() {
    Stats s = summarise({4, -1, 9, 3});
    std::cout << "sum=" << s.sum << " n=" << s.count
              << " min=" << s.lo << " max=" << s.hi << "\n";
    return 0;
}
```

```
sum=15 n=4 min=-1 max=9
```

Four answers, one traversal, $O(1)$ space. Four separate loops would also be $O(n)$ — but they would read the array four times, and on large data that is four times the memory traffic.

Cost summary

Problem	Time	Space	Running state
Largest	$O(n)$	$O(1)$	`best`
Second largest	$O(n)$	$O(1)$	`best` , `second`
Is sorted	$O(n)$	$O(1)$	none
Max consecutive ones	$O(n)$	$O(1)$	`current` , `best`
Sum / min / max / count	$O(n)$	$O(1)$	four accumulators

INTERVIEW ANGLE | **What this pattern is really testing**

A linear-traversal question is never about the loop. It is about whether you:

- ask what "second largest" means when there are duplicates;
- test the empty array and the all-equal array without being prompted;
- initialise from a real element rather than from 0;
- notice that `{1, 1, 1}` never ends its run.

Interviewers use these problems precisely because the algorithm is trivial, so nothing distracts from how carefully you think.

CHAPTER RECAP | **Chapter 8 in five lines**

- One pass, with a small fixed **running state**, solves any problem whose answer depends on a summary of everything seen so far.
- Designing the running state *is* the algorithm; write down the notebook before you write the loop.
- Demote before you promote when maintaining a top-two.
- Update `best` inside the run, not when the run ends, so the final run is not lost.
- Early exit improves the best case only; the complexity is unchanged.

EXERCISES | **Practise**

1. Find the **third** largest distinct value in one pass. How many variables does the notebook need, and in what order must they be updated?
2. Count how many elements are strictly greater than both of their neighbours. What happens at the two ends?
3. Check whether an array is sorted in **either** direction (ascending or descending) in one pass. What should `{3, 3, 3}` return?
4. Find the longest run of **equal** values, and return both the value and the length. What is the running state now?
5. Given an array of 0s and 1s, find the longest run of 1s you could get if you were allowed to flip **one** 0 to a 1. (This is a genuine step up — try the one-pass version, and if it resists, read Chapter 10 and come back.)
6. Explain why counting elements above the average cannot be done in one pass, and name a quantity that *can* be compared against as you go.
7. Write a one-pass function returning the index of the first element that is larger than every element before it and smaller than every element after it. What makes this harder than it looks, and which pattern would make it easy?

CHAPTER

09

Pattern 2: Two Pointers

Two indices, moving with purpose, turning $O(n^2)$ into $O(n)$

CORE PREREQS: Chapters 6 and 8

The two-pointer pattern replaces a nested loop with two indices that between them cross the array only once. It comes in two shapes — pointers converging from the ends, and pointers chasing each other from the same end — and almost every $O(n)$ solution to a problem that "obviously" needs $O(n^2)$ is one of them. This chapter derives both shapes from brute force, and proves why nothing is missed.

WHAT YOU WILL LEARN

- Recognise the two shapes: converging pointers and same-direction pointers
- Derive the converging solution to Two Sum on a sorted array, and justify it
- Check palindromes and reverse arrays with converging pointers
- Use a read pointer and a write pointer to filter an array in place
- Explain why the pattern is $O(n)$ even though it has two moving indices
- Know the precondition each shape needs, and what breaks without it

The two shapes

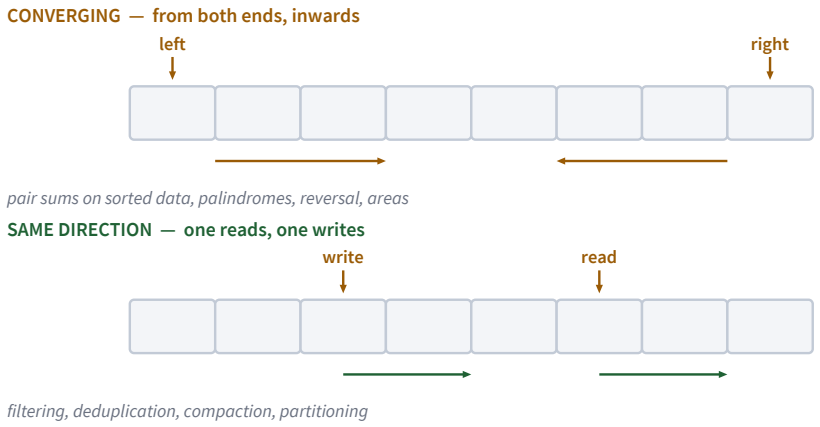


Figure 9.1 The two shapes of the pattern. **Converging**: one pointer at each end, moving towards each other, usually requiring sorted input. **Same direction**: both start at the left, one reads ahead while the other writes behind, requiring nothing.

DEFINITION | The two-pointer pattern
Keep **two indices** into the array and move them according to a rule, so that between them they traverse the array a constant number of times.
Converging (`left` at 0, `right` at $n-1$, moving inwards): used for pair problems on sorted data, palindromes, reversal, and area problems. Usually needs sorted or symmetric input.
Same direction (`slow` / `write` and `fast` / `read` , both starting at 0): used for filtering, deduplication, compaction and partitioning. Needs nothing.

How to recognise a two-pointer problem:

Signal in the problem	Shape to reach for
sorted array, and you need a **pair**	converging
"is it a palindrome", "mirror", "reverse"	converging
"maximum area / water between two lines"	converging
"remove / keep / move elements in place"	same direction
"partition into two groups"	same direction
"$O(1)$ extra space" with an $O(n^2)$ obvious answer	one of the two

Shape 1: Converging pointers

The motivating problem: Two Sum on a sorted array

DEFINITION | The problem
Given a **sorted** array and a target, return the indices of two elements that add to the target, or report that there is no such pair.
`{2, 7, 11, 15}` , target 9 → indices 0 and 1, because 2+7=9.

Step 1: the brute force

Try every pair.

Brute force: try every pair

```
for (int i = 0; i < n; ++i)
    for (int j = i + 1; j < n; ++j)
        if (v[i] + v[j] == target)
            std::cout << i << " " << j;    // found - but only after
                n^2/2 tries
```

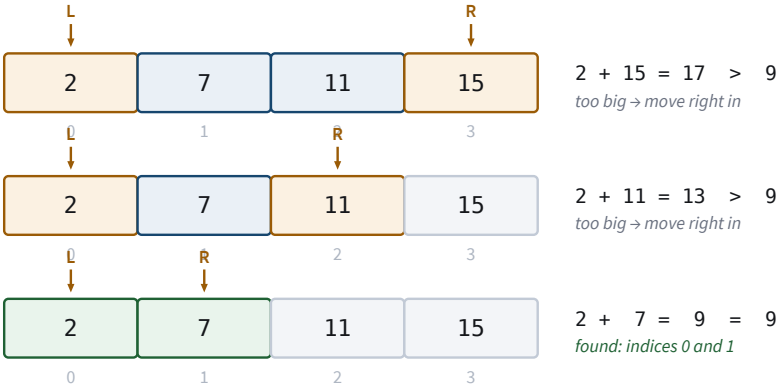
Correct, and $O(n^2)$: about $n^2/2$ pairs. For n=100,000 that is 5×10^9 comparisons — roughly a minute. Too slow.

Step 2: find the waste

The brute force never uses the fact that the array is **sorted**. That is always the sign that a better solution exists: an unused precondition is a wasted asset.

Step 3: the insight

Put `left` at the smallest element and `right` at the largest, and look at their sum.



moving L past an element rules out every pair that used it — n steps cover $n^2/2$ pairs

Figure 9.2 Two Sum by converging pointers. The sum is too small, so the only way to increase it is to raise the smaller end; the sum is too large, so the only way to decrease it is to lower the larger end. Each comparison eliminates an entire row or column of the brute-force table.

> **KEY INSIGHT | Why moving a pointer discards nothing**
> Suppose `v[left] + v[right] < target`.
> Consider every pair that uses `left`: its partner can be anything from `left+1` to `right`.
> The **largest** such sum is `v[left] + v[right]`, because `v[right]` is the largest remaining element. That largest sum is already too small — so **no pair using `left` can ever reach the target.** `left` is eliminated entirely, and we advance it.
> The mirror argument applies when the sum is too large: `right`'s best partner is `v[left]`, the smallest remaining, and if even that overshoots, no pair using `right` can work.
> Each step removes one element from consideration for good, so after at most n steps we have either found the answer or ruled out every pair. **One pass covers $n^2/2$ possibilities**, and that is the whole magic of the pattern.

The trace

`{2, 7, 11, 15}`, target 9.

`left`	`right`	`v[left]`	`v[right]`	sum	vs target	Action
0	3	2	15	17	too big	`--right`
0	2	2	11	13	too big	`--right`
0	1	2	7	9	equal	found: (0, 1)

Table 9.1 Each row eliminates one element permanently. Three comparisons replace the six the brute force would make — and on a large array, n replaces $n^2/2$.

Two Sum on a sorted array, O(n)

```cpp
#include <iostream>
#include <vector>

// Returns {i, j} with i < j and v[i] + v[j] == target, or {-1, -1}.
std::pair<int, int> twoSumSorted(const std::vector<int>& v, int target) {
    int left = 0, right = (int)v.size() - 1;
    while (left < right) {
        long long sum = (long long)v[left] + v[right];   // no overflow
        if (sum == target) return {left, right};
        if (sum < target) ++left;      // need a bigger sum
        else              --right;     // need a smaller sum
    }
    return {-1, -1};
}

int main() {
    std::vector<int> v = {2, 7, 11, 15};
    auto [i, j] = twoSumSorted(v, 9);
    std::cout << i << " " << j << "\n";

    auto [a, b] = twoSumSorted(v, 100);
    std::cout << a << " " << b << "\n";

    std::vector<int> neg = {-8, -3, 0, 4, 9};
    auto [c, d] = twoSumSorted(neg, 1);
    std::cout << c << " " << d << "\n";
    return 0;
}
```

```
0 1
-1 -1
0 4
```

$O(n)$ time, $O(1)$ space — down from $O(n^2)$.

COMMON PITFALL | The precondition is not optional
On **unsorted** input this returns wrong answers confidently. `{7, 2, 11}` with target 9 moves `right` first and misses the pair entirely.
If the input is unsorted you have two choices: sort it first ($O(n \log n)$, but the original indices are lost unless you track them), or use a hash map ($O(n)$ time, $O(n)$ space — Chapter 14). Which is right depends on whether the problem wants indices or values, and whether extra space is allowed. State that trade-off out loud; it is half of what the question is testing.

Palindrome check

The same shape, with equality instead of a sum.

Is this a palindrome?

```cpp
#include <iostream>
#include <string>
```

```cpp
#include <vector>

bool isPalindrome(const std::string& s) {
    int left = 0, right = (int)s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) return false;
        ++left;
        --right;
    }
    return true;
}

int main() {
    std::cout << isPalindrome("racecar") << "\n";   // odd length
    std::cout << isPalindrome("abba")    << "\n";   // even length
    std::cout << isPalindrome("abc")     << "\n";
    std::cout << isPalindrome("")        << "\n";   // vacuously true
    std::cout << isPalindrome("x")       << "\n";
    return 0;
}
```

```
1
1
0
1
1
```

COMMON PITFALL | `while (left <= right)` **here is a real bug, not a harmless one**
In Chapter 6's reversal, `<=` merely swapped the middle element with itself — wasteful but harmless.
Here it is different in kind. With `<=`, an odd-length string ends by comparing `s[mid]` with `s[mid]`, which is always equal, so the function still returns the right answer — and you conclude the bug is harmless. But change the comparison to a *case-insensitive* or *skip-punctuation* variant, as the real LeetCode problem does, and the middle character gets processed twice, which can throw off an index you are also advancing.
The habit to build: `left < right` means *"there are still at least two distinct positions to compare"*, which is exactly the condition you want. Use it and the question never arises.

Container with most water

A harder converging problem, included because the greedy argument is the same one, one level deeper.

DEFINITION | **The problem**
Each element is the height of a vertical line. Choosing two lines, the water held between them is $\min(h_i, h_j) \times (j - i)$ — the shorter line sets the depth, and the distance sets the width. Find the maximum.

the SHORTER wall caps the depth — moving the taller one in can only lose width

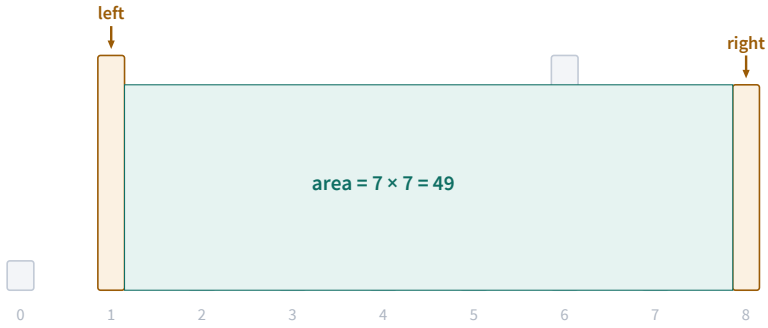


Figure 9.3 Container with most water. Starting at the widest possible pair, every later pair is narrower — so the only way to do better is a taller shortest line, which means moving the shorter wall inwards.

KEY INSIGHT | Why you always move the shorter wall

Start with the widest pair: `left = 0`, `right = n-1`. Any other pair is narrower, so to beat the current area a future pair must be **taller**.

The area is limited by the shorter wall. If you move the **taller** wall inwards, the width shrinks and the height is still capped by the same short wall, so the area cannot improve — that move is pure loss.

Moving the **shorter** wall inwards also shrinks the width, but it is the only move that can possibly raise the cap. So it is the only move worth making, and every pair involving the discarded short wall is correctly eliminated.

Container with most water, O(n)

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

int maxArea(const std::vector<int>& h) {
    int left = 0, right = (int)h.size() - 1, best = 0;
    while (left < right) {
        int width = right - left;
        int depth = std::min(h[left], h[right]);
        best = std::max(best, width * depth);
        if (h[left] < h[right]) ++left;    // move the SHORTER wall
        else                    --right;
    }
    return best;
}

int main() {
    std::cout << maxArea({1, 8, 6, 2, 5, 4, 8, 3, 7}) << "\n";
    std::cout << maxArea({1, 1})                      << "\n";
    std::cout << maxArea({4, 3, 2, 1, 4})             << "\n";
    return 0;
}
```

```
49
```

```
1
16
```

Brute force is $O(n^2)$; this is $O(n)$ time and $O(1)$ space.

Shape 2: Same-direction pointers

Here both pointers start on the left. One runs ahead reading; the other lags behind writing. You have already used this in Chapter 6 — now we name it.

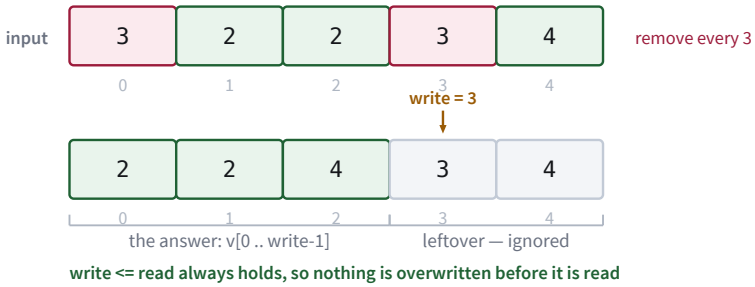


Figure 9.4 The read/write pair. `read` visits every element; `write` only advances when something is kept. The gap between them is exactly the number of elements discarded so far.

> **DEFINITION | The read/write (slow/fast) shape**
> `read` scans every element. For each one, decide **keep or discard**. On keep, copy it to `write` and advance `write`. On discard, do nothing.
> At the end, `v[0 .. write-1]` is the filtered array, and `write` is its length. Everything from `write` onwards is leftover rubbish you simply ignore.

Remove all occurrences of a value

Remove every occurrence of val, in place

```cpp
#include <iostream>
#include <vector>

// Returns the new length k; v[0..k-1] holds the kept elements.
int removeElement(std::vector<int>& v, int val) {
    int write = 0;
    for (int read = 0; read < (int)v.size(); ++read)
        if (v[read] != val)
            v[write++] = v[read];
    return write;
}

int main() {
    std::vector<int> v = {3, 2, 2, 3, 4};
    int k = removeElement(v, 3);
    std::cout << "k = " << k << " -> ";
```

```
    for (int i = 0; i < k; ++i) std::cout << v[i] << " ";
    std::cout << "\n";

    std::vector<int> all = {7, 7, 7};
    std::cout << "all removed -> " << removeElement(all, 7) << "\n";

    std::vector<int> none = {1, 2, 3};
    std::cout << "none removed -> " << removeElement(none, 9) << "\n";
    return 0;
}
```

```
k = 3 -> 2 2 4
all removed -> 0
none removed -> 3
```

$O(n)$ time, $O(1)$ space, one pass. Compare with the `erase` -in-a-loop version of Chapter 5, which is $O(n^2)$ for the same job.

> **KEY INSIGHT | Why `write` never overtakes `read`**
> `write` advances only when `read` advances, and `read` advances every iteration. So `write <= read` always holds, which means `v[write] = v[read]` never writes to a position `read` has not already passed.
> That is the safety proof for every algorithm of this shape, and it is worth being able to state. It is also why the pattern needs no extra array: the space freed by the discarded elements is exactly the space the kept ones move into.

Partition: evens before odds

Move all even numbers before all odd ones

```
#include <iostream>
#include <vector>

void partitionEvenOdd(std::vector<int>& v) {
    int write = 0;
    for (int read = 0; read < (int)v.size(); ++read)
        if (v[read] % 2 == 0)
            std::swap(v[write++], v[read]);
}

int main() {
    std::vector<int> v = {3, 1, 2, 4, 7, 6};
    partitionEvenOdd(v);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
2 4 6 1 7 3
```

Note that `swap` is used rather than assignment, because nothing may be thrown away — the odd numbers still have to end up somewhere. The evens keep their relative order;

the odds do not. If you need **both** groups to keep their order, $O(1)$ space is no longer possible in one pass, and you need a second array ($O(n)$ space) or a stable in-place algorithm that is $O(n \log n)$. Knowing that limit exists is worth as much as knowing the algorithm.

Why two moving pointers is still O(n)

A reasonable worry: there are two indices, so is this not $O(n^2)$?

No, and the reason is worth stating precisely.

> **KEY INSIGHT | The counting argument**
> In the **converging** shape, every iteration either increments `left` or decrements `right`. The gap between them starts at $n-1$ and shrinks by at least 1 each time, so there can be at most $n-1$ iterations.
> In the **same-direction** shape, `read` increments every iteration and never goes back, so there are exactly n iterations.
> In both, the total number of index movements is bounded by a constant times n. Two pointers are only $O(n^2)$ if one of them **restarts** — which is precisely what the brute-force nested loop does.
> *"Does any pointer ever go backwards or restart?"* If no, it is linear.

Cost summary

Problem	Brute force	Two pointers	Precondition
Two Sum (sorted)	$O(n^2)$	$O(n)$	sorted
Palindrome	$O(n)$	$O(n)$, $O(1)$ space	none
Reverse in place	—	$O(n)$, $O(1)$ space	none
Container with most water	$O(n^2)$	$O(n)$	none
Remove element	$O(n^2)$ with `erase`	$O(n)$	none
Remove duplicates (sorted)	$O(n^2)$	$O(n)$	sorted
Partition even/odd	$O(n)$ with a copy	$O(n)$, $O(1)$ space	none

CHAPTER RECAP | Chapter 9 in six lines

- **Converging** pointers start at both ends and move inwards; **same-direction** pointers both start left, one reading and one writing.
- Converging works because each comparison eliminates an entire family of pairs — be ready to explain *which* family.
- Converging pair-sum problems need **sorted** input; without it the answers are silently wrong.
- `write <= read` always holds, which is why in-place filtering is safe.
- Use `left < right`, not `left <= right`, so the loop means "two distinct positions remain".
- Two pointers are $O(n)$ because neither ever restarts. A restart is what makes a nested loop quadratic.

EXERCISES | Practise

1. Trace `twoSumSorted({1, 3, 4, 6, 8, 11}, 10)` by hand. How many iterations? How many pairs did the brute force avoid?
2. Modify Two Sum to return **all** distinct pairs summing to the target, not just the first. How do you skip duplicates without using extra space?
3. Write `isPalindrome` for a string, ignoring case and non-letters — `"A man, a plan, a canal: Panama"` is a palindrome. Which pointer moves, and when?
4. Three Sum: find all distinct triples summing to zero. (Hint: sort, fix one element, and run Two Sum on the rest. What is the complexity, and why is it not $O(n^3)$?)
5. For "container with most water", construct an input where moving the taller wall would find a better answer — or explain why no such input exists.
6. Squares of a sorted array: given `{-4, -1, 0, 3, 10}`, return the sorted squares. Sorting afterwards is $O(n \log n)$; two pointers make it $O(n)$. Where do the two pointers start, and why does the output get filled from the **back**?
7. Reverse only the vowels of a string, leaving everything else where it is.
8. Explain, in two sentences, why the converging shape would be useless on an unsorted array, using the elimination argument rather than an example.

CHAPTER

10

Pattern 3: Sliding Window

A sub-array that grows and shrinks instead of being rebuilt

CORE PREREQS: Chapter 9

Whenever a problem asks about contiguous sub-arrays, the brute force rebuilds every one of them from scratch — and almost all of that work is re-reading elements it has already read. A sliding window keeps one sub-array and edits it at the edges, which turns $O(n^2)$ or $O(n^3)$ into $O(n)$. This chapter covers both kinds, fixed-size and variable-size, and the single question that tells you which one you need.

WHAT YOU WILL LEARN

- Recognise a sliding-window problem from the word “contiguous” or “sub-array”
- Implement a fixed-size window with the add-one / remove-one update
- Implement a variable-size window with the grow-while-valid, shrink-while-invalid loop
- Explain why the variable window is $O(n)$ despite having a loop inside a loop
- Solve: maximum sum of k elements, longest sub-array with sum ≤ target,
- longest sub-string without repeats, minimum window that reaches a sum
- Say precisely when the pattern breaks, and what to use instead

The waste the pattern removes

Take a concrete problem.

DEFINITION | Maximum sum of any k consecutive elements
Given `{2, 1, 5, 1, 3, 2}` and k=3, the sums of the four possible windows are 2+1+5=8, 1+5+1=7, 5+1+3=9, 1+3+2=6. The answer is 9.

The obvious solution computes each window’s sum from scratch:

Brute force: recompute every window

```
int best = INT_MIN;
for (int i = 0; i + k <= n; ++i) {
    int sum = 0;
    for (int j = i; j < i + k; ++j) sum += v[j];   // k additions, every
        time
```

```
    best = std::max(best, sum);
}
```

That is $O(n \cdot k)$. Now look at what it is doing.

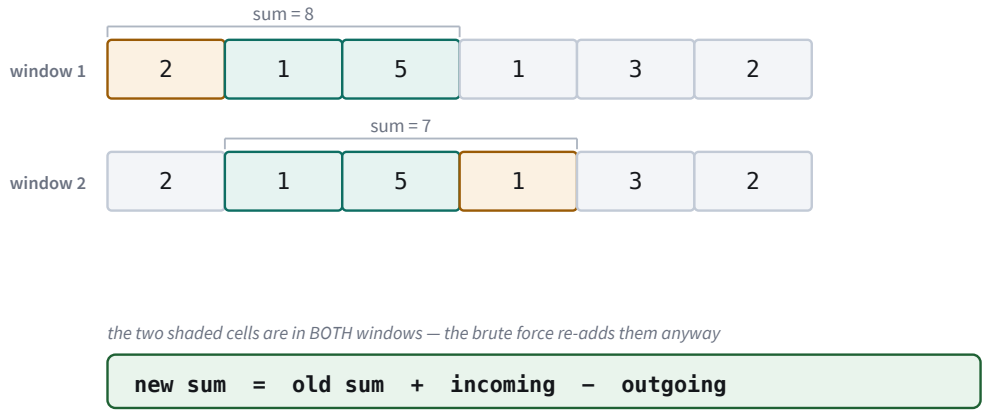


Figure 10.1 Consecutive windows overlap almost entirely. Going from one window to the next, $k-2$ of the k elements are unchanged — yet the brute force re-adds all k of them. The sliding window edits only the two ends.

KEY INSIGHT | The whole pattern in one sentence

Two neighbouring windows share all but **two** elements, so the second sum can be obtained from the first with **one addition and one subtraction** instead of k additions.

$$\text{sum}_{\text{new}} = \text{sum}_{\text{old}} + v[\text{incoming}] - v[\text{outgoing}]$$

Nothing else about the pattern is hard. Everything else is bookkeeping about *when* to move each edge.

Kind 1: Fixed-size window

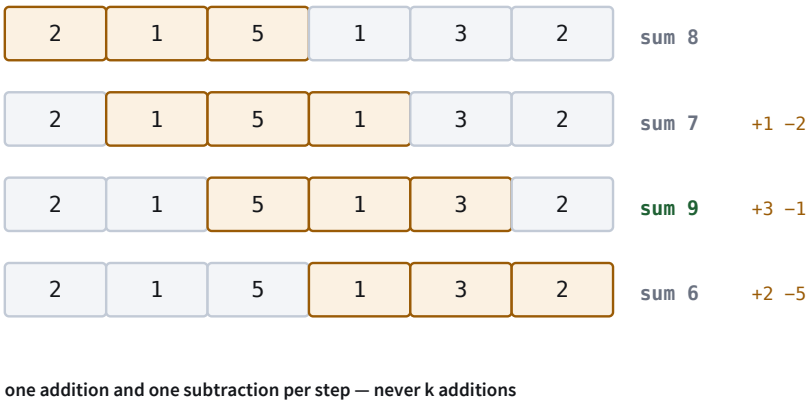


Figure 10.2 A fixed window of size 3 moving across the array. Each step adds the element entering on the right and subtracts the one leaving on the left — two operations, not k.

The trace

`{2, 1, 5, 1, 3, 2}`, k=3.

Step	Window	Enters	Leaves	sum	best
build	2 1 5	—	—	8	8
1	1 5 1	1	2	8+1−2=7	8
2	5 1 3	3	1	7+3−1=9	9
3	1 3 2	2	5	9+2−5=6	9

Table 10.1 Each step is one addition and one subtraction, regardless of how large k is.

Maximum sum of k consecutive elements, O(n)

```cpp
#include <algorithm>
#include <climits>
#include <iostream>
#include <vector>

long long maxSumOfK(const std::vector<int>& v, int k) {
    int n = (int)v.size();
    if (k <= 0 || k > n) return 0;          // no valid window exists

    long long sum = 0;
    for (int i = 0; i < k; ++i) sum += v[i];   // build the first window
    long long best = sum;

    for (int i = k; i < n; ++i) {
        sum += v[i];                            // the element entering,
            on the right
        sum -= v[i - k];                        // the element leaving, on
            the left
        best = std::max(best, sum);
    }
    return best;
}

int main() {
    std::cout << maxSumOfK({2, 1, 5, 1, 3, 2}, 3) << "\n";
    std::cout << maxSumOfK({-3, -1, -4, -1}, 2)   << "\n";
    std::cout << maxSumOfK({5}, 1)                << "\n";
    std::cout << maxSumOfK({1, 2}, 5)             << "\n";
    return 0;
}
```

```
9
-4
5
0
```

$O(n)$ time, $O(1)$ space, down from $O(n \cdot k)$.

COMMON PITFALL | The index of the outgoing element
When `i` is the index entering the window, the element leaving is at `i - k` — **not** `i - k + 1`, and not `i - k - 1`.
Derive it rather than memorising it: after adding `v[i]`, the window covers `i-k+1 .. i`, which is *k* elements. The element that was in the previous window but is not in this one is the one just before the new left edge, namely `i - k`. Deriving it takes five seconds and you will never get it wrong again.

Kind 2: Variable-size window

Fixed windows are the easy half. The interesting problems let the window **choose** its own size.

DEFINITION | The shape of a variable window
Both edges start at 0. Then, for each position of the right edge:

1. **Grow:** add `v[right]` to the window.
2. **Shrink:** while the window is *invalid*, remove `v[left]` and advance `left`.
3. **Record:** the window is now valid, so update the answer.

The condition that defines "invalid" is the whole problem. Everything else is the same three lines every time.

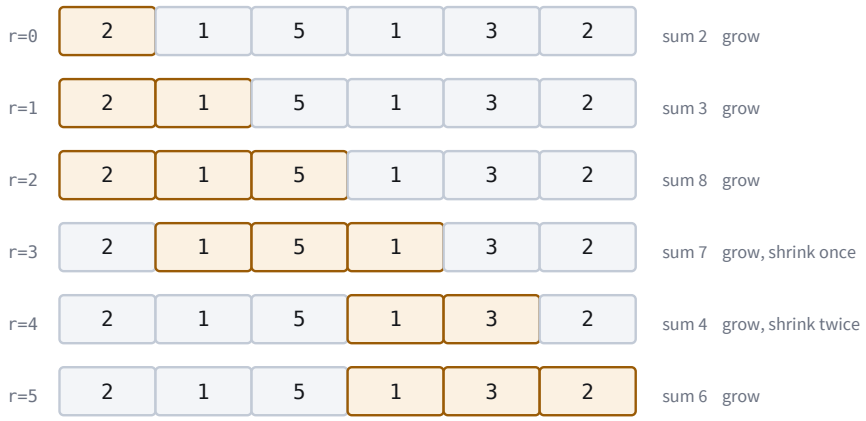


Figure 10.3 The grow–shrink loop. The right edge advances once per iteration; the left edge chases it, moving only as far as it must to restore validity. Neither edge ever moves backwards.

Problem: longest sub-array with sum at most a target

Longest contiguous sub-array with sum <= target

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

// All values must be non-negative - see the pitfall below.
int longestAtMost(const std::vector<int>& v, long long target) {
    long long sum = 0;
```

```
    int left = 0, best = 0;
    for (int right = 0; right < (int)v.size(); ++right) {
        sum += v[right];                        // 1. grow
        while (sum > target && left <= right) {
            sum -= v[left];                     // 2. shrink while invalid
            ++left;
        }
        best = std::max(best, right - left + 1);  // 3. record
    }
    return best;
}

int main() {
    std::cout << longestAtMost({2, 1, 5, 1, 3, 2}, 8) << "\n";
    std::cout << longestAtMost({4, 4, 4}, 3)          << "\n";
    std::cout << longestAtMost({1, 1, 1}, 100)        << "\n";
    return 0;
}
```

```
3
0
3
```

The trace

`{2, 1, 5, 1, 3, 2}`, target 8. The crucial habit: record `sum` and `left` after **every individual edge movement**, not once per iteration. The state carries across iterations, and that is where careless traces go wrong.

right	enters	sum	shrink steps	left	window	length	best
0	2	2	none	0	`2`	1	1
1	1	3	none	0	`2 1`	2	2
2	5	8	none — 8 is not >8	0	`2 1 5`	3	**3**
3	1	9→7	drop `2`	1	`1 5 1`	3	3
4	3	10→9→4	drop `1`, then `5`	3	`1 3`	2	3
5	2	6	none	3	`1 3 2`	3	3

Table 10.2 Each row is one value of `right`. The arrows inside a `sum` cell show the shrinking steps, one per removed element. Note that `left` moves twice in a single iteration at `right = 4`, and not at all in three of the six.

The answer is 3, which matches the program. Three rows of that table repay study:

Row 2. The sum is exactly 8 and the condition is `sum > target`, so nothing shrinks. If the problem had said "strictly less than", this row would shrink and the answer would change. The comparison operator *is* the specification.

Row 4. `left` moves twice in one iteration. This is why the inner loop is a `while` and not an `if` — a single element entering can invalidate the window by more than one element's worth.

Rows 0, 1 and 5. `left` does not move at all. Over the whole run `left` advanced 3 times and `right` advanced 6 times: nine movements for six elements, which is the linear behaviour the next section proves.

Why the variable window is O(n)

There is a `while` loop inside a `for` loop. That *looks* quadratic.

> **KEY INSIGHT | The amortised argument**
> Count how many times `++left` can ever execute, across the whole run.
> `left` starts at 0, only increases, and never exceeds n. So **over the entire algorithm**, the inner `while` body runs at most n times in total — not n times per iteration.
> The outer loop runs n times. Total work: at most $2n$ steps. $O(n)$.
> This is the same accounting as amortised analysis in Chapter 5: a particular iteration may be expensive, but the total across all iterations is bounded. Being able to give this argument out loud is the difference between "I think it's linear" and a convincing answer.

Problem: longest sub-string without repeating characters

The canonical variable-window problem, and the one most often asked.

> **DEFINITION | The problem**
> Find the length of the longest sub-string with no repeated character. `"abcabcbb"` → 3, from `"abc"`. `"bbbbb"` → 1. `"pwwkew"` → 3, from `"wke"`.

The window must stay free of duplicates. We need to know, in $O(1)$, whether the incoming character is already inside — so we carry a count per character.

Longest substring without repeating characters

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int longestUnique(const std::string& s) {
    std::vector<int> count(256, 0);            // how many of each char
        in window
    int left = 0, best = 0;
    for (int right = 0; right < (int)s.size(); ++right) {
        ++count[(unsigned char)s[right]];      // 1. grow
        while (count[(unsigned char)s[right]] > 1) {
            --count[(unsigned char)s[left]];   // 2. shrink until no
                duplicate
            ++left;
        }
        best = std::max(best, right - left + 1);  // 3. record
    }
    return best;
}

```

```cpp
int main() {
    std::cout << longestUnique("abcabcbb") << "\n";
    std::cout << longestUnique("bbbbb")    << "\n";
    std::cout << longestUnique("pwwkew")   << "\n";
    std::cout << longestUnique("")         << "\n";
    return 0;
}
```

```
3
1
3
0
```

The shrink condition is worth reading twice: we shrink while **the character that just arrived** appears more than once. We do not need to search the window for any duplicate, because the window was duplicate-free before `s[right]` arrived — so the only possible duplicate involves `s[right]` . Maintaining an invariant like that, and then exploiting it, is the heart of window problems.

$O(n)$ time — each character enters once and leaves at most once. $O(1)$ space, because the count array is a fixed 256 entries regardless of the input length.

When the pattern does not apply

> **WATCH OUT | The non-negative requirement**
> `longestAtMost` relies on an assumption that is easy to miss: **growing the window never decreases the sum, and shrinking never increases it.** That is only true when every value is non-negative.
> With negatives, `{5, -3, 4}` and target 5: the window `5` is valid, `5 -3` sums to 2 and is also valid, and `5 -3 4` sums to 6 and is not — but shrinking from the left gives `-3 4` = 1, which *is* valid and longer than you might have kept. The monotonic relationship between window size and sum is gone, so the shrink rule can no longer be trusted.
> For sub-array-sum problems with negative numbers, use **prefix sums plus a hash map** — which is exactly Chapter 11.

Checklist for applying the pattern at all:

Question	If no
Must the answer be **contiguous**?	not a window problem — think sorting or hashing
Does extending the window change the quantity **monotonically**?	prefix sums (Chapter 11)
Can you update the window state in $O(1)$ per edge move?	the window will not be $O(n)$

Cost summary

Problem	Brute force	Window	Space
Max sum of k elements	$O(n \cdot k)$	$O(n)$	$O(1)$
Longest sub-array, sum ≤ target	$O(n^2)$	$O(n)$	$O(1)$
Longest sub-string, no repeats	$O(n^2)$ or worse	$O(n)$	$O(1)$
Smallest window with sum ≥ target	$O(n^2)$	$O(n)$	$O(1)$
Max consecutive ones with one flip	$O(n^2)$	$O(n)$	$O(1)$

CHAPTER RECAP | Chapter 10 in six lines

- Neighbouring windows differ by two elements, so update at the edges instead of rebuilding.
- **Fixed size:** add `v[i]`, subtract `v[i-k]`, record. One addition, one subtraction.
- **Variable size:** grow by one, shrink while invalid, record. The "invalid" condition is the entire problem.
- The inner `while` does not make it quadratic: `left` advances at most n times in total across the whole run.
- Keep the window state updatable in $O(1)$ — a running sum, a count array, a hash map.
- Non-negative values are a hidden precondition of sum-based windows. With negatives, use prefix sums.

EXERCISES | Practise

1. Re-run the trace above with target 9 instead of 8. Which row changes first, and what is the new answer?
2. Find the **minimum** length of a contiguous sub-array whose sum is at least a target. Which part of the three-line shape changes, and which stays the same?
3. Find the average of every window of size k and return the largest. Why must you avoid computing the average inside the loop?
4. Maximum consecutive ones if you may flip at most **one** zero. (Hint: the window is invalid when it contains two zeroes. What is the window state?)
5. Generalise exercise 4 to at most k flips. How much of your code changes?
6. Longest sub-string with at most two distinct characters. What is the window state, and what is the shrink condition?
7. Explain, with a concrete three-element example, why `longestAtMost` is wrong when the array may contain negative numbers.
8. Permutation in a string: given `s1` and `s2`, does `s2` contain a permutation of `s1` as a sub-string? (Hint: a fixed window of size `s1.size()`, and a cheap way to compare two character-count arrays.)

CHAPTER

11

Pattern 4: Prefix Sums

Pay O(n) once so every range question afterwards costs O(1)

CORE PREREQS: Chapters 3 and 10

A prefix sum array stores the running total up to each position. From it, the sum of any range comes out as a single subtraction — which turns a stream of range queries from $O(n)$ each into $O(1)$ each, and turns the sub-array-sum problems that sliding windows cannot handle into one-pass hash-map problems. This is the clearest time-for-space trade in the book.

WHAT YOU WILL LEARN

- Build a prefix sum array and explain the off-by-one that the extra leading zero removes
- Answer any range-sum query in $O(1)$
- Explain when prefix sums beat a sliding window, and why
- Solve "sub-array sum equals K" in $O(n)$ with a hash map
- Use the same idea for counts, not just sums
- Extend prefix sums to two dimensions

The problem it solves

You are given an array and then asked, many times: *what is the sum of the elements from index i to index j?*

The obvious answer loops from i to j: $O(j-i)$ per query, so $O(n)$ in the worst case. With q queries that is $O(n \cdot q)$. For $n=10^5$ and $q=10^5$ that is 10^{10} operations — far too slow.

KEY INSIGHT | The trade

Spend $O(n)$ time and $O(n)$ space **once**, up front. Every query afterwards costs $O(1)$.
Total: $O(n+q)$ instead of $O(n \cdot q)$. For the numbers above, 2×10^5 instead of 10^{10} — fifty thousand times less work.
This shape — *precompute once, answer cheaply forever* — is one of the most valuable ideas in algorithms, and prefix sums are its simplest instance.

Building the array

DEFINITION | Prefix sum
`prefix[i]` is the sum of the first i elements, that is `v[0] + v[1] + ... + v[i-1]`. Note that `prefix` has $n+1$ entries, and `prefix[0] = 0` — the sum of no elements. That leading zero looks like a detail. It is the difference between code with an edge case and code without one.

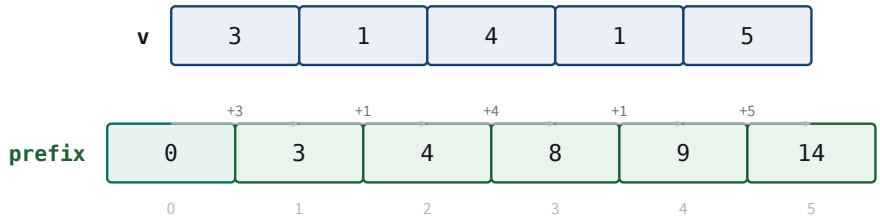


Figure 11.1 Building the prefix array. Each entry is the previous entry plus one element, so the whole array is built in one pass. The leading zero means `prefix[i]` can always be read as "everything strictly before index i".

Building prefix sums

```cpp
#include <iostream>
#include <vector>

std::vector<long long> buildPrefix(const std::vector<int>& v) {
    std::vector<long long> prefix(v.size() + 1, 0);   // n+1 entries,
        p[0] = 0
    for (int i = 0; i < (int)v.size(); ++i)
        prefix[i + 1] = prefix[i] + v[i];
    return prefix;
}

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    auto p = buildPrefix(v);
    for (long long x : p) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
0 3 4 8 9 14
```

Read the output against the input: `0`, then 3, then 3+1=4, then 3+1+4=8, then 9, then 14. Each entry is the total of everything before that point.

The range-sum formula



sum(1, 3) = prefix[4] − prefix[1] = 9 − 3 = 6

that is v[1] + v[2] + v[3] = 1 + 4 + 1 — the shared part cancels exactly

Figure 11.2 The range-sum subtraction. The sum of `v[i..j]` is everything up to and including *j*, minus everything strictly before *i*. The overlapping part cancels exactly.

DEFINITION | Range sum in O(1)

$$\text{sum}(i,j) = \text{prefix}[j+1] - \text{prefix}[i]$$

for the inclusive range `v[i..j]`.

Why it works: `prefix[j+1]` is everything up to and including index *j*. `prefix[i]` is everything strictly before index *i*. Subtracting removes precisely the part you did not ask for.

Range sum queries in O(1) each

```cpp
#include <iostream>
#include <vector>

class RangeSum {
public:
    explicit RangeSum(const std::vector<int>& v) : prefix(v.size() + 1,
        0) {
        for (int i = 0; i < (int)v.size(); ++i)
            prefix[i + 1] = prefix[i] + v[i];
    }
    // Inclusive sum of v[i..j]. O(1).
    long long query(int i, int j) const { return prefix[j + 1] -
        prefix[i]; }

private:
    std::vector<long long> prefix;
};

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    RangeSum rs(v);
    std::cout << rs.query(1, 3) << "\n";   // 1 + 4 + 1
    std::cout << rs.query(0, 4) << "\n";   // everything
    std::cout << rs.query(2, 2) << "\n";   // a single element
    std::cout << rs.query(0, 0) << "\n";
    return 0;
```

```
25  }
```

```
6
14
4
3
```

Check `query(2, 2)` : `prefix[3] - prefix[2]` is $8-4=4$, which is `v[2]` . A single-element range works with no special case, and that is the leading zero earning its place.

> **COMMON PITFALL** | **The version without the leading zero**
> A very common alternative defines `prefix[i]` as the sum **up to and including** `v[i]` , with `prefix[0] = v[0]` . The range formula then becomes
>
> $$\text{sum}(i,j)=\text{prefix}[j]-\text{prefix}[i-1]$$
>
> which is wrong when $i=0$, because `prefix[-1]` does not exist. You must write
>
> ```
> long long sum = (i == 0) ? prefix[j] : prefix[j] - prefix[i - 1];
> ```
>
> Both conventions are correct. The $n+1$ version needs no `if` , and an `if` you never write is an `if` you cannot get wrong. Use the leading zero.

Where prefix sums beat sliding windows

Chapter 10 ended with a warning: sliding windows break when the array contains negative numbers, because growing the window no longer increases the sum monotonically.

Prefix sums do not care. They make no assumption about signs at all, because they never rely on growing or shrinking anything.

Sub-array sum equals K

> **DEFINITION** | **The problem**
> Count the contiguous sub-arrays whose sum is exactly K. Values may be negative. `{1, 2, 3}` with $K=3 \to 2$ (the sub-arrays `{1,2}` and `{3}`). `{1, -1, 0}` with $K=0 \to 3$ (the sub-arrays `{1,-1}` , `{1,-1,0}` and `{0}`).

From brute force to the insight

Brute force tries every (i,j) pair: $O(n^2)$, and with a prefix array each sum is $O(1)$, so $O(n^2)$ overall. Better than $O(n^3)$, still too slow.

Now the key rearrangement:

KEY INSIGHT | The rearrangement that makes it linear
A sub-array ending at index j with sum K means

$$\text{prefix}[j+1] - \text{prefix}[i] = K$$

Rearrange for the thing you are looking for:

$$\text{prefix}[i] = \text{prefix}[j+1] - K$$

So as you sweep j from left to right, the question *"how many sub-arrays ending here sum to K?"* becomes *"how many earlier prefix values equal `current - K`?"*
That is a **lookup**, and a hash map answers it in $O(1)$. One pass, counting prefix values as you go, and the whole problem is $O(n)$.

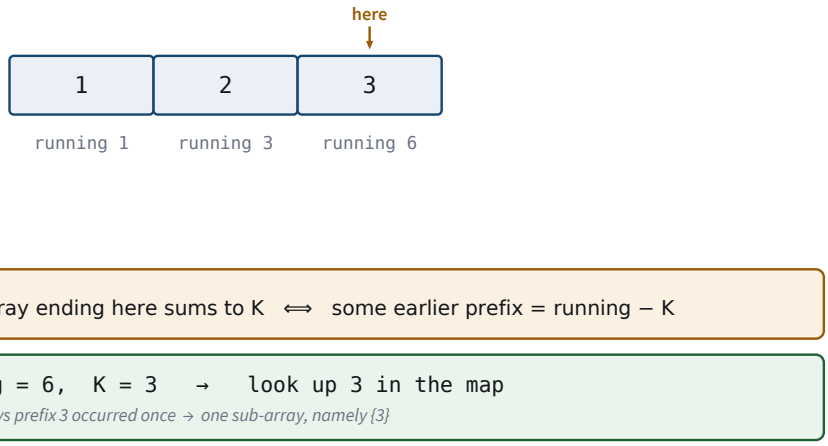


Figure 11.3 Sub-array sum equals K. At each position, the current running total is known; subtracting K gives the earlier running total we would need to have seen. The hash map says how many times that value has occurred, and each occurrence is one qualifying sub-array.

The trace

`{1, 2, 3}`, K=3. `seen` maps a prefix value to how many times it has occurred, and starts as `{0: 1}` — the empty prefix, which must be counted or sub-arrays starting at index 0 are missed.

`j`	`v[j]`	`running`	need `running - K`	found	`count`	`seen` afterwards
start	—	0	—	—	0	`{0:1}`
0	1	1	1−3=−2	0	0	`{0:1, 1:1}`
1	2	3	3−3=0	1	1	`{0:1, 1:1, 3:1}`
2	3	6	6−3=3	1	2	`{0:1, 1:1, 3:1, 6:1}`

Table 11.1 Tracing sub-array sum equals K. The map is consulted *before* the current prefix is inserted, so a sub-array can never be matched against itself.

Sub-array sum equals K, O(n), works with negatives

```cpp
#include <iostream>
#include <unordered_map>
#include <vector>

```

```cpp
int subarraySum(const std::vector<int>& v, int k) {
    std::unordered_map<long long, int> seen;
    seen[0] = 1;                        // the empty prefix, before any
        element
    long long running = 0;
    int count = 0;
    for (int x : v) {
        running += x;
        auto it = seen.find(running - k);
        if (it != seen.end()) count += it->second;   // look up BEFORE
            inserting
        ++seen[running];
    }
    return count;
}

int main() {
    std::cout << subarraySum({1, 2, 3}, 3)       << "\n";
    std::cout << subarraySum({1, 1, 1}, 2)       << "\n";
    std::cout << subarraySum({1, -1, 0}, 0)      << "\n";
    std::cout << subarraySum({3, 4, 7, 2}, 7)    << "\n";
    std::cout << subarraySum({1, 2, 3}, 100)     << "\n";
    return 0;
}
```

```
2
2
3
2
0
```

$O(n)$ time, $O(n)$ space. A sliding window cannot solve this at all once negatives are allowed.

COMMON PITFALL | The two bugs that make this problem hard

Forgetting `seen[0] = 1`. Without it, any sub-array that starts at index 0 is missed, because the prefix it needs to subtract is the empty one. `{3}` with K=3 would return 0.

Inserting before looking up. If you write `++seen[running];` first and then look up `running - k`, then when K=0 the current prefix matches itself and every position contributes a spurious count. Look up first, insert second.

Prefix counts, not just sums

The idea generalises to anything you can accumulate.

Prefix counts: how many even numbers in a range?

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {4, 7, 2, 9, 6, 1};
    std::vector<int> evens(v.size() + 1, 0);
```

```
    for (int i = 0; i < (int)v.size(); ++i)
        evens[i + 1] = evens[i] + (v[i] % 2 == 0 ? 1 : 0);

    // evens in v[1..4] = 7, 2, 9, 6  ->  2 of them
    std::cout << evens[5] - evens[1] << "\n";
    // evens in the whole array
    std::cout << evens[6] - evens[0] << "\n";
    return 0;
}
```

```
2
3
```

The same subtraction works for *any* quantity that adds up across a range: counts of a particular value, counts matching a predicate, XOR totals (where subtraction becomes XOR again), and products if there are no zeroes.

Two-dimensional prefix sums

For grids, the same idea gives the sum of any rectangle in $O(1)$.

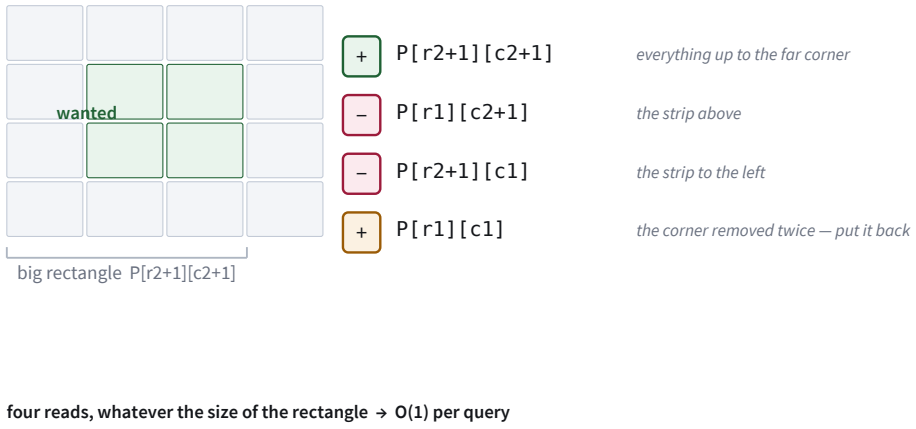


Figure 11.4 Inclusion–exclusion on a grid. Taking the big rectangle and removing the two overhanging strips subtracts their shared corner twice, so it has to be added back once. Four array reads, whatever the size of the rectangle.

DEFINITION | The 2-D range formula

With `P[r][c]` the sum of the rectangle from (0,0) to $(r-1, c-1)$, the sum of the rectangle with corners (r_1, c_1) and (r_2, c_2) inclusive is

$$P[r_2+1][c_2+1] - P[r_1][c_2+1] - P[r_2+1][c_1] + P[r_1][c_1]$$

The final + is **inclusion–exclusion**: the top-left region was removed twice, once by each of the two subtractions, so it must be restored once.

2-D prefix sums: any rectangle in O(1)

```
#include <iostream>
#include <vector>

int main() {
```

```
    std::vector<std::vector<int>> g = {{1, 2, 3},
                                       {4, 5, 6},
                                       {7, 8, 9}};
    int R = 3, C = 3;
    std::vector<std::vector<long long>> P(R + 1, std::vector<long long>(C
        + 1, 0));
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c)
            P[r + 1][c + 1] = g[r][c] + P[r][c + 1] + P[r + 1][c] -
                P[r][c];

    auto rect = [&](int r1, int c1, int r2, int c2) {
        return P[r2 + 1][c2 + 1] - P[r1][c2 + 1] - P[r2 + 1][c1] +
            P[r1][c1];
    };

    std::cout << rect(0, 0, 2, 2) << "\n";   // whole grid: 45
    std::cout << rect(1, 1, 2, 2) << "\n";   // 5+6+8+9
    std::cout << rect(0, 0, 0, 0) << "\n";   // just g[0][0]
    return 0;
}
```

```
45
28
1
```

Building `P` is $O(R \cdot C)$; each query is $O(1)$, with four reads regardless of the rectangle's size.

Choosing between the three range techniques

Situation	Technique	Time	Space
Many range-sum queries, array never changes	prefix sums	$O(n)$ build, $O(1)$ query	$O(n)$
Longest/shortest window, all values non-negative	sliding window	$O(n)$	$O(1)$
Count sub-arrays with an exact sum, values may be negative	prefix + hash map	$O(n)$	$O(n)$
Range sums **and** the array changes between queries	Fenwick / segment tree	$O(\log n)$ each	$O(n)$
Maximum sub-array sum	Kadane (Chapter 16)	$O(n)$	$O(1)$

Table 11.2 Which tool for which range problem. Read this table as a decision procedure when a problem mentions sub-arrays.

The fourth row is outside this book; it is mentioned so you know that prefix sums have a limit. **Prefix sums assume the array is frozen.** One update to `v[i]` invalidates every prefix entry from i onwards, so rebuilding is $O(n)$. If updates are frequent, you need a different structure.

CHAPTER RECAP | Chapter 11 in six lines

- `prefix[i]` is the sum of everything strictly before index i, and `prefix[0] = 0`.
- sum(i,j)=prefix$[j+1]$−prefix$[i]$ — one subtraction, $O(1)$.
- The leading zero removes the i=0 special case. Always use it.
- Prefix sums work with negative numbers; sliding windows do not.
- "Sub-array sums to K" becomes "has prefix value current−K been seen?" — a hash-map lookup, so $O(n)$.
- Seed the map with `{0: 1}`, and look up before you insert.

EXERCISES | Practise

1. Build the prefix array for `{5, -2, 7, 1}` by hand. Then use it to find the sum of `v[1..3]`.
2. Why does `prefix` have n+1 entries rather than n? Give the specific query that breaks without the extra entry.
3. Find the **longest** sub-array with sum exactly K (not the count). What must the hash map store instead of a count, and why do you keep only the *earliest* occurrence of each prefix value?
4. Count the sub-arrays with an **even** sum. (Hint: you do not need the sums themselves, only their parity. How many distinct prefix values are there then?)
5. Given an array of 0s and 1s, find the longest sub-array with equal numbers of each. (Hint: map 0 to −1, then find the longest sub-array with sum 0.)
6. Explain why `subarraySum` must look up before inserting, using K=0 and the input `{0}` as your example.
7. Build a 2-D prefix array for a 3×3 grid of all 1s and use it to find the sum of the middle 2×2 square. Verify the inclusion–exclusion by hand.
8. An array changes between queries. Explain why prefix sums are the wrong tool, and state what the rebuild would cost.

CHAPTER

12

Pattern 5: Binary Search

Twenty steps to find one item among a million

CORE PREREQS: Chapters 3 and 4

Binary search is the highest-value algorithm in this book: four lines of code that turn a million comparisons into twenty. It is also the algorithm most people write almost correctly — the off-by-one that loops forever, the overflow that appears only on huge inputs, the variant that finds an answer rather than the first one. This chapter writes it once, proves why it terminates, and then derives the two variants you actually need in interviews.

WHAT YOU WILL LEARN

- Implement binary search correctly, and say why each line is as it is
- State the loop invariant and use it to prove the algorithm cannot miss an element
- Explain why `mid = lo + (hi - lo) / 2` and not `(lo + hi) / 2`
- Avoid the infinite loop and the off-by-one, and know which variant causes which
- Find the **first** and **last** occurrence of a repeated value
- Recognise "binary search on the answer", where the array is not what you search
- Use `std::lower_bound` and `std::upper_bound` correctly

The guessing game

I am thinking of a number from 1 to 1000. You may ask "is it higher or lower than x?" How many questions do you need?

If you guess 1, then 2, then 3, you may need 1000 questions. If you always guess the **middle**, each answer eliminates half the remaining possibilities, and you need at most 10 — because $2^{10}=1024>1000$.

DEFINITION | Binary search
Given a **sorted** array, repeatedly compare the target with the middle element of the region still under consideration, and discard the half that cannot contain it.
Each comparison halves the search space, so the number of comparisons is $\lceil \log_2 n \rceil$.

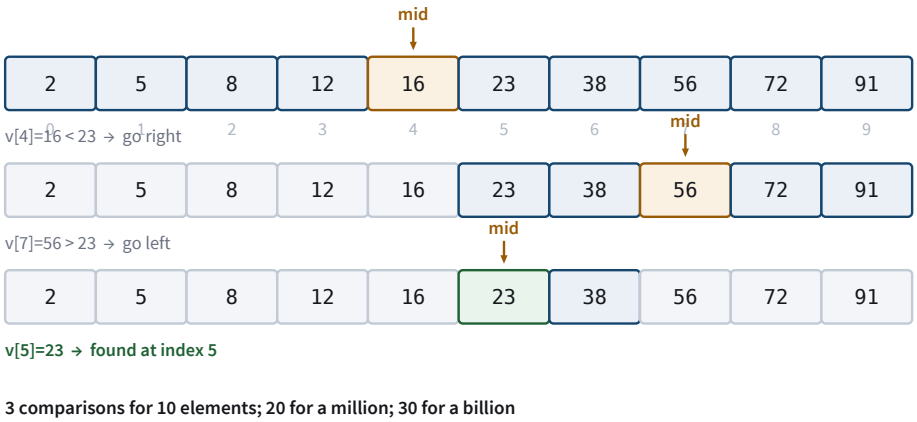


Figure 12.1 Searching for 23 in a sorted array of 10. Three comparisons settle it. On a million elements it would be twenty, and on a billion, thirty.

n	Linear search (worst)	Binary search (worst)
100	100	7
10,000	10,000	14
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

That last row is the one to remember. A billion elements, thirty comparisons.

The code, line by line

Binary search — the canonical form

```cpp
#include <iostream>
#include <vector>

// Returns an index of target, or -1 if it is absent. v must be sorted.
int binarySearch(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;      // inclusive bounds: [lo, hi]
    while (lo <= hi) {                       // while the region is non-empty
        int mid = lo + (hi - lo) / 2;        // midpoint, overflow-safe
        if (v[mid] == target) return mid;
        if (v[mid] < target) lo = mid + 1;   // target must be to the RIGHT
        else                 hi = mid - 1;   // target must be to the LEFT
    }
    return -1;
}

int main() {
    std::vector<int> v = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    std::cout << binarySearch(v, 23) << "\n";
    std::cout << binarySearch(v, 2)  << "\n";
    std::cout << binarySearch(v, 91) << "\n";
```

```
    std::cout << binarySearch(v, 50) << "\n";
    std::cout << binarySearch({}, 1) << "\n";
    return 0;
}
```

```
5
0
9
-1
-1
```

Now the five decisions in that function, each of which has a wrong alternative that *almost* works.

Decision 1: `hi = size() - 1`, and the bounds are inclusive

The region under consideration is `v[lo .. hi]` **including both ends**. That choice forces the other decisions, so make it consciously and stick to it. (The alternative — half-open `[lo, hi)` with `hi = size()` — is equally valid, and mixing the two is how people produce bugs.)

Decision 2: `while (lo <= hi)`

With inclusive bounds, `lo == hi` describes a region containing **one** element, which still has to be examined. Writing `while (lo < hi)` skips it, so searching for the only element in a one-element array returns −1.

Decision 3: `mid = lo + (hi - lo) / 2`

The natural `(lo + hi) / 2` is correct mathematics and wrong code.

> **COMMON PITFALL | The overflow that took nine years to find**
> If `lo` and `hi` are both near `INT_MAX`, `lo + hi` overflows. Signed overflow in C++ is undefined behaviour; in practice you get a large negative number, and `v[mid]` reads far outside the array.
> `lo + (hi - lo) / 2` is algebraically identical — $lo + hi - lo/2 = lo + hi/2$ — but `hi - lo` is at most the array length, so nothing can overflow.
> This bug sat undetected in the Java standard library's binary search, and in the binary search of the standard reference textbook, for about nine years. It only shows up on arrays of more than a billion elements, which nobody had in
>
> 1. Write it the safe way out of habit.

Decision 4: `mid + 1` and `mid - 1`, never plain `mid`

We have just established that `v[mid]` is not the target. Leaving `mid` inside the region means it may be examined again — and if the region never shrinks, the loop never ends.

COMMON PITFALL | The infinite loop

```
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (v[mid] == target) break;
    if (v[mid] < target) lo = mid;      // BUG: should be mid + 1
    else                 hi = mid;      // BUG: should be mid - 1
}
```

With `lo = 0, hi = 1`, `mid` is 0. If `v[0] < target` we set `lo = 0` — which is what it already was. Same `lo`, same `hi`, same `mid`, forever.

The test that catches it: run your binary search on a **two-element** array searching for the larger element. Almost every binary search bug shows up at n=2, and the test takes ten seconds to write.

Decision 5: integer division rounds down

`(hi - lo) / 2` truncates, so `mid` is the **left** of the two middles when the region has even length. That is not arbitrary: combined with `lo = mid + 1` it guarantees the region strictly shrinks every iteration, which is the termination proof. We will see in a moment that one variant needs the *other* rounding, and why.

Why it cannot miss an element

DEFINITION | The loop invariant

If the target is in the array at all, it is inside `v[lo .. hi]`.

This is true at the start, because `[0, n-1]` is the whole array. Each step preserves it: if `v[mid] < target`, then by sortedness every element at index ≤ *mid* is also < *target*, so the target cannot be there, and discarding `[lo, mid]` cannot discard it. The mirror argument covers the other branch.

When the loop ends, `lo > hi`, so the region is empty. The invariant then says the target is inside an empty region — which is only possible if it was never in the array. Hence `return -1` is correct.

An invariant plus a decreasing quantity is how you *prove* a loop, and it is worth saying out loud in an interview: *"the invariant is that the answer, if it exists, is in `[lo, hi]`; each iteration halves that range, so it terminates in* $\log n$ *steps."*

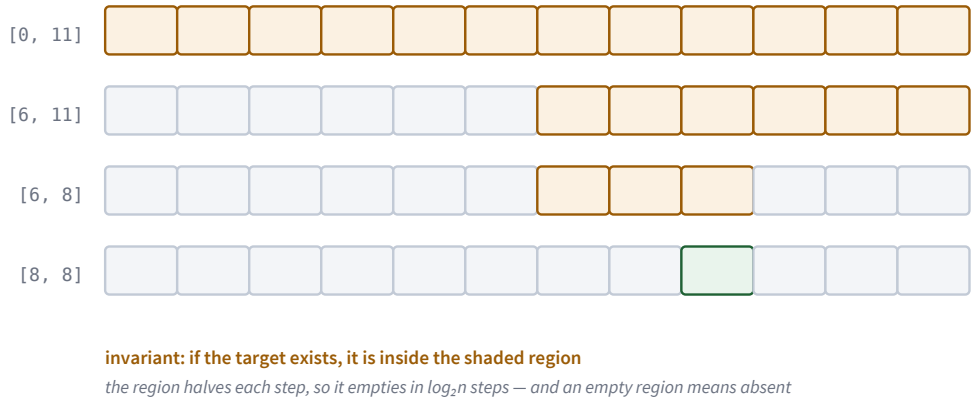


Figure 12.2 The invariant in pictures. The shaded region is everywhere the target could still be. It only ever shrinks, it never loses the target, and when it becomes empty the answer is settled.

Variant: first and last occurrence

Plain binary search returns *an* index of the target. With duplicates, which one it returns is unpredictable.

> **DEFINITION** | **The problem**
> In `{5, 7, 7, 7, 8, 8, 10}`, the value 7 occupies indices 1, 2 and 3. Find the **first** and the **last** index, or −1 if absent.

The fix is small but subtle: **when you find the target, do not stop.** Record it and keep searching in the half that could hold an earlier (or later) occurrence.

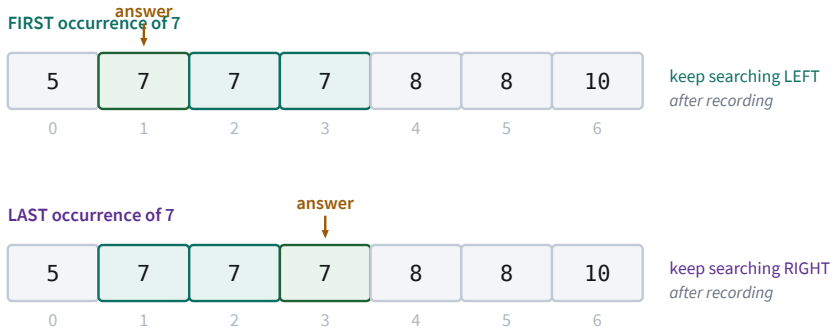


Figure 12.3 Finding the first and last occurrence. On a match, the first-occurrence search continues leftwards and the last-occurrence search continues rightwards. The recorded answer is overwritten only by a better one.

First and last occurrence of a value

```cpp
#include <iostream>
#include <vector>

int firstOccurrence(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1, answer = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
```

```
        if (v[mid] == target) {
            answer = mid;               // record it...
            hi = mid - 1;               // ...then keep looking LEFT
        } else if (v[mid] < target) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return answer;
}

int lastOccurrence(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1, answer = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == target) {
            answer = mid;               // record it...
            lo = mid + 1;               // ...then keep looking RIGHT
        } else if (v[mid] < target) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return answer;
}

int main() {
    std::vector<int> v = {5, 7, 7, 7, 8, 8, 10};
    std::cout << firstOccurrence(v, 7) << " " << lastOccurrence(v, 7) <<
        "\n";
    std::cout << firstOccurrence(v, 8) << " " << lastOccurrence(v, 8) <<
        "\n";
    std::cout << firstOccurrence(v, 5) << " " << lastOccurrence(v, 5) <<
        "\n";
    std::cout << firstOccurrence(v, 9) << " " << lastOccurrence(v, 9) <<
        "\n";
    // count of a value, in O(log n)
    int f = firstOccurrence(v, 7), l = lastOccurrence(v, 7);
    std::cout << "count of 7 = " << (f == -1 ? 0 : l - f + 1) << "\n";
    return 0;
}
```

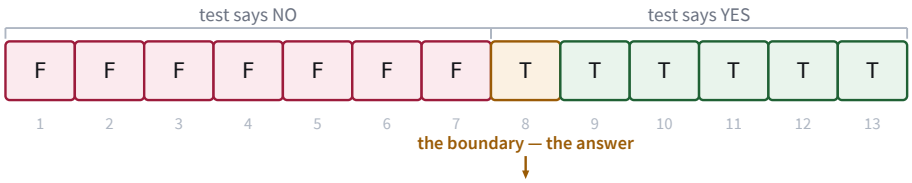
```
1 3
4 5
0 0
-1 -1
count of 7 = 3
```

That last line is the real pay-off. Counting occurrences by scanning is $O(n)$; with two binary searches it is $O(\log n)$. On a sorted array of ten million elements that is 46 comparisons instead of ten million.

Binary search on the answer

Here is the idea that turns binary search from a lookup into a general technique, and it is what separates a mid-level answer from a strong one.

> **KEY INSIGHT | You are not searching the array — you are searching a *decision***
> Binary search needs only one thing: a range of candidate answers, and a **yes/no test** whose answers are *monotonic* — false, false, false, true, true, true — so that once it turns true it stays true.
> The array itself need not exist. If you can ask *"is an answer of x good enough?"* and know that a larger x is never worse, you can binary search for the smallest good x in $O(\log(\text{range}) \times \text{cost of the test})$.



the candidates are every possible ANSWER, not the array
all that is required is that once the test turns true, it stays true

Figure 12.4 Binary searching a decision instead of an array. The candidates are all possible answers; the test splits them into a false region and a true region; binary search finds the boundary. The array is only used inside the test.

Integer square root by binary search on the answer

```cpp
#include <iostream>

// Largest x with x*x <= n. The test "x*x <= n" is false once and for all
// beyond the answer, which is exactly the monotonicity we need.
int integerSqrt(int n) {
    if (n < 2) return n;
    int lo = 1, hi = n / 2, answer = 1;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (mid * mid <= (long long)n) {
            answer = (int)mid;          // good enough - try larger
            lo = (int)mid + 1;
        } else {
            hi = (int)mid - 1;          // too big - try smaller
        }
    }
    return answer;
}

int main() {
    std::cout << integerSqrt(16) << "\n";
    std::cout << integerSqrt(17) << "\n";
    std::cout << integerSqrt(1)  << "\n";
```

```
    std::cout << integerSqrt(0)  << "\n";
    std::cout << integerSqrt(2147395600) << "\n";
    return 0;
}
```

```
4
4
1
0
46340
```

No array was searched. The "array" was the conceptual list 1, 2, 3, … and the test was `mid * mid <= n`.

> **INTERVIEW ANGLE | The sentence that unlocks these problems**
> Whenever a problem asks for *"the minimum X such that …"* or *"the maximum X such that …"*, ask yourself:
>
> > *"If I guess a value of X, can I check in reasonable time whether it works? And if X works, does every larger X also work?"*
>
> If both answers are yes, binary search the answer. This single observation solves "minimum capacity to ship packages in D days", "split array largest sum", "Koko eating bananas", "minimum days to make bouquets" and a dozen other problems that look nothing like searching.

The standard library

You rarely need to hand-write binary search in production.

std::lower_bound, upper_bound, binary_search, equal_range

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {5, 7, 7, 7, 8, 8, 10};

    auto lb = std::lower_bound(v.begin(), v.end(), 7);   // FIRST >= 7
    auto ub = std::upper_bound(v.begin(), v.end(), 7);   // FIRST > 7

    std::cout << "lower_bound(7) index = " << (lb - v.begin()) << "\n";
    std::cout << "upper_bound(7) index = " << (ub - v.begin()) << "\n";
    std::cout << "count of 7           = " << (ub - lb) << "\n";
    std::cout << "contains 8?          = " << std::binary_search(v.begin(
        ), v.end(), 8) << "\n";
    std::cout << "contains 9?          = " << std::binary_search(v.begin(
        ), v.end(), 9) << "\n";

    // Where would 9 be inserted to keep the array sorted?
    std::cout << "insert 9 at index    = "
              << (std::lower_bound(v.begin(), v.end(), 9) - v.begin()) <<
                  "\n";
```

```
    return 0;
}
```

```
lower_bound(7) index = 1
upper_bound(7) index = 4
count of 7           = 3
contains 8?          = 1
contains 9?          = 0
insert 9 at index    = 6
```

Call	Returns	Use it for
`lower_bound(b, e, x)`	first element $\geq x$	first occurrence; insertion point
`upper_bound(b, e, x)`	first element $> x$	one past the last occurrence
`equal_range(b, e, x)`	both of the above	the whole run of equal values
`binary_search(b, e, x)`	`bool`	presence only; discards the position

Table 12.1 The standard library's binary searches. `lower_bound` is the one you will use most, because it answers both "where is it?" and "where would it go?" with the same call.

COMMON PITFALL | These are $O(\log n)$ only on random-access containers
On a `std::vector` or an array, `lower_bound` is $O(\log n)$.
On a `std::list` — a linked list — it still performs $O(\log n)$ *comparisons*, but reaching the middle element requires walking, so the total is $O(n)$. The algorithm is correct; the data structure cannot support it.
This is the cleanest illustration of the book's central trade-off: **binary search is an array algorithm.** Chapter 33 returns to exactly why.

Cost summary

Operation	Time	Space	Precondition
Binary search	$O(\log n)$	$O(1)$	sorted
First / last occurrence	$O(\log n)$	$O(1)$	sorted
Count occurrences	$O(\log n)$	$O(1)$	sorted
Insertion point	$O(\log n)$	$O(1)$	sorted
Binary search on the answer	$O(\log R \cdot C)$	$O(1)$	monotonic test
Sort, then binary search once	$O(n \log n)$	depends	—

That last row is a warning: if you will search only once, sorting first is $O(n \log n)$, which is *worse* than a single $O(n)$ linear scan. Sorting pays off when you will search many times.

CHAPTER RECAP | Chapter 12 in seven lines

- Each comparison halves the region, so $n=10^9$ needs 30 comparisons.
- Inclusive bounds `[lo, hi]` go with `while (lo <= hi)` and `mid ± 1`. Pick one convention and never mix.
- `mid = lo + (hi - lo) / 2` avoids an overflow that hid in standard libraries for nine years.
- `lo = mid` or `hi = mid` without the ±1 gives an infinite loop. Test with $n=2$.
- The invariant — *the answer, if any, is in* `[lo, hi]` — is the proof, and worth stating aloud.
- On a match, record and continue to find the **first** or **last** occurrence.
- If a monotonic yes/no test exists, binary search the **answer**, not the array.

EXERCISES | Practise

1. Trace `binarySearch({1,3,5,7,9,11}, 11)` by hand, writing `lo`, `hi` and `mid` each iteration. How many comparisons?
2. Run the same search for a value that is absent, say 6. Where does the loop stop, and what are `lo` and `hi` at that moment?
3. Write binary search with **half-open** bounds (`hi = size()`, `while (lo < hi)`). Which of the five decisions change?
4. In `firstOccurrence`, why is `hi = mid - 1` after a match correct, rather than `hi = mid`? What would `hi = mid` do on a two-element array of equal values?
5. Given a sorted array rotated at an unknown point, such as `{4,5,6,7,0,1,2}`, find a target in $O(\log n)$. (Hint: one half is always properly sorted — work out which, then decide whether the target lies in it. Full treatment in Chapter 20.)
6. Find the **peak** of an array that increases and then decreases, in $O(\log n)$. What is the monotonic test here? Note that the array is **not** sorted — which tells you something about what binary search really needs.
7. "Koko eating bananas": given piles of bananas and H hours, find the smallest eating speed k such that all piles can be finished in H hours. What is the candidate range, what is the test, and why is it monotonic?
8. Explain why `std::lower_bound` on a `std::list` is $O(n)$ even though it makes only $O(\log n)$ comparisons.

CHAPTER

13

Pattern 6: In-Place Modification

Rearranging an array using only the array

CORE PREREQS: Chapters 6 and 9

"Solve it in $O(1)$ extra space" is one of the most common follow-up questions an interviewer asks, and it is never about saving memory — it is about whether you can see that the array already contains everywhere you need to put things. This chapter collects the four techniques: the write pointer, the swap, the reversal trick, and the genuinely clever one of storing extra information inside the values themselves.

WHAT YOU WILL LEARN

- Say precisely what "in place" and "$O(1)$ auxiliary space" mean
- Use a write pointer to filter, compact or partition in one pass
- Rotate an array by k positions using three reversals, and explain why it works
- Use the array's own indices as a lookup table
- Encode a second piece of information inside an element without losing the first
- Recognise when in-place is impossible and say so

What "in place" means

DEFINITION | In place

An algorithm is **in place** if it uses $O(1)$ auxiliary space — a fixed number of extra variables, no matter how large the input.

A handful of `int`s is in place. A second array of size n is not. A `std::vector` you `push_back` into n times is not.

The *input itself* does not count against you: you are allowed to modify it, and you are allowed to use it as workspace. That permission is the whole trick.

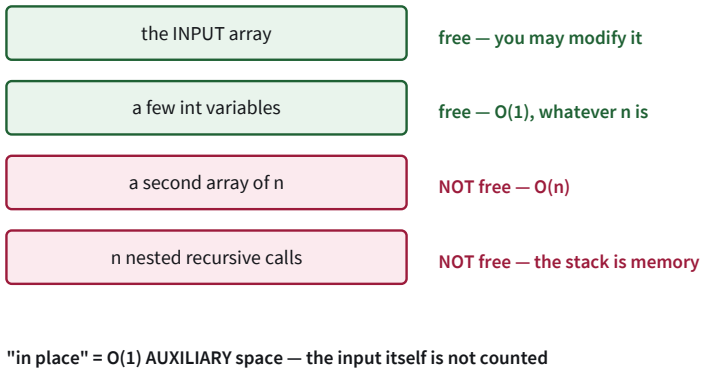


Figure 13.1 What counts as auxiliary space. The input is free; a constant number of variables is free; anything that grows with n is not. Recursion depth counts, which catches people out.

Three techniques, in increasing order of cleverness.

Technique 1: The write pointer

You have met this in Chapters 6 and 9. It is the workhorse, and it solves any problem of the form *"keep some elements, discard the rest, preserve order"*.

The general shape — keep what the predicate accepts

```
int write = 0;
for (int read = 0; read < n; ++read)
    if (keep(v[read]))
        v[write++] = v[read];
// v[0 .. write-1] is the answer; write is its length
```

Specialise `keep` and you get a family of problems:

Problem	`keep(x)` is
Remove all occurrences of `val`	`x != val`
Remove duplicates from a sorted array	`x != v[write - 1]`
Move zeroes to the end	`x != 0`, then pad with zeroes
Keep only even numbers	`x % 2 == 0`
Compact out every `nullptr`	`x != nullptr`

All of them are $O(n)$ time, $O(1)$ space, one pass. If you recognise the shape, you have solved them all at once.

Technique 2: The reversal trick for rotation

DEFINITION | The problem
Rotate an array right by k positions. `{1,2,3,4,5,6,7}` with k=3 becomes `{5,6,7,1,2,3,4}` — the last three elements move to the front.

The approaches that do not satisfy the constraint

Use a second array. Copy each element to its new index $(i+k) \bmod n$. $O(n)$ time, but $O(n)$ space — rejected.

Rotate by one, k times. $O(1)$ space, but $O(n \cdot k)$ time, which for $k \approx n$ is $O(n^2)$ — rejected.

We want both $O(n)$ time and $O(1)$ space, and the solution is one of the prettiest ideas in elementary algorithms.

The insight

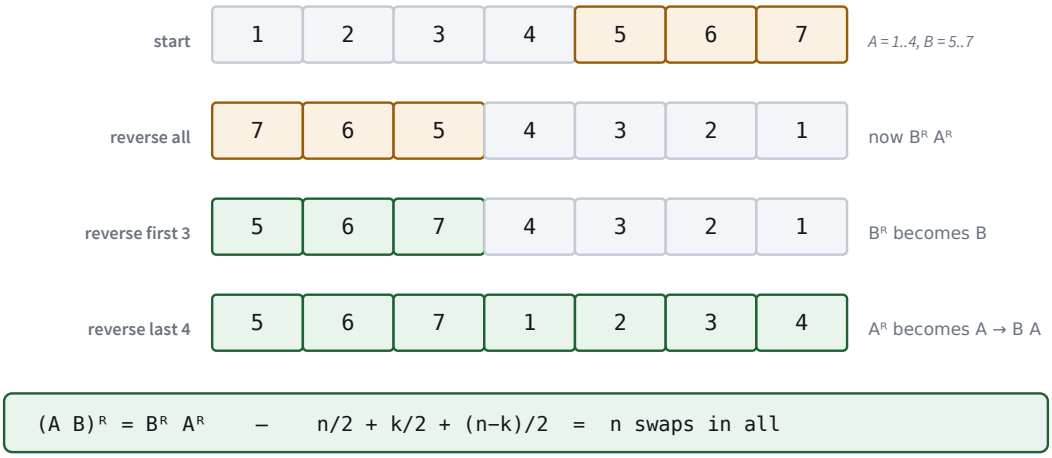


Figure 13.2 Rotation by three reversals. Reversing the whole array puts the right block at the front, but both blocks are now backwards. Reversing each block separately repairs them. Three passes, no extra memory.

KEY INSIGHT | Why reversing three times rotates
Write the array as two blocks: A (the first $n-k$ elements) and B (the last k). You currently have AB and you want BA.
Reversing the whole array gives $(AB)^{\mathrm{R}}$, and reversing a concatenation reverses the order of the parts as well as each part: $(AB)^{\mathrm{R}} = B^{\mathrm{R}} A^{\mathrm{R}}$.
So after one reversal you have $B^{\mathrm{R}} A^{\mathrm{R}}$ — the blocks are in the right order, each one backwards. Reverse the first k elements to turn B^{R} into B, and reverse the remaining $n-k$ to turn A^{R} into A. Result: BA.
Three reversals, $n/2 + k/2 + (n-k)/2 = n$ swaps in total. $O(n)$ time, $O(1)$ space.

The trace

`{1,2,3,4,5,6,7}`, $k=3$.

Step	Action	Array
start	—	1 2 3 4 5 6 7
1	reverse all	7 6 5 4 3 2 1
2	reverse first $k=3$	5 6 7 4 3 2 1
3	reverse last $n-k=4$	5 6 7 1 2 3 4

Table 13.1 The three reversals. After step 1 the blocks are in the right places and the wrong order internally; steps 2 and 3 fix each block.

Rotate right by k, in O(n) time and O(1) space

```cpp
#include <iostream>
#include <vector>

void reverseRange(std::vector<int>& v, int lo, int hi) {
    while (lo < hi) std::swap(v[lo++], v[hi--]);
}

void rotateRight(std::vector<int>& v, int k) {
    int n = (int)v.size();
    if (n == 0) return;
    k %= n;                              // k = 10 on 7 elements is k = 3
    if (k < 0) k += n;                   // tolerate a negative k
    reverseRange(v, 0, n - 1);           // 1. reverse everything
    reverseRange(v, 0, k - 1);           // 2. reverse the first k
    reverseRange(v, k, n - 1);           // 3. reverse the rest
}

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7};
    rotateRight(v, 3);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> w = {1, 2, 3};
    rotateRight(w, 10);                  // 10 % 3 == 1
    for (int x : w) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> z = {1, 2, 3};
    rotateRight(z, 3);                   // a full turn changes nothing
    for (int x : z) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
5 6 7 1 2 3 4
3 1 2
1 2 3
```

COMMON PITFALL | **Forgetting `k %= n`**
If $k \geq n$, `reverseRange(v, 0, k - 1)` reads past the end of the array — undefined behaviour, and on a big enough k, a crash.
A rotation by n is the identity, so only $k \bmod n$ matters. This one line turns a function that works on the examples into a function that works.
Rotating **left** by k is the same as rotating right by $n-k$. Ask which direction is wanted; the problem statements are often ambiguous and interviewers notice when you check.

Technique 3: Using indices as storage

Now the genuinely clever family. These problems share a precondition that looks like a detail and is actually the whole solution:

> *the array contains n values, each in the range 1 to n.*

When you see that, the array has exactly as many **slots** as it has possible **values** — so index i can carry information about the value i+1. The array becomes its own hash table, for free.

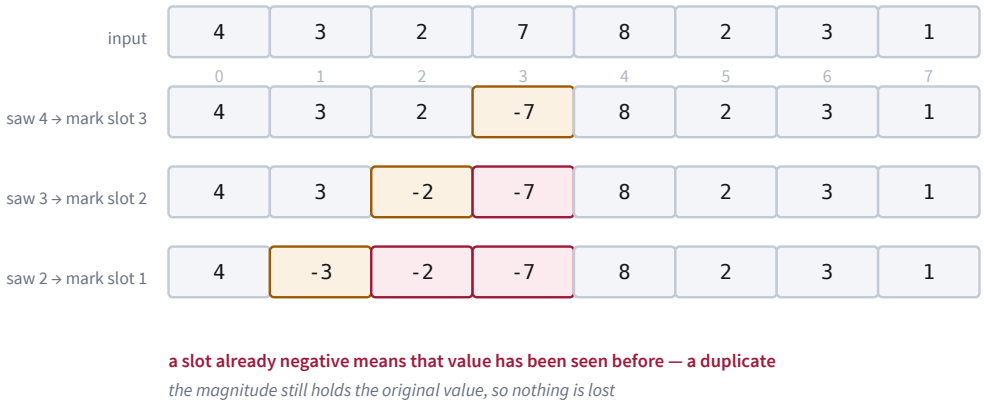


Figure 13.3 Using the sign bit as a visited mark. Seeing value v means flipping the sign at index $

Find all duplicates in an array of 1..n

All duplicates, O(n) time, O(1) space

```cpp
#include <cmath>
#include <iostream>
#include <vector>

// v has n elements, each in [1, n]. Each value appears once or twice.
// Returns the values appearing twice. v is modified and then restored.
std::vector<int> findDuplicates(std::vector<int>& v) {
    std::vector<int> result;
    for (int i = 0; i < (int)v.size(); ++i) {
        int idx = std::abs(v[i]) - 1;       // the slot this value owns
        if (v[idx] < 0)
            result.push_back(std::abs(v[i]));   // already marked -> a
                duplicate
        else
            v[idx] = -v[idx];                   // mark it as seen
    }
    for (int& x : v) x = std::abs(x);           // tidy up after ourselves
    return result;
}

int main() {
    std::vector<int> v = {4, 3, 2, 7, 8, 2, 3, 1};
    for (int x : findDuplicates(v)) std::cout << x << " ";
```

```
    std::cout << "\n";
    for (int x : v) std::cout << x << " ";      // restored
    std::cout << "\n";

    std::vector<int> none = {1, 2, 3};
    std::cout << "duplicates: " << findDuplicates(none).size() << "\n";
    return 0;
}
```

```
2 3
4 3 2 7 8 2 3 1
duplicates: 0
```

The sign of `v[idx]` carries a boolean — *have I seen the value* `idx + 1` *?* — without costing a single extra byte. The magnitude still carries the original value, so nothing is lost.

> **KEY INSIGHT | Is the output vector not extra space?**
> A fair question, and the standard answer is: **space used purely to hold the output does not count** against an in-place claim. The working space is $O(1)$; the result has to go somewhere.
> Say this explicitly in an interview. Candidates who notice the subtlety and name the convention look considerably better than candidates who do not notice it at all.

Encoding two values in one slot

A variation worth knowing, because it looks impossible until you see it.

> **DEFINITION | The trick**
> To store a second value b (where $0 \le b < n$) alongside an existing value a (also $< n$) in one slot, write $a + b \times n$.
> Recover them with division and remainder: $a = x \bmod n$, $b = \lfloor x / n \rfloor$.
> It works because $a < n$ means a never spills into the multiples-of-n part. Watch for overflow: use `long long` when n^2 might exceed `int`.

Two values in one slot: in-place permutation

```
#include <iostream>
#include <vector>

// v is a permutation of 0..n-1. Replace each v[i] with v[v[i]], in place.
void permuteInPlace(std::vector<int>& v) {
    int n = (int)v.size();
    for (int i = 0; i < n; ++i)
        v[i] += n * (v[v[i]] % n);      // old value in the remainder,
            new in the quotient
    for (int i = 0; i < n; ++i)
        v[i] /= n;                      // keep only the new value
}

int main() {
    std::vector<int> v = {0, 2, 1, 5, 3, 4};
    permuteInPlace(v);
```

```
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
0 1 2 4 5 3
```

In the first loop, `v[v[i]] % n` extracts the *original* value at the target slot even if that slot has already been updated — because the original is still sitting in the remainder. That is the point of the encoding: the first pass can read old values and write new ones simultaneously.

COMMON PITFALL | The two ways this trick bites

Overflow. `v[i] + n * something` can reach n^2. For n=100,000 that is 10^{10}, well past `int`. Use `long long` or check the constraints.

Negative numbers. Both the sign trick and the encoding trick assume non-negative values. A single negative element breaks `abs(v[i]) - 1` as an index. Always check the stated range before reaching for either.

When in-place is impossible

Not every problem can be done in $O(1)$ space, and recognising that is as valuable as finding a clever encoding.

Problem	In place?	Why
Reverse an array	yes	swaps only
Rotate by k	yes	three reversals
Remove duplicates (sorted)	yes	write pointer
Remove duplicates (unsorted, order kept)	**no**	you must remember every value seen: $O(n)$
Merge two sorted arrays into a third	**no**	the output is $O(n)$ by definition
Stable partition, both groups ordered	**no** in $O(n)$ time	needs $O(n)$ space or $O(n \log n)$ time
Sort	yes, with heapsort or quicksort	but quicksort uses $O(\log n)$ stack

INTERVIEW ANGLE | **How to handle the follow-up**

When an interviewer says *"now do it in O(1) space"*, they are asking one of three questions, and your first job is to work out which:

1. **"Can you use a write pointer?"** — most of the time, this is it.
2. **"Have you spotted the 1..n constraint?"** — if the problem mentions a value range equal to the array length, index marking is intended.
3. **"Do you know this is impossible?"** — occasionally the honest answer is *"not without losing the order"* or *"not in linear time"*, and saying so with a reason beats inventing something broken.

CHAPTER RECAP | **Chapter 13 in six lines**

- In place means $O(1)$ **auxiliary** space; the input is free to use and to modify.
- A **write pointer** solves every keep-or-discard problem in one pass.
- Rotation by k is three reversals, because $(AB)^{\mathrm{R}}=B^{\mathrm{R}}A^{\mathrm{R}}$. Remember `k %= n`.
- When values are in $1..n$, the array has a slot per value: use **signs** as visited marks.
- $a+b\times n$ stores two values in one slot; recover with `% n` and `/ n`, and watch for overflow.
- Some problems genuinely need $O(n)$ space. Knowing which ones is part of the skill.

EXERCISES | **Practise**

1. Rotate `{1,2,3,4,5}` **left** by 2 using three reversals. Which ranges do you reverse, and in what order?
2. Why does `rotateRight` need `k %= n`? What exactly goes wrong with `{1,2,3}` and $k=5$ if you leave it out?
3. Given an array of n values each in $1..n$ with exactly one missing and one duplicated, find both in $O(n)$ time and $O(1)$ space.
4. First missing positive: given an unsorted array, find the smallest positive integer not present, in $O(n)$ time and $O(1)$ space. (Hint: the answer is always in $1..n+1$ — why? Then put each value x into slot $x-1$ by swapping.)
5. Sort an array containing only 0s, 1s and 2s in one pass with $O(1)$ space. (This is the Dutch National Flag problem; it needs **three** pointers.)
6. Explain why removing duplicates from an *unsorted* array while preserving order cannot be done in $O(1)$ space. What is the minimum you must remember?
7. The sign-marking trick fails if the array may contain 0. Why? Suggest a fix that still uses $O(1)$ space.
8. You are asked to merge two sorted arrays `a` (with m real elements and n spare slots at the end) and `b` (with n elements) into `a`, in place. Why does filling from the **front** fail, and what does filling from the **back** fix?

CHAPTER

14

Pattern 7: Frequency Counting

Trading memory for time by remembering what you have seen

CORE PREREQS: Chapters 3 and 8

Most $O(n^2)$ array solutions are quadratic for one reason: for each element they search the rest of the array for a partner. A frequency table removes that inner search entirely by remembering what has already gone past. This chapter covers the array counter and the hash map, when each is right, and the complement trick that makes Two Sum linear on unsorted data.

WHAT YOU WILL LEARN

- Choose between a fixed-size count array and a hash map, with a reason
- Build a frequency table in one pass and use it to answer counting questions
- Solve unsorted Two Sum in $O(n)$ with the complement trick
- Find the majority element, the single number, and anagram pairs
- State the real cost of `std::unordered_map`, including when it is not $O(1)$
- Recognise when a `std::set` or `std::unordered_set` is enough

The waste the pattern removes

Here is a quadratic solution, and the reason it is quadratic.

Does this array contain a duplicate? The slow way

```
for (int i = 0; i < n; ++i)
    for (int j = i + 1; j < n; ++j)
        if (v[i] == v[j])
            std::cout << "duplicate";
```

The inner loop asks *"have I seen `v[i]` anywhere else?"* and answers it by looking. But the information was available all along — we **had** already seen those elements, we simply did not write anything down.

KEY INSIGHT | The trade, stated plainly
Spend $O(n)$ memory to remember what you have seen. In exchange, every *"have I seen this?"* question becomes $O(1)$ instead of $O(n)$.
$O(n^2)$ time and $O(1)$ space becomes $O(n)$ time and $O(n)$ space. This is the single most common optimisation in interview problems, and the single most common follow-up is *"now do it without the extra space"* — which usually means sorting (Chapter 15) or index marking (Chapter 13).

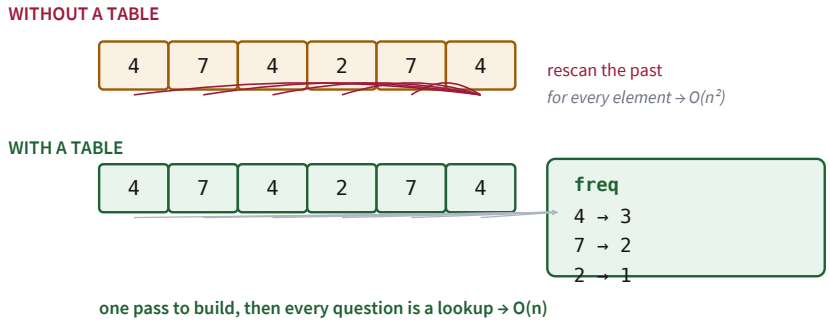
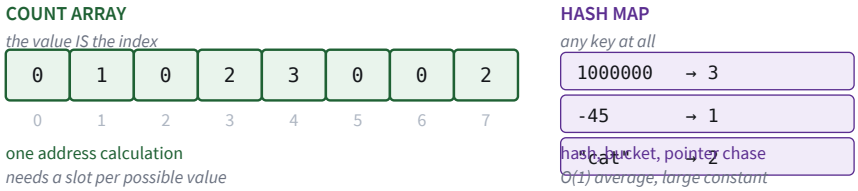


Figure 14.1 Replacing a search with a lookup. The nested loop re-scans the past for every element; the frequency table records it once and answers in constant time.

Two tools



if the value can be used as an index, use the array — typically 5–20× faster

Figure 14.2 The two implementations. A count array is a direct index — fastest possible, but it needs one slot per possible value. A hash map handles any key at the cost of hashing and indirection.

DEFINITION | Count array versus hash map
A **count array** is `std::vector<int> count(K, 0)`, where the value itself is the index. Use it when the values are small non-negative integers, or characters, and K is manageable. Access is one address calculation: genuinely $O(1)$, with a tiny constant.
A **hash map** is `std::unordered_map<Key, int>`. Use it when the keys are large, negative, sparse, or not integers at all. Access is $O(1)$ *on average*, with a much larger constant — a hash computation plus a pointer chase.
Rule of thumb: if you can index by the value directly, do. A count array is typically 5–20 times faster than a hash map for the same job.

Situation	Reach for
lowercase letters	`std::vector<int> count(26, 0)`
any `char`	`std::vector<int> count(256, 0)`
digits, or values 0..1000	a count array of that size
values up to 10^9, or negative	`std::unordered_map<int, int>`
strings, pairs, or custom types	`std::unordered_map<Key, int>`
you only need presence, not counts	`std::unordered_set` or a `vector<bool>`

Building and using a frequency table

One pass to build, then answer anything

```cpp
#include <iostream>
#include <unordered_map>
#include <vector>

int main() {
    std::vector<int> v = {4, 7, 4, 2, 7, 4};

    std::unordered_map<int, int> freq;
    for (int x : v) ++freq[x];          // operator[] creates a 0 if absent

    std::cout << "4 appears " << freq[4] << " times\n";
    std::cout << "9 appears " << freq[9] << " times\n";

    // Most frequent value, breaking ties by smaller value for determinism.
    int bestVal = 0, bestCount = 0;
    for (const auto& [value, count] : freq)
        if (count > bestCount || (count == bestCount && value < bestVal)) {
            bestVal = value;
            bestCount = count;
        }
    std::cout << "most frequent: " << bestVal << " (" << bestCount << ")\n";
    return 0;
```

```
23  }
```

```
4 appears 3 times
9 appears 0 times
most frequent: 4 (3)
```

COMMON PITFALL | `operator[]` **inserts, even when you are only looking**
`freq[9]` on a key that is absent **creates** the entry with value 0 and returns a reference to it. So the map above now contains a 9, and `freq.size()` has grown by one.
Usually harmless. Occasionally a real bug: inside a loop over a map while checking keys, insertion can invalidate iterators, and in a `const` context it will not compile at all.
To look without inserting:

```cpp
auto it = freq.find(9);
int c = (it == freq.end()) ? 0 : it->second;
// or, more briefly:
int c2 = freq.count(9) ? freq.at(9) : 0;
```

Note the tie-break in the program above. Iteration order over an `unordered_map` is unspecified, so without a deterministic rule the "most frequent" answer could differ between runs or compilers. Interviewers notice when you handle that.

The complement trick: Two Sum, unsorted

Chapter 9 solved Two Sum on **sorted** input with two pointers. Here the array is unsorted and we must return the original indices, so sorting would destroy the answer.

KEY INSIGHT | **Ask for the partner, not for the pair**
For each element x, the partner you need is exactly $target - x$ — its **complement**. There is no searching to do: you know precisely what you want.
So the question becomes *"have I already seen $target - x$?"*, which a hash map answers in $O(1)$.
Crucially, you check the map **before** inserting the current element. That way an element can never pair with itself, and the stored index is always strictly earlier.

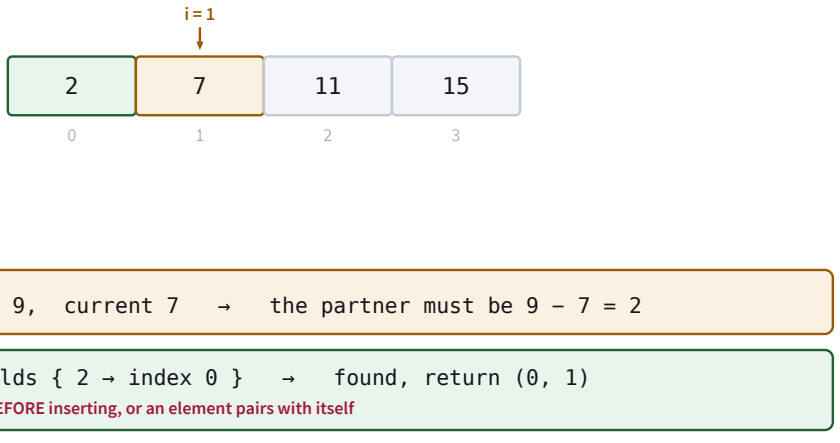


Figure 14.3 Two Sum by complement. At each element, compute what is missing and ask the map whether it has already gone past. One pass, no sorting, original indices preserved.

The trace

`{2, 7, 11, 15}`, target 9.

i	v[i]	complement	in map?	map afterwards
0	2	7	no	`{2→0}`
1	7	2	**yes, at 0**	return (0, 1)

Table 14.1 Each row does one lookup and one insert. The pair is found the moment the second member arrives.

Two Sum on unsorted data, O(n)

```cpp
#include <iostream>
#include <unordered_map>
#include <vector>

std::pair<int, int> twoSum(const std::vector<int>& v, int target) {
    std::unordered_map<int, int> seen;       // value -> index
    for (int i = 0; i < (int)v.size(); ++i) {
        int complement = target - v[i];
        auto it = seen.find(complement);
        if (it != seen.end()) return {it->second, i};   // look up FIRST
        seen[v[i]] = i;                                 // then insert
    }
    return {-1, -1};
}

int main() {
    auto [a, b] = twoSum({2, 7, 11, 15}, 9);
    std::cout << a << " " << b << "\n";

    auto [c, d] = twoSum({3, 2, 4}, 6);
    std::cout << c << " " << d << "\n";

    auto [e, f] = twoSum({3, 3}, 6);        // duplicates are fine
    std::cout << e << " " << f << "\n";

    auto [g, h] = twoSum({1, 2}, 100);
    std::cout << g << " " << h << "\n";
    return 0;
}
```

```
0 1
1 2
0 1
-1 -1
```

COMMON PITFALL | Inserting before looking up
With `{3, 2, 4}` and target 6, inserting first means that at i=0 the map already contains `3 → 0`; the complement of 3 is 3, the lookup succeeds, and you return `(0, 0)` — one element used twice.
Look up, then insert. The same rule appeared in Chapter 11 for sub-array sums, and it will appear again. It is worth recognising as a rule rather than rediscovering it each time.

Approach	Time	Space	Needs sorted?	Returns original indices?
Brute force	$O(n^2)$	$O(1)$	no	yes
Sort, then two pointers	$O(n \log n)$	$O(1)$ or $O(n)$	sorts it	only with extra bookkeeping
Hash map	$O(n)$	$O(n)$	no	yes

Table 14.2 Two Sum, three ways. The right answer depends on whether indices are required and whether extra space is allowed — which is why a good candidate asks before coding.

Counting with a plain array: anagrams

Are two strings anagrams of each other?

```cpp
#include <iostream>
#include <string>
#include <vector>

bool areAnagrams(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;    // cheap rejection first
    std::vector<int> count(26, 0);
    for (char c : a) ++count[c - 'a'];         // add for a...
    for (char c : b) --count[c - 'a'];         // ...subtract for b
    for (int x : count)
        if (x != 0) return false;              // anything left over
            means no
    return true;
}

int main() {
    std::cout << areAnagrams("listen", "silent") << "\n";
    std::cout << areAnagrams("hello",  "world")  << "\n";
    std::cout << areAnagrams("a",      "ab")     << "\n";
    std::cout << areAnagrams("",       "")       << "\n";
    return 0;
}
```

```
1
0
0
1
```

Three things worth copying from this function:

`c - 'a'` **maps a letter to 0..25.** Characters are small integers; `'c' - 'a'` is 2. This is the cheapest possible frequency table.

One array, not two. Adding for the first string and subtracting for the second means the array ends as all zeroes exactly when the strings match. Two arrays and a comparison would also work and would need twice the memory and a second loop.

The length check comes first. It costs $O(1)$ and eliminates most negative cases immediately. Cheap rejections before expensive work is a habit worth having.

$O(n)$ time, and $O(1)$ space — the count array is 26 entries whatever the input length, which is a constant.

Sets: when you only need “have I seen it?”

Has this array a duplicate? Does it contain a target?

```cpp
#include <iostream>
#include <unordered_set>
#include <vector>

bool hasDuplicate(const std::vector<int>& v) {
    std::unordered_set<int> seen;
    for (int x : v)
        if (!seen.insert(x).second)      // insert returns {iterator,
            inserted?}
            return true;                 // already present
    return false;
}

int main() {
    std::cout << hasDuplicate({1, 2, 3, 1}) << "\n";
    std::cout << hasDuplicate({1, 2, 3})    << "\n";
    std::cout << hasDuplicate({})           << "\n";
    return 0;
}
```

```
1
0
0
```

`insert` returns a pair whose `.second` is `true` if the element was new. That gives you *check and insert* in one operation, instead of a `count` followed by an `insert` — half the hashing work.

What a hash map really costs

WATCH OUT | `unordered_map` **is $O(1)$ on average, not always**
The worst case for a single lookup is $O(n)$, when every key lands in the same bucket. With adversarially chosen keys this is achievable, and competitive programmers do exploit it. Beyond complexity, the **constant factor** is large. Every access computes a hash, indexes a bucket array, and follows a pointer chain — typically a cache miss. A `std::vector<int>` lookup is a single address calculation.
Practical guidance:

- Prefer a **count array** whenever the value range allows it.
- `reserve()` the map if you know roughly how many keys there will be; it avoids rehashing, which is the same amortised story as Chapter 5.
- Do not use a map when a sorted array plus binary search would do and the data is static.

	count array	`unordered_map`	`map`
Lookup	$O(1)$, tiny constant	$O(1)$ average, $O(n)$ worst	$O(\log n)$
Memory	one slot per possible value	per stored key	per stored key
Keys allowed	small non-negative integers	anything hashable	anything orderable
Iterates in sorted order	yes, by index	no	yes
Good when	the range is small	the range is large or sparse	you need order

Table 14.3 Choosing a container. The first column is almost always the right answer when it is available at all.

Cost summary

Problem	Without counting	With counting
Contains duplicate	$O(n^2)$	$O(n)$ time, $O(n)$ space
Two Sum (unsorted)	$O(n^2)$	$O(n)$ time, $O(n)$ space
Most frequent element	$O(n^2)$	$O(n)$ time, $O(n)$ space
Anagram check	$O(n \log n)$ by sorting	$O(n)$ time, $O(1)$ space
First non-repeating character	$O(n^2)$	$O(n)$ time, $O(1)$ space
Intersection of two arrays	$O(n \cdot m)$	$O(n+m)$ time, $O(n)$ space

CHAPTER RECAP | Chapter 14 in six lines

- A frequency table replaces *searching the past* with *looking it up*, turning $O(n^2)$ into $O(n)$.
- Use a **count array** when the value is usable as an index; it is far faster than a hash map.
- **Look up before you insert** — otherwise an element pairs with itself.
- `operator[]` on a map inserts a zero entry for a missing key. Use `find` to look without inserting.
- `unordered_map` is $O(1)$ **average** with a large constant, not a free lookup.
- The usual follow-up is "now in $O(1)$ space", which means sorting or index marking instead.

EXERCISES | Practise

1. Find the first character in a string that appears exactly once. What size is your count array, and how many passes do you need?
2. Given two arrays, return their intersection without duplicates. What is the complexity, and which array should go into the set if their sizes differ a lot?
3. Single number: every value appears twice except one. Solve it with a hash map, then solve it again in $O(1)$ space using XOR. Why does XOR work?
4. Group a list of words into anagram groups. What makes a good map **key** for this, and what are the two standard choices?
5. Majority element — a value appearing more than $n/2$ times. Solve with a frequency map, then look up the Boyer–Moore voting algorithm and explain how it reaches $O(1)$ space.
6. Why must `twoSum` look up before inserting? Give a concrete input where the other order returns a wrong answer.
7. You must count the frequency of values in the range 0 to 10^9, but there are only 10^5 elements. Which container, and why not the other one?
8. Longest consecutive sequence: given an unsorted array, find the length of the longest run of consecutive integers, in $O(n)$. (Hint: a set, and start counting only from values that have no predecessor in the set — why does that make it linear rather than quadratic?)

CHAPTER

15

Pattern 8: Sorting-Based Approaches

When $O(n \log n)$ of preparation makes the rest trivial

CORE PREREQS: Chapters 9 and 12

Sorting is not a pattern in itself — it is an enabler. Once an array is sorted, duplicates are adjacent, binary search becomes available, two pointers work, and "closest pair" questions collapse to neighbouring comparisons. This chapter is about recognising when paying $O(n \log n)$ up front is the right move, and when it is a mistake that destroys information you needed.

WHAT YOU WILL LEARN

- Use `std::sort` correctly, including with custom comparators
- Explain what sorting *gives* you: adjacency, monotonicity, searchability
- Decide between sorting, hashing, and a direct scan, with reasons
- Sort pairs and structures without losing the association between fields
- Recognise when sorting destroys the answer (original indices, stability)
- Know the cost and the guarantees of `std::sort`, `std::stable_sort` and `std::nth_element`

What sorting buys you

DEFINITION | The three gifts of a sorted array

Adjacency. Equal values become neighbours, so duplicate detection is a single pass comparing `v[i]` with `v[i-1]` — no memory needed.

Monotonicity. The array only goes up, so two pointers from the ends behave predictably (Chapter 9) and binary search becomes available (Chapter 12).

Locality of closeness. The numerically nearest value to `v[i]` is `v[i-1]` or `v[i+1]`. Nothing far away can be closer. This turns many "closest pair" questions from $O(n^2)$ into $O(n \log n)$.

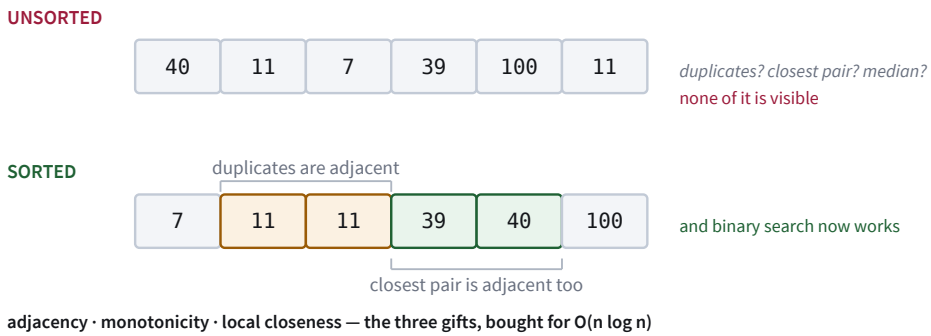


Figure 15.1 What sorting gives you. In the unsorted array the duplicates, the closest pair and the median are all invisible without searching. In the sorted array each is a local observation.

The cost is $O(n \log n)$ and, usually, the loss of the original order. Both of those matter, and the rest of the chapter is about weighing them.

Using std::sort

Sorting: ascending, descending, and by a custom rule

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<int> v = {5, 2, 9, 1, 7};

    std::sort(v.begin(), v.end());                          // ascending
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    std::sort(v.begin(), v.end(), std::greater<int>());  // descending
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    // Custom rule: by distance from 6, nearest first.
    // 5 and 7 are both 1 away, so the comparator needs a tie-break -
    // without one, std::sort may order them either way. See the pitfall
        below.
    std::sort(v.begin(), v.end(), [](int a, int b) {
        int da = std::abs(a - 6), db = std::abs(b - 6);
        if (da != db) return da < db;
        return a < b;
    });
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    // Sorting strings by length, then alphabetically.
    std::vector<std::string> words = {"pear", "fig", "apple", "kiwi"};
```

```
    std::sort(words.begin(), words.end(),
              [](const std::string& a, const std::string& b) {
                  if (a.size() != b.size()) return a.size() < b.size();
                  return a < b;
              });
    for (const auto& w : words) std::cout << w << " ";
    std::cout << "\n";
    return 0;
}
```

```
1 2 5 7 9
9 7 5 2 1
5 7 9 2 1
fig kiwi pear apple
```

COMMON PITFALL | The comparator must be a *strict* ordering

`std::sort` requires its comparator to return `false` when the two arguments are equivalent. Writing `<=` instead of `<` breaks that rule, and the result is not a wrong order — it is undefined behaviour, which in practice means `std::sort` reads outside the array and crashes, sometimes only on large inputs.

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a <= b; });  // BAD
std::sort(v.begin(), v.end(), [](int a, int b) { return a <  b; });  // OK
```

The same applies inside multi-key comparators: compare the first key, and only if it ties, move to the next — exactly as the `words` comparator above does. Never write `return a.size() <= b.size()`.

Sorting structures without losing the association

A common beginner error is sorting one array and forgetting that a second array was lined up with it.

Keep related fields together — sort the pairs, not the parts

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

struct Student {
    std::string name;
    int score;
};

int main() {
    std::vector<Student> students = {{"Ana", 72}, {"Ben", 91}, {"Cy",
    ↪ 72}};

    // Highest score first; ties broken alphabetically for determinism.
    std::sort(students.begin(), students.end(),
```

```
            [](const Student& a, const Student& b) {
                if (a.score != b.score) return a.score > b.score;
                return a.name < b.name;
            });
    for (const auto& s : students)
        std::cout << s.name << ":" << s.score << " ";
    std::cout << "\n";

    // When you must keep original positions, sort (value, index) pairs.
    std::vector<int> v = {40, 10, 30};
    std::vector<std::pair<int, int>> indexed;
    for (int i = 0; i < (int)v.size(); ++i) indexed.push_back({v[i], i});
    std::sort(indexed.begin(), indexed.end());
    for (const auto& [value, idx] : indexed)
        std::cout << value << "(was at " << idx << ") ";
    std::cout << "\n";
    return 0;
}
```

```
Ben:91 Ana:72 Cy:72
10(was at 1) 30(was at 2) 40(was at 0)
```

The second technique is the standard answer to *"sorting would lose the original indices, which the problem wants"*. Pair each value with its index before sorting, and the association survives.

When sorting is the right move

Closest pair of values

Smallest absolute difference between any two elements

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <vector>

int smallestDifference(std::vector<int> v) {     // by value: we will sort it
    if (v.size() < 2) return -1;
    std::sort(v.begin(), v.end());
    int best = INT_MAX;
    for (int i = 1; i < (int)v.size(); ++i)
        best = std::min(best, v[i] - v[i - 1]);  // only NEIGHBOURS can be closest
    return best;
}

int main() {
    std::cout << smallestDifference({40, 11, 7, 39, 100}) << "\n";
    std::cout << smallestDifference({5, 5})               << "\n";
    std::cout << smallestDifference({3})                  << "\n";
    return 0;
```

```
}
```

```
1
0
-1
```

Brute force is $O(n^2)$. Sorting makes it $O(n \log n)$, and the loop that follows is $O(n)$ — dominated by the sort.

> **KEY INSIGHT | Why only neighbours matter**
> After sorting, suppose $i<j$ and they are not adjacent. Then there is some k with $i<k<j$, and because the array is sorted, $v[j]-v[i] \geq v[j]-v[k]$ and $v[j]-v[i] \geq v[k]-v[i]$.
> So any non-adjacent pair is at least as far apart as some adjacent pair inside it. **The minimum must be between neighbours**, and checking $n-1$ pairs suffices instead of $n^2/2$.
> Being able to give that argument is the difference between "I sorted it because that usually helps" and understanding why the algorithm is correct.

Merging and intersecting sorted arrays

Union and intersection of two sorted arrays, in O(n + m)

```cpp
#include <iostream>
#include <vector>

std::vector<int> unionSorted(const std::vector<int>& a,
                             const std::vector<int>& b) {
    std::vector<int> out;
    int i = 0, j = 0;
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] < b[j])      { if (out.empty() || out.back() != a[i])
            out.push_back(a[i]); ++i; }
        else if (b[j] < a[i]) { if (out.empty() || out.back() != b[j])
            out.push_back(b[j]); ++j; }
        else                  { if (out.empty() || out.back() != a[i])
            out.push_back(a[i]); ++i; ++j; }
    }
    while (i < (int)a.size()) { if (out.empty() || out.back() != a[i])
        out.push_back(a[i]); ++i; }
    while (j < (int)b.size()) { if (out.empty() || out.back() != b[j])
        out.push_back(b[j]); ++j; }
    return out;
}

std::vector<int> intersectSorted(const std::vector<int>& a,
                                 const std::vector<int>& b) {
    std::vector<int> out;
    int i = 0, j = 0;
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] < b[j])      ++i;
        else if (b[j] < a[i]) ++j;
        else {
            if (out.empty() || out.back() != a[i]) out.push_back(a[i]);
            ++i; ++j;
```

```
        }
    }
    return out;
}

int main() {
    std::vector<int> a = {1, 2, 2, 4, 6};
    std::vector<int> b = {2, 3, 4, 4, 9};
    for (int x : unionSorted(a, b))     std::cout << x << " ";
    std::cout << "\n";
    for (int x : intersectSorted(a, b)) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
1 2 3 4 6 9
2 4
```

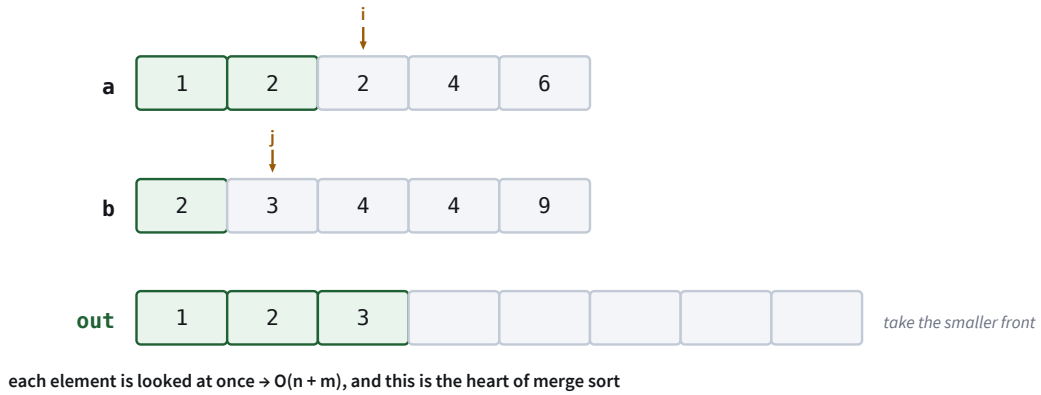


Figure 15.2 Merging two sorted arrays. Two pointers advance independently, always taking the smaller front element. Each element is examined once, so the whole merge is $O(n+m)$ — and this is the core of merge sort.

The `out.back() != x` guard is how duplicates are suppressed, and it works precisely because the output is being built in sorted order — the first gift of sorting, used on the result rather than the input.

When sorting is the wrong move

WATCH OUT | **Three times you should not sort**

You need the original indices. Two Sum must return positions in the input. Sorting destroys them unless you pair values with indices first, which costs $O(n)$ space — at which point a hash map is simpler and faster.

Order within equal elements matters. `std::sort` is **not stable**: equal elements may be reordered. If you have already sorted by score and now sort by name, the score order within each name is not preserved. Use `std::stable_sort` ($O(n \log n)$ guaranteed, but it allocates $O(n)$ memory).

You only need one element. To find the median, or the k-th largest, a full sort is wasteful. `std::nth_element` puts the k-th element in its final position in **average** $O(n)$ time, leaving everything else merely partitioned.

nth_element: the k-th smallest without a full sort

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {7, 2, 9, 4, 1, 8, 3};
    int k = 2;                  // 0-based: the 3rd smallest
    std::nth_element(v.begin(), v.begin() + k, v.end());
    std::cout << "3rd smallest = " << v[k] << "\n";

    // Everything before position k is <= v[k]; everything after is >= v[k].
    // The two halves are NOT individually sorted, and that is the saving.
    std::vector<int> w = {7, 2, 9, 4, 1, 8, 3};
    std::sort(w.begin(), w.end());
    std::cout << "check       = " << w[k] << "\n";
    return 0;
}
```

```
3rd smallest = 3
check        = 3
```

Choosing between the three approaches

	Sorting	Hashing	Direct scan
Time	$O(n \log n)$	$O(n)$ average	$O(n)$ or $O(n^2)$
Extra space	$O(1)$ to $O(n)$	$O(n)$	$O(1)$
Keeps original order	no	yes	yes
Gives you order statistics	yes	no	no
Enables binary search after	yes	no	no
Worst-case guarantee	yes	no	yes

Table 15.1 The standard decision. Most array problems can be attacked all three ways, and interviewers are usually asking you to compare them rather than to pick one.

INTERVIEW ANGLE | **Say all three**

A strong answer to an array problem sounds like this:

> *"The brute force is $O(n^2)$. I could sort and use two pointers for $O(n \log n)$ with no extra space, or use a hash map for $O(n)$ time and $O(n)$ space. Since the problem asks for the original indices, the hash map is the better fit — but if memory were constrained I would sort pairs of (value, index) instead."*

That paragraph demonstrates all three patterns, the trade-off between them, and attention to the specific requirement. It is a better answer than any single piece of code.

What std::sort actually is

Function	Complexity	Stable?	Extra memory
`std::sort`	$O(n \log n)$ worst case	no	$O(\log n)$ stack
`std::stable_sort`	$O(n \log n)$	yes	$O(n)$
`std::partial_sort`	$O(n \log k)$	no	$O(1)$
`std::nth_element`	$O(n)$ average	no	$O(1)$
`std::is_sorted`	$O(n)$	—	$O(1)$

`std::sort` is **introsort**: quicksort, which switches to heapsort if the recursion gets too deep (so the worst case stays $O(n \log n)$ rather than quicksort's $O(n^2)$), and to insertion sort for small ranges (because for $n < 16$ the simpler algorithm's smaller constant wins). That design is itself a lesson: Big O chooses the algorithm, and constants choose the special cases.

CHAPTER RECAP | **Chapter 15 in six lines**

- Sorting gives **adjacency**, **monotonicity** and **local closeness** — then other patterns become available.
- It costs $O(n \log n)$ and usually the original order. Weigh both.
- After sorting, the closest pair is always adjacent, so one pass finishes the job.
- Comparators must be strict (`<`, never `<=`), or `std::sort` is undefined behaviour.
- To keep positions, sort `(value, index)` pairs; to keep ties in order, use `stable_sort`.
- For one order statistic, `nth_element` is $O(n)$ average — do not sort the whole array.

EXERCISES | **Practise**

1. Given an array, find whether any value appears more than twice, using sorting and $O(1)$ extra space.
2. Merge two sorted arrays into one sorted array **keeping** duplicates. How does the code differ from `unionSorted`?

3. Three Sum: find all distinct triples summing to zero. Sort first; then for each i, run two pointers on the rest. Why is the total $O(n^2)$ and not $O(n^3)$? How do you skip duplicate triples?
4. Meeting rooms: given intervals $[s_i, e_i)$, find the minimum number of rooms needed. (Hint: sort the starts and the ends into two separate arrays and sweep. Why is it valid to separate them?)
5. Why is `[](int a, int b){ return a <= b; }` dangerous? What does the standard require of a comparator, and what actually happens when you break it?
6. You must find the median of 10^7 numbers once. Compare `std::sort` with `std::nth_element` in both complexity and real work done.
7. Sort an array of strings so that concatenating them gives the largest possible number: `{3, 30, 34, 5, 9}` → `9534330`. What is the comparator? (This one is famous for having a comparator that looks wrong and is right.)
8. Give a problem where sorting makes the answer **impossible** to recover, and say what you would do instead.

CHAPTER

16

Pattern 9: Kadane's Algorithm

The maximum sub-array sum in one pass, and the idea behind it

CORE PREREQS: Chapters 8 and 11

Kadane's algorithm finds the largest sum of any contiguous sub-array in a single pass with two variables. It is four lines long, and almost nobody derives it on first encounter — but the idea behind it is simple enough to rediscover, and it generalises to a whole family of problems. This chapter derives it rather than presenting it, which is the only way it will stay in your memory.

WHAT YOU WILL LEARN

- State the maximum sub-array problem precisely, including the all-negative case
- Derive Kadane's algorithm from the brute force by asking one question
- Explain the "extend or restart" decision and why it is always correct
- Trace the algorithm and recover the *indices* of the best sub-array, not just its sum
- Adapt it to the maximum **product** sub-array, and say why that needs a second variable
- Recognise the shape in problems that do not look like sub-array sums

The problem

DEFINITION | Maximum sub-array sum

Given an array, find the largest sum obtainable from any **contiguous**, non-empty sub-array.

`{-2, 1, -3, 4, -1, 2, 1, -5, 4}` → **6**, from `{4, -1, 2, 1}`.

"Non-empty" matters: if the empty sub-array were allowed, an all-negative array would answer 0, and most versions of the problem do not want that. Ask.

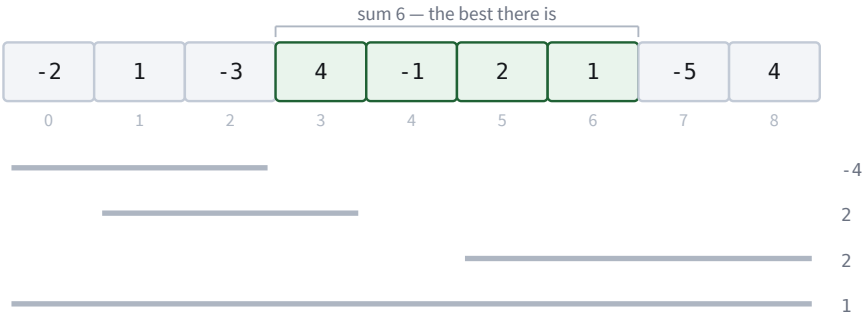


Figure 16.1 The problem. Every contiguous stretch has a sum; we want the largest. The answer is rarely the whole array, and with all-negative input it is the single least-bad element.

From brute force to the insight

Attempt 1: every sub-array, summed from scratch — $O(n^3)$

All starts, all ends, sum each one

```
int best = INT_MIN;
for (int i = 0; i < n; ++i)
    for (int j = i; j < n; ++j) {
        int sum = 0;
        for (int k = i; k <= j; ++k) sum += v[k];
        best = std::max(best, sum);
    }
```

Attempt 2: extend the inner sum — $O(n^2)$

The innermost loop re-adds elements it already added. Carry the sum instead:

Grow the sum as j advances

```
int best = INT_MIN;
for (int i = 0; i < n; ++i) {
    int sum = 0;
    for (int j = i; j < n; ++j) {
        sum += v[j];                // extend by one element
        best = std::max(best, sum);
    }
}
```

Better. Still quadratic, because we restart from every possible i.

Attempt 3: the question that removes the outer loop

Instead of asking *"what is the best sub-array overall?"*, ask a smaller question at each position:

> ***"What is the best sub-array that ends exactly here?"***

Call that `endingHere[i]`. If you knew it for every i, the answer would be the largest of them — because every sub-array ends somewhere.

And `endingHere[i]` has a two-line answer:

KEY INSIGHT | Extend, or restart
A sub-array ending at i either

- **extends** the best sub-array ending at $i-1$, giving `endingHere[i-1] + v[i]`, or
- **restarts** at i, giving just `v[i]`.

Those are the only two possibilities, so

$$\text{endingHere}[i] = \max(v[i], \text{endingHere}[i-1] + v[i])$$

Now notice *when* restarting wins: exactly when `endingHere[i-1] < 0`. A negative prefix can only drag down whatever follows it, so you drop it. A non-negative prefix can only help, so you keep it.

Kadane's algorithm in one sentence: carry the running sum forward, and throw it away the moment it goes negative.

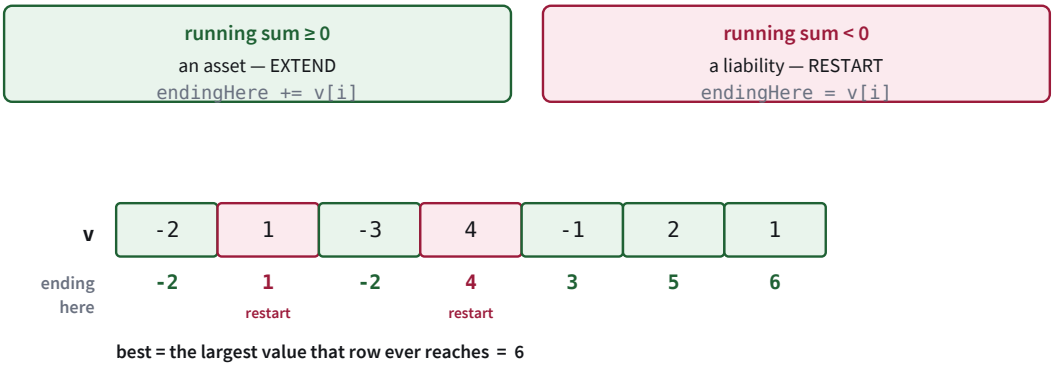


Figure 16.2 The extend-or-restart decision at each element. When the running sum is negative it is a liability, so the next element starts fresh; otherwise it is an asset and the next element inherits it.

Each `endingHere[i]` depends only on the previous one, so we need a single variable, not an array. That is the whole algorithm.

The code

Kadane's algorithm

```cpp
#include <algorithm>
#include <climits>
#include <iostream>
#include <vector>

long long maxSubarraySum(const std::vector<int>& v) {
    if (v.empty()) return 0;
    long long endingHere = v[0];      // best sub-array ending at index 0
    long long best = v[0];            // best seen anywhere so far
    for (int i = 1; i < (int)v.size(); ++i) {
```

```
        endingHere = std::max((long long)v[i], endingHere + v[i]);  //
            extend or restart
        best = std::max(best, endingHere);                          //
            record
    }
    return best;
}

int main() {
    std::cout << maxSubarraySum({-2, 1, -3, 4, -1, 2, 1, -5, 4}) << "\n";
    std::cout << maxSubarraySum({-3, -1, -7})                    << "\n";
    std::cout << maxSubarraySum({5})                             << "\n";
    std::cout << maxSubarraySum({1, 2, 3})                       << "\n";
    return 0;
}
```

```
6
-1
5
6
```

$O(n)$ time, $O(1)$ space.

The trace

`{-2, 1, -3, 4, -1, 2, 1, -5, 4}`.

i	v[i]	endingHere + v[i]	v[i]	decision	endingHere	best
0	−2	—	—	start	−2	−2
1	1	−1	1	**restart**	1	1
2	−3	−2	−3	extend	−2	1
3	4	2	4	**restart**	4	4
4	−1	3	−1	extend	3	4
5	2	5	2	extend	5	5
6	1	6	1	extend	6	**6**
7	−5	1	−5	extend	1	6
8	4	5	4	extend	5	6

Table 16.1 Kadane traced. The decision column is the heart of it: a negative running sum is always abandoned, because carrying it forward can only make the next sub-array worse.

Look at rows 1 and 3. Both restart, and both do so because the previous `endingHere` was negative. Look at row 7: `endingHere` drops from 6 to 1 but stays positive, so it is kept — and `best` is unaffected, because `best` only ever increases.

COMMON PITFALL | Initialising to 0 breaks all-negative input

```
long long endingHere = 0, best = 0;          // BUG for all-negative
    arrays
for (int x : v) {
    endingHere = std::max(0LL, endingHere + x);
    best = std::max(best, endingHere);
}
```

On `{-3, -1, -7}` this returns 0, which is the sum of the *empty* sub-array. If the problem demands a non-empty sub-array, the answer should be −1.
Initialising both to `v[0]` and starting the loop at index 1 makes the non-empty requirement automatic. If the empty sub-array *is* allowed, the 0-version is correct — so **ask which it is**, and say which you assumed.

Recovering the sub-array itself

Interviewers often follow up with *"now tell me where it is"*. Two extra variables do it.

Kadane with indices

```
#include <algorithm>
#include <iostream>
#include <vector>

struct Result { long long sum; int from; int to; };

Result maxSubarray(const std::vector<int>& v) {
    Result r{v[0], 0, 0};
    long long endingHere = v[0];
    int start = 0;                              // start of the current
        run
    for (int i = 1; i < (int)v.size(); ++i) {
        if (endingHere + v[i] < v[i]) {         // restarting
            endingHere = v[i];
            start = i;                          // the run begins here
        } else {
            endingHere += v[i];                 // extending
        }
        if (endingHere > r.sum) {               // a new overall best
            r.sum = endingHere;
            r.from = start;
            r.to = i;
        }
    }
    return r;
}

int main() {
    auto r = maxSubarray({-2, 1, -3, 4, -1, 2, 1, -5, 4});
    std::cout << "sum " << r.sum << " from index " << r.from
              << " to " << r.to << "\n";
    auto s = maxSubarray({-3, -1, -7});
    std::cout << "sum " << s.sum << " from index " << s.from
```

```
                << " to " << s.to << "\n";
    return 0;
}
```

```
sum 6 from index 3 to 6
sum -1 from index 1 to 1
```

The key detail: `start` is updated **only when restarting**, and `r.from` is copied from it **only when a new best is found**. Updating `r.from` on every extension would be wrong, and updating `start` on every element would make it meaningless.

The variation: maximum product sub-array

> **DEFINITION** | **The problem**
> Same question, but with products. `{2, 3, -2, 4}` → **6**, from `{2, 3}`.

Kadane does not transfer directly, and the reason is instructive.

> **KEY INSIGHT** | **Why products need two running values**
> With sums, a large running total is good and a negative one is bad. With products, **a very negative running product is an asset**, because the next negative number will flip it to a very large positive.
> `{-2, 3, -4}`: the best product is $(-2)\times3\times(-4)=24$, and the running product was -6 at the middle. A Kadane that discards negatives would throw away exactly the value it needs.
> So you carry **both** the maximum and the minimum product ending here. On a negative element the two swap roles, because multiplying by a negative turns the largest into the smallest and vice versa.

Maximum product sub-array

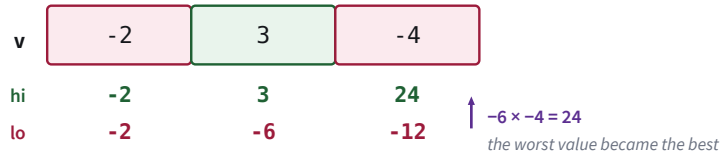
```cpp
#include <algorithm>
#include <iostream>
#include <vector>

long long maxProduct(const std::vector<int>& v) {
    long long best = v[0], hi = v[0], lo = v[0];
    for (int i = 1; i < (int)v.size(); ++i) {
        long long x = v[i];
        if (x < 0) std::swap(hi, lo);          // a negative swaps the
            roles
        hi = std::max(x, hi * x);
        lo = std::min(x, lo * x);
        best = std::max(best, hi);
    }
    return best;
}

int main() {
    std::cout << maxProduct({2, 3, -2, 4})  << "\n";
    std::cout << maxProduct({-2, 0, -1})    << "\n";
    std::cout << maxProduct({-2, 3, -4})    << "\n";
```

```
    std::cout << maxProduct({-2})           << "\n";
    return 0;
}
```

```
6
0
24
-2
```



with sums, a negative running total is useless — with products it is an asset in waiting

Figure 16.3 Why the product version carries two values. A negative element exchanges the largest and smallest running products, so the "worst" value a moment ago becomes the best one now. Zeroes reset both.

Note that `{-2, 0, -1}` gives 0: a zero wipes out both running values, and the `std::max(x, hi * x)` form handles it automatically by restarting at `x`.

The family of problems this shape solves

Problem	What `endingHere` means
Maximum sub-array sum	best sum ending at i
Maximum product sub-array	best and worst product ending at i
Best time to buy and sell stock	best profit if we sell on day i
Longest increasing run	length of the run ending at i
Maximum circular sub-array sum	Kadane twice: max, and total minus min
House robber	best loot using houses up to i

Table 16.2 Kadane is one instance of a general shape — *the best thing ending here, from the best thing ending just before here*. Recognising the shape is more useful than memorising the four lines.

> **KEY INSIGHT | The shape in general terms**
> Define a quantity that depends on **position i and ends there**. Find a rule expressing it from the value at $i-1$. Keep a single running variable, and a separate `best`.
> That is also the definition of one-dimensional dynamic programming — Kadane is usually a student's first DP algorithm, taught before the name. If the DP chapters of a later course feel sudden, come back and re-read this one.

Best time to buy and sell stock

The same shape, in different clothes.

Buy once, sell once, maximise the profit

```cpp
#include <algorithm>
#include <climits>
#include <iostream>
#include <vector>

int maxProfit(const std::vector<int>& price) {
    int cheapest = INT_MAX, best = 0;
    for (int p : price) {
        cheapest = std::min(cheapest, p);          // best day to have bought
        best = std::max(best, p - cheapest);       // profit if we sell today
    }
    return best;
}

int main() {
    std::cout << maxProfit({7, 1, 5, 3, 6, 4}) << "\n";
    std::cout << maxProfit({7, 6, 4, 3, 1})    << "\n";
    std::cout << maxProfit({2, 4, 1})          << "\n";
    return 0;
}
```

```
5
0
2
```

`best` is *"the best profit achievable selling on this day"* — which is exactly `endingHere`. The running state is the cheapest price so far instead of a running sum, but the structure is identical. Here 0 is the right initial value, because you are allowed not to trade at all.

CHAPTER RECAP | Chapter 16 in six lines

- Ask *"what is the best sub-array **ending here**?"* — that question removes the outer loop.
- `endingHere = max(v[i], endingHere + v[i])`: extend if the running sum helps, restart if it does not.
- Restarting happens exactly when the running sum has gone negative.
- Initialise from `v[0]`, not 0, unless the empty sub-array is explicitly allowed.
- To report the indices, update `start` only on a restart and copy it only on a new best.
- Products need **two** running values, because a negative element swaps the largest and smallest.

EXERCISES | **Practise**

1. Trace Kadane on `{5, -2, 3}` and on `{-1, -2, -3}`. Which rows restart?
2. Modify Kadane to allow the empty sub-array (answer 0 for all-negative input). Which two lines change?
3. Maximum sum of a **circular** sub-array, where the sub-array may wrap around the end. (Hint: either it does not wrap — plain Kadane — or it does, in which case the elements *not* taken form a contiguous minimum sub-array. What is the edge case when every element is negative?)
4. Longest increasing contiguous run. What is `endingHere` here, and what is the restart condition?
5. House robber: maximise the sum of a subset with no two adjacent elements. Define `endingHere` carefully — you will need two variables.
6. Explain, with `{-2, 3, -4}`, why maximum product needs both `hi` and `lo`.
7. Buy and sell stock with **two** transactions allowed. What does the running state become?
8. Why is Kadane often called the simplest example of dynamic programming? State the sub-problem and the recurrence explicitly.

CHAPTER

17

Pattern 10: Matrix Patterns

Grid problems, and the index arithmetic that makes them easy

CORE PREREQS: Chapters 7 and 13

Matrix problems look intimidating because they seem to need careful case analysis at every boundary. Almost always, they do not: the right move is to find the single index relationship that describes the whole transformation, or to shrink the grid layer by layer. This chapter covers rotation, spiral traversal, boundary walking, the set-matrix-zeroes trap, and searching a sorted grid.

WHAT YOU WILL LEARN

- Rotate a square matrix by 90° in place, via transpose and reflect
- Traverse a grid in spiral order with four moving boundaries
- Explain why "set matrix zeroes" cannot be done naively in one pass
- Search a row-and-column-sorted matrix in $O(R+C)$ using the staircase walk
- Flood-fill a region, and see why it is really a graph problem
- Derive index relationships instead of guessing them

Rotating a matrix by 90°

DEFINITION | **The problem**

Rotate an $n \times n$ matrix 90° **clockwise**, in place.

$$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix} \rightarrow \begin{pmatrix}7&4&1\\8&5&2\\9&6&3\end{pmatrix}$$

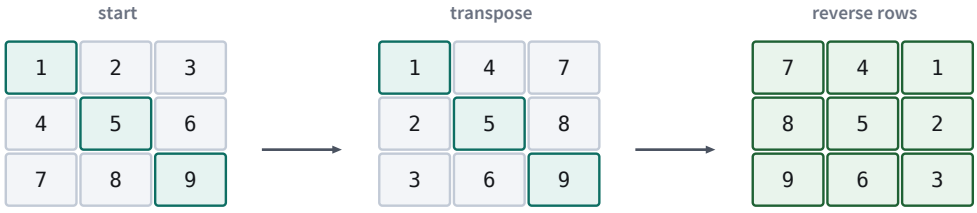
Finding the index relationship

Do not guess. Take one element and watch where it goes.

The element at $(0,0)$ — the value 1 — ends up at $(0,2)$, the top-right. The element at $(0,2)$ — the value 3 — ends up at $(2,2)$, the bottom-right. The element at $(2,2)$ — the value 9 — ends up at $(2,0)$.

So $(r, c) \rightarrow (c, n-1-r)$. Check it: $(0,0) \rightarrow (0,2)$ ✓. $(0,2) \rightarrow (2,2)$ ✓. $(1,0) \rightarrow (0,1)$: the value 4 does indeed end up in the top row, second column ✓.

You could implement that directly by rotating four elements at a time in cycles. It works, and the loop bounds are fiddly. There is a much easier route.



`(r, c) → (c, r) → (c, n−1−r)` which is exactly a 90° clockwise turn

both steps are things you can already write, and neither has an awkward boundary

Figure 17.1 Rotation as transpose then reflect. Transposing flips the matrix about its main diagonal; reversing each row then flips it left-to-right. The composition of the two is exactly a 90° clockwise rotation, and each step is code you already have.

KEY INSIGHT | Rotation = transpose, then reverse each row
Transposing sends $(r,c)\to(c,r)$. Reversing each row sends $(r,c)\to(r,n-1-c)$. Compose them: $(r,c)\to(c,r)\to(c,n-1-r)$ — exactly the relationship we derived. Both steps are things you can already write, both are in place, and neither has an awkward boundary case. This is a general tactic: **look for a decomposition into operations you have already got right.**

Rotate a square matrix 90° clockwise, in place

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

void rotate90(std::vector<std::vector<int>>& g) {
    int n = (int)g.size();
    for (int r = 0; r < n; ++r)                  // 1. transpose
        for (int c = r + 1; c < n; ++c)          //    upper triangle only
            std::swap(g[r][c], g[c][r]);
    for (auto& row : g)                          // 2. reverse each row
        std::reverse(row.begin(), row.end());
}

int main() {
    std::vector<std::vector<int>> g = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    rotate90(g);
    for (const auto& row : g) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }
    return 0;
}
```

```
7 4 1
8 5 2
9 6 3
```

$O(n^2)$ time — every element is touched a constant number of times — and $O(1)$ space.

COMMON PITFALL | Anti-clockwise is not the same two steps
Rotating **anti-clockwise** is transpose then reverse each **column** — or, equivalently, reverse each row *first* and then transpose. Doing the clockwise steps in the wrong order gives you the anti-clockwise rotation, which is a very easy mistake to make and an even easier one to test for.
Always check with a non-symmetric 2×2 before trusting it: `{{1,2},{3,4}}` must become `{{3,1},{4,2}}` clockwise.

Spiral traversal

DEFINITION | The problem
Visit every cell in spiral order: left to right across the top, down the right side, right to left across the bottom, up the left side, then inwards and repeat.

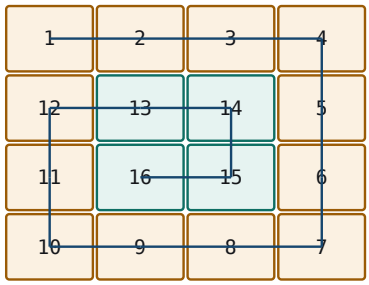


Figure 17.2 Spiral order as four shrinking boundaries. After each edge is consumed, its boundary moves inwards by one. The loop ends when the boundaries cross, which handles rectangular grids and single rows without any special case.

KEY INSIGHT | Four variables, not a pile of cases
Keep `top`, `bottom`, `left` and `right` — the edges of the untraversed rectangle. Each pass consumes one edge and moves that boundary inwards.
The only subtlety is the two `if` guards in the second half of the loop: after `++top`, the rectangle may already be empty, and walking the bottom row would then revisit the row you just consumed. Those guards are what make single-row and single-column grids work.

Spiral order traversal

```cpp
#include <iostream>
#include <vector>

std::vector<int> spiralOrder(const std::vector<std::vector<int>>& g) {
    std::vector<int> out;
    if (g.empty() || g[0].empty()) return out;
    int top = 0, bottom = (int)g.size() - 1;
    int left = 0, right = (int)g[0].size() - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; ++c) out.push_back(g[top][c]);
        ++top;
        for (int r = top; r <= bottom; ++r) out.push_back(g[r][right]);
        --right;
        if (top <= bottom) {                       // guard: any row left?
            for (int c = right; c >= left; --c) out.push_back(
                g[bottom][c]);
            --bottom;
        }
        if (left <= right) {                       // guard: any column
            left?
            for (int r = bottom; r >= top; --r) out.push_back(g[r][left]);
            ++left;
        }
    }
    return out;
}

int main() {
    for (int x : spiralOrder({{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}))
        std::cout << x << " ";
    std::cout << "\n";
    for (int x : spiralOrder({{1, 2, 3, 4}}))      // a single row
        std::cout << x << " ";
    std::cout << "\n";
    for (int x : spiralOrder({{1}, {2}, {3}}))     // a single column
        std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
1 2 3 6 9 8 7 4 5
1 2 3 4
1 2 3
```

Remove either guard and the single-row case prints `1 2 3 4 4 3 2 1`. Those two `if`s are the entire difficulty of the problem, and they are the reason to test non-square inputs.

Set matrix zeroes: the trap

> **DEFINITION** | **The problem**
> If any cell is 0, set its entire row and column to 0. In place.

WATCH OUT | Why the obvious loop is wrong

```
for (int r = 0; r < R; ++r)
    for (int c = 0; c < C; ++c)
        if (g[r][c] == 0) { /* zero out row r and column c now */ }
```

The zeroes you write are indistinguishable from the zeroes that were there originally. The very next iteration sees one of your new zeroes and clears another row — and the whole grid cascades to zero.
This is the same class of bug as deleting inside a forward loop (Chapter 6): **you are modifying the thing you are still reading.**

The fix is to separate *finding* from *applying*:

1. **First pass:** record which rows and which columns must be cleared.
2. **Second pass:** apply the clearing.

Two boolean arrays of size R and C cost $O(R+C)$ space, which is usually accepted. The $O(1)$-space version uses the matrix's own first row and first column as those marker arrays — the index-marking idea of Chapter 13, applied to a grid.

Set matrix zeroes, O(R + C) space

```
#include <iostream>
#include <vector>

void setZeroes(std::vector<std::vector<int>>& g) {
    int R = (int)g.size(), C = (int)g[0].size();
    std::vector<bool> clearRow(R, false), clearCol(C, false);

    for (int r = 0; r < R; ++r)                    // pass 1: find
        for (int c = 0; c < C; ++c)
            if (g[r][c] == 0) { clearRow[r] = true; clearCol[c] = true; }

    for (int r = 0; r < R; ++r)                    // pass 2: apply
        for (int c = 0; c < C; ++c)
            if (clearRow[r] || clearCol[c]) g[r][c] = 0;
}

int main() {
    std::vector<std::vector<int>> g = {{1, 1, 1}, {1, 0, 1}, {1, 1, 1}};
    setZeroes(g);
    for (const auto& row : g) {
        for (int x : row) std::cout << x << " ";
        std::cout << "\n";
    }
    return 0;
}
```

```
1 0 1
0 0 0
1 0 1
```

Searching a sorted matrix: the staircase walk

DEFINITION | The problem
A matrix has each **row** sorted left to right and each **column** sorted top to bottom. Find a target.

Binary searching every row is $O(R \log C)$. There is something better, and the idea is elegant.

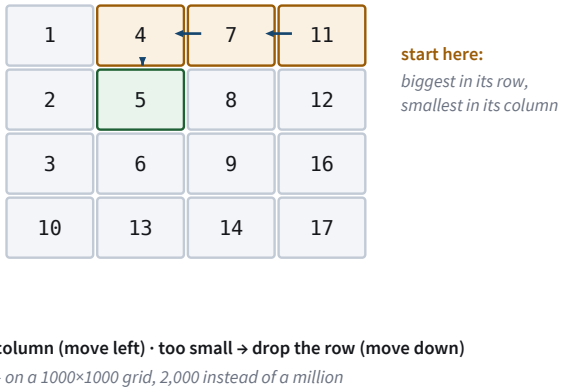


Figure 17.3 The staircase search. Starting at the top-right corner, every comparison eliminates an entire row or an entire column — because that corner is simultaneously the largest in its row and the smallest in its column.

KEY INSIGHT | Start at a corner where the comparison is decisive
At the **top-right** cell, the value is the largest in its row and the smallest in its column. So:

- target **smaller** than it → it cannot be anywhere in this column (everything below is larger). Discard the column: move left.
- target **larger** than it → it cannot be anywhere in this row (everything to the left is smaller). Discard the row: move down.

Either way one full row or column disappears, so at most $R+C$ steps are needed. The bottom-left corner works equally well by the mirror argument; the top-left and bottom-right corners do **not**, because there both comparisons leave two regions in play.

Search a row-and-column-sorted matrix in O(R + C)

```cpp
#include <iostream>
#include <vector>

bool searchMatrix(const std::vector<std::vector<int>>& g, int target) {
    if (g.empty() || g[0].empty()) return false;
    int r = 0, c = (int)g[0].size() - 1;        // start at the top-right
    while (r < (int)g.size() && c >= 0) {
        if (g[r][c] == target) return true;
        if (g[r][c] > target) --c;              // discard this column
        else                  ++r;              // discard this row
    }
    return false;
}
```

```
14
15 int main() {
16     std::vector<std::vector<int>> g = {{1,  4,  7, 11},
17                                        {2,  5,  8, 12},
18                                        {3,  6,  9, 16},
19                                        {10, 13, 14, 17}};
20     std::cout << searchMatrix(g, 5)  << "\n";
21     std::cout << searchMatrix(g, 20) << "\n";
22     std::cout << searchMatrix(g, 1)  << "\n";
23     std::cout << searchMatrix(g, 17) << "\n";
24     return 0;
25 }
```

```
1
0
1
1
```

$O(R + C)$ time, $O(1)$ space. On a 1000×1000 grid that is at most 2,000 steps instead of a million.

Flood fill and the four-neighbour walk

Many grid problems — counting islands, flood fill, shortest path through a maze — are **graph** problems wearing a grid costume. The grid is the graph; each cell is a node, and its up/down/left/right neighbours are its edges.

Flood fill with an explicit stack

```
1  #include <iostream>
2  #include <stack>
3  #include <vector>
4
5  void floodFill(std::vector<std::vector<int>>& g, int sr, int sc, int
   ↪      newColour) {
6      int R = (int)g.size(), C = (int)g[0].size();
7      int old = g[sr][sc];
8      if (old == newColour) return;                 // guard against
   ↪          looping forever
9
10     const int dr[4] = {-1, 1, 0, 0};              // the four directions
11     const int dc[4] = {0, 0, -1, 1};
12
13     std::stack<std::pair<int, int>> todo;
14     todo.push({sr, sc});
15     while (!todo.empty()) {
16         auto [r, c] = todo.top();
17         todo.pop();
18         if (r < 0 || r >= R || c < 0 || c >= C) continue;   // off the
   ↪              grid
19         if (g[r][c] != old) continue;                        // wrong
   ↪              colour
20         g[r][c] = newColour;                                 // paint it
21         for (int d = 0; d < 4; ++d)
```

```
            todo.push({r + dr[d], c + dc[d]});
        }
    }

    int main() {
        std::vector<std::vector<int>> g = {{1, 1, 1},
                                           {1, 1, 0},
                                           {1, 0, 1}};
        floodFill(g, 1, 1, 2);
        for (const auto& row : g) {
            for (int x : row) std::cout << x << " ";
            std::cout << "\n";
        }
        return 0;
    }
```

```
2 2 2
2 2 0
2 0 1
```

Two details worth stealing:

The direction arrays. `dr` / `dc` replace four near-identical blocks of code with one loop. For eight-way movement, extend them to eight entries and nothing else changes.

`if (old == newColour) return;` Without it, repainting a region in its existing colour never marks anything as done, and the stack grows until memory runs out. Painting *is* the visited-marker here, which is efficient and fragile — the guard is what makes it safe.

Cost summary

Problem	Time	Space	Technique
Rotate 90°	$O(n^2)$	$O(1)$	transpose + reverse rows
Spiral order	$O(R \cdot C)$	$O(1)$ extra	four shrinking boundaries
Set matrix zeroes	$O(R \cdot C)$	$O(R+C)$, or $O(1)$	find, then apply
Search sorted matrix	$O(R+C)$	$O(1)$	staircase from a corner
Flood fill / count islands	$O(R \cdot C)$	$O(R \cdot C)$ worst	stack or queue
Transpose	$O(n^2)$	$O(1)$	upper triangle only

CHAPTER RECAP | Chapter 17 in six lines

- Derive the index relationship from one concrete element; never guess it.
- Rotation 90° clockwise = transpose, then reverse each row.
- Spiral traversal is four boundaries moving inwards, plus two guards that make single rows and columns work.
- Never modify a grid you are still reading: find first, apply second.
- In a row-and-column-sorted matrix, start at the top-right so each comparison kills a whole row or column.
- Grid walking problems are graph problems; use `dr` / `dc` direction arrays and mark cells as visited.

EXERCISES | Practise

1. Derive the index relationship for an **anti-clockwise** 90° rotation by tracking (0,0), (0,2) and (2,2) in a 3×3 grid. Then write it as two steps.
2. Rotate a square matrix 180°. What is the simplest decomposition?
3. Write spiral traversal for a 3×5 grid by hand. At which point does the first guard matter?
4. Solve "set matrix zeroes" in $O(1)$ extra space using the first row and first column as markers. What is the one cell that needs special handling, and why?
5. In the staircase search, explain precisely why starting at the top-left corner fails. What is ambiguous there?
6. Count the number of islands — connected groups of 1s — in a grid of 0s and 1s. How do you avoid counting the same island twice?
7. Given a grid, return the cells in diagonal order. What index relationship describes a diagonal? (Hint: Chapter 7 gave you two.)
8. A 1000×1000 sorted matrix is searched a million times. Compare the staircase walk ($O(R+C)$ per query) with binary searching each row ($O(R \log C)$). Which wins, and does the answer change if the matrix is 10×100,000?

PART IV

The Array Problem Set

Twenty-five problems, graded from your first day to a Google on-site, each one solved the long way first so that you can watch the short way being discovered rather than recited.

18 Foundational Array Problems

19 Intermediate Array Problems

20 Advanced Array Problems

CHAPTER

18

Foundational Array Problems

Eleven problems that should become automatic

PRACTICE PREREQS: Part III

These eleven problems are the vocabulary of array work. None is hard once you see it, and every one of them appears inside harder problems later, so the goal here is not merely to solve them but to make them automatic — to reach the point where "find the second largest" produces a correct one-pass solution without conscious thought.

WHAT YOU WILL LEARN

- Solve all eleven foundational problems correctly, including their edge cases
- Recognise which pattern from Part III each one uses
- Give the brute force, the optimisation, and the reason for the optimisation
- Handle empty arrays, single elements, all-equal input and negative numbers
- Explain the XOR trick, and why it gives $O(1)$ space

How these are presented

Every problem follows the same six headings. Read **Think** before looking at the code, and try to finish the derivation yourself.

Heading	What it contains
Problem	the precise statement, including what the awkward inputs should do
Think	brute force, then the observation that improves it
Code	complete, compilable C++ with its real output
Cost	time and auxiliary space, with the reason
Watch for	the specific mistakes this problem invites
Next	a closely related problem to try alone

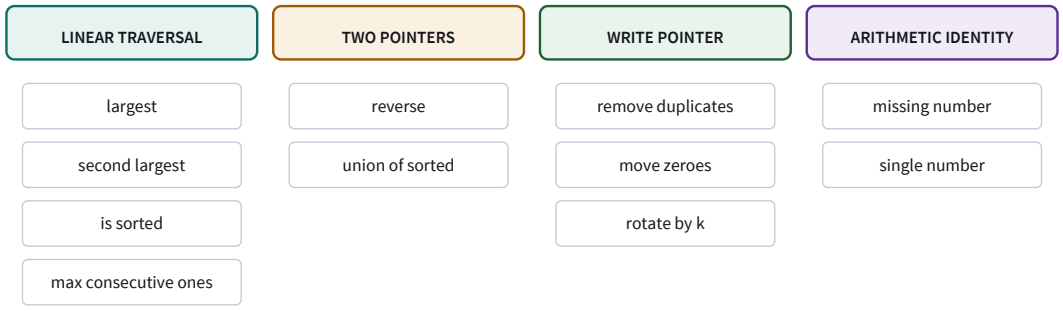


Figure 18.1 The eleven problems and the pattern each one rests on. Four of the ten patterns cover all of them, which is the point of having learned the patterns first.

1. Largest element

Problem. Return the largest value. Empty input has no answer — decide whether to throw or to return a sentinel, and say which.

Think. One pass, one variable. The only decision is the initial value, and it must be a real element so that all-negative arrays work.

Code. See Chapter 8. **Cost.** $O(n)$ / $O(1)$.

Watch for. `int best = 0;` fails on `{-5, -2}`.

Next. Largest *and* smallest in one pass, using $\lceil 3n/2 \rceil$ comparisons instead of $2n$ — compare elements in pairs first.

2. Second-largest element

Problem. Return the second-largest **distinct** value, or report that there is none. `{5, 5, 4}` → 4. `{7, 7, 7}` → none.

Think. Covered fully in Chapter 8. The two ideas: carry `best` and `second`; demote before you promote.

Cost. $O(n)$ / $O(1)$.

Watch for. Using `INT_MIN` as the "none" marker when `INT_MIN` could be a real value. Use a wider accumulator type.

Next. k-th largest distinct value for small k. At what k does this approach stop being sensible, and what replaces it?

3. Check whether an array is sorted

Problem. Return whether the array is non-decreasing. Empty and single-element arrays are sorted.

Think. Compare each element with its predecessor; one violation is enough. Covered in Chapter 8.

Cost. $O(n)$ / $O(1)$.

Watch for. Starting the loop at `i = 0` and reading `v[-1]`. Deciding whether equal neighbours are allowed — `{2, 2}` is sorted under "non-decreasing" and not under "strictly increasing". Ask.

Next. Check whether an array is a **rotated** sorted array, such as `{3, 4, 5, 1, 2}`. How many "descents" may there be, and where?

4. Reverse an array

Problem. Reverse in place.

Think. Converging two pointers, swapping and stepping inwards. Covered in Chapter 6.

Cost. $O(n)$ / $O(1)$ — exactly $\lfloor n/2 \rfloor$ swaps.

Watch for. Looping to `n` instead of `n/2`, which reverses twice and changes nothing.

Next. Reverse only a given range `[i, j]`. This is the building block of the rotation trick in Chapter 13.

5. Remove duplicates from a sorted array

Problem. Remove repeats in place, keeping order; return the new length.

Think. Sorted input means duplicates are adjacent, so you only compare with the last value you kept. Write pointer. Covered in Chapter 6.

Cost. $O(n)$ / $O(1)$.

Watch for. Assuming it works on unsorted data — it does not.

Next. Allow each value to appear at most **twice**. Compare against `v[w - 2]`, and think carefully about where `w` starts.

6. Move zeroes to the end

Problem. Move all zeroes to the end, preserving the order of the non-zero values, in place.

Think. Write pointer: compact the non-zeroes, then pad. Covered in Chapter 6.

Cost. $O(n)$ / $O(1)$.

Watch for. Swapping the non-zeroes into place changes which elements move but not the complexity; both are acceptable, and you should be able to compare them.

Next. Move all instances of a given value to the **front** instead, with the rest in order.

7. Rotate an array by k

Problem. Rotate right by k in place. k may exceed n.

Think. Three reversals, because $(AB)^{\mathrm{R}}=B^{\mathrm{R}}A^{\mathrm{R}}$. Covered in Chapter 13.

Cost. $O(n)$ / $O(1)$.

Watch for. Forgetting `k %= n`, which reads past the end.

Next. Rotate a **matrix** by 90° — Chapter 17 uses the same decompose-into-known-operations tactic.

8. Union of two sorted arrays

Problem. Return the sorted union of two sorted arrays, without duplicates.

Think. Two pointers, one per array, always advancing the one with the smaller front value. Covered in Chapter 15.

Cost. $O(n+m)$ time, $O(n+m)$ for the output.

Watch for. Duplicates within one array as well as across both; the `out.back() != x` guard handles both at once. And do not forget the two tail loops — when one array runs out, the other still has elements.

Next. The **intersection**, and the **difference** $A \setminus B$. All three are the same loop with different branches.

9. Missing number

Problem. An array holds n distinct values from the range $0 \ldots n$, so exactly one is missing. Find it. `{3, 0, 1}` → 2.

Think. Sorting is $O(n \log n)$; a seen-array is $O(n)$ space. Both work. But there are two $O(n)$-time, $O(1)$-space solutions, and they are worth knowing because the same tricks reappear.

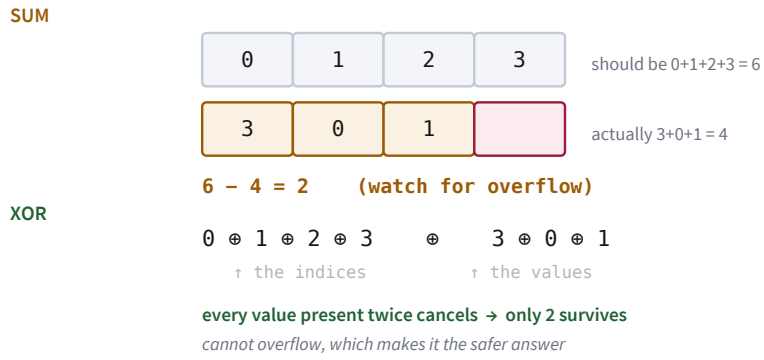


Figure 18.2 Two ways to find the missing number. The sum method subtracts what is present from what should be present. The XOR method cancels every value that appears in both lists, leaving only the one that does not.

KEY INSIGHT | The sum trick and the XOR trick
Sum. The numbers $0 \ldots n$ add up to $n(n+1)/2$. Subtract the actual sum and the difference is the missing value. Simple, but the expected sum can overflow `int` for large n — use `long long`.
XOR. Exclusive-or has two properties that make it ideal: $x \oplus x = 0$ and $x \oplus 0 = x$. So if you XOR together *all* the indices $0 \ldots n$ **and** all the array values, every number that appears in both cancels to zero, and only the missing one survives.
XOR cannot overflow, which makes it the more robust answer — and it is the key to problem 11 as well.

Missing number, two ways

```cpp
#include <iostream>
#include <vector>

int missingBySum(const std::vector<int>& v) {
    long long n = (long long)v.size();
    long long expected = n * (n + 1) / 2;     // 0 + 1 + ... + n
    long long actual = 0;
    for (int x : v) actual += x;
    return (int)(expected - actual);
}

int missingByXor(const std::vector<int>& v) {
    int result = (int)v.size();               // start with n itself
    for (int i = 0; i < (int)v.size(); ++i)
        result ^= i ^ v[i];                   // every present value
            cancels
    return result;
}

int main() {
    std::vector<int> v = {3, 0, 1};
    std::cout << missingBySum(v) << " " << missingByXor(v) << "\n";
    std::vector<int> w = {0, 1};
    std::cout << missingBySum(w) << " " << missingByXor(w) << "\n";
    std::vector<int> z = {1};                 // 0 is missing
    std::cout << missingBySum(z) << " " << missingByXor(z) << "\n";
    return 0;
}
```

```
2 2
2 2
0 0
```

Cost. $O(n)$ / $O(1)$ for both.

Watch for. Overflow in the sum version. Forgetting that the range is $0 \ldots n$ inclusive, which is $n+1$ possible values for n array slots — that asymmetry is the whole problem.

Next. Two numbers are missing from $1 \ldots n$. The sum alone is no longer enough; what second equation do you need?

10. Maximum consecutive ones

Problem. Longest run of 1s in a binary array.

Think. Current-run and best-run. Covered in Chapter 8.

Cost. $O(n)$ / $O(1)$.

Watch for. Updating `best` only when the run ends, which loses a run that reaches the end of the array.

Next. Allow flipping at most one 0 — that becomes a sliding window (Chapter 10), and generalises to k flips with no extra difficulty.

11. Single number

Problem. Every value appears exactly twice except one, which appears once. Find it. `{4, 1, 2, 1, 2}` → 4.

Think. A hash map solves it in $O(n)$ time and $O(n)$ space. The interviewer will then ask for $O(1)$ space, and XOR is the answer.

> **KEY INSIGHT | Why XOR solves it in one line**
> XOR is **commutative** and **associative**, so the order of operations does not matter. XOR-ing the whole array therefore groups the pairs however it likes, and each pair becomes $x \oplus x = 0$. Everything cancels except the lonely value, which ends XOR-ed with 0 and so survives unchanged.
>
> $$4 \oplus 1 \oplus 2 \oplus 1 \oplus 2 = 4 \oplus (1 \oplus 1) \oplus (2 \oplus 2) = 4 \oplus 0 \oplus 0 = 4$$

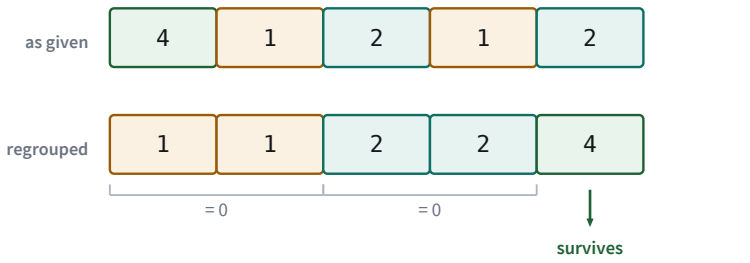


Figure 18.3 XOR cancellation. Rearranging the array so pairs sit together is allowed because XOR is commutative and associative — and once rearranged, every pair vanishes.

Single number in one line

```cpp
#include <iostream>
#include <vector>

int singleNumber(const std::vector<int>& v) {
    int result = 0;
    for (int x : v) result ^= x;     // pairs cancel; the lonely value
        survives
```

```
    return result;
}

int main() {
    std::cout << singleNumber({4, 1, 2, 1, 2}) << "\n";
    std::cout << singleNumber({2, 2, 7})       << "\n";
    std::cout << singleNumber({99})            << "\n";
    std::cout << singleNumber({-3, 5, 5})      << "\n";   // negatives
        are fine
    return 0;
}
```

```
4
7
99
-3
```

Cost. $O(n)$ time, $O(1)$ space.

Watch for. This works **only** when every other value appears exactly *twice*. If the others appear three times, XOR no longer cancels them, and you need a different technique (counting the bits set at each position, modulo 3). Check the exact wording of the problem.

Next. Two values appear once and the rest twice. (Hint: XOR everything, find any bit where the result is 1, and use that bit to split the array into two groups — each containing one of the loners.)

The whole set, at a glance

#	Problem	Pattern	Time	Space
1	Largest element	linear traversal	$O(n)$	$O(1)$
2	Second largest	linear traversal	$O(n)$	$O(1)$
3	Is sorted	linear traversal	$O(n)$	$O(1)$
4	Reverse	two pointers	$O(n)$	$O(1)$
5	Remove duplicates (sorted)	write pointer	$O(n)$	$O(1)$
6	Move zeroes	write pointer	$O(n)$	$O(1)$
7	Rotate by k	three reversals	$O(n)$	$O(1)$
8	Union of sorted arrays	two pointers	$O(n+m)$	$O(n+m)$ out
9	Missing number	sum or XOR	$O(n)$	$O(1)$
10	Max consecutive ones	linear traversal	$O(n)$	$O(1)$
11	Single number	XOR	$O(n)$	$O(1)$

Table 18.1 The eleven foundational problems. Four patterns cover all of them, and every one is $O(n)$ or better with $O(1)$ space.

INTERVIEW ANGLE | **What "foundational" means to an interviewer**
Nobody will hire you for solving these. But everybody will *decline* to hire you for getting one wrong — because an interviewer reads a bug in "reverse an array" as a signal about everything you will write afterwards.
The bar here is not cleverness; it is **reliability**. You should be able to write any of these, correctly, first time, while talking. That only comes from writing them more than once.

EXERCISES | **Consolidate**

1. Write all eleven from memory, with no reference, and test each one on: the empty array, one element, all elements equal, and all elements negative.
2. For each problem, name its pattern without looking at the table.
3. Which three could be solved by sorting first? For each, say what sorting costs you and whether it is ever worth it.
4. Which two use a mathematical identity rather than a traversal strategy?
5. Take problems 5 and 6 and write a single function that does both in one pass over a sorted array.
6. For problem 9, work out the largest n for which `int` arithmetic is safe in the sum method. Does the XOR method have an equivalent limit?

CHAPTER

19

Intermediate Array Problems

Ten problems where the good solution is not the obvious one

PRACTICE PREREQS: Chapter 18

These are the problems that interviews are actually built from. Each has an obvious $O(n^2)$ solution and a linear one that requires an idea, and in every case the idea is one of the ten patterns. Work through them with the derivations covered up; the goal is to reach the point where you can find the idea, not recall it.

WHAT YOU WILL LEARN

- Solve the ten standard intermediate array problems
- Derive each linear solution from its brute force
- Apply Boyer–Moore voting, Dutch National Flag, and Floyd's cycle detection to arrays
- Build prefix and suffix products without using division
- Explain why merging two sorted arrays in place must work backwards
- Choose correctly between hashing, sorting and two pointers under stated constraints

1. Two Sum

Problem. Return the **indices** of two elements summing to a target. Exactly one solution exists; you may not use an element twice.

Think. Brute force is every pair, $O(n^2)$. The improvement: you are not searching for *any* partner, you are searching for a *specific* one, namely $target - x$. That is a lookup, not a search. Covered in Chapter 14.

Cost. $O(n)$ time, $O(n)$ space.

Watch for. Look up before inserting, or an element pairs with itself. If the array were sorted and *values* were wanted instead of indices, two pointers would give $O(1)$ space — be ready to say that.

2. Best Time to Buy and Sell Stock

Problem. One buy and one later sell. Maximise the profit; 0 if no profitable trade exists.

Think. Brute force tries every pair of days, $O(n^2)$. The insight: when considering selling on day i, the only thing that matters about the past is the **cheapest price so far**. Carry it. Covered in Chapter 16.

Cost. $O(n)$ / $O(1)$.

Watch for. You must buy before you sell, so update the running minimum *and* the running best in the right order. Returning a negative profit when the prices only fall — 0 is usually correct, but confirm.

3. Maximum sub-array sum

Problem. Largest sum of any contiguous non-empty sub-array.

Think. Kadane: ask *"what is the best sub-array ending here?"* Covered in Chapter 16.

Cost. $O(n)$ / $O(1)$.

Watch for. Initialising to 0 breaks all-negative input.

4. Majority element

Problem. One value appears **more than** $n/2$ times. Find it. You may assume it exists.

Think. A frequency map is $O(n)$ time and $O(n)$ space. Sorting puts the majority at the middle, $O(n \log n)$ and $O(1)$ space. Both are fine answers. The $O(n)$ time *and* $O(1)$ space answer is Boyer–Moore voting, and it is beautiful.

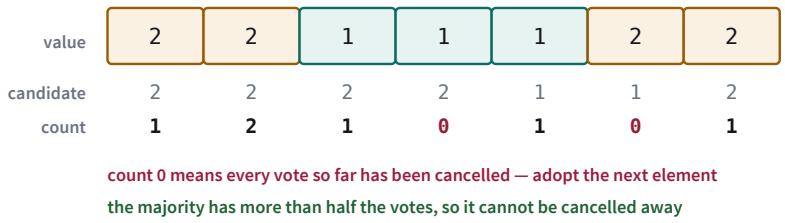


Figure 19.1 Boyer–Moore voting. Think of each element as a vote, and of two differing votes as cancelling each other out. Because the majority has more than half the votes, it cannot be fully cancelled by all the others combined — so whatever survives must be it.

KEY INSIGHT | Why the voting algorithm works
Keep a `candidate` and a `count`. On each element: if `count` is 0, adopt the element as the new candidate; then add 1 if it matches the candidate and subtract 1 if it does not.
The pairing argument: every decrement pairs one majority element with one non-majority element and removes both from consideration. There are fewer than $n/2$ non-majority elements and more than $n/2$ majority ones, so the non-majority elements run out first. Whatever is left standing is the majority.
This only works if a majority genuinely exists. With `{1, 2, 3}` it returns some arbitrary value. If existence is not guaranteed, do a second pass to verify the candidate really does appear more than $n/2$ times — and say so unprompted.

Boyer-Moore majority vote, with verification

```cpp
#include <iostream>
#include <vector>

// Returns the majority element, or -1 if there is none.
int majorityElement(const std::vector<int>& v) {
    int candidate = 0, count = 0;
    for (int x : v) {                       // pass 1: find the survivor
        if (count == 0) candidate = x;
        count += (x == candidate) ? 1 : -1;
    }
    int freq = 0;                           // pass 2: verify it
    for (int x : v) if (x == candidate) ++freq;
    return freq > (int)v.size() / 2 ? candidate : -1;
}

int main() {
    std::cout << majorityElement({3, 2, 3})             << "\n";
    std::cout << majorityElement({2, 2, 1, 1, 1, 2, 2}) << "\n";
    std::cout << majorityElement({1, 2, 3})             << "\n";  // none
    std::cout << majorityElement({7})                   << "\n";
    return 0;
}
```

```
3
2
-1
7
```

Cost. $O(n)$ time, $O(1)$ space, two passes.

Next. All values appearing more than $n/3$ times. There can be at most two, and the algorithm generalises to two candidates and two counters.

5. Sort an array of 0s, 1s and 2s

Problem. Sort an array containing only the values 0, 1 and 2, in one pass with $O(1)$ space.

Think. Counting the three values and rewriting is two passes and perfectly acceptable. One pass requires the **Dutch National Flag** algorithm, which is three pointers rather

than two.

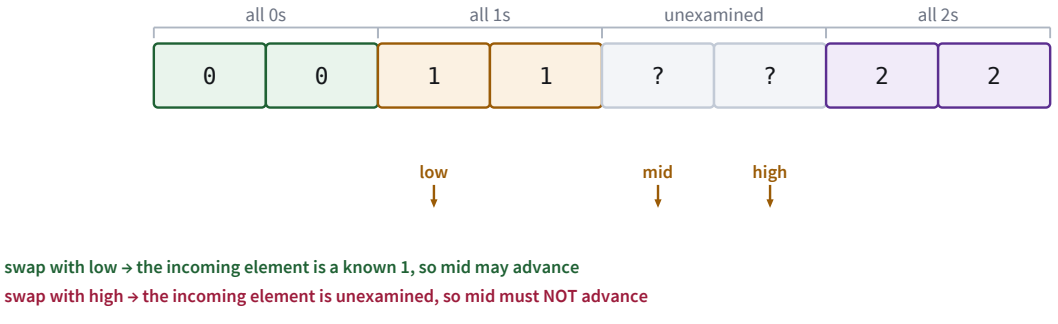


Figure 19.2 The Dutch National Flag partition. Three boundaries divide the array into four regions: settled 0s, settled 1s, unexamined, and settled 2s. The unexamined region shrinks to nothing, and each element is placed in one step.

KEY INSIGHT | **The three-region invariant**

- `v[0 .. low-1]` are all 0
- `v[low .. mid-1]` are all 1
- `v[mid .. high]` is unexamined
- `v[high+1 .. n-1]` are all 2

Look at `v[mid]`:

- **0** → swap it into the 0 region: `swap(v[low], v[mid]); ++low; ++mid;`
- **1** → it is already in the right region: `++mid;`
- **2** → swap it into the 2 region: `swap(v[mid], v[high]); --high;`

The asymmetry matters. After swapping with `low`, the incoming element is known to be a 1 (it came from the 1 region), so `mid` can advance. After swapping with `high`, the incoming element has **never been examined**, so `mid` must stay put and look at it next time.
Getting that asymmetry right is the entire problem.

Dutch National Flag: one pass, O(1) space

```cpp
#include <iostream>
#include <vector>

void sortColours(std::vector<int>& v) {
    int low = 0, mid = 0, high = (int)v.size() - 1;
    while (mid <= high) {
        if (v[mid] == 0)      { std::swap(v[low], v[mid]); ++low; ++mid; }
        else if (v[mid] == 1) { ++mid; }
        else                  { std::swap(v[mid], v[high]); --high; }
    }
}

int main() {
    std::vector<int> v = {2, 0, 2, 1, 1, 0};
```

```
    sortColours(v);
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> w = {2, 0, 1};
    sortColours(w);
    for (int x : w) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> z = {0, 0, 0};
    sortColours(z);
    for (int x : z) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
0 0 1 1 2 2
0 1 2
0 0 0
```

Cost. $O(n)$ / $O(1)$, one pass.

Watch for. `while (mid <= high)`, not `<`. And do **not** advance `mid` after swapping with `high`.

6. Merge two sorted arrays in place

Problem. `a` has m real elements followed by n empty slots; `b` has n elements. Merge `b` into `a`, sorted, in place.

Think. Merging from the front would overwrite elements of `a` that have not been placed yet. The fix is to fill from the **back**, where the slots are already empty.

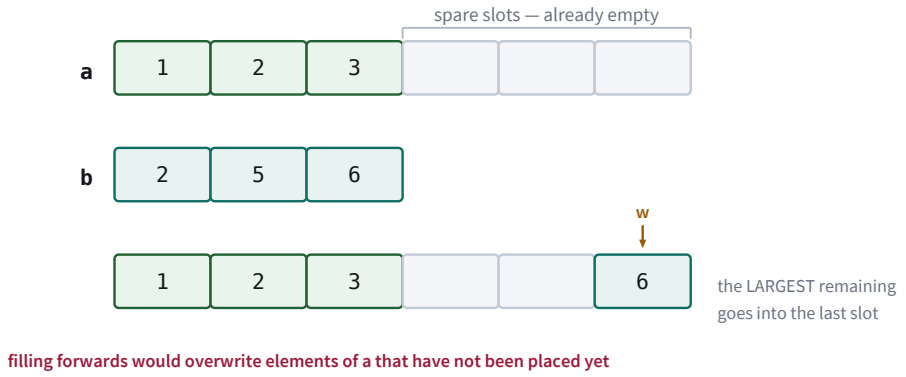


Figure 19.3 Merging backwards. The spare slots are at the end, so writing the **largest** remaining element each time never overwrites anything that still matters.

Merge b into a, filling from the back

```
#include <iostream>
#include <vector>
```

```cpp

void merge(std::vector<int>& a, int m, const std::vector<int>& b, int n) {
    int i = m - 1, j = n - 1, w = m + n - 1;   // w: the last empty slot
    while (j >= 0) {                           // while b has elements
        left
        if (i >= 0 && a[i] > b[j]) a[w--] = a[i--];
        else                       a[w--] = b[j--];
    }
    // No tail loop for a: anything left in a is already in the right
        place.
}

int main() {
    std::vector<int> a = {1, 2, 3, 0, 0, 0};
    std::vector<int> b = {2, 5, 6};
    merge(a, 3, b, 3);
    for (int x : a) std::cout << x << " ";
    std::cout << "\n";

    std::vector<int> c = {0};
    std::vector<int> d = {1};
    merge(c, 0, d, 1);                         // a is empty
    for (int x : c) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

```
1 2 2 3 5 6
1
```

Cost. $O(m+n)$ time, $O(1)$ space.

Watch for. The loop condition is `j >= 0`, not `i >= 0 && j >= 0`. If `a` runs out first, the rest of `b` must still be copied; if `b` runs out first, the rest of `a` is already in place and needs no work. That asymmetry is why there is only one tail case.

7. Find the duplicate number

Problem. $n+1$ values, each in $1 \dots n$, with exactly one value repeated. Find it **without modifying the array** and in $O(1)$ space.

Think. Sorting modifies. A hash set uses $O(n)$. The constraints are deliberately ruling out every easy answer, which is a signal that an unusual idea is wanted.

KEY INSIGHT | The array is a linked list in disguise
Read `v[i]` as *"the next index after i"*. Starting from index 0 and following $i \to v[i]$ gives a sequence. Because every value is in $1 \dots n$, the sequence can never leave the array — so it must eventually repeat, which means it contains a **cycle**.
The duplicated value is exactly the index that two different positions point to, which makes it the **entry point of the cycle**.
So this is cycle detection, and Floyd's tortoise-and-hare algorithm finds the entry point in $O(n)$ time and $O(1)$ space. The same algorithm appears for real linked lists in Chapter 30; here it is being applied to an array that merely behaves like one.

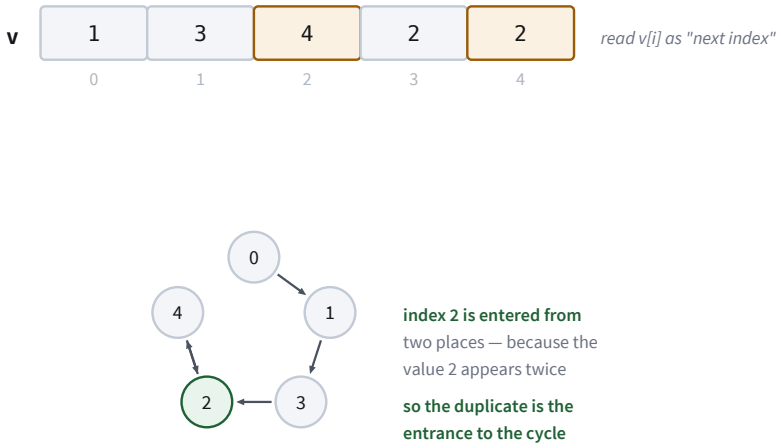


Figure 19.4 The array read as a linked list. Index i points to index `v[i]`. Two positions holding the same value point to the same place, which creates a cycle whose entrance is the duplicated number.

Find the duplicate with Floyd's cycle detection

```cpp
#include <iostream>
#include <vector>

int findDuplicate(const std::vector<int>& v) {
    int slow = v[0], fast = v[0];
    do {                                    // phase 1: find a meeting point
        slow = v[slow];
        fast = v[v[fast]];
    } while (slow != fast);

    slow = v[0];                            // phase 2: find the cycle
        entrance
    while (slow != fast) {
        slow = v[slow];
        fast = v[fast];
    }
    return slow;
}

int main() {
    std::cout << findDuplicate({1, 3, 4, 2, 2}) << "\n";
    std::cout << findDuplicate({3, 1, 3, 4, 2}) << "\n";
    std::cout << findDuplicate({2, 2, 2, 2, 2}) << "\n";
    return 0;
```

```
24  }
```

```
2
3
2
```

Cost. $O(n)$ time, $O(1)$ space, array unmodified.

Watch for. Phase 2 is **not** optional, and both pointers move at the *same* speed in it. Chapter 30 proves why that finds the entrance.

8. Product of array except self

Problem. Return an array where `out[i]` is the product of every element except `v[i]`. **Division is not allowed**, and the answer must be $O(n)$.

Think. Division would make it trivial — but it fails when any element is 0, which is exactly why the problem forbids it.

> **KEY INSIGHT | Split the product in two**
> Everything except `v[i]` is *everything to the left of i* multiplied by *everything to the right of i*. Both are prefix products — one built left to right, one built right to left. Chapter 11's idea, with multiplication instead of addition.
> The trick that gets it to $O(1)$ extra space: write the left products into the output array during the first pass, then sweep right to left carrying a single running product and multiply it in. No second array is needed.



Figure 19.5 Prefix and suffix products. Each output cell is the product of everything before it times the product of everything after it — two passes, no division, and the zeroes take care of themselves.

Product except self, O(n) time, O(1) extra space

```
#include <iostream>
#include <vector>

std::vector<long long> productExceptSelf(const std::vector<int>& v) {
    int n = (int)v.size();
    std::vector<long long> out(n, 1);
```

```cpp

    long long left = 1;                     // pass 1: products of the
        left
    for (int i = 0; i < n; ++i) {
        out[i] = left;
        left *= v[i];
    }
    long long right = 1;                    // pass 2: multiply in the
        right
    for (int i = n - 1; i >= 0; --i) {
        out[i] *= right;
        right *= v[i];
    }
    return out;
}

int main() {
    for (long long x : productExceptSelf({1, 2, 3, 4})) std::cout << x <<
        " ";
    std::cout << "\n";
    for (long long x : productExceptSelf({-1, 1, 0, -3, 3})) std::cout <<
        x << " ";
    std::cout << "\n";
    return 0;
}
```

```
24 12 8 6
0 0 9 0 0
```

Cost. $O(n)$ time; $O(1)$ extra space, not counting the output.

Watch for. Note how the zero case works out with no special handling: the single 0 contributes to every other cell's product and is itself excluded. A division-based solution would need an explicit count of zeroes and three branches.

9. Sub-array sum equals K

Problem. Count contiguous sub-arrays summing to exactly K. Values may be negative.

Think. Prefix sums plus a hash map. The negatives rule out a sliding window. Covered in Chapter 11.

Cost. $O(n)$ time, $O(n)$ space.

Watch for. Seed the map with `{0: 1}`; look up before inserting.

10. Longest sub-array with a given sum

Problem. Return the **length** of the longest contiguous sub-array summing to K.

Think. Same prefix-sum idea, but the map stores *where* a prefix value was first seen rather than how many times.

KEY INSIGHT | Keep the earliest occurrence only
For a *count* you want every occurrence. For the *longest* you want the **earliest**, because an earlier start makes a longer sub-array.
So insert into the map only if the prefix value is not already there. One `if` is the difference between the two problems.

Longest sub-array with sum exactly K

```cpp
#include <algorithm>
#include <iostream>
#include <unordered_map>
#include <vector>

int longestSubarrayWithSum(const std::vector<int>& v, long long k) {
    std::unordered_map<long long, int> firstSeen;   // prefix value -> index
    firstSeen[0] = -1;                              // empty prefix, before index 0
    long long running = 0;
    int best = 0;
    for (int i = 0; i < (int)v.size(); ++i) {
        running += v[i];
        auto it = firstSeen.find(running - k);
        if (it != firstSeen.end())
            best = std::max(best, i - it->second);
        if (firstSeen.find(running) == firstSeen.end())
            firstSeen[running] = i;                 // EARLIEST occurrence only
    }
    return best;
}

int main() {
    std::cout << longestSubarrayWithSum({1, 2, 3, 1, 1, 1, 1}, 3) << "\n";
    std::cout << longestSubarrayWithSum({-1, 1, 1, -1, 2}, 2)     << "\n";
    std::cout << longestSubarrayWithSum({1, 2, 3}, 100)           << "\n";
    return 0;
}
```

```
3
5
0
```

Cost. $O(n)$ time, $O(n)$ space.

Watch for. `firstSeen[0] = -1`, not 0 — the empty prefix sits *before* index 0, and using 0 makes every length one too short.

The whole set, at a glance

#	Problem	Brute force	Best	Idea
1	Two Sum	$O(n^2)$	$O(n)$	complement lookup
2	Buy and sell stock	$O(n^2)$	$O(n)$	running minimum
3	Max sub-array sum	$O(n^2)$	$O(n)$	Kadane
4	Majority element	$O(n^2)$	$O(n)$, $O(1)$ sp.	Boyer–Moore voting
5	Sort 0s, 1s, 2s	$O(n \log n)$	$O(n)$ one pass	Dutch flag, 3 pointers
6	Merge sorted in place	$O((m+n)\log)$	$O(m+n)$	fill from the back
7	Find the duplicate	$O(n^2)$	$O(n)$, $O(1)$ sp.	Floyd's cycle detection
8	Product except self	$O(n^2)$	$O(n)$	prefix × suffix
9	Sub-array sum = K	$O(n^2)$	$O(n)$	prefix + hash map
10	Longest sub-array sum K	$O(n^2)$	$O(n)$	prefix + earliest index

Table 19.1 The ten intermediate problems. Note how many of them have an $O(n^2)$ answer that is correct and a linear answer that needs one specific idea.

EXERCISES | Consolidate

1. For each of the ten, state the brute force and the single observation that improves it, without looking.
2. Problems 4 and 7 both achieve $O(1)$ space by a non-obvious argument. Write out both arguments in your own words.
3. Why does problem 6 fail when filled from the front? Construct a small input that breaks it.
4. Problem 8 forbids division. Write the division-based solution anyway, then list every input that breaks it.
5. Problems 9 and 10 differ by one `if`. State exactly which, and why.
6. Three Sum: find all distinct triples summing to zero. Which two patterns combine, and what is the complexity?
7. Problem 5 generalises to k distinct values. At what k does the three-pointer approach stop being practical, and what replaces it?

CHAPTER

20

Advanced Array Problems

Four problems that reward thinking and punish memorisation

ADVANCED PREREQS: Chapter 19

These four are the ones that separate candidates. Each has a short solution that looks obvious once seen and is almost impossible to guess — and in every case there is a derivation that makes it inevitable rather than magical. Trapping Rain Water in particular is worth an hour of your life, because the reasoning behind its two-pointer solution is the most transferable argument in this book.

WHAT YOU WILL LEARN

- Search a rotated sorted array in $O(\log n)$ by deciding which half is sorted
- Find the first and last position of a value with two modified binary searches
- Solve Trapping Rain Water four ways, and derive the two-pointer argument
- Solve maximum product sub-array, and say why it needs two running values
- Explain, for each, why the obvious solution fails and what replaces it

1. Search in a rotated sorted array

DEFINITION | The problem
A sorted array was rotated at some unknown pivot: `{4,5,6,7,0,1,2}`. Find a target in $O(\log n)$. All values are distinct.

Why the obvious thing fails. Plain binary search compares the target with `v[mid]` and concludes which side to go. Here that conclusion is invalid: the array is not sorted, so `v[mid] < target` tells you nothing about where the target is.

KEY INSIGHT | One half is always properly sorted
Cut a rotated sorted array anywhere. The rotation point lies in exactly one of the two halves — so **the other half is an ordinary sorted array**.
Which one? Compare `v[lo]` with `v[mid]`:

- `v[lo] <= v[mid]` → the **left** half is sorted.
- otherwise → the **right** half is sorted.

Once you know which half is sorted, you can test membership in it reliably, because within a sorted range a simple range check is exact: *is* `v[lo] <= target < v[mid]` *?*

- **Yes** → the target is in the sorted half; search there.
- **No** → it can only be in the other half; search there instead.

Either way, half the array is eliminated, so the whole thing is $O(\log n)$.

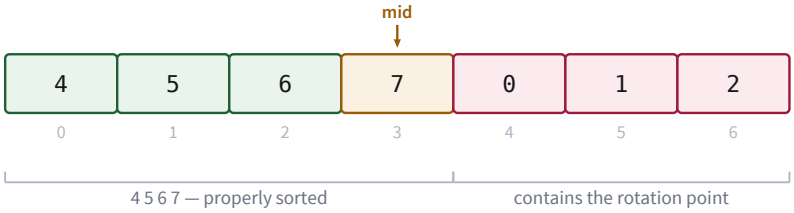


Figure 20.1 A rotated sorted array cut at the midpoint. Whichever side does not contain the rotation point is a plain sorted range, and inside a sorted range a range check settles the question exactly.

Search in a rotated sorted array, O(log n)

```cpp
#include <iostream>
#include <vector>

int searchRotated(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == target) return mid;

        if (v[lo] <= v[mid]) {                        // LEFT half is sorted
            if (v[lo] <= target && target < v[mid]) hi = mid - 1;
            else                                    lo = mid + 1;
        } else {                                      // RIGHT half is sorted
            if (v[mid] < target && target <= v[hi])  lo = mid + 1;
            else                                     hi = mid - 1;
        }
    }
    return -1;
```

```
}

int main() {
    std::vector<int> v = {4, 5, 6, 7, 0, 1, 2};
    std::cout << searchRotated(v, 0) << "\n";
    std::cout << searchRotated(v, 4) << "\n";
    std::cout << searchRotated(v, 3) << "\n";
    std::cout << searchRotated({1}, 1)    << "\n";
    std::cout << searchRotated({3, 1}, 1) << "\n";
    return 0;
}
```

```
4
0
-1
0
1
```

Cost. $O(\log n)$ time, $O(1)$ space.

> **COMMON PITFALL | The three details that break this**
> `v[lo] <= v[mid]`, **not** `<`. When `lo == mid` (a two-element range) the two are equal, and `<` would send you down the wrong branch. Test `{3, 1}` searching for 1 — it is the smallest input that catches this.
> **The range checks must use the right endpoints.** The left-sorted branch checks against `v[lo]` and `v[mid]`; the right-sorted branch checks against `v[mid]` and `v[hi]`. Mixing them up produces code that works on half the inputs.
> **Duplicates break the whole argument.** With `{1,1,1,0,1}`, `v[lo] == v[mid]` tells you nothing about which half is sorted. The standard fix is: when `v[lo] == v[mid] == v[hi]`, increment `lo` and decrement `hi` and try again — which degrades the worst case to $O(n)$. If the problem allows duplicates, say this out loud; it is the point of the variant.

Next. Find the **minimum** of a rotated sorted array in $O(\log n)$. The same "which half is sorted" question, with a simpler answer.

2. First and last position of an element

Problem. In a sorted array with duplicates, return the first and last index of a target, or `{-1, -1}`.

Think. Covered in Chapter 12: on a match, record the index and keep searching — leftwards for the first, rightwards for the last.

Cost. $O(\log n)$ for both, $O(1)$ space.

Watch for. Scanning outwards from a match found by plain binary search is tempting and is $O(n)$ in the worst case — on an array that is entirely the target value, it scans everything. Two modified binary searches stay logarithmic whatever the input.

In production you would write `std::equal_range`, which returns both bounds in one call.

3. Trapping Rain Water

DEFINITION | The problem
Each element is the height of a bar of width 1. After rain, how much water is trapped between the bars?
`{0,1,0,2,1,0,1,3,2,1,2,1}` → **6** units.

```
water[i]  =  max(0,  min(maxLeft[i], maxRight[i])  −  h[i])
```

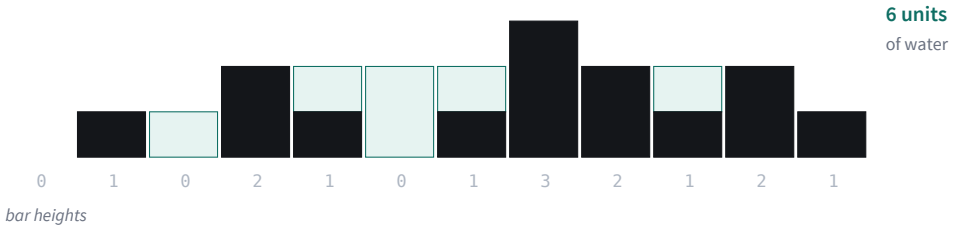


Figure 20.2 Trapping Rain Water. Above each bar, the water level is set by the **shorter** of the tallest bar to its left and the tallest bar to its right. Subtract the bar's own height and you have that column's contribution.

Step 1: solve one column

Forget the whole array. Ask about a single position i: how much water sits **directly above bar** i?

Water at i is held in by the tallest bar somewhere to its left and the tallest bar somewhere to its right. The water level is the **lower** of those two — the shorter wall is where it would spill over. So:

$$\text{water}[i] = \max\left(0, \min\left(\text{maxLeft}[i], \text{maxRight}[i]\right) - h[i]\right)$$

The $\max(0, \cdot)$ handles the case where the bar is itself the tallest, so nothing sits on it.

That formula is the entire problem. Everything that follows is about computing it efficiently.

Step 2: four implementations

A. Brute force — $O(n^2)$. For each i, scan left for the maximum, scan right for the maximum, apply the formula.

B. Precomputed arrays — $O(n)$ time, $O(n)$ space. The scans repeat work, so precompute `maxLeft` and `maxRight` with two passes — prefix maxima, exactly Chapter 11's idea with `max` instead of `+`.

C. Two pointers — $O(n)$ time, $O(1)$ space. The clever one, below.

D. Monotonic stack — $O(n)$ time, $O(n)$ space. Fills the water in horizontal layers instead of vertical columns. Correct and elegant; it is not simpler, and the two-pointer argument generalises better.

B — precomputed prefix maxima, the clearest O(n) solution

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

long long trapTwoArrays(const std::vector<int>& h) {
    int n = (int)h.size();
    if (n < 3) return 0;                        // need a wall on each side
    std::vector<int> maxLeft(n), maxRight(n);

    maxLeft[0] = h[0];
    for (int i = 1; i < n; ++i) maxLeft[i] = std::max(maxLeft[i - 1],
        h[i]);

    maxRight[n - 1] = h[n - 1];
    for (int i = n - 2; i >= 0; --i) maxRight[i] = std::max(maxRight[i +
        1], h[i]);

    long long total = 0;
    for (int i = 0; i < n; ++i)
        total += std::min(maxLeft[i], maxRight[i]) - h[i];
    return total;
}

int main() {
    std::cout << trapTwoArrays({0,1,0,2,1,0,1,3,2,1,2,1}) << "\n";
    std::cout << trapTwoArrays({4,2,0,3,2,5})             << "\n";
    std::cout << trapTwoArrays({3,2,1})                   << "\n";
    std::cout << trapTwoArrays({})                        << "\n";
    return 0;
}
```

```
6
9
0
0
```

Note that the final loop needs no `std::max(0, ...)`: since `maxLeft[i]` and `maxRight[i]` both include `h[i]` itself, their minimum is never below `h[i]`. Spotting that is a small thing, and noticing *why* it is safe is the kind of reasoning that makes code trustworthy.

Step 3: removing the two arrays

KEY INSIGHT | Why two pointers suffice

Put `left` at 0 and `right` at $n-1$, and carry `maxLeft` and `maxRight` as running values — but only of the parts already seen.

The problem: at `left`, you know `maxLeft` exactly, but you only know a *lower bound* on the true maximum to the right. Normally that is not enough.

Here is the resolution. Suppose `maxLeft < maxRight`. The true maximum to the right of `left` is at least `maxRight`, hence strictly greater than `maxLeft`. So min(true left max, true right max) is `maxLeft` — the right-hand side cannot be the limiting wall, whatever else is out there.

The shorter side is always the one you know exactly. So process that side, and the uncertainty about the other side never matters.

That is the same argument as "container with most water" in Chapter 9, and it is worth recognising as a reusable move: *when one of two unknowns cannot possibly be the binding constraint, you can act on the other with certainty.*

C — two pointers, O(n) time and O(1) space

```cpp
#include <iostream>
#include <vector>

long long trap(const std::vector<int>& h) {
    int left = 0, right = (int)h.size() - 1;
    int maxLeft = 0, maxRight = 0;
    long long total = 0;
    while (left < right) {
        if (h[left] < h[right]) {              // the left wall is the
            shorter
            if (h[left] >= maxLeft) maxLeft = h[left];
            else                    total += maxLeft - h[left];
            ++left;
        } else {                               // the right wall is the
            shorter
            if (h[right] >= maxRight) maxRight = h[right];
            else                      total += maxRight - h[right];
            --right;
        }
    }
    return total;
}

int main() {
    std::cout << trap({0,1,0,2,1,0,1,3,2,1,2,1}) << "\n";
    std::cout << trap({4,2,0,3,2,5})             << "\n";
    std::cout << trap({3,2,1})                   << "\n";
    std::cout << trap({2,0,2})                   << "\n";
    std::cout << trap({})                        << "\n";
    return 0;
}
```

```
6
9
0
2
```

```
0
```

Approach	Time	Space	Comment
A · rescan for each column	$O(n^2)$	$O(1)$	the honest starting point
B · two prefix-max arrays	$O(n)$	$O(n)$	clearest; derive this first
C · two pointers	$O(n)$	$O(1)$	the expected answer
D · monotonic stack	$O(n)$	$O(n)$	fills in horizontal layers

Table 20.1 Trapping Rain Water, four ways. All four are worth being able to describe; the two-pointer version is the one to be able to write.

> **INTERVIEW ANGLE | How to present this problem**
> Do **not** jump to the two-pointer version. The strong performance is:
>
> > *"For a single column, the water is* $\min(\text{maxLeft}, \text{maxRight}) - h[i]$*. Naively that is* $O(n^2)$*. I can precompute both maxima in two passes for O(n) time and O(n) space. And I can drop the arrays entirely with two pointers, because whichever side is shorter is the side whose maximum I know exactly — so its water is already determined."*
>
> That shows the formula, the optimisation, and the insight, in that order. A candidate who writes the two-pointer code immediately has either seen it before or is guessing, and the interviewer cannot tell which.

Watch for. `n < 3` returns 0 — you need a wall on each side of a gap. The comparison `h[left] < h[right]` decides which side to process, and getting it backwards produces answers that are right on symmetric inputs and wrong otherwise.

4. Maximum product sub-array

Problem. Largest product of any contiguous non-empty sub-array. `{2,3,-2,4}` → 6.

Think. Covered in Chapter 16. The key observation: a very negative running product is an asset, because the next negative flips it to a large positive. So carry both the maximum and the minimum product ending here, and swap them when the current element is negative.

Cost. $O(n)$ / $O(1)$.

Watch for. Zeroes reset both running values — the `std::max(x, hi * x)` form handles this automatically. Overflow is real: 30 elements of value 10 overflow `int` easily; use `long long` and read the constraints.

Next. Maximum product of a **non-contiguous** subsequence. It becomes a different problem entirely — count the negatives and the zeroes. Try it.

Where to go from here

Problem	Technique	Natural follow-up
Search rotated	binary search + which-half-is-sorted	minimum of a rotated array; rotated with duplicates
First/last position	two modified binary searches	count occurrences; `equal_range`
Trapping Rain Water	per-column formula, then two pointers	largest rectangle in a histogram
Max product sub-array	two running values	maximum circular sub-array sum

Table 20.2 These four, and the natural follow-up each one leads to. Working the follow-ups is worth more than finding four new problems.

CHAPTER RECAP | Part IV in five lines

- Every problem here has an $O(n^2)$ solution that is correct. Say it first, then improve it.
- The improvement is always one of the ten patterns, not a new idea.
- Constraints are hints: "do not modify the array", "$O(1)$ space", "no division" each rule out the easy answer and point at the intended one.
- Derive the formula for a *single position* before worrying about the whole array — that is what makes Trapping Rain Water tractable.
- When one of two unknowns cannot be the binding constraint, you may act on the other with certainty. That one argument solves three problems in this book.

EXERCISES | Practise

1. Trace `searchRotated({4,5,6,7,0,1,2}, 0)`. At each step, say which half is sorted and why.
2. Find the minimum of a rotated sorted array in $O(\log n)$. What is the comparison that drives it?
3. Write Trapping Rain Water approach A, then B, then C, in that order, testing each against the others on random inputs.
4. For `{4,2,0,3,2,5}`, compute the water above **each** column by hand using the formula, and check that they sum to 9.
5. Implement Trapping Rain Water with a monotonic stack. How do the layers it fills differ from the columns of the other three methods?
6. Why does `searchRotated` degrade to $O(n)$ when duplicates are allowed? Construct the worst case.
7. Largest rectangle in a histogram: for each bar, find how far it can extend left and right before hitting a shorter bar. Which of the four Trapping Rain Water approaches does this most resemble?
8. For maximum product sub-array, construct an input where the answer uses an even number of negatives and one where it must stop before a zero.

PART V

Pointers and Memory

Linked lists are impossible without pointers and trivial with them. So we stop, and we learn pointers properly — addresses, the heap, nullptr, ownership and leaks — before going on.

21 Pointers, References and Dereferencing

22 The Stack, the Heap, new and delete

CHAPTER

21

Pointers, References and Dereferencing

A variable whose value is an address

FOUNDATION PREREQS: Chapter 2

Almost nobody struggles with linked lists. They struggle with pointers, and then meet linked lists before anybody has explained pointers properly. This chapter does that: what a pointer is, what `&` and `*` really do, why `nullptr` exists, how `->` differs from `.`, and the exact difference between passing by value, by reference and by pointer. Nothing here is advanced, and nothing after it works without it.

WHAT YOU WILL LEARN

- Say what a pointer stores, in one sentence
- Use `&` and `*` correctly, and explain why they are opposites
- Read any pointer declaration out loud
- Explain `nullptr`, and what happens when you dereference it
- Use `->` on a pointer to a struct, and say why `.` would not compile
- Choose between passing by value, by reference and by pointer, with a reason
- Recognise a dangling pointer and an uninitialised pointer

The one-sentence definition

DEFINITION | **Pointer**
A **pointer** is a variable whose value is a memory address.
That is the whole idea. A normal `int` variable holds a number like 42. A pointer holds a number like 1000, and that number is understood as *"the place where something lives"*.

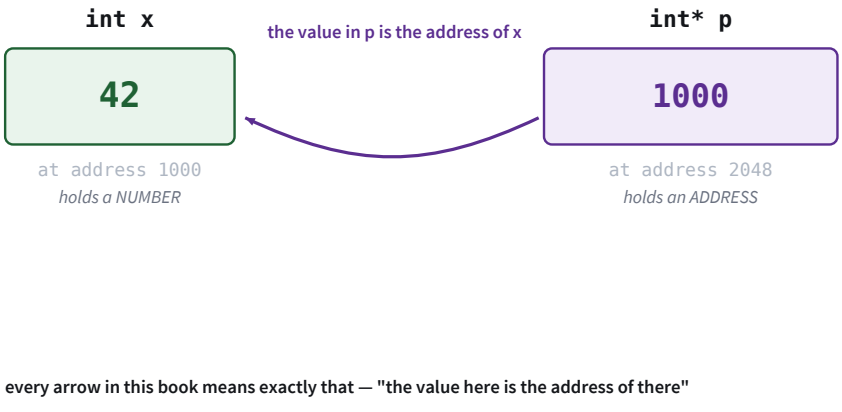


Figure 21.1 A variable and a pointer to it. `x` holds 42 and lives at address 1000. `p` holds 1000 and lives somewhere else entirely. The arrow in every diagram in this book means "the value in this box is the address of that box".

Your first pointer

```cpp
#include <iostream>

int main() {
    int x = 42;
    int* p = &x;          // p holds the ADDRESS of x

    std::cout << "x  = " << x  << "\n";   // 42       - the value
    std::cout << "&x = " << &x << "\n";   // 0x...    - where x lives
    std::cout << "p  = " << p  << "\n";   // the same address
    std::cout << "*p = " << *p << "\n";   // 42       - follow p, read
        what is there
    return 0;
}
```

A possible run:

```
x  = 42
&x = 0x7ffd3b2a1c4c
p  = 0x7ffd3b2a1c4c
*p = 42
```

`p` and `&x` print the same thing, because that is exactly what `p` was set to. And `*p` prints 42, because following the address leads to `x`.

The two operators

Everything about pointers is these two symbols, and they are **opposites**.

DEFINITION | `&` and `*`
`&x` — the **address-of** operator. Read it as *"the address of x"*. It takes a variable and gives you where it lives.
`*p` — the **dereference** operator. Read it as *"the thing at p"*. It takes an address and gives you the variable there.
They undo each other: `*(&x)` is just `x`.

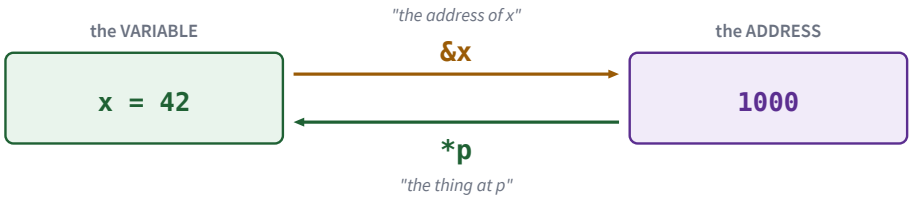


Figure 21.2 The two operators as a round trip. `&` goes from a variable to its address; `*` goes from an address back to the variable. Applying one and then the other returns you to where you started.

COMMON PITFALL | `*` means two different things

```
int* p = &x;      // in a DECLARATION: "p is a pointer to int"
*p = 99;          // in an EXPRESSION:  "the thing at p"
```

Same character, unrelated jobs. In a declaration it is part of the type. In an expression it is the dereference operator.

This is genuinely confusing and there is no trick — you simply learn to ask *"am I declaring something, or doing something?"*

One more wrinkle. These two declarations are identical:

```
int* a;
int *b;
```

The space means nothing. This book writes `int* p` because it groups the `*` with the type, which is how you should *think* about it. But beware:

```
int* a, b;        // a is int*, b is a plain int. The * applies to a
    only.
```

Declare one pointer per line and the problem never arises.

Dereferencing changes the original

This is the point of pointers, so it deserves its own demonstration.

Writing through a pointer

```
#include <iostream>

int main() {
    int x = 42;
    int* p = &x;

    *p = 99;                    // write 99 to WHEREVER p points - that is
        x

```

```
    std::cout << "x  = " << x  << "\n";
    std::cout << "*p = " << *p << "\n";
    return 0;
}
```

```
x  = 99
*p = 99
```

We never mentioned `x` on the line that changed it. We changed the thing at address `p`, and that thing *is* `x`. This is how a function can modify a variable belonging to its caller, and it is how a linked list node can be rewired without copying anything.

nullptr: a pointer to nothing

> **DEFINITION | nullptr**
> `nullptr` is a special pointer value meaning **"this points at nothing"**.
> It is not address 0 in any meaningful sense, and it is not an uninitialised pointer. It is a deliberate, checkable marker for *absent*.
> Use it for: the end of a linked list; a function that may fail to find something; an optional reference to an object that might not exist.

nullptr, and checking before you follow

```
#include <iostream>

int main() {
    int* p = nullptr;

    if (p == nullptr) std::cout << "p points at nothing\n";
    if (!p)           std::cout << "and this is the same check\n";

    int x = 7;
    p = &x;                          // now it points somewhere real
    if (p) std::cout << "*p = " << *p << "\n";
    return 0;
}
```

```
p points at nothing
and this is the same check
*p = 7
```

A pointer converts to `bool` as *"do I point at something?"*, so `if (p)` and `if (p != nullptr)` are the same. Both are common; pick one and be consistent.

WATCH OUT | Dereferencing nullptr

```
int* p = nullptr;
std::cout << *p;        // undefined behaviour - typically a crash
```

Following a pointer to nothing is undefined behaviour. In practice you get a segmentation fault, which at least tells you immediately.
The rule that prevents it: before you write `*p` or `p->something`, satisfy yourself that `p` is not null. In linked list code this becomes the habit of writing `while (curr != nullptr)` rather than `while (true)`, and it is why Chapters 25 and 26 check for the empty list before touching anything.

NOTE | Use `nullptr`, not `NULL` or `0`
`NULL` is inherited from C and is really the integer 0 in disguise, which means it can select the wrong overload and produce surprising errors.
`nullptr` has its own type and cannot be confused with an integer. It has been the correct choice since C++11. This book uses it everywhere, and so should you.

Pointers to structs, and the arrow

A pointer can point to anything, including a struct.

The arrow operator

```
#include <iostream>
#include <string>

struct Point {
    int x;
    int y;
};

int main() {
    Point a{3, 4};
    Point* p = &a;

    std::cout << a.x << " " << a.y << "\n";        // on the object: dot
    std::cout << (*p).x << " " << (*p).y << "\n";  // dereference, then
        dot
    std::cout << p->x << " " << p->y << "\n";      // shorthand for the
        above

    p->x = 10;                                    // writes through the
        pointer
    std::cout << a.x << "\n";
    return 0;
}
```

```
3 4
3 4
3 4
```

```
10
```

DEFINITION | `.` versus `->`
`a.x` — `a` **is** the object. Use the dot.
`p->x` — `p` is a **pointer** to the object. Use the arrow.
`p->x` is exactly shorthand for `(*p).x`: dereference, then take the member. The brackets are required in the long form because `.` binds tighter than `*`, so `*p.x` would try to take `.x` of the pointer and fail to compile.
Writing `p.x` on a pointer is a compile error, and the error message says something like *"request for member 'x' in 'p', which is of pointer type"*. When you see that, the fix is always the same: change the dot to an arrow.

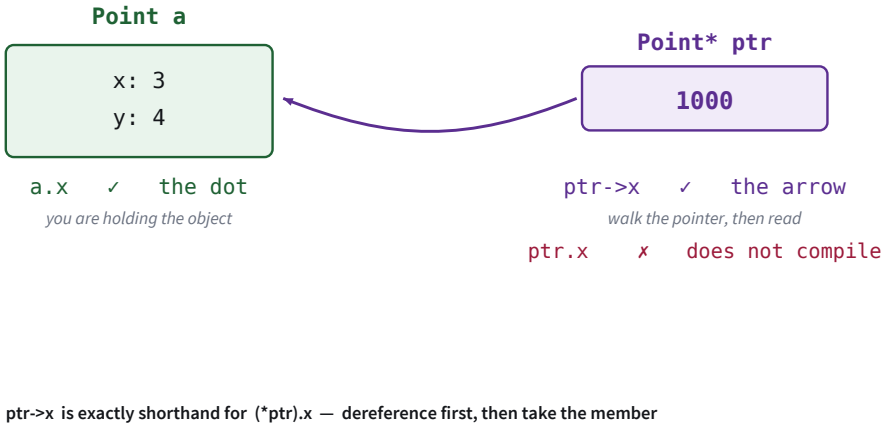


Figure 21.3 Dot versus arrow. The dot reads a member of an object you are holding; the arrow walks the pointer first and then reads the member. Every linked list traversal in this book is a chain of arrows.

This is the operator you will use most in Part VI. `node->next` means *"go to the node, and read its `next` field"*, and `node->next->next` means *"go two nodes along"*.

References: an alias, not a pointer

C++ has a second way to refer to an existing object.

References compared with pointers

```cpp
#include <iostream>

int main() {
    int x = 42;

    int& r = x;         // r IS x - another name for the same bytes
    int* p = &x;        // p POINTS AT x - a separate variable holding an
        address

    r = 99;             // no star needed: r is x
    std::cout << x << "\n";

    *p = 7;             // star needed: follow the address first
    std::cout << x << " " << r << "\n";

```

```
    std::cout << &x << " == " << &r << "\n";   // the same address
    return 0;
}
```

A possible run:

```
99
7 7
0x7ffc1a2b3c4c == 0x7ffc1a2b3c4c
```

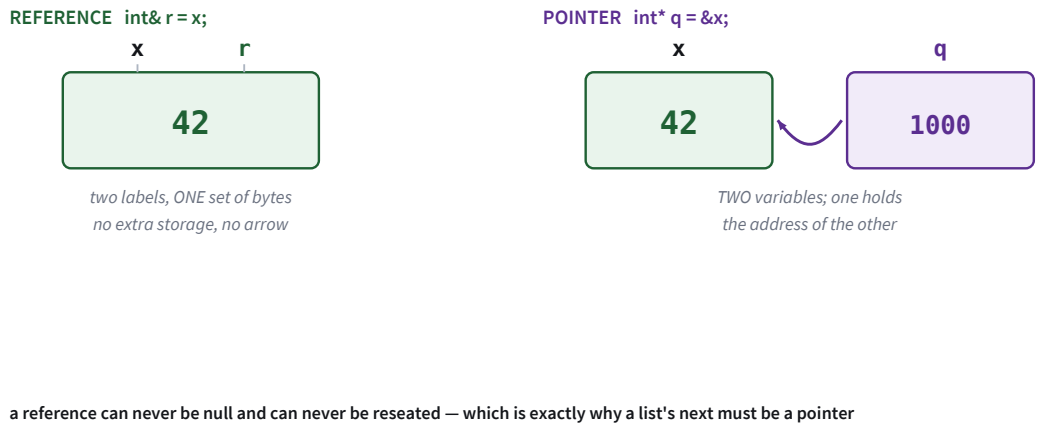


Figure 21.4 A reference and a pointer. The reference is a second label on the same bytes — there is no extra storage and no arrow to follow. The pointer is a separate variable whose contents happen to be an address.

	Reference `int&`	Pointer `int*`
Can be null	no	yes (`nullptr`)
Must be initialised	yes, always	no (and that is a hazard)
Can be reseated later	no, bound for life	yes
Syntax to use the value	`r`	`*p`
Syntax for a member	`r.x`	`p->x`
Pointer arithmetic	no	yes
Use it for	a parameter you will not rebind	optional things, linked structures

Table 21.1 Reference versus pointer. The short version: a reference is simpler and safer, so prefer it; a pointer is more capable, so use it when you need absence or rebinding.

> **KEY INSIGHT | When each one is right**
> Use a **reference** when the thing definitely exists and you will refer to that same thing for the whole of its life. Function parameters are the main case.
> Use a **pointer** when the thing might be absent (`nullptr`), or when you need to change what you are referring to (walk a list), or when you are managing a heap object.
> A linked list node's `next` is a pointer and could not be a reference: it has to be able to mean "nothing follows", and it has to be reassignable.

Passing to functions: three ways

This is the most practically important section in the chapter.

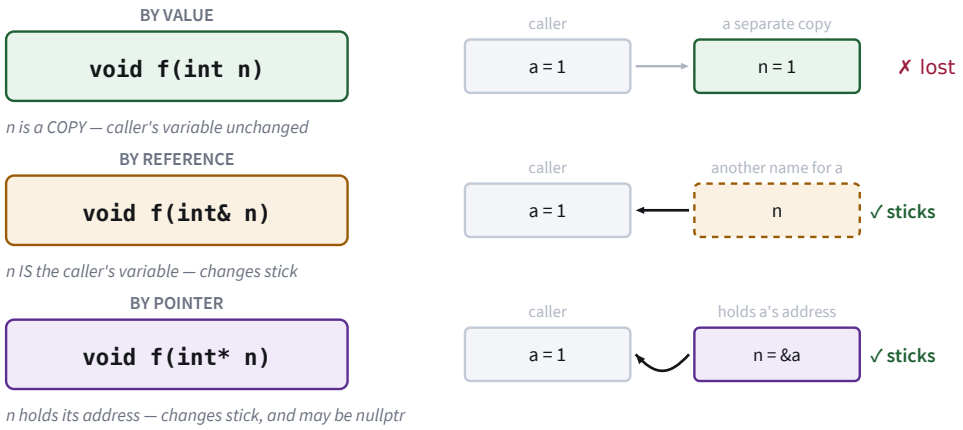


Figure 21.5 The three ways to pass an argument. By value copies it, so changes are lost. By reference names the caller's variable, so changes stick. By pointer passes the address, so changes stick and the argument may be absent.

By value, by reference, by pointer

```cpp
#include <iostream>

void byValue(int n)     { n = 99; }      // changes a COPY - caller unaffected
void byReference(int& n){ n = 99; }      // changes the caller's variable
void byPointer(int* n)  { if (n) *n = 99; }   // same, but may be given nullptr

int main() {
    int a = 1, b = 1, c = 1;

    byValue(a);
    byReference(b);
    byPointer(&c);                       // note the & at the call site

    std::cout << a << " " << b << " " << c << "\n";

    byPointer(nullptr);                  // legal, and does nothing
    std::cout << "survived the nullptr call\n";
    return 0;
}
```

```
1 99 99
survived the nullptr call
```

`a` is unchanged because `byValue` received a copy. `b` and `c` changed because both functions were given a way back to the original.

COMMON PITFALL | Copying large objects by accident

```
void process(std::vector<int> v);          // copies all n elements: O(
    n) per call
void process(const std::vector<int>& v);   // no copy, and cannot modify
    it
void process(std::vector<int>& v);         // no copy, and may modify it
```

The first version is a silent $O(n)$ cost on every call. For a vector of a million integers, that is 4 MB copied each time.

The rule this book follows:

- small types (`int` , `double` , a pointer) → **by value**, it is cheapest;
- containers and structs you only read → `const&` ;
- anything you intend to modify → `&` , or `*` if it may be absent.

Pointer arithmetic, and the link back to arrays

Pointers and arrays are more closely related than they look.

An array name is almost a pointer

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* p = arr;                    // the array name converts to &arr[0]

    std::cout << *p << "\n";         // 10
    std::cout << *(p + 2) << "\n";   // 30 - two ELEMENTS along, not two
        bytes
    std::cout << p[2] << "\n";       // 30 - exactly the same thing
    std::cout << *(arr + 3) << "\n"; // 40 - and this works on the array
        too

    // Walking with a pointer instead of an index
    for (int* q = arr; q != arr + 5; ++q)
        std::cout << *q << " ";
    std::cout << "\n";
    return 0;
}
```

```
10
30
30
40
10 20 30 40 50
```

KEY INSIGHT | `arr[i]` *is* `*(arr + i)`
The subscript operator is defined as pointer arithmetic. `arr[2]` means *"start at* `arr`*, move 2 elements, read"* — which is exactly $B+i\times S$ from Chapter 2.
Adding 1 to an `int*` moves **4 bytes**, not 1. The compiler scales by `sizeof(int)` automatically, which is why `p + 2` lands on element 2 rather than somewhere in the middle of element 0.
A curiosity that follows: since `arr[i]` is `*(arr + i)` and addition commutes, `2[arr]` is legal C++ and means `arr[2]`. Never write it. Knowing why it works means you have understood the definition.

The three pointer bugs

WATCH OUT | **The ones that will actually happen to you**
Uninitialised pointer.

```
int* p;            // contains whatever bytes were there
*p = 42;           // writes 42 to a random address
```

It may crash, or it may silently corrupt something else. Always initialise — to `nullptr` if you have nothing better.
Dangling pointer — pointing at something that no longer exists.

```
int* p;
{
    int local = 5;
    p = &local;
}                  // local is destroyed here
std::cout << *p;   // p now points at reclaimed memory
```

Very often this still prints 5, because nothing has reused the bytes yet. It will fail later, somewhere else, for no visible reason. This is also what happens to every pointer into a `std::vector` after it reallocates (Chapter 5), and to every pointer to a node after you `delete` it (Chapter 22).
Null dereference.

```
Node* n = findNode(head, 42);   // returns nullptr when not found
std::cout << n->data;           // crash if it was not found
```

The fix is the habit: check before you follow.

What you should take away

CHAPTER RECAP | Chapter 21 in seven lines

- A pointer is a **variable whose value is an address**.
- `&x` gives the address of `x`; `*p` gives the thing at `p`. They are opposites.
- `*` in a declaration is part of the type; `*` in an expression dereferences.
- `nullptr` means "points at nothing". Check it before you follow a pointer.
- `p->x` is `(*p).x`. Dot for objects, arrow for pointers.
- A **reference** is another name for an existing object: no null, no reseating, no star.
- Pass small things by value, read-only containers by `const&`, and things you will modify by `&` or `*`.

EXERCISES | Practise

1. Write a program with `int x = 5;` and `int* p = &x;` that prints `x`, `&x`, `p`, `*p` and `&p`. Explain in words what each of the five is.
2. Predict the output of the snippet below the box, then run it.
3. Write `swap(int* a, int* b)` using pointers, then write it again using references. Which is easier to call, and which could be given `nullptr`?
4. Why does `p.x` fail to compile when `p` is a `Point*`? What is the exact fix?
5. Given `int arr[4]` and `int* p = arr;`, what is the difference between `p + 1` and `(char*)p + 1`? How many bytes does each move?
6. Explain the difference between a pointer that is `nullptr` and a pointer that is dangling. Which one is more dangerous, and why?
7. Write a function that takes a pointer to an `int` and sets it to 0 **only if** the pointer is not null. Why can this function not be written with a reference parameter?
8. Draw the memory diagram for `int x = 1; int* p = &x; int** pp = &p;`. What is `**pp`? Where would you need a pointer to a pointer in real code?

For exercise 2 — predict before you run

```
int a = 1, b = 2;
int* p = &a;
*p = 10;
p = &b;
*p = 20;
std::cout << a << " " << b;
```

CHAPTER

22

The Stack, the Heap, new and delete

Where objects live, who owns them, and how memory leaks

FOUNDATION PREREQS: Chapter 21

Every linked list node you will ever create lives on the heap, which means you are responsible for its life and its death. This chapter explains the two regions of memory, what `new` and `delete` really do, what a memory leak is and why it is invisible, and the three bugs — leak, dangling pointer, double free — that heap code invites. It ends with the ownership rules this book follows.

WHAT YOU WILL LEARN

- Describe what lives on the stack and what lives on the heap, and why
- Use `new` and `delete`, and `new[]` and `delete[]`, correctly
- Explain what a memory leak is and why the program keeps working anyway
- Recognise dangling pointers, double frees and use-after-free
- State who owns a heap object, and what ownership obliges you to do
- Know what smart pointers do, and why this book still uses raw pointers

Two regions

Chapter 2 previewed this. Now we do it properly.

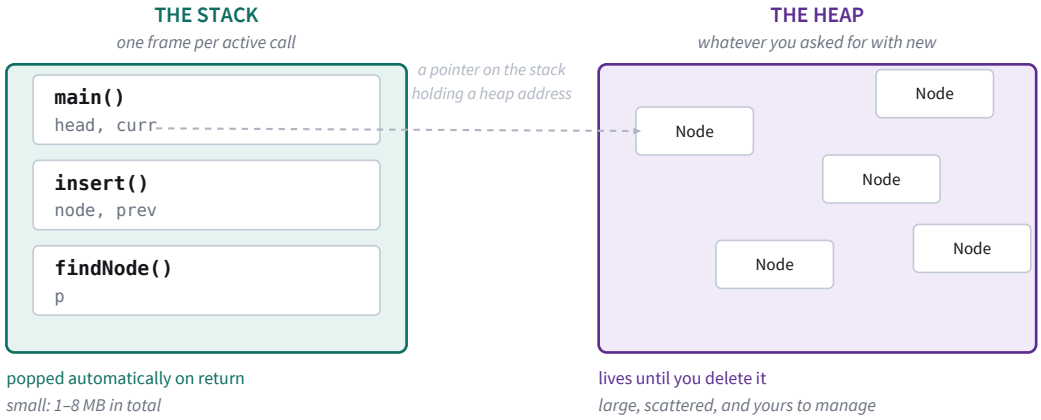


Figure 22.1 The two regions of a running program. The stack holds one frame per active function call and is cleaned up automatically. The heap holds whatever you asked for with `new` and stays until you say otherwise.

DEFINITION | The stack
The **stack** holds the local variables of every function currently running. Each call pushes a **frame** containing its parameters and locals; each return pops that frame and the memory is reclaimed immediately.
Advantages: no cost to manage, impossible to leak, very fast (the next frame is already in cache).
Limits: the size must be known at compile time, the object dies when the function returns, and the whole stack is small — typically 1 to 8 MB. Too many nested calls, or one huge local array, produces a **stack overflow**.

DEFINITION | The heap
The **heap** is a large pool of memory you request from at run time with `new` and return with `delete`.
Advantages: the size can be decided while the program runs, the object lives exactly as long as you want, and the space is large — effectively all available RAM.
Limits: allocation is comparatively slow, the objects are scattered (bad for cache), and **you** must return the memory. If you forget, it is lost until the program exits.

Where things live

```cpp
#include <iostream>

int main() {
    int onStack = 42;                 // stack: dies at the end of main
    int arr[3] = {1, 2, 3};           // stack: 12 bytes, size fixed at
        compile time

    int* onHeap = new int(42);        // heap: lives until we delete it
    int* heapArr = new int[3]{1, 2, 3};  // heap: size could have been a
        variable

    std::cout << onStack << " " << arr[1] << "\n";
    std::cout << *onHeap << " " << heapArr[1] << "\n";

    delete onHeap;                    // give the single object back
    delete[] heapArr;                 // give the ARRAY back - note the []
    return 0;
}
```

```
42 2
42 2
```

What new actually does

DEFINITION | new and delete

`new T(args)` does three things:

1. asks the system for `sizeof(T)` bytes of heap memory;
2. constructs a `T` in those bytes using `args`;
3. returns a `T*` — the address of that object.

`delete p` does two things, in the opposite order:

1. destroys the object that `p` points at;
2. returns its bytes to the system.

`delete` does **not** change `p`. After `delete p`, the variable `p` still holds the same address, but the object there is gone. That is what makes a dangling pointer so easy to create.

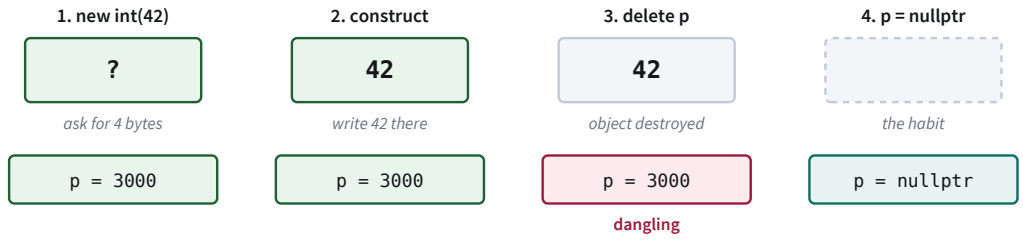


Figure 22.2 What `new` and `delete` do, in order. Note the last panel: `delete` frees the object but leaves the pointer holding the old address, which is why setting it to `nullptr` afterwards is a habit worth having.

COMMON PITFALL | `delete` and `delete[]` are not interchangeable

```
int* one = new int(5);
int* many = new int[5];

delete one;       // OK
delete[] many;    // OK

delete many;      // BAD undefined behaviour
delete[] one;     // BAD undefined behaviour
```

`new[]` records how many objects it made so that `delete[]` can destroy all of them. Using plain `delete` on an array destroys only the first and corrupts the allocator's bookkeeping. The rule is mechanical: `new` **pairs with** `delete`; `new[]` **pairs with** `delete[]`. Match them at the moment you write the `new`, and you will never get it wrong.

Memory leaks

DEFINITION | Memory leak
A **leak** is heap memory that is no longer reachable — you have lost every pointer to it — and has not been given back.
The memory cannot be reused, because the program still "owns" it; and it cannot be freed, because nothing knows where it is. It is lost for the lifetime of the process.

Three ways to leak, all of which run perfectly

```cpp
#include <iostream>

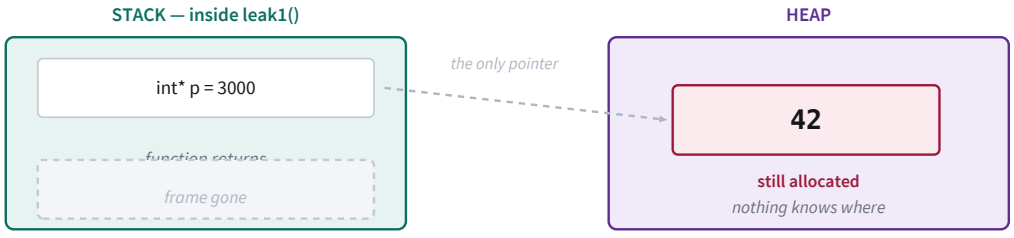
void leak1() {
    int* p = new int(42);
    // no delete - the pointer dies with the function, the memory does not
}

void leak2() {
    int* p = new int(1);
    p = new int(2);          // the first object is now unreachable
    delete p;                // frees only the second
}

void leak3(bool bad) {
    int* p = new int(3);
    if (bad) return;         // the early return skips the delete
    delete p;
}

int main() {
    for (int i = 0; i < 3; ++i) { leak1(); leak2(); leak3(true); }
    std::cout << "no crash, no error, no warning\n";
    return 0;
}
```

```
no crash, no error, no warning
```



no crash · no exception · no compiler warning — which is exactly what makes leaks dangerous

Figure 22.3 A leak. The pointer variable lived on the stack and vanished when the function returned; the object it pointed to is still on the heap, and now nothing in the program knows where it is.

WATCH OUT | Why leaks are so dangerous

Notice that the program above ran correctly and printed exactly what it should. **A leak produces no error.** There is no crash, no exception, no compiler warning.

The symptom appears much later: a long-running process gradually consumes more memory until the machine starts swapping, or the allocator fails. In a program that runs for a second, you will never notice. In a server that runs for a month, it is fatal.

The tool that finds them:

```
g++ -std=c++17 -fsanitize=address -g prog.cpp -o prog && ./prog
```

AddressSanitizer reports every leak at exit, with the stack trace of the `new` that created it. Valgrind (`valgrind --leak-check=full ./prog`) does the same. Run one of them on every linked list exercise in Part VI — the whole point of those exercises is memory discipline, and this is how you check it.

The other two heap bugs

WATCH OUT | Dangling pointers and double frees

Use after free.

```
int* p = new int(42);
delete p;
std::cout << *p;        // undefined behaviour - the object is gone
```

Very often this still prints 42, because the bytes have not been reused yet. Then one day another allocation claims them and the value changes under you.

Double free.

```
int* p = new int(42);
delete p;
delete p;               // undefined behaviour - usually a crash, often
    worse
```

The habit that prevents both:

```
delete p;
p = nullptr;            // now a second delete is harmless, and *p
    crashes loudly
```

`delete nullptr` is explicitly defined to do nothing, so nulling after deleting makes double frees safe. And dereferencing `nullptr` crashes *immediately and visibly*, which is far better than reading stale data for an hour.

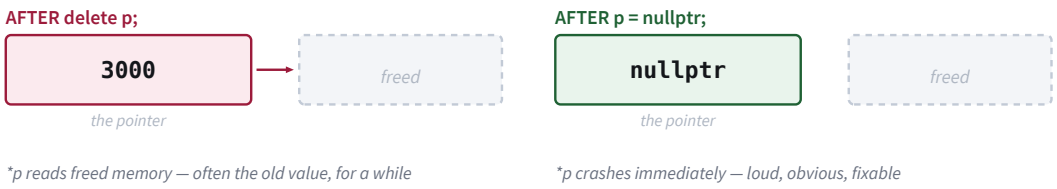


Figure 22.4 A dangling pointer. After `delete`, the object is gone but the pointer still holds its address. Setting the pointer to `nullptr` turns a silent, intermittent bug into an immediate, obvious one.

Ownership

The concept that makes heap code manageable.

> **DEFINITION | Ownership**
> The **owner** of a heap object is the piece of code responsible for deleting it.
> Every heap object must have **exactly one** owner. Zero owners is a leak. Two owners is a double free.
> Other code may hold pointers to the object and use it — those are *non-owning*, or *observing*, pointers. They must not delete, and they must not outlive the owner.

In a linked list, this is simple and absolute:

> **KEY INSIGHT | Ownership in a linked list**
> **The list owns its nodes.** Each node is created with `new` when it is inserted and destroyed with `delete` when it is removed. When the whole list is destroyed, every remaining node is deleted.
> Pointers you hold temporarily while traversing — `curr`, `prev`, `temp` — are **non-owning**. They look at nodes; they never delete them. The only exception is the `temp` you deliberately set aside *in order to* delete it, and that is always two lines long:
>
> ```
> Node* doomed = curr;        // remember the node we are about to orphan
> curr = curr->next;          // move past it
> delete doomed;              // now destroy it
> ```
>
> Reversing those lines — deleting before moving — reads `curr->next` through a freed pointer. That specific bug is the subject of Chapter 26, and it is worth arriving there having already seen it.

A list that owns and frees its nodes

```
1  #include <iostream>
2
3  struct Node {
4      int data;
```

```
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

int main() {
    // Build 10 -> 20 -> 30 on the heap.
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    for (Node* curr = head; curr != nullptr; curr = curr->next)
        std::cout << curr->data << " ";
    std::cout << "\n";

    // Destroy every node. Three nodes created, three deleted.
    Node* curr = head;
    while (curr != nullptr) {
        Node* doomed = curr;        // hold on to it
        curr = curr->next;          // step past it FIRST
        delete doomed;              // now it is safe to destroy
    }
    head = nullptr;                 // the list is empty, say so
    std::cout << "all nodes freed\n";
    return 0;
}
```

```
10 20 30
all nodes freed
```

Count them: three `new`s and three `delete`s. That balance is what you are aiming for in every program in Part VI, and AddressSanitizer will tell you whether you achieved it.

Smart pointers, and why this book uses raw ones

Modern C++ rarely writes `new` and `delete` by hand.

The same object, managed automatically

```
#include <iostream>
#include <memory>

int main() {
    std::unique_ptr<int> p = std::make_unique<int>(42);
    std::cout << *p << "\n";
    // no delete needed - p frees it when p goes out of scope
    return 0;
}
```

```
42
```

DEFINITION | RAII
Resource Acquisition Is Initialisation: tie a resource's lifetime to an object's lifetime, so that the destructor releases it automatically.
`std::unique_ptr` is a tiny object holding one pointer, whose destructor calls `delete`. Because the destructor runs whenever the `unique_ptr` goes out of scope — including when an exception unwinds the stack — the memory cannot leak.
This is why `std::vector` never leaks either: it owns its block, and its destructor frees it.

KEY INSIGHT | Why Part VI still uses raw pointers
In production code, a linked list node should be managed by a smart pointer or, more sensibly, you should use `std::list` and not write one at all.
This book uses raw `new` and `delete` for three reasons:

1. **Every interview uses raw pointers.** The linked list questions in Chapter 30 onwards are posed with `ListNode*`, universally.
2. **You cannot appreciate what RAII saves you from until you have done it by hand.** Learning `unique_ptr` first teaches you a tool; learning it second teaches you a relief.
3. **The pointer manipulation is the lesson.** `unique_ptr` would add move semantics to every reversal and obscure the three-line pointer dance that is the entire point.

So: raw pointers here, deliberately, with the discipline to match. In your own projects, prefer the standard containers and smart pointers.

Cost summary

	Stack	Heap
Allocation cost	essentially free	relatively slow
Size known at	compile time	run time
Lifetime	until the function returns	until you `delete`
Typical total size	1–8 MB	all available RAM
Can leak	no	yes
Cache behaviour	excellent	scattered
Used for	locals, fixed arrays, the pointer itself	linked list nodes, `std::vector`'s block

Table 22.1 Stack versus heap. Note that "slow" and "fast" here are relative — a heap allocation is perhaps 20–100 nanoseconds, which only matters when you do millions of them.

CHAPTER RECAP | Chapter 22 in seven lines

- The **stack** is automatic, fast, small and short-lived; the **heap** is manual, large and lives as long as you choose.
- `new` allocates and constructs; `delete` destroys and frees. They must be balanced exactly.
- `new[]` pairs with `delete[]`, always.
- A **leak** is unreachable, unfreed memory. It produces no error at all — use `-fsanitize=address`.
- `delete` does not change the pointer. Set it to `nullptr` afterwards and two classes of bug disappear.
- Every heap object has **exactly one owner**; everybody else holds non-owning pointers.
- When deleting a list node: step past it **first**, then delete.

EXERCISES | Practise

1. Which of these live on the stack and which on the heap? `int x;` · `int* p = new int;` · `std::vector<int> v;` · the elements inside `v` · `Node* head;` · the node `head` points to.
2. Write a program that leaks exactly 40 bytes. Compile it with `-fsanitize=address` and read the report.
3. Fix `leak2` and `leak3` from this chapter. For `leak3`, give two different fixes and say which you prefer.
4. What does `delete nullptr;` do? Why is that useful?
5. Build a list of five nodes and free it. Verify with AddressSanitizer that nothing leaks. Then deliberately break the free loop by deleting before advancing, and read the error it produces.
6. Why can a recursive function cause a stack overflow when an iterative one with the same complexity does not? Roughly how deep can recursion go with a 1 MB stack and a 64-byte frame?
7. Rewrite the three-node list of this chapter using `std::unique_ptr`. What becomes easier, and what becomes harder?
8. Explain why the snippet below leaks, even though it contains a `delete`.

For exercise 8 — one delete is not enough

```
Node* head = new Node(1);
head->next = new Node(2);
delete head;
```

PART VI

Linked Lists From Scratch

We build the structure one node at a time, draw every pointer before we write it, and handle every empty-list and one-node case that has ever broken somebody's interview.

23 What a Linked List Is, and Why It Exists
24 Building a Linked List From Scratch
25 Insertion
26 Deletion
27 Doubly and Circular Linked Lists
28 Mastering Pointer Manipulation: Reversal

CHAPTER

23

What a Linked List Is, and Why It Exists

Giving up calculation to buy cheap insertion

FOUNDATION PREREQS: Chapters 21 and 22

A linked list is what you get when you refuse to keep your data in one block. Every property it has — good and bad — follows from that one decision. This chapter builds the idea from the array's weakness, defines node, head, tail and nullptr precisely, draws the memory, and sets out the trade-off against arrays that the rest of Part VI depends on.

WHAT YOU WILL LEARN

- Explain why linked lists exist, starting from what arrays are bad at
- Define node, data, next, head, tail and nullptr, and point to each in a diagram
- Explain why a linked list does not need contiguous memory
- State the cost of every basic linked list operation, and justify it
- Compare arrays and linked lists honestly, including where the list loses
- Recognise the three variants: singly, doubly and circular

The array's weakness, restated

Chapter 6 established the cost table for arrays. Two rows were bad:

Operation	Array	Why
Insert at the front	$O(n)$	every element shifts right
Delete from the front	$O(n)$	every element shifts left

And the reason, exactly, was this: **the array promises that its elements sit side by side with no gaps.** Keeping that promise after an insertion means moving everything.

KEY INSIGHT | The question that produces the linked list

"What if we simply stopped making that promise?"

Let each value live wherever there happens to be room. Then inserting is just creating a new value somewhere — nothing has to move, because nothing was guaranteed to be anywhere in particular.

But now we have a new problem: if the values are scattered, **how do we find the next one?**

The answer is the entire idea: **store the address of the next value alongside each value.** Each piece carries a signpost to the one after it.

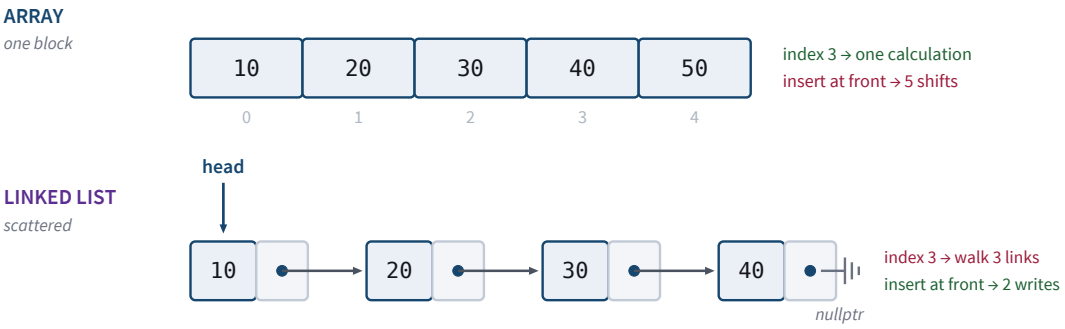


Figure 23.1 The same five values, two ways. The array keeps a promise about position and pays for it on every insertion. The linked list keeps no promise about position and pays for it on every lookup.

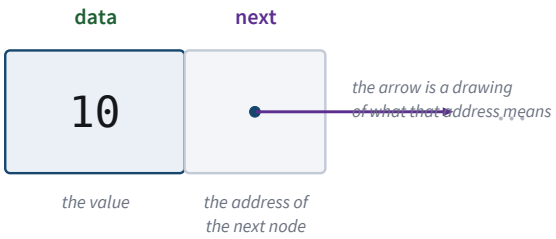
The node

DEFINITION | Node

A **node** is one piece of a linked list. It holds two things:

- **data** — the value being stored;
- **next** — a pointer to the following node, or `nullptr` if there is none.

A node is an object on the heap. It knows nothing about the list as a whole: it does not know its own position, how many nodes follow it, or what came before it. All it knows is its value and where to go next.



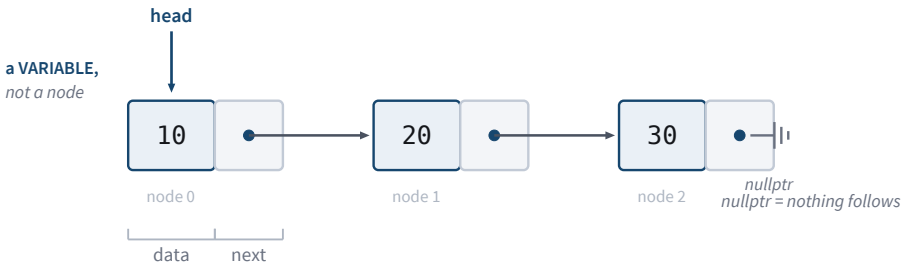
a node knows its value and where to go next — nothing about its position or the list as a whole

Figure 23.2 One node, labelled. The left half holds the value; the right half holds the address of the next node. The arrow is a drawing of what that address means.

That is deliberately minimal, and the minimalism is the source of both the strengths and the limitations. A node cannot tell you its index, because it does not have one.

The list

DEFINITION | Head, tail and nullptr
The **head** is a pointer to the first node. It is not itself a node — it is a variable, usually on the stack, holding the address of the first node.
The head is the list. If you lose it, every node becomes unreachable and the whole list is leaked. That is why so much linked list code is careful never to move `head` itself, and uses a separate `curr` to walk.
The **tail** is the last node — the one whose `next` is `nullptr`. Some implementations also keep a `tail` pointer so that appending is $O(1)$ instead of $O(n)$.
`nullptr` **in** `next` **means "nothing follows".** It is how a traversal knows when to stop, and it is the only thing distinguishing the end of the list from a node that continues.
An **empty list** is `head == nullptr`. Not a list with one empty node — no nodes at all.



head is a pointer · the nodes are heap objects · nullptr marks the end

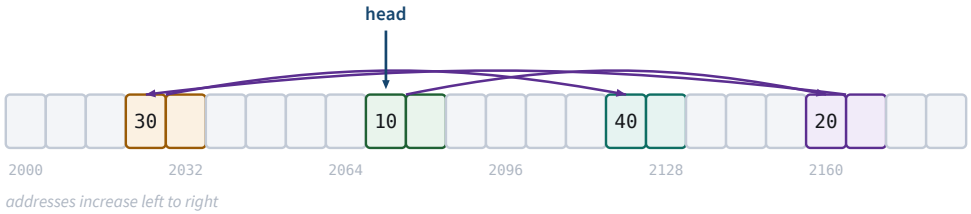
Figure 23.3 A complete three-node list, fully labelled. Four things to keep distinct: the head pointer (a variable), the nodes (heap objects), the data fields, and the next pointers — with `nullptr` marking the end.

COMMON PITFALL | `head` **is a pointer, not a node**
A persistent beginner confusion:

```
head->data        // OK  the value in the FIRST NODE
head.data         // BAD does not compile - head is a pointer
```

And a more subtle one: `head` and `head->next` are both `Node*` . They are the same *type*. What differs is which node they point at. Once that clicks, every traversal becomes obvious; until it does, nothing will.

Why contiguity is not needed



list order is 10 → 20 → 30 → 40, and the addresses go the other way — the pointers are the only order there is

Figure 23.4 The same list in memory. The nodes are at unrelated addresses, in an order that has nothing to do with the list order — node 3 sits before node 1 in memory. The `next` pointers are the only thing defining the sequence.

KEY INSIGHT | **Order in memory is not order in the list**
In an array, element 3 is *physically* after element 2. The order is the layout.
In a linked list, the order is the **pointers**, and nothing else. The second node of the list could be at a lower address than the first; it makes no difference whatsoever to how the list behaves.
This has two consequences, one good and one bad:
Good. New nodes can be created anywhere there is a free four-or-sixteen bytes. You never need a large contiguous block, so a linked list cannot fail because memory is fragmented — and inserting never disturbs anything.
Bad. There is no address formula. To reach node i you must follow i pointers. And each of those jumps is likely to be a **cache miss**, because the processor cannot predict where the next node will be. This is why linked lists are usually slower in practice than their complexity suggests — see Chapter 33.

Reaching the i-th element

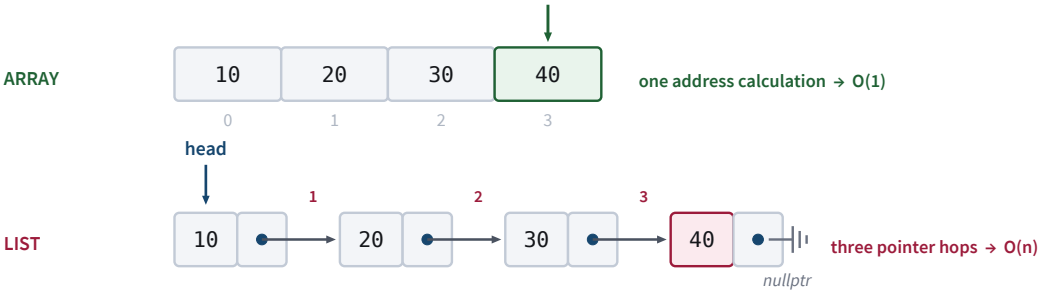


Figure 23.5 Reaching index 3. The array computes an address; the list follows three pointers, each one a separate memory access to an unpredictable location.

This is the trade in its sharpest form:

	Array	Linked list
Reach index i	compute $B+i\times S$	follow i pointers
Cost	$O(1)$	$O(i)$, so $O(n)$

Everything else about the two structures follows from this one difference and its mirror image on insertion.

The complete comparison

Operation	Array / vector	Singly linked list
Access by index	$O(1)$	$O(n)$
Search for a value	$O(n)$	$O(n)$
Insert at the front	$O(n)$	$O(\mathbf{1})$
Insert at the back	$O(1)$ amortised	$O(n)$, or $O(1)$ with a tail pointer
Insert after a node you already hold	$O(n)$	$O(\mathbf{1})$
Delete from the front	$O(n)$	$O(\mathbf{1})$
Delete a node you already hold	$O(n)$	$O(n)$ singly; $O(\mathbf{1})$ doubly
Memory per element	the value	the value + a pointer (often 2× or more)
Cache behaviour	excellent	poor
Needs one contiguous block	yes	no
Binary search possible	yes	no

Table 23.1 Arrays versus singly linked lists. Read the whole table before concluding anything — the list wins two rows decisively and loses most of the others.

WATCH OUT | The row people misquote
"Insertion in a linked list is $O(1)$" is **only true if you already hold a pointer to the node you are inserting after.**
If you have to *find* that position first, finding it is $O(n)$, and the total is $O(n)$ — exactly the same as an array, which also does $O(n)$ work (shifting instead of walking).
So the honest statement is: *a linked list is O(1) for the **modification** and O(n) for the **navigation**; an array is O(1) for the navigation and O(n) for the modification.* Which one wins depends entirely on whether you already have a pointer.
Chapter 33 works this through properly. It is the single most common mistake in interview answers about linked lists, and saying the precise version will distinguish you.

PERFORMANCE NOTE | Why real code usually picks the array
Textbooks present this as a balanced trade. In practice, for most workloads, `std::vector` wins — even for front insertions on small data — for three reasons:

1. **Cache.** Walking an array reads memory sequentially, which the processor prefetches. Walking a list is a chain of dependent, unpredictable loads, and a cache miss costs roughly as much as a hundred arithmetic operations.
2. **Memory.** A node of one `int` costs 4 bytes of data plus 8 bytes of pointer plus allocator overhead — often 32 bytes for 4 bytes of payload.
3. **Allocation.** Every `new` has a cost; a vector amortises one allocation across many elements.

Linked lists genuinely win when: elements are large, you hold pointers directly to nodes, you splice whole ranges between lists, or you need iterators that stay valid across insertions. They are also the right *teaching* structure, because pointer manipulation is the skill being built — and they remain a staple of interviews.

The three variants

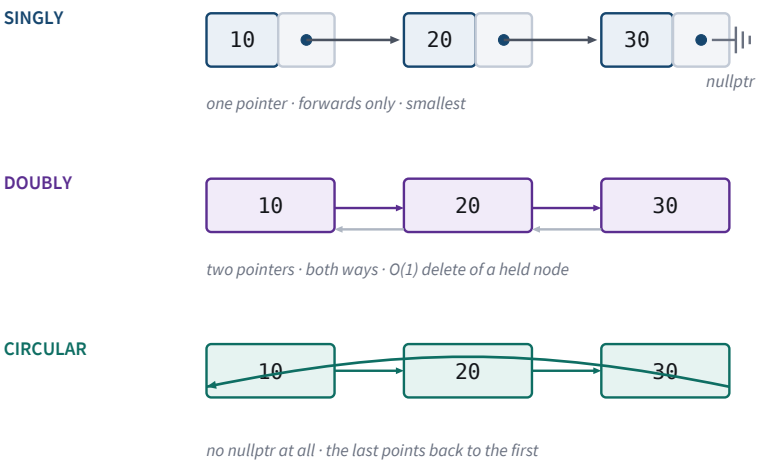


Figure 23.6 The three shapes. A singly linked list points forwards only; a doubly linked list points both ways at the cost of one more pointer per node; a circular list has no `nullptr` at all, because the last node points back to the first.

DEFINITION | Singly, doubly and circular

Singly linked. Each node has one pointer, `next`. You can only move forwards, and you can never get back. Smallest memory footprint. This is the default, and Chapters 24 to 26 use it exclusively.

Doubly linked. Each node has `next` and `prev`. You can walk in either direction, and you can delete a node you hold a pointer to in $O(1)$ — because you can reach its predecessor. The cost is one extra pointer per node and twice as many pointer updates on every operation. `std::list` is a doubly linked list.

Circular. The last node's `next` points back to the first instead of to `nullptr`. Useful for round-robin scheduling and ring buffers. The hazard: every loop condition that said `while (curr != nullptr)` now never terminates, and must become `do { ... } while (curr != head)`.

Chapter 27 covers the latter two properly.

What is coming

Chapter	What you will build
24	the `Node` struct, line by line; creating, connecting, traversing
25	insertion at the front, the back, a position, and around a given node
26	deletion of the first, the last, by position and by value, without leaking
27	doubly and circular lists
28	reversal — the single most important pointer exercise there is
29–32	the patterns, and sixteen interview problems

CHAPTER RECAP | Chapter 23 in six lines

- A linked list gives up contiguity, and with it the address formula, to make insertion and deletion cheap.
- A **node** holds data and a pointer to the next node; it knows nothing else.
- The **head** is a pointer to the first node, and losing it leaks the list. `head == nullptr` means empty.
- `nullptr` in `next` marks the end, and is how every traversal terminates.
- Order in the list is defined by pointers, not by addresses. Nodes may sit anywhere.
- "$O(1)$ insertion" is true only when you already hold the node to insert after. Finding it is $O(n)$.

EXERCISES | **Before you go on**

1. Draw, by hand, a four-node list holding 5, 10, 15, 20 — including the head pointer and the `nullptr`. Label the data and next fields of each node.
2. In your drawing, what is `head->next->next->data`? What about `head->next->next->next->next`?
3. Why can a linked list not support binary search, even when the values are in sorted order?
4. An array of a million `int`s uses about 4 MB. Roughly how much would a singly linked list of the same million `int`s use on a 64-bit machine? Account for the pointer and some allocator overhead.
5. Give a concrete situation where a linked list is clearly the right choice, and one where it is clearly wrong. Justify each in one sentence.
6. The chapter says losing `head` leaks the whole list. Write the two lines of code that would cause that.
7. What goes wrong if you write `while (curr != nullptr)` to walk a **circular** list? What should it be instead?

CHAPTER

24

Building a Linked List From Scratch

Every line, every symbol, explained

FOUNDATION PREREQS: Chapter 23

We now write the `Node` struct that the rest of this book uses, and explain every single symbol in it — `struct`, `Node*`, the constructor, `nullptr`, and the difference between a node and a pointer to a node. Then we create nodes, connect them, walk the list, count it, search it, update it, and free it, drawing the memory at every step.

WHAT YOU WILL LEARN

- Explain every line and symbol of the `Node` definition
- Distinguish a node from a pointer to a node, and say why it matters
- Create nodes on the heap and connect them into a list
- Traverse a list correctly, and explain why `curr` is used rather than `head`
- Count, search and update nodes
- Free an entire list without leaking and without using a freed pointer

The Node definition

Here it is. Every linked list in this book uses exactly this.

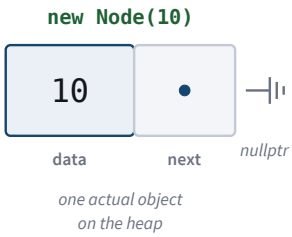
The Node struct — used unchanged for the rest of the book

```
struct Node {
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};
```

Six lines. We will spend the next two pages on them, because every later chapter assumes you understand all six completely.

```
struct Node {                    — a new type grouping two values
    int data;                    — the value stored here
    Node* next;                  — a POINTER to the next node
    Node(int value) {            — the constructor — runs on new
        data = value;            — store what was passed in
        next = nullptr;          — so it is never garbage
    }
};                               — the semicolon is required
```



the struct is a blueprint · new builds one object from it

Figure 24.1 The `Node` definition mapped onto the object it creates. The struct is a *blueprint*; `new Node(10)` uses the blueprint to build one actual object on the heap.

`struct Node {`

> **DEFINITION** | **struct**
> A **struct** defines a new type by grouping several values together under one name. After this definition, `Node` is a type, exactly like `int` is a type. You can declare variables of it, make pointers to it, and allocate it with `new`.
> (In C++, `struct` and `class` are the same thing with one difference: a `struct`'s members are public by default and a `class`'s are private. For a plain data holder like this, `struct` is the conventional choice.)

The `{` opens the list of members, and the matching `};` at the end closes it. **That semicolon is required** — leaving it off produces a confusing error on whatever line comes next, which is a classic five-minute waste.

`int data;`

The value this node stores. It is an ordinary `int`, living inside the node object. If you wanted a list of strings you would write `std::string data;` instead, and nothing else about the structure would change.

The name `data` is conventional, not required. LeetCode calls it `val`, and some books call it `info`. Pick one and stay with it.

`Node* next;`

The important one. Read it as "`next` **is a pointer to a** `Node`".

KEY INSIGHT | A node contains a pointer, not a node
`Node* next;` could not be `Node next;` — and the reason is worth understanding rather than accepting.
If a `Node` contained a `Node`, that inner `Node` would contain another `Node`, and so on forever. The type would have infinite size, and the compiler rejects it outright.
A **pointer** to a `Node` is always the same size (8 bytes on a 64-bit machine) no matter what it points at — because it holds only an address. So the struct's size is settled: 4 bytes for `data`, 8 for `next`, plus padding. Finite, and the recursion lives in the *data*, not in the *type*.
This is why every self-referential structure in programming — lists, trees, graphs — is built from pointers. It is not a style choice; it is the only thing that can work.

Note also that `Node*` is usable *inside* the definition of `Node` itself, even though `Node` is not finished yet. That is allowed precisely because the compiler does not need to know how big a `Node` is to know how big a `Node*` is.

`Node(int value) { ... }`

DEFINITION | Constructor
A **constructor** is a function that runs automatically when an object is created. It has the same name as the type and no return type at all — not even `void`.
`Node(int value)` says: *"to make a Node, you must supply one* `int` *"*. So `new Node(10)` calls it with `value = 10`.
Its job here is to leave the new object in a sensible state: `data` set to the supplied value, and `next` set to `nullptr`.

COMMON PITFALL | Without the constructor, `next` is garbage

```
struct Node {
    int data;
    Node* next;
};

Node* n = new Node;      // no constructor: next contains random bytes
if (n->next == nullptr)  // almost certainly false, and meaningless
```

An uninitialised pointer does not contain `nullptr` — it contains whatever was in those 8 bytes before. Following it is undefined behaviour, and the bug appears as a random crash much later in the traversal.
The constructor makes that impossible. **Write it every time.**
The modern form uses an initialiser list, which is slightly more efficient and is what you will see in production code:

```
Node(int value) : data(value), next(nullptr) {}
```

The two versions are equivalent for `int` and `Node*`. This book uses the assignment form because it reads more plainly for a beginner; use either.

Node versus pointer-to-Node

This distinction causes more confusion than anything else in Part VI.

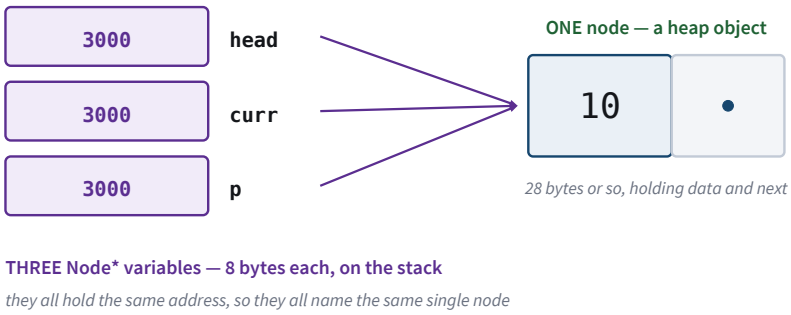


Figure 24.2 A node and a pointer to a node. The node is the object with the two fields; the pointer is an 8-byte variable holding its address. Three `Node*` variables can all point at the same single node.

	`Node n(10);`	`Node* p = new Node(10);`
What it is	an object	a pointer to an object
Where it lives	the stack	the pointer on the stack, the object on the heap
Lifetime	until the enclosing block ends	until you `delete` it
Access a member	`n.data`	`p->data`
Can be `nullptr`	no	yes
Used in this book	almost never	always

Every node in a linked list is created with `new`, because it must outlive the function that created it. The pointers (`head`, `curr`, `prev`) are stack variables; the nodes are heap objects. Chapter 22's diagram, exactly.

Building the smallest possible list

Start with one node, and build up.

One node, then two, then three

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

int main() {
    Node* head = new Node(10);       // one node; head->next is nullptr
    std::cout << head->data << "\n";
```

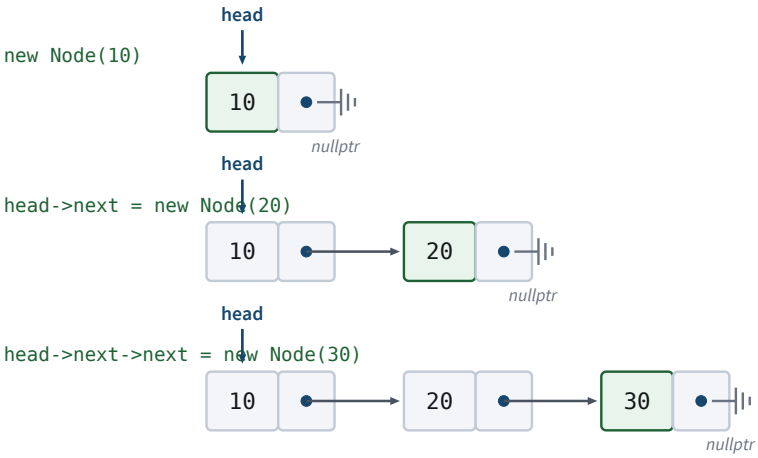
```
    std::cout << (head->next == nullptr ? "end of list" : "more") << "\n";

    head->next = new Node(20);       // attach a second node
    std::cout << head->next->data << "\n";

    head->next->next = new Node(30); // and a third
    std::cout << head->next->next->data << "\n";

    delete head->next->next;         // free from the back
    delete head->next;
    delete head;
    return 0;
}
```

```
10
end of list
20
30
```



at every step the last node's next is nullptr — so the list is valid the whole way through

Figure 24.3 Building the list one node at a time. At every step the final node's `next` is `nullptr`, so the list is always valid — you could traverse it correctly at any point in the middle of construction.

Read `head->next->next = new Node(30);` carefully, left to right:

1. `head` — the first node.
2. `head->next` — the second node.
3. `head->next->next` — the second node's `next` field.
4. `= new Node(30)` — make a third node and store its address there.

That chain of arrows is how all linked list code reads. It becomes automatic sooner than you expect.

COMMON PITFALL | Freeing in the wrong order

```
delete head;
delete head->next;     // head is already freed - undefined behaviour
```

After `delete head`, reading `head->next` is a use-after-free. The address you get back may be the right one, or rubbish; it is undefined either way.
This is why the free loop later in this chapter saves the next pointer **before** deleting. It is the same "read before you overwrite" principle from Chapter 6, applied to memory instead of array slots.

Traversing: the loop you will write a hundred times

Print every node

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

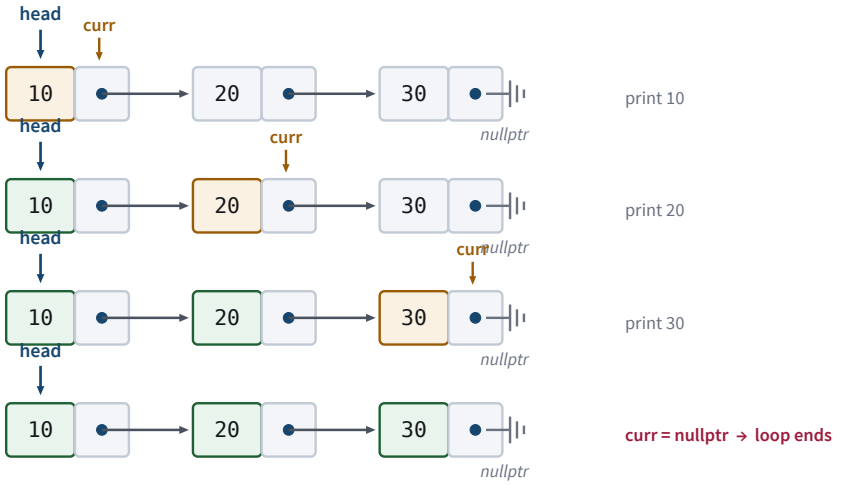
void printList(Node* head) {
    Node* curr = head;                  // a WALKER - head must not move
    while (curr != nullptr) {           // until we fall off the end
        std::cout << curr->data;
        if (curr->next != nullptr) std::cout << " -> ";
        curr = curr->next;              // step forward
    }
    std::cout << "\n";
}

int main() {
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    printList(head);
    printList(nullptr);                 // an empty list prints nothing

    delete head->next->next;
    delete head->next;
    delete head;
    return 0;
}
```

```
10 -> 20 -> 30
```



head never moves — it is the list's identity. curr does the walking.

Figure 24.4 The traversal, step by step. `curr` moves along; `head` never does. When `curr` becomes `nullptr` the loop ends — and notice that it ends *after* the last node has been processed, not before.

KEY INSIGHT | Why `curr`, and not `head`

```
while (head != nullptr) {        // BAD destroys the list
    std::cout << head->data;
    head = head->next;
}
```

Inside the function this works and prints the right thing. But `head` was a copy of the caller's pointer, so the caller still has its list — this time.

Write the same loop on a `head` you actually own, and after it runs `head` is `nullptr`: you have walked it off the end and lost every node. The nodes are still on the heap, unreachable. A complete leak, from one missing variable.

The rule: `head` identifies the list, so it never moves. `curr` does the walking. Create a separate walker, always, even when it feels unnecessary.

Note also that an empty list needs no special case: `curr` starts as `nullptr`, the loop body never runs, and the function prints an empty line. Code where the edge case falls out of the structure is better than code that tests for it.

Counting, searching, updating

The three read-only operations

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

int length(Node* head) {
    int n = 0;
    for (Node* curr = head; curr != nullptr; curr = curr->next) ++n;
```

```
    return n;
}

// Returns the node holding target, or nullptr if absent.
Node* findNode(Node* head, int target) {
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        if (curr->data == target) return curr;
    return nullptr;
}

// Returns the node at index i (0-based), or nullptr if i is out of range.
Node* nodeAt(Node* head, int i) {
    Node* curr = head;
    for (int k = 0; k < i && curr != nullptr; ++k) curr = curr->next;
    return curr;
}

int main() {
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);

    std::cout << "length  = " << length(head) << "\n";
    std::cout << "find 20 = " << (findNode(head, 20) ? "found" :
        "absent") << "\n";
    std::cout << "find 99 = " << (findNode(head, 99) ? "found" :
        "absent") << "\n";
    std::cout << "node[1] = " << nodeAt(head, 1)->data << "\n";
    std::cout << "node[9] = " << (nodeAt(head, 9) ? "exists" : "out of
        range") << "\n";

    Node* n = findNode(head, 20);        // updating is just a field
        assignment
    if (n) n->data = 99;
    std::cout << "after update, node[1] = " << nodeAt(head, 1)->data <<
        "\n";

    while (head) { Node* doomed = head; head = head->next; delete doomed;
        }
    return 0;
}
```

```
length  = 3
find 20 = found
find 99 = absent
node[1] = 20
node[9] = out of range
after update, node[1] = 99
```

All three are $O(n)$ and $O(1)$ space. Note what `nodeAt` makes vivid: **a linked list has no index.** There is nothing to compute; you simply count your way along. That function is the whole reason `list[i]` does not exist.

COMMON PITFALL | **The `&& curr != nullptr` in `nodeAt`**
Without it, asking for index 9 in a 3-node list would dereference `nullptr` on the fourth step. The extra condition stops the walk at the end and lets the function return `nullptr`, which the caller is then obliged to check.
The general rule, worth saying out loud every time: **before you follow a pointer, know that it is not null.**

Building from an array, and freeing everything

Two utilities we will reuse in every later chapter.

Build a list from values, print it, and free it

```cpp
#include <iostream>
#include <vector>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Builds a list from the values, in order. Returns the head.
Node* buildList(const std::vector<int>& values) {
    Node* head = nullptr;
    Node* tail = nullptr;                // tracks the last node: O(1) append
    for (int v : values) {
        Node* node = new Node(v);
        if (head == nullptr) head = tail = node;   // the first node
        else { tail->next = node; tail = node; }   // attach and advance
    }
    return head;
}

void printList(Node* head) {
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        std::cout << curr->data << (curr->next ? " -> " : "\n");
    if (head == nullptr) std::cout << "(empty)\n";
}

// Deletes every node. The list must not be used afterwards.
void freeList(Node* head) {
    Node* curr = head;
    while (curr != nullptr) {
        Node* doomed = curr;        // 1. remember the node to destroy
        curr = curr->next;          // 2. step past it FIRST
        delete doomed;              // 3. now destroy it
    }
}

int main() {
    Node* head = buildList({1, 2, 3, 4, 5});
    printList(head);
```

```
    std::cout << "empty list: ";
    printList(nullptr);
    freeList(head);
    head = nullptr;                 // say that the list is gone
    std::cout << "freed\n";
    return 0;
}
```

```
1 -> 2 -> 3 -> 4 -> 5
empty list: (empty)
freed
```

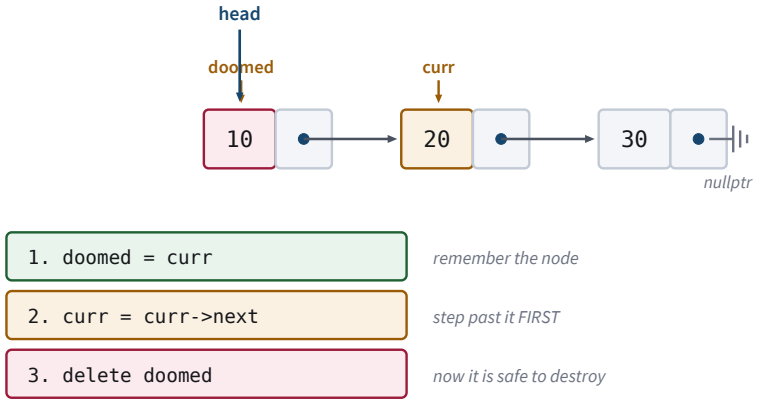


Figure 24.5 The free loop. The next pointer is read **before** the node is destroyed. Reversing those two lines reads a freed object, which is the most common linked list memory bug there is.

KEY INSIGHT | The tail pointer in `buildList`
Without `tail`, appending each new node would mean walking to the end of the list every time — $O(n)$ per append, so $O(n^2)$ to build a list of n elements.
Keeping `tail` makes each append $O(1)$, so building is $O(n)$. For 100,000 elements that is the difference between instant and about five seconds.
This is the first appearance of a theme: **a linked list operation is $O(1)$ exactly when you already hold the relevant node.** Keeping a `tail` pointer is how you arrange to hold it.

Cost summary

Operation	Time	Space	Note
Create one node	$O(1)$	$O(1)$	one `new`
Attach a node you hold	$O(1)$	$O(1)$	one assignment
Traverse / print	$O(n)$	$O(1)$	the standard `curr` loop
Length	$O(n)$	$O(1)$	no stored count unless you keep one
Search by value	$O(n)$	$O(1)$	no shortcut, even if sorted
Node at index i	$O(i)$	$O(1)$	counting, not calculating

Operation	Time	Space	Note
Update a node you hold	$O(1)$	$O(1)$	a field assignment
Build from n values	$O(n)$	$O(n)$ nodes	$O(n^2)$ without a tail pointer
Free the whole list	$O(n)$	$O(1)$	save `next` before deleting

Table 24.1 Everything in this chapter. Note that the only $O(1)$ operations are the ones where you already have the node in your hand.

CHAPTER RECAP | Chapter 24 in seven lines

- `Node* next;` must be a **pointer**, or the type would have infinite size.
- The constructor exists so that `next` is `nullptr` rather than garbage.
- A **node** is a heap object; a `Node*` is a stack variable holding its address.
- `head` identifies the list and never moves. `curr` walks.
- `while (curr != nullptr)` handles the empty list with no special case.
- Before following a pointer, know it is not null.
- When freeing: save `next`, step forward, **then** delete.

EXERCISES | Practise

1. Write out, in English, what each of the six lines of the `Node` struct does.
2. Why can `Node` not contain a `Node` by value? Give the two-sentence answer.
3. Build a 4-node list holding 7, 14, 21, 28 without using `buildList`. Then print the third value using only arrows from `head`.
4. Write `sum(Node* head)` returning the total of all values, and `maxValue` returning the largest. What should each do on an empty list?
5. Write `nodeAtFromEnd(Node* head, int k)` returning the k-th node from the **end**. The obvious way takes two passes — write that first.
6. What is wrong with the loop below, and what does it leak?
7. Write `freeList` recursively. For what list length would it overflow a 1 MB stack, and what does that tell you about recursion on linked lists?

For exercise 6 — find the bug

```
while (head != nullptr) {
    delete head;
    head = head->next;
}
```

CHAPTER

25

Insertion

Four places to insert, and the order of the pointer changes

CORE PREREQS: Chapter 24

Every insertion is two or three pointer assignments, and the only thing that can go wrong is doing them in the wrong order — which silently loses the rest of the list. This chapter draws each insertion before coding it, states the order rule that makes all of them correct, and handles every empty-list and one-node case explicitly.

WHAT YOU WILL LEARN

- Insert at the front, at the back, at a position, and after a given node
- State the order rule: connect the new node forwards before rewiring backwards
- Draw the before, during and after state of each insertion
- Handle the empty list and the single-node list in every case
- Explain why inserting at the front is $O(1)$ and at the back is $O(n)$
- Recognise the lost-list bug from its symptom

The rule that makes every insertion correct

Before any code, here is the principle. It is short, it is the whole chapter, and it is worth memorising.

KEY INSIGHT | The order rule

Point the new node at the rest of the list *first*. Only then point the previous node at the new node.

Do it the other way round and the only pointer to the rest of the list is overwritten, so the remainder is orphaned — unreachable and leaked.

In code: `node->next = X;` comes **before** `prev->next = node;` — always.

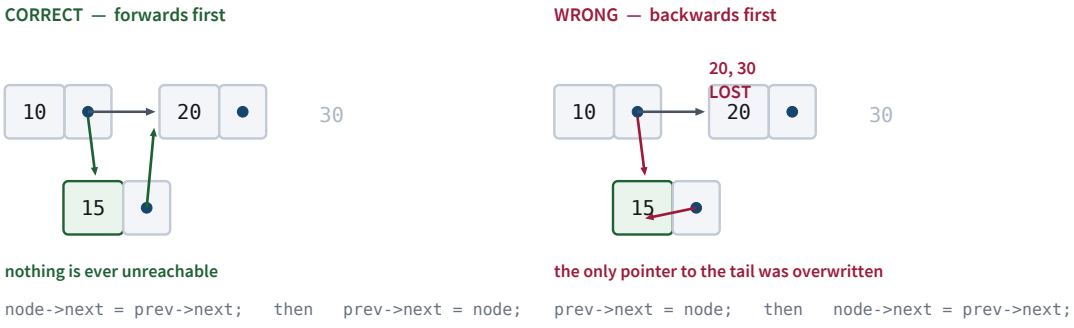
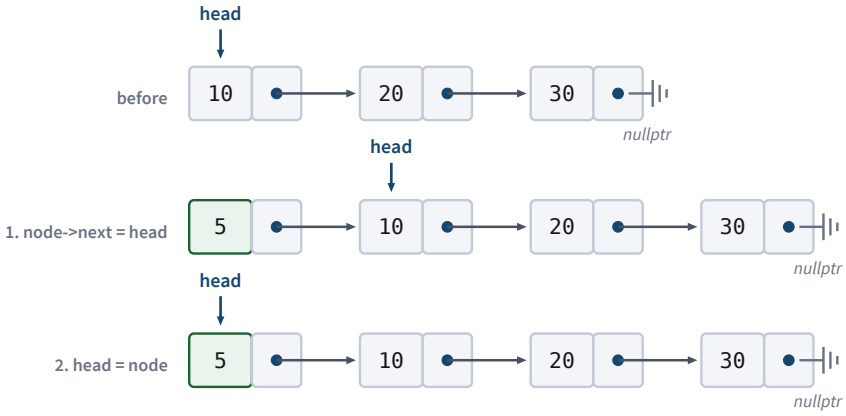


Figure 25.1 The order rule. On the left, the new node is connected forwards first, so nothing is ever unreachable. On the right, rewiring backwards first overwrites the only pointer to the tail, and 20 and 30 are lost.

If you understand that diagram, every insertion in this chapter is a corollary.

1. Insert at the front

> **DEFINITION | The operation**
> Make a new node the first node of the list. The old first node becomes the second.



O(1) — nothing is traversed, two assignments

the empty list needs no special case: node->next = nullptr is exactly right

Figure 25.2 Inserting 5 at the front. Two assignments: the new node points at the old head, and then `head` points at the new node. The empty list needs no special case, because `head` being `nullptr` is exactly what the new node should point at.

Insert at the front — O(1)

```
#include <iostream>
#include <vector>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
```

```
};

// head is passed BY REFERENCE, because we are going to change it.
void insertFront(Node*& head, int value) {
    Node* node = new Node(value);
    node->next = head;      // 1. the new node points at the old first
        node
    head = node;            // 2. head now names the new node
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = nullptr;
    printList(head);
    insertFront(head, 30);
    insertFront(head, 20);
    insertFront(head, 10);
    printList(head);
    freeList(head);
    return 0;
}
```

```
(empty)
10 -> 20 -> 30
```

Cost. $O(1)$ time, $O(1)$ space. Nothing is traversed; two assignments.

COMMON PITFALL | `Node*& head` **— the ampersand is not optional**

```
void insertFront(Node* head, int value) {   // BAD no ampersand
    Node* node = new Node(value);
    node->next = head;
    head = node;                            // changes only the local
        copy
}
```

Without the `&`, `head` is a **copy** of the caller's pointer (Chapter 21). The function builds the node, links it correctly, and then assigns to its own local variable. The caller's `head` is unchanged, the new node is unreachable, and you have a leak plus a list that did not grow.

The symptom is distinctive: the function appears to do nothing at all. Whenever an insertion silently fails to happen, check for a missing `&`.

Two alternatives you will meet:

- `Node** head` — a pointer to the pointer, used in C and in some C++ code. Then you write `*head = node;`.

- `Node* insertFront(Node* head, int value)` — return the new head and make the caller write `head = insertFront(head, 10);`. This is common on LeetCode.

All three work. Pick one per codebase. This book uses `Node*&`.

Notice that the empty list required no special case: when `head` is `nullptr`, `node->next = head` sets the new node's `next` to `nullptr`, which is exactly right for a one-node list.

2. Insert at the back

DEFINITION | The operation
Append a new node after the current last node.

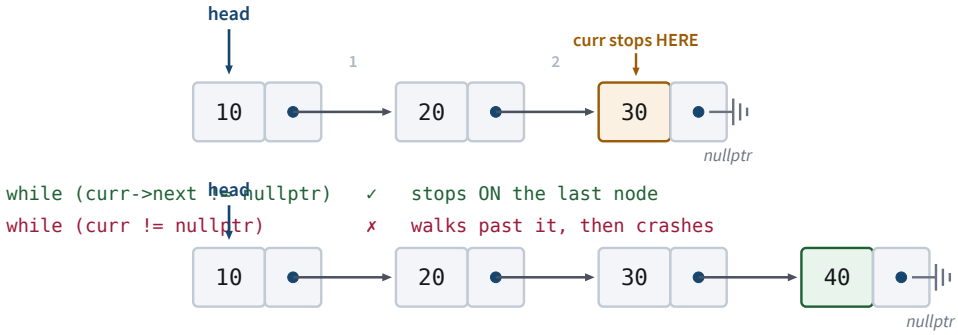


Figure 25.3 Appending 40. The list must be walked to find the last node — the one whose `next` is `nullptr` — and only then can the new node be attached. The empty list is the one genuine special case, because there is no last node to attach to.

Insert at the back — O(n) without a tail pointer

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

void insertBack(Node*& head, int value) {
    Node* node = new Node(value);
    if (head == nullptr) {       // SPECIAL CASE: the empty list
        head = node;
        return;
    }
    Node* curr = head;
    while (curr->next != nullptr)   // walk to the LAST node, not past it
        curr = curr->next;
```

```
    curr->next = node;
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = nullptr;
    insertBack(head, 10);            // into an empty list
    insertBack(head, 20);
    insertBack(head, 30);
    printList(head);
    freeList(head);
    return 0;
}
```

```
10 -> 20 -> 30
```

KEY INSIGHT | `curr->next != nullptr`, **not** `curr != nullptr`
This is the single most common error in `insertBack`.
`while (curr != nullptr)` walks until `curr` falls **off** the end, leaving `curr` as `nullptr`. Then `curr->next = node;` dereferences null and crashes.
`while (curr->next != nullptr)` stops **on** the last node, which is exactly the node you need to modify.
The general principle: **to modify a node, you must stop on it, not past it.** That is also why deletion needs a `prev` pointer, and why the dummy-node trick of Chapter 29 exists.

Cost. $O(n)$ time because of the walk, $O(1)$ space.

PERFORMANCE NOTE | **Keeping a tail pointer**
If you store a `tail` pointer alongside `head`, appending becomes:

```
void insertBackFast(Node*& head, Node*& tail, int value) {
    Node* node = new Node(value);
    if (tail == nullptr) head = tail = node;    // the list was empty
    else { tail->next = node; tail = node; }
}
```

$O(1)$ instead of $O(n)$. Building an n-element list goes from $O(n^2)$ to $O(n)$ — the difference between five seconds and instant at n=100,000.
The cost is that **every** operation must now maintain `tail`: deleting the last node must move it back, and clearing the list must null it. A `tail` pointer that is sometimes stale is worse than no `tail` pointer, because the bug appears far from the function that caused it.

3. Insert at a position

DEFINITION | The operation
Insert a new node so that it ends up at index `pos` (0-based). `pos == 0` means the front; `pos == length` means the back; anything larger is an error.

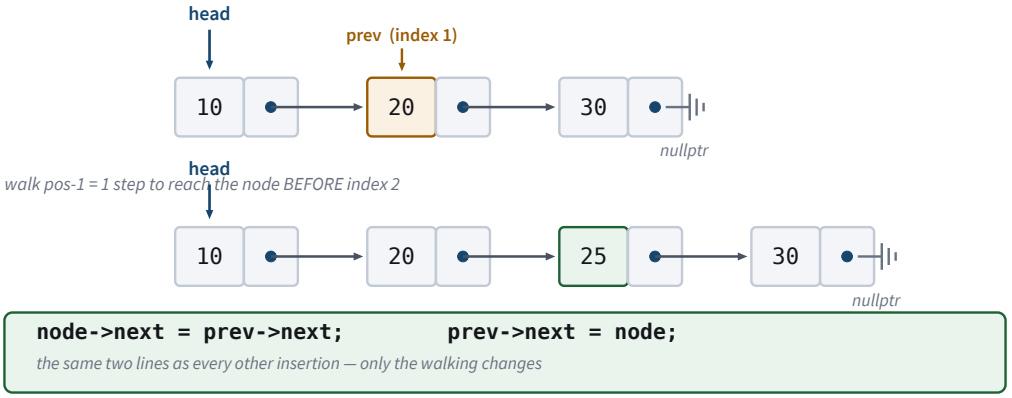


Figure 25.4 Inserting 25 at index 2. Walk to the node *before* the target position, then perform the same two assignments as always — new node forwards first, predecessor afterwards.

The key realisation: once you hold the node *before* the insertion point, this is the same two-line operation as every other insertion. All the work is in the walking.

Insert at a given position

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Returns true on success, false if pos is out of range.
bool insertAt(Node*& head, int pos, int value) {
    if (pos < 0) return false;
    if (pos == 0) {                      // the front: head itself changes
        Node* node = new Node(value);
        node->next = head;
        head = node;
        return true;
    }
    Node* prev = head;                   // walk to the node BEFORE pos
    for (int i = 0; i < pos - 1; ++i) {
        if (prev == nullptr) return false;   // ran off the end
        prev = prev->next;
    }
    if (prev == nullptr) return false;    // pos is beyond the end

```

```
    Node* node = new Node(value);
    node->next = prev->next;            // 1. forwards first
    prev->next = node;                  // 2. then backwards
    return true;
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = nullptr;
    insertAt(head, 0, 10);
    insertAt(head, 1, 20);
    insertAt(head, 2, 30);
    printList(head);

    insertAt(head, 1, 15);              // into the middle
    printList(head);

    insertAt(head, 0, 5);               // at the front
    printList(head);

    insertAt(head, 5, 40);              // at the very end
    printList(head);

    std::cout << "pos 99: " << (insertAt(head, 99, 1) ? "ok" :
        "rejected") << "\n";
    std::cout << "pos -1: " << (insertAt(head, -1, 1) ? "ok" :
        "rejected") << "\n";
    freeList(head);
    return 0;
}
```

```
10 -> 20 -> 30
10 -> 15 -> 20 -> 30
5 -> 10 -> 15 -> 20 -> 30
5 -> 10 -> 15 -> 20 -> 30 -> 40
pos 99: rejected
pos -1: rejected
```

COMMON PITFALL | The three range checks
Look at how much of that function is validation rather than work. That is normal and correct.
`pos == 0` **is separate** because it is the only case that changes `head` itself. Chapter 29's dummy-node trick exists precisely to remove this split.
`prev == nullptr` **inside the loop** catches a `pos` larger than the list. Without it, the walk dereferences `nullptr`.
`prev == nullptr` **after the loop** catches `pos` exactly one past the end of the walk. Both checks are needed; neither covers the other.
Beginners write the happy path and stop. The validation is what makes it work on inputs you did not think of, and an interviewer is watching for exactly that.

Cost. $O(pos)$ to walk, so $O(n)$ worst case; $O(1)$ space. Inserting at index 0 is $O(1)$.

4. Insert after a given node

DEFINITION | The operation
You already hold a pointer to some node. Insert a new node directly after it.

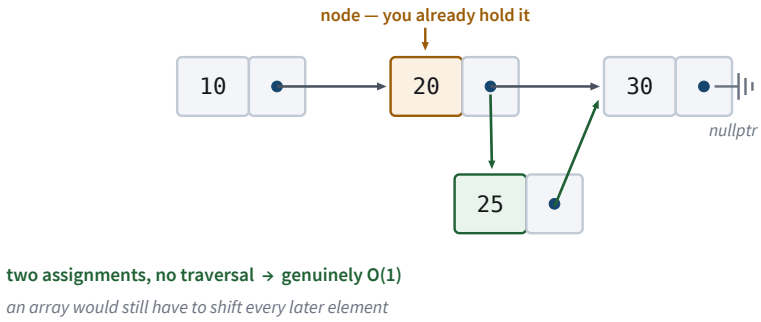


Figure 25.5 Inserting after a node you already hold. No walking at all — two assignments, $O(1)$. This is the case where linked lists genuinely beat arrays, because an array would still shift everything.

Insert after a node — genuinely O(1)

```
void insertAfter(Node* node, int value) {
    if (node == nullptr) return;         // nothing to insert after
    Node* fresh = new Node(value);
    fresh->next = node->next;            // 1. forwards first
    node->next = fresh;                  // 2. then backwards
}
```

Two assignments, no traversal, no `head` involved. $O(1)$ time and space.

KEY INSIGHT | This is the whole advantage, and its limit

This is what people mean by "linked lists have $O(1)$ insertion". Not insertion at an arbitrary index — insertion *given a pointer*.

In an array, inserting after a known index still costs $O(n)$, because every later element must shift. Here it is two writes regardless of list length.

But notice what the function cannot do: insert *before* a node. To do that you would need the node's predecessor, and a singly linked list cannot go backwards. That limitation is exactly what a doubly linked list removes, at the cost of one extra pointer per node — Chapter 27.

Insert-before, in a singly linked list, with only the node: there is a famous trick. Insert *after* the node, then **swap the two nodes' data**. The structure ends up right, and no predecessor was needed. It works only when copying the data is cheap and legitimate. The same trick solves "delete a node given only that node" in Chapter 31.

All four, compared

Insertion	Walk needed	Time	Changes `head`?	Special cases
At the front	none	$O(1)$	yes	none — empty works naturally
At the back	to the last node	$O(n)$	only if empty	the empty list
At position p	$p-1$ steps	$O(p)$, $O(n)$ worst	only if $p=0$	$p=0$; p out of range
After a known node	none	$O(1)$	no	a null node pointer
At the back, with a `tail`	none	$O(1)$	only if empty	`tail` must be maintained everywhere

Table 25.1 The four insertions. The pattern is clear: the two assignments never change; what changes is how much walking you must do to find the node to attach to.

The symptom of getting the order wrong

WATCH OUT | What a lost list looks like

If you write the two assignments backwards:

```
prev->next = node;          // BAD backwards first
node->next = prev->next;    // now this is node itself!
```

the second line reads `prev->next`, which you have just set to `node` — so the new node points at **itself**. The list now has a one-node cycle, and everything that used to follow `prev` is unreachable.

The symptom: printing the list hangs forever, repeating the same value. If a traversal ever fails to terminate, suspect a self-referential or cyclic `next` pointer, and check the order of your assignments first.

Chapter 30 turns cycle detection into a problem of its own. Here it is a bug.

CHAPTER RECAP | **Chapter 25 in six lines**

- **Connect the new node forwards first, then rewire the predecessor.** Always.
- Front insertion is $O(1)$ and needs `Node*&` (or a returned head).
- Back insertion is $O(n)$ without a `tail` pointer, and $O(1)$ with one.
- Stop **on** the node you must modify (`curr->next != nullptr`), not past it.
- Insert-at-position is the same two lines, plus a walk and three range checks.
- Insert-after-a-known-node is the genuinely $O(1)$ case, and is the real advantage of a linked list.

EXERCISES | **Practise**

1. Draw the before and after state for inserting 25 at index 2 of `10 -> 20 -> 30`. Number the pointer changes in the order they must happen.
2. Write `insertBack` keeping a `tail` pointer. Which other functions in this chapter would then need changing, and why?
3. Write `insertSorted(Node*& head, int value)` that inserts into a sorted list so it stays sorted. Which case needs `head` to change?
4. What happens if `insertAt` is called with `pos` equal to the list's length? Should it succeed? Check your implementation agrees with your answer.
5. Rewrite `insertFront` using `Node** head` instead of `Node*& head`. Which do you find clearer, and why do you think C code uses the first form?
6. Explain what `insertAfter(nullptr, 5)` should do, and what the version without the null check would do.
7. Implement insert-**before** given only a pointer to a node, using the data-swap trick. When is this trick unacceptable?
8. Build a list of 100,000 elements with `insertBack` and no `tail` pointer, and time it. Then add a `tail` pointer and time it again. Explain the ratio you observe.

CHAPTER

26

Deletion

Unlink, then free — and never in the other order

CORE PREREQS: Chapters 25 and 22

Deletion is insertion in reverse plus one extra obligation: the removed node must be given back to the heap, exactly once, and not before you have finished reading from it. This chapter covers deleting the first node, the last, a position, and a value, with the memory discipline that keeps every one of them leak-free.

WHAT YOU WILL LEARN

- Delete the first node, the last node, a node at a position, and by value
- State the order rule for deletion: unlink first, then free
- Explain why singly linked deletion needs the **predecessor**, not the node
- Handle the empty list, the single-node list, and deleting the head
- Free a whole list without leaking and without use-after-free
- Recognise the three deletion bugs: leak, dangling, double free

The two rules

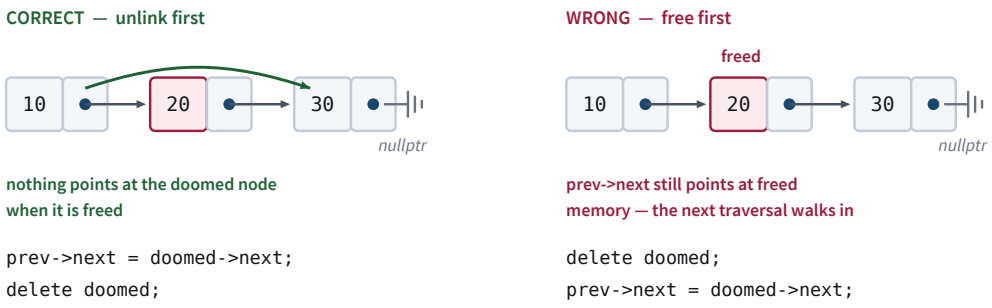


Figure 26.1 The order rule for deletion. Unlink the node by rerouting its predecessor around it, and only then `delete` it. Deleting first leaves `prev->next` pointing at freed memory, and the next traversal walks into it.

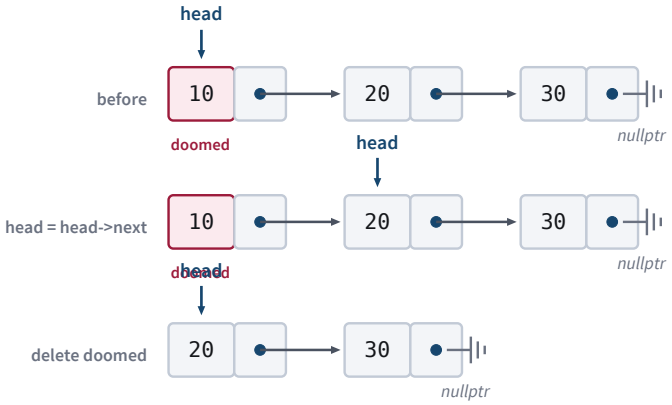
KEY INSIGHT | The two rules of deletion

Rule 1 — unlink before you free. Reroute `prev->next` past the doomed node *first*. Only when nothing points at it may you `delete` it.

Rule 2 — you need the predecessor, not the node. To unlink node *X* you must change *X*'s predecessor's `next`. A singly linked list cannot go backwards, so you must either walk with a trailing `prev` pointer, or stop one node early.

Rule 2 is why every deletion function below looks more complicated than its insertion counterpart.

1. Delete the first node



O(1) · the one-node list works naturally, because head->next is already nullptr

Figure 26.2 Deleting the first node. Save the head, move `head` on by one, then free what you saved. Three lines, *O*(1), and the one-node list falls out correctly because `head->next` is already `nullptr`.

Delete the first node — O(1)

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

void deleteFront(Node*& head) {
    if (head == nullptr) return;    // nothing to delete
    Node* doomed = head;            // 1. remember it
    head = head->next;              // 2. unlink: head skips past it
    delete doomed;                  // 3. now free it
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
```

```
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = buildList({10, 20, 30});
    deleteFront(head);
    printList(head);
    deleteFront(head);
    deleteFront(head);
    printList(head);                // now empty
    deleteFront(head);              // deleting from empty is harmless
    printList(head);
    freeList(head);
    return 0;
}
```

```
20 -> 30
(empty)
(empty)
```

COMMON PITFALL | The two ways to get these three lines wrong

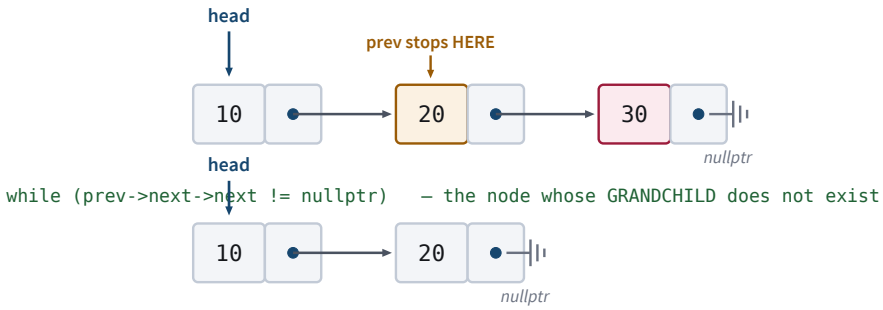
```
delete head;            // BAD freed first...
head = head->next;      //   ...then read through the freed pointer
```

Classic use-after-free. It often appears to work, because the bytes still hold the old value.

```
head = head->next;      // BAD unlinked...
// ...and never freed: the old first node is now unreachable
```

A leak. No error, no crash, and invisible until you run AddressSanitizer.
Save, unlink, free — in that order. It is three lines because it needs to be three lines.

2. Delete the last node



the single-node list is a genuine special case: there is no second-to-last node

O(n) even with a tail pointer — you still have to FIND the new tail

Figure 26.3 Deleting the last node. You must stop on the **second-to-last** node, because that is the node whose `next` must become `nullptr` . The single-node list is a genuine special case, because there is no second-to-last node.

Delete the last node — O(n)

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

void deleteBack(Node*& head) {
    if (head == nullptr) return;              // empty: nothing to do
    if (head->next == nullptr) {              // SPECIAL CASE: exactly
        one node
        delete head;
        head = nullptr;                       // the list is now empty
        return;
    }
    Node* prev = head;
    while (prev->next->next != nullptr)       // stop on the
        SECOND-TO-LAST
        prev = prev->next;
    delete prev->next;                        // free the last node
    prev->next = nullptr;                     // and mark the new end
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
```

```
int main() {
    Node* head = buildList({10, 20, 30});
    deleteBack(head);
    printList(head);
    deleteBack(head);
    printList(head);
    deleteBack(head);                          // down to empty
    printList(head);
    deleteBack(head);                          // and again: harmless
    printList(head);
    return 0;
}
```

```
10 -> 20
10
(empty)
(empty)
```

KEY INSIGHT | `prev->next->next != nullptr` **— read it slowly**
We need the node *before* the last one. Equivalently: the node whose **grandchild** does not exist.

- `prev->next == nullptr` would stop on the last node — too far, and then `prev->next = nullptr` would do nothing useful.
- `prev->next->next == nullptr` stops one earlier, on exactly the node we must modify.

This is Chapter 25's principle again: *to modify a node, stop on it.* Here the node to modify is the second-to-last, so the condition looks one step further ahead.
The single-node case must be handled separately precisely because `head->next` is already `nullptr`, so `prev->next->next` would dereference null.

Cost. $O(n)$ time, $O(1)$ space. This is a weakness of the singly linked list: even with a `tail` pointer, deleting the last node is $O(n)$, because you still have to *find* the new tail. A **doubly** linked list makes it $O(1)$, which is one of the main reasons that variant exists (Chapter 27).

3. Delete at a position

Delete the node at index pos

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

bool deleteAt(Node*& head, int pos) {
    if (head == nullptr || pos < 0) return false;
```

```

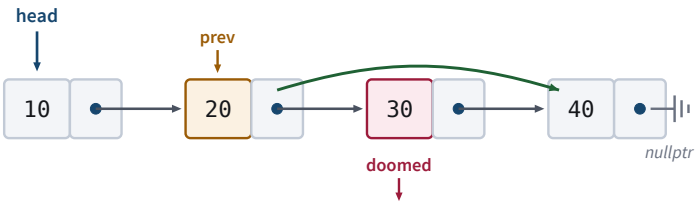
    if (pos == 0) {                     // the head itself changes
        Node* doomed = head;
        head = head->next;
        delete doomed;
        return true;
    }
    Node* prev = head;                  // walk to the node BEFORE pos
    for (int i = 0; i < pos - 1; ++i) {
        if (prev->next == nullptr) return false;   // pos is past the end
        prev = prev->next;
    }
    if (prev->next == nullptr) return false;       // nothing at pos

    Node* doomed = prev->next;
    prev->next = doomed->next;          // 1. unlink: skip over it
    delete doomed;                      // 2. then free
    return true;
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = buildList({10, 20, 30, 40, 50});
    deleteAt(head, 2);                  // remove 30
    printList(head);
    deleteAt(head, 0);                  // remove the head
    printList(head);
    deleteAt(head, 2);                  // remove the last
    printList(head);
    std::cout << "pos 9: " << (deleteAt(head, 9) ? "ok" : "rejected") <<
        "\n";
    freeList(head);
    return 0;
}
```

```
10 -> 20 -> 40 -> 50
20 -> 40 -> 50
20 -> 40
pos 9: rejected
```



`prev->next = doomed->next;` **then** **`delete doomed;`**

index 0 still needs its own branch, because the head has no predecessor

Figure 26.4 Deleting at index 2. `prev` stops on index 1; the node at index 2 is unlinked by pointing `prev->next` at its successor, and only then freed.

4. Delete by value

The version you will actually be asked for, and it shows the `prev` / `curr` walking pattern clearly.

Delete the first node holding a value

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Removes the FIRST node equal to value. Returns whether one was found.
bool deleteValue(Node*& head, int value) {
    if (head == nullptr) return false;

    if (head->data == value) {           // the head matches
        Node* doomed = head;
        head = head->next;
        delete doomed;
        return true;
    }
    Node* prev = head;
    Node* curr = head->next;             // curr is always prev->next
    while (curr != nullptr) {
        if (curr->data == value) {
            prev->next = curr->next;     // 1. unlink
            delete curr;                 // 2. free
            return true;
        }
        prev = curr;                     // advance BOTH, keeping them
            adjacent
        curr = curr->next;
    }
```

```
    return false;                       // not present
}

// Removes EVERY node equal to value. Returns how many were removed.
int deleteAllValues(Node*& head, int value) {
    int removed = 0;
    while (head != nullptr && head->data == value) {   // strip matching
        heads
        Node* doomed = head;
        head = head->next;
        delete doomed;
        ++removed;
    }
    if (head == nullptr) return removed;

    Node* prev = head;
    while (prev->next != nullptr) {
        if (prev->next->data == value) {
            Node* doomed = prev->next;
            prev->next = doomed->next;
            delete doomed;
            ++removed;
            // do NOT advance prev - the new prev->next has not been
                checked
        } else {
            prev = prev->next;
        }
    }
    return removed;
}

void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* head = buildList({10, 20, 30, 20});
    std::cout << deleteValue(head, 20) << " ";
    printList(head);
    std::cout << deleteValue(head, 99) << " ";
    printList(head);

    Node* h2 = buildList({7, 7, 1, 7, 2, 7, 7});
    std::cout << "removed " << deleteAllValues(h2, 7) << ": ";
    printList(h2);

```

```
    Node* h3 = buildList({5, 5, 5});
    std::cout << "removed " << deleteAllValues(h3, 5) << ": ";
    printList(h3);

    freeList(head);
    freeList(h2);
    freeList(h3);
    return 0;
}
```

```
1 10 -> 30 -> 20
0 10 -> 30 -> 20
removed 5: 1 -> 2
removed 3: (empty)
```

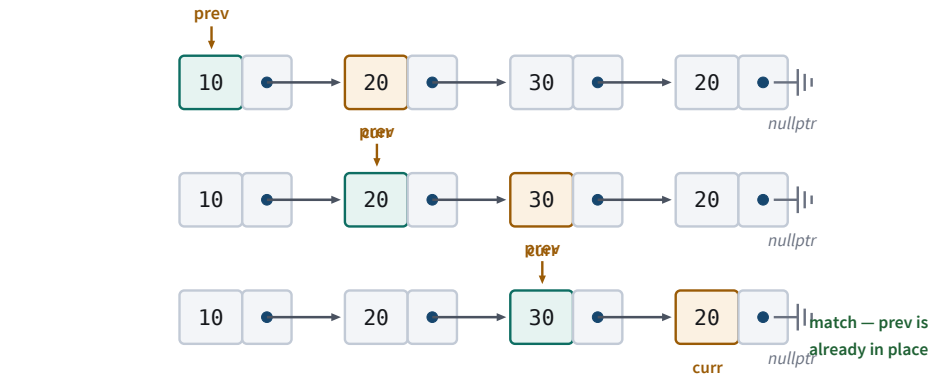


Figure 26.5 The `prev` / `curr` pair. They move together, always adjacent, so that the moment `curr` matches, `prev` is already the node that must be rerouted. This is the standard walking shape for any operation that must modify a node's predecessor.

COMMON PITFALL | Advancing `prev` after a deletion

In `deleteAllValues`, look at the comment: after removing a node, `prev` **must not advance**.

The new `prev->next` is a node that has never been examined. Advancing past it skips it, so `{7, 7}` would lose only the first 7 — exactly the array bug from Chapter 6, in a new costume.

The symptom is distinctive: consecutive duplicates are only partly removed. If your output has one leftover of each run, this is why.

KEY INSIGHT | Why the head needs a separate branch, every time
Every function in this chapter begins with a special case for the head, and the reason is structural: **the head has no predecessor.** Every other node is unlinked by modifying `prev->next`; the first node is unlinked by modifying `head` itself, which is a different variable of a different kind.
That asymmetry is irritating, it doubles the number of branches, and it is where most linked list bugs live.
It is also completely removable. Chapter 29 introduces the **dummy node**: a fake node placed before the real head, so that every real node — including the first — has a predecessor. With it, `deleteValue` loses its special case and becomes a single uniform loop. It is the single most useful trick in linked list programming, and it exists precisely because of the paragraph above.

Clearing the whole list

Free every node, exactly once

```
void clearList(Node*& head) {
    Node* curr = head;
    while (curr != nullptr) {
        Node* doomed = curr;        // 1. remember
        curr = curr->next;          // 2. advance PAST it first
        delete doomed;              // 3. then free
    }
    head = nullptr;                 // 4. the list is empty - say so
}
```

Step 4 matters. Without it, `head` dangles: it holds the address of a freed node, and the next `printList(head)` walks into freed memory. One line turns a subtle bug into an impossible one.

The three deletion bugs

Bug	Cause	Symptom
Leak	unlinked but never `delete`d	nothing visible; memory grows; AddressSanitizer reports it
Use after free	`delete` before reading `next`	usually works, then fails later, seemingly at random
Double free	two `delete`s of one node	crash, or heap corruption
Dangling head	freed the nodes, left `head` set	the next traversal reads freed memory

Table 26.1 What each deletion bug looks like from the outside. The first is silent, the second is intermittent, and the third usually crashes — which makes the third the least dangerous.

WATCH OUT | **Check your work mechanically**

```
g++ -std=c++17 -fsanitize=address,undefined -g list.cpp -o list && ./list
```

Every program in this chapter passes that cleanly. If yours does not, the report names the exact line of the `new` that leaked or the `delete` that was repeated.
Run it on every exercise in this chapter. Linked lists are the first place most people meet manual memory management, and this is how you find out whether you got it right.

Cost summary

Deletion	Walk needed	Time	Changes `head`?	Special cases
First node	none	$O(1)$	yes	empty list
Last node	to the second-to-last	$O(n)$	if one node	empty; exactly one node
At position p	$p-1$ steps	$O(n)$	if $p=0$	$p=0$; p out of range
By value	until found	$O(n)$	if the head matches	head matches; not found
All of one value	the whole list	$O(n)$	if heads match	runs of matches
Whole list	the whole list	$O(n)$	yes	none

Table 26.2 The four deletions. Only the front is $O(1)$, and that is the mirror image of insertion, where only the front and a held node were $O(1)$.

CHAPTER RECAP | **Chapter 26 in six lines**

- **Unlink first, then free.** Reroute `prev->next` before `delete`.
- To delete a node you need its **predecessor**, which is why deletion walks with `prev` and `curr` together.
- The head always needs its own branch, because it has no predecessor — and the dummy node of Chapter 29 removes that.
- When removing *all* matches, do **not** advance `prev` after a removal.
- After freeing a list, set `head = nullptr`.
- Compile every linked list exercise with `-fsanitize=address` and believe what it tells you.

EXERCISES | **Practise**

1. Draw the three pointer states for `deleteFront` on `10 -> 20 -> 30`. Which line would be wrong to move, and what would happen?
2. Why does `deleteBack` need the **second-to-last** node? What would `while (prev->next != nullptr)` leave `prev` pointing at?
3. Write `deleteLastOccurrence(Node*& head, int value)`. How does it differ from deleting the first, and can it be done in one pass?
4. In `deleteAllValues`, what goes wrong if you advance `prev` after a removal? Give the smallest input that shows it.
5. Write `removeDuplicates(Node*& head)` for a **sorted** list, freeing the removed nodes. Which array problem is this?
6. Run every program in this chapter under `-fsanitize=address`. Then deliberately remove one `delete` and confirm the report names it.
7. Explain why deleting the last node is $O(n)$ even with a `tail` pointer. What exactly would a doubly linked list give you here?
8. Write `deleteFront` so that it **returns** the deleted value rather than discarding it. What should it return on an empty list, and what are the three reasonable designs?

CHAPTER

27

Doubly and Circular Linked Lists

One more pointer, and one missing nullptr

CORE PREREQS: Chapter 26

Two variations, each solving one specific weakness of the singly linked list. A doubly linked list adds a `prev` pointer, which makes backwards traversal possible and deletion of a held node $O(1)$. A circular list removes the final `nullptr`, which suits anything that goes round. Both change every loop condition you have learned, and this chapter is mostly about getting those right.

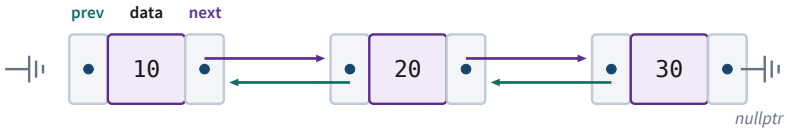
WHAT YOU WILL LEARN

- Define a doubly linked node and keep both pointers consistent
- Insert and delete in a doubly linked list, and count the pointer updates
- Explain why deleting a held node is $O(1)$ doubly and $O(n)$ singly
- Write the correct traversal loop for a circular list
- Build a circular doubly linked list, which is what `std::list` actually is
- Decide which variant a problem needs

The doubly linked node

A node that points both ways

```
struct DNode {
    int data;
    DNode* next;
    DNode* prev;
    DNode(int value) { data = value; next = nullptr; prev = nullptr; }
};
```



the invariant: a->next == b ⟺ b->prev == a

every operation must leave this true in BOTH directions

Figure 27.1 A doubly linked list. Every link exists twice, once in each direction, and the invariant is that they always agree: if `a->next == b` then `b->prev == a`. The first node's `prev` and the last node's `next` are both `nullptr`.

DEFINITION | The doubly linked invariant

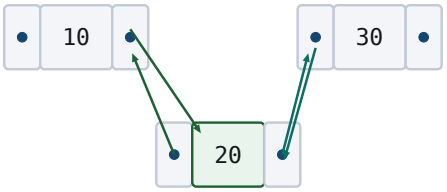
For every pair of adjacent nodes a and b:

$$a\to\text{next}=b \Leftrightarrow b\to\text{prev}=a$$

Every operation must leave this true. That is the entire difficulty of a doubly linked list: there are twice as many pointers to update, and updating only one side produces a list that traverses correctly forwards and corrupts going backwards — a bug that can hide for a long time.

Check both directions when you test. Print the list forwards and backwards and compare; if they are not mirror images, an update was missed.

Insertion in a doubly linked list




```
1. fresh->next = node->next     3. node->next->prev = fresh
2. fresh->prev = node           4. node->next = fresh
```

four assignments instead of two — and missing one leaves the list correct forwards and broken backwards

Figure 27.2 Inserting into a doubly linked list. Four pointers change, not two: the new node's `next` and `prev`, the predecessor's `next`, and the successor's `prev`. The order still follows the rule — connect the new node to its neighbours first.

Insert at the front and after a node

```cpp
#include <iostream>

struct DNode {
    int data;
    DNode* next;
    DNode* prev;
    DNode(int value) { data = value; next = nullptr; prev = nullptr; }
};

void insertFront(DNode*& head, int value) {
    DNode* node = new DNode(value);
    node->next = head;                        // 1. new node forwards
    if (head != nullptr) head->prev = node;   // 2. old head points back
    head = node;                              // 3. head moves
    // node->prev stays nullptr - it is the first node
}

// Insert after a node you already hold. O(1), no walking.
void insertAfter(DNode* node, int value) {
    if (node == nullptr) return;
    DNode* fresh = new DNode(value);
    fresh->next = node->next;                 // 1. forwards
    fresh->prev = node;                        // 2. backwards
    if (node->next != nullptr)
        node->next->prev = fresh;              // 3. the successor points back
    node->next = fresh;                        // 4. the predecessor points on
}

void printForward(DNode* head) {
    for (DNode* c = head; c; c = c->next) std::cout << c->data << (c->next ? " <-> " : "\n");
    if (!head) std::cout << "(empty)\n";
}

void printBackward(DNode* head) {
    if (head == nullptr) { std::cout << "(empty)\n"; return; }
    DNode* tail = head;
    while (tail->next != nullptr) tail = tail->next;      // walk to the end
    for (DNode* c = tail; c; c = c->prev) std::cout << c->data << (c->prev ? " <-> " : "\n");
}

void freeList(DNode* head) { while (head) { DNode* d = head; head = head->next; delete d; } }

int main() {
    DNode* head = nullptr;
    insertFront(head, 30);
    insertFront(head, 20);
    insertFront(head, 10);
    printForward(head);
```

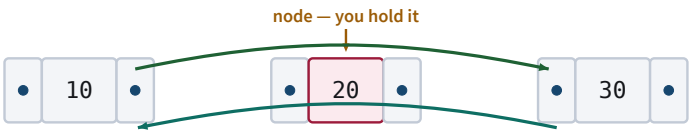
```
    printBackward(head);                    // must be the exact mirror

    insertAfter(head->next, 25);            // after the node holding 20
    printForward(head);
    printBackward(head);
    freeList(head);
    return 0;
}
```

```
10 <-> 20 <-> 30
30 <-> 20 <-> 10
10 <-> 20 <-> 25 <-> 30
30 <-> 25 <-> 20 <-> 10
```

Printing both directions and checking they mirror is the test that catches a missed `prev` update. Make it a habit.

Deletion: the reason doubly linked lists exist




```
node->prev->next = node->next;    node->next->prev = node->prev;
```

no walking at all → O(1). In a singly linked list you would have to find the predecessor: O(n).

this one fact is why LRU caches and std::list are doubly linked

Figure 27.3 Deleting a node you hold, in $O(1)$. Because the node knows its own predecessor, you can reroute both neighbours around it without walking anywhere. In a singly linked list this is $O(n)$, because finding the predecessor means starting from the head.

Delete a node you already hold — O(1)

```
#include <iostream>

struct DNode {
    int data;
    DNode* next;
    DNode* prev;
    DNode(int value) { data = value; next = nullptr; prev = nullptr; }
};

// Unlink and free `node`. O(1) - no traversal at all.
void deleteNode(DNode*& head, DNode* node) {
    if (node == nullptr) return;

```

```
    if (node->prev != nullptr) node->prev->next = node->next;
    else                       head = node->next;        // it was the
        head

    if (node->next != nullptr) node->next->prev = node->prev;
    // if it was the tail, there is nothing after it to fix

    delete node;
}

DNode* buildD(std::initializer_list<int> vs) {
    DNode* head = nullptr; DNode* tail = nullptr;
    for (int v : vs) {
        DNode* n = new DNode(v);
        if (!head) head = tail = n;
        else { tail->next = n; n->prev = tail; tail = n; }
    }
    return head;
}
void printForward(DNode* head) {
    for (DNode* c = head; c; c = c->next) std::cout << c->data << (
        c->next ? " <-> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(DNode* head) { while (head) { DNode* d = head; head =
    head->next; delete d; } }

int main() {
    DNode* head = buildD({10, 20, 30, 40});
    deleteNode(head, head->next);            // remove 20, a middle node
    printForward(head);
    deleteNode(head, head);                  // remove the head
    printForward(head);
    DNode* last = head; while (last->next) last = last->next;
    deleteNode(head, last);                  // remove the tail
    printForward(head);
    deleteNode(head, head);                  // the only node left
    printForward(head);
    freeList(head);
    return 0;
}
```

```
10 <-> 30 <-> 40
30 <-> 40
30
(empty)
```

KEY INSIGHT | **The one genuine superpower**
Compare the two signatures:

```
void deleteNode(Node*&  head, Node*  node);   // singly: must FIND the
    predecessor, O(n)
void deleteNode(DNode*& head, DNode* node);   // doubly: node->prev is
    right there, O(1)
```

That is the whole justification for the extra pointer. And it is not academic — it is exactly what an **LRU cache** needs: when an entry is used, you must move its node to the front, which means deleting it from the middle in $O(1)$. A hash map gives you the node pointer; a doubly linked list lets you unlink it immediately. Neither structure can do it alone.
If you ever wondered why `std::list` is doubly linked despite the memory cost, this is why: `erase(iterator)` is $O(1)$, and it could not be otherwise.

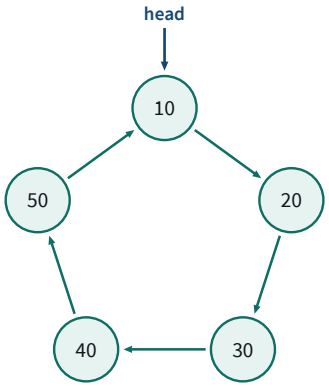
COMMON PITFALL | **The two-branch shape**
Both branches in `deleteNode` are necessary and are easy to half-write:

- `node->prev == nullptr` means the node is the **head**, so `head` must move.
- `node->next == nullptr` means the node is the **tail**, so there is no successor to fix.

A node that is both — the only node in the list — hits both branches, and the list correctly becomes empty. Test that case; it is the one people forget.

Circular linked lists

DEFINITION | **Circular linked list**
A list in which the last node's `next` points back to the **first** node instead of to `nullptr`. There is no end. Every node has a successor, and traversal must be stopped by a condition other than reaching null.




```
while (curr != nullptr)
✗ there is no nullptr — this never ends

do {
    visit(curr);
    curr = curr->next;
} while (curr != head);

✓ stops when it comes back round

a do-while, because curr == head
is already true at the start
```

Figure 27.4 A circular list. Because the last node points back to the head, `while (curr != nullptr)` never terminates. The correct shape is a `do ... while (curr != head)`, which visits every node exactly once.

WATCH OUT | Every loop condition changes

```
while (curr != nullptr) { visit(curr); curr = curr->next; }   // BAD
    runs forever
```

There is no `nullptr` to find. The program hangs, or prints the same values until you stop it.
The correct form is:

```
if (head != nullptr) {
    Node* walker = head;
    do {
        process(walker->data);
        walker = walker->next;
    } while (walker != head);        // stop when we come back round
}
```

A `do ... while` is required rather than a `while`, because `curr == head` is true at the very start — a plain `while` would process nothing. And the `if (head != nullptr)` is required because a `do` body always runs once, which would dereference null on an empty list.

A circular list, traversed correctly

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Build a circular list; returns the head.
Node* buildCircular(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) {
        Node* n = new Node(v);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    if (tail) tail->next = head;            // close the ring
    return head;
}

void printCircular(Node* head) {
    if (head == nullptr) { std::cout << "(empty)\n"; return; }
    Node* curr = head;
    do {
        std::cout << curr->data << " ";
        curr = curr->next;
    } while (curr != head);
    std::cout << "\n";
}

int countCircular(Node* head) {
```

```
    if (head == nullptr) return 0;
    int n = 0;
    Node* curr = head;
    do { ++n; curr = curr->next; } while (curr != head);
    return n;
}

void freeCircular(Node* head) {
    if (head == nullptr) return;
    Node* curr = head->next;
    while (curr != head) {                  // free everything but the
        head
        Node* doomed = curr;
        curr = curr->next;
        delete doomed;
    }
    delete head;                            // then the head itself
}

int main() {
    Node* head = buildCircular({10, 20, 30});
    printCircular(head);
    std::cout << "count = " << countCircular(head) << "\n";

    // Walking twice round shows there is no end.
    Node* c = head;
    for (int i = 0; i < 7; ++i) { std::cout << c->data << " "; c =
        c->next; }
    std::cout << "\n";

    freeCircular(head);
    printCircular(nullptr);
    return 0;
}
```

```
10 20 30
count = 3
10 20 30 10 20 30 10
(empty)
```

Note `freeCircular`: breaking the ring correctly requires freeing everything *except* the head first, then the head. Using the ordinary `freeList` loop on a circular list would run forever, freeing nodes it had already freed.

KEY INSIGHT | Where circular lists are genuinely used

Round-robin scheduling. Each process gets a time slice; when the last one finishes, you are already back at the first with no special case.

Ring buffers. A fixed set of slots reused endlessly — audio buffers, network packet queues, replay buffers in reinforcement learning. (Though these are usually implemented as a circular *array*, which gets the ring behaviour with array cache performance.)

The Josephus problem, and anything else where you repeatedly go round eliminating members.

`std::list` is a *circular doubly linked* list with one sentinel node. The sentinel's `next` is the first element and its `prev` is the last, which means `begin()`, `end()`, insertion and deletion all work with no null checks and no special cases anywhere. It is the dummy-node trick of Chapter 29, taken to its conclusion.

Choosing a variant

	Singly	Doubly	Circular (singly)
Pointers per node	1	2	1
Traverse backwards	no	yes	no (but you can loop round)
Delete a held node	$O(n)$	$O(\mathbf{1})$	$O(n)$
Delete the last node	$O(n)$	$O(1)$ with a tail	$O(n)$
End-of-list test	`== nullptr`	`== nullptr`	`== head`
Empty-list test	`head == nullptr`	`head == nullptr`	`head == nullptr`
Memory for one `int`	4 + 8 = 12 bytes	4 + 16 = 20 bytes	4 + 8 = 12 bytes
Typical use	the default; interviews	LRU caches, `std::list`, editors	round-robin, ring buffers

Table 27.1 The three variants compared. Note the last row: the extra memory of a doubly linked list is not small when the payload is a single `int`.

CHAPTER RECAP | **Chapter 27 in six lines**

- A doubly linked node adds `prev`, and the invariant is `a->next == b` ⇔ `b->prev == a`.
- Doubly linked insertion and deletion update **four** pointers, not two. Test by printing both directions.
- Deleting a node you hold is $O(1)$ doubly and $O(n)$ singly. That single fact is why LRU caches and `std::list` use it.
- A circular list has no `nullptr`, so `while (curr != nullptr)` hangs. Use `do { ... } while (curr != head)`.
- Freeing a circular list must break the ring, or it frees nodes twice.
- `std::list` is a circular doubly linked list with a sentinel, which is how it avoids special cases entirely.

EXERCISES | **Practise**

1. Draw a three-node doubly linked list and label all six pointers, including the two `nullptr`s.
2. Write `insertBefore(DNode* node, int value)` for a doubly linked list. Why is this impossible in $O(1)$ for a singly linked list?
3. Write `deleteNode` for a **singly** linked list taking only a `Node*`. What must you do, and what is the complexity? (Chapter 31 has a famous trick for this.)
4. Write a function that checks the doubly linked invariant holds for a whole list, returning `false` at the first violation.
5. Convert a singly linked list into a circular one, then back again. How do you detect that a list is already circular?
6. Solve the Josephus problem with a circular list: n people in a ring, every k-th is removed, who is last? What does your solution cost?
7. Why is `std::list::erase(it)` $O(1)$ while `std::vector::erase(it)` is $O(n)$? Why is `std::list::operator[]` not provided at all?
8. A doubly linked list of one `int` per node uses 20 bytes for 4 bytes of data. At what payload size does the overhead stop mattering, in your judgement?

CHAPTER

28

Mastering Pointer Manipulation: Reversal

Four lines, traced to death, until they are obvious

CORE PREREQS: Chapters 24 to 26

Reversing a linked list is the most-asked linked list question there is, and the most instructive. Four lines, three pointers, and every single one of them matters. This chapter traces the loop on a three-node list showing every variable at every moment, explains exactly why the saved pointer is necessary, demonstrates what each wrong ordering actually produces, and then does it recursively.

WHAT YOU WILL LEARN

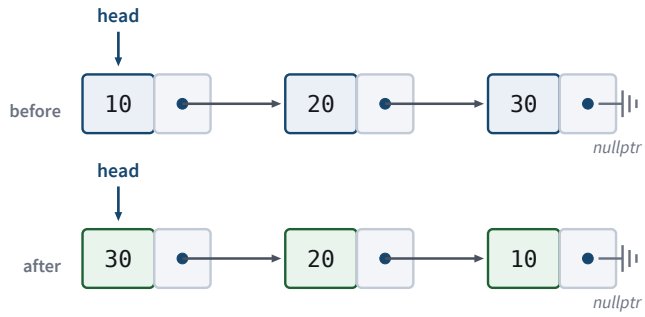
- Reverse a singly linked list iteratively, and explain every line
- State the value of `prev`, `curr` and `nextNode` at every point of every iteration
- Explain precisely why `nextNode` must be saved before `curr->next` is changed
- Predict what each incorrect ordering of the four lines produces
- Reverse a list recursively, and say what that costs
- Reverse only part of a list

The problem

DEFINITION | Reverse a linked list

Turn `10 -> 20 -> 30 -> nullptr` into `30 -> 20 -> 10 -> nullptr`, in place, without allocating any new nodes.

"In place" means the nodes stay where they are in memory; only the `next` pointers change. The node holding 10 is still the same object at the same address — it is simply no longer first.



no node moves in memory and none is created — only the next pointers change

head ends up naming what used to be the last node

Figure 28.1 Before and after. No node moves in memory and no node is created; every `next` pointer is turned around, and `head` ends up naming what used to be the last node.

Why this is harder than reversing an array

In Chapter 6 we reversed an array with two pointers swapping inwards. That worked because an array can be read from either end.

A singly linked list cannot. You can only go forwards, and the moment you turn a node's `next` pointer around, **you have destroyed your only route to the rest of the list**.

> **KEY INSIGHT | The difficulty, stated exactly**
> To reverse the link from node *A* to node *B*, you must set `A->next` to point at whatever came *before* *A*.
> But `A->next` is currently the only thing in the entire program that knows where *B* is. Overwrite it, and *B* and everything after it is unreachable.
> **Therefore: before you overwrite `A->next`, you must save it somewhere.**
> That single sentence is the whole algorithm. The three pointers are: where you came from (`prev`), where you are (`curr`), and where you were going (`nextNode`).

The code

Iterative reversal — the four lines

```
Node* prev = nullptr;
Node* curr = head;

while (curr != nullptr) {
    Node* nextNode = curr->next;   // 1. SAVE the rest of the list
    curr->next = prev;             // 2. REVERSE this node's pointer
    prev = curr;                   // 3. prev moves forward
    curr = nextNode;               // 4. curr moves forward
}

head = prev;                       // prev is now the last node we saw
```

Four lines inside the loop. Learn what each one is *for*, not what order they come in — the order then becomes forced.

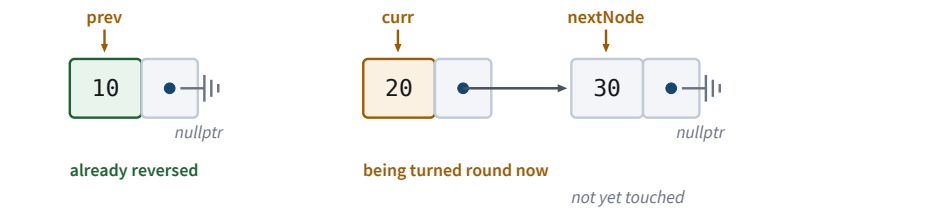


Figure 28.2 The three pointers at the moment the loop body begins. `prev` is the part already reversed, `curr` is the node being turned around now, and `nextNode` is the part not yet touched. The loop advances all three by one node each iteration.

The complete trace

List: `10 -> 20 -> 30 -> nullptr`. We show every variable at the start and end of every iteration, exactly as asked.

Before the loop

Variable	Value
`prev`	`nullptr`
`curr`	node(10)
the list	`10 -> 20 -> 30 -> nullptr`

`prev` starts as `nullptr` because the node that ends up **last** must have `next == nullptr` — and the first node we process is exactly the node that will end up last. Starting `prev` at `nullptr` makes that happen automatically.

Iteration 1 — `curr` is node(10)

Line	Statement	`prev`	`curr`	`nextNode`	Links now
—	at entry	`nullptr`	10	—	`10->20->30->∅`
1	`nextNode = curr->next`	`nullptr`	10	**20**	`10->20->30->∅`
2	`curr->next = prev`	`nullptr`	10	20	`10->∅`, `20->30->∅`
3	`prev = curr`	**10**	10	20	unchanged

Line	Statement	prev	curr	nextNode	Links now
4	`curr = nextNode`	10	**20**	20	unchanged

Table 28.1 Iteration 1 line by line. After line 2, node(10) points at nothing, which is correct: it is destined to be the last node.

Look carefully at line 2. The list has just been **split in two**: node(10) now points at `nullptr`, and `20 -> 30` is a separate chain. The *only* thing still holding on to that second chain is `nextNode`.

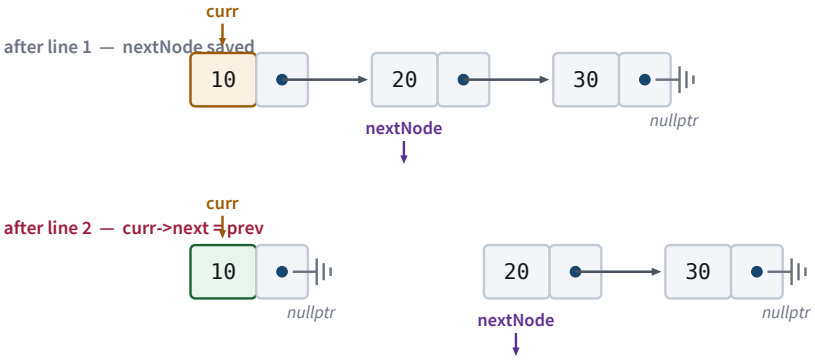


Figure 28.3 Iteration 1. After line 2 the list is temporarily in two pieces. `nextNode` is the only pointer to the second piece — which is precisely why it had to be saved on line 1.

Iteration 2 — `curr` is node(20)

Line	Statement	prev	curr	nextNode	Links now
—	at entry	10	20	20	`10->∅`, `20->30->∅`
1	`nextNode = curr->next`	10	20	**30**	unchanged
2	`curr->next = prev`	10	20	30	`20->10->∅`, `30->∅`
3	`prev = curr`	**20**	20	30	unchanged
4	`curr = nextNode`	20	**30**	30	unchanged

The reversed portion is growing: `20 -> 10 -> nullptr`. `prev` always points at the **head of the reversed portion so far**.

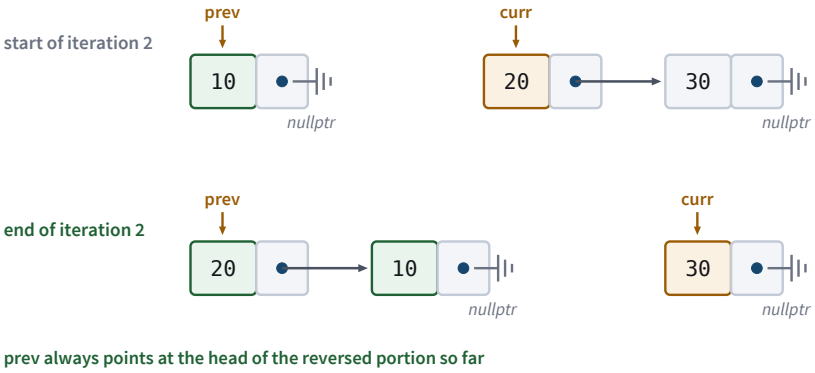


Figure 28.4 Iteration 2. The reversed chain `20 -> 10` is being built behind `prev`, and the untouched remainder sits ahead of `curr`.

Iteration 3 — `curr` is node(30)

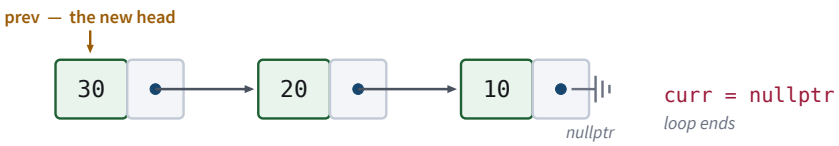
Line	Statement	`prev`	`curr`	`nextNode`	Links now
—	at entry	20	30	30	`20->10->∅`, `30->∅`
1	`nextNode = curr->next`	20	30	`nullptr`	unchanged
2	`curr->next = prev`	20	30	`nullptr`	`30->20->10->∅`
3	`prev = curr`	**30**	30	`nullptr`	unchanged
4	`curr = nextNode`	30	`nullptr`	`nullptr`	unchanged

The loop ends

`curr` is `nullptr`, so the condition fails. The list is fully reversed, and `prev` points at node(30) — the new first node.

```
head = prev;      // without this line, head still names node(10),
                  // which is now the LAST node: head would be a one-node list
```

COMMON PITFALL | **Forgetting `head = prev;`**
This is the most common way to get reversal "nearly right". Every pointer is correctly turned around, but `head` still names the old first node — which is now the tail.
The symptom: the function appears to delete everything but one element. Printing gives `10` instead of `30 -> 20 -> 10`. If you see that exact symptom, you have forgotten the last line.



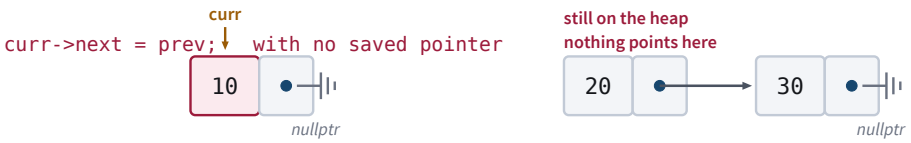

```
head = prev;
```
forget this and head still names node(10), which is now the LAST node — you get a one-element list

Figure 28.5 The end state. `curr` has fallen off the end, and `prev` has been left pointing at the last node processed — which is the new head.

Why `nextNode` must be saved first

We have now seen it happen; here it is as an argument.

> **KEY INSIGHT | The necessity of line 1**
> At the moment line 2 runs, `curr->next` is overwritten.
> Before that instant, the only pointer in the whole program to the unreversed remainder of the list is `curr->next` itself. `prev` points backwards. `head` points at the original first node, which is behind us. Nothing else exists.
> So overwriting `curr->next` without first copying it elsewhere makes the entire remainder unreachable: a leak, and a reversal that produces a list of length 1.
> `nextNode` is a lifeline thrown ahead before the rope behind is cut.



prev points backwards · head is behind us · curr->next has just been overwritten

nextNode is a lifeline thrown ahead before the rope behind is cut

Figure 28.6 What happens without `nextNode`. Overwriting `curr->next` severs the only connection to the rest of the list; nodes 20 and 30 are still on the heap, with nothing pointing at them.

What each wrong ordering actually produces

The brief way to learn this is to try it. Here is what each mistake gives you on `10 -> 20 -> 30`.

Mistake A: reverse before saving

```
curr->next = prev;              // BAD the rest of the list is now unreachable
Node* nextNode = curr->next;    //   this reads prev, not the next node!
prev = curr;
curr = nextNode;
```

After the first iteration, `nextNode` is `prev`, which is `nullptr`. So `curr` becomes `nullptr`, the loop ends immediately, and `head = prev` gives a **one-node list containing 10**. Nodes 20 and 30 are leaked.

Symptom: reversal returns a single element.

Mistake B: advance `curr` before saving `prev`

```
Node* nextNode = curr->next;
curr->next = prev;
curr = nextNode;                // BAD curr moved on...
prev = curr;                    //   ...so prev is now the SAME node as curr
```

`prev` and `curr` end up equal every iteration, so each node's `next` is set to itself. The result is a node pointing at itself: an infinite loop the next time anything traverses the list.

Symptom: printing hangs, repeating one value forever.

Mistake C: forget `head = prev`

Covered above. **Symptom:** the list becomes one element long.

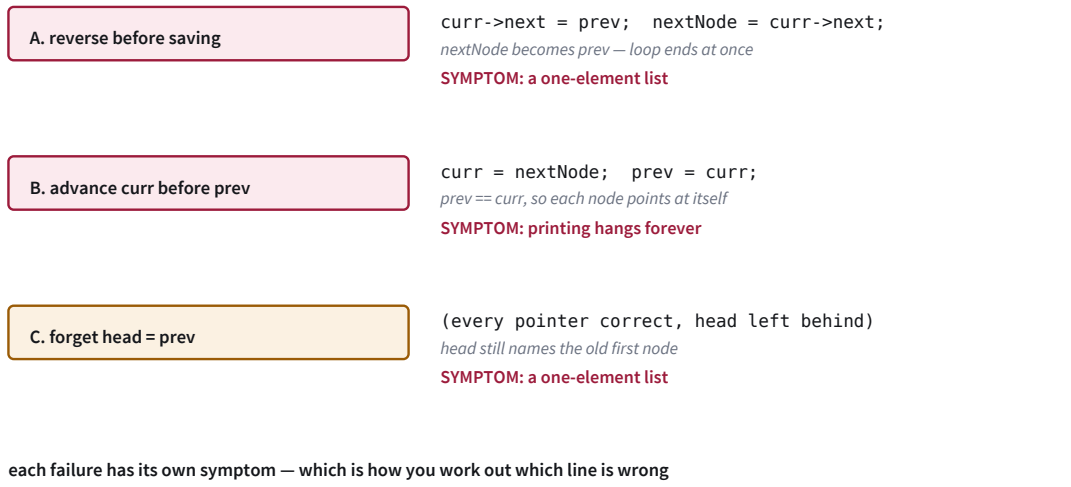


Figure 28.7 The three wrong orderings and what each produces. Each failure has a distinctive symptom, which is worth knowing: the symptom tells you which line is wrong.

KEY INSIGHT | Why the correct order is forced
Read the four lines as a dependency chain:

- Line 2 **destroys** `curr->next`, so anything needing it must come first → line 1 before line 2.
- Line 3 needs `curr` to still be the current node, so it must come before line 4 moves `curr`.
- Line 4 needs `nextNode`, which line 1 set.

There is exactly one order satisfying all three constraints. It is not arbitrary and it is not worth memorising — derive it from "what does each line destroy, and who still needs that?" and you will reconstruct it correctly every time, including under interview pressure.

The complete program

Iterative reversal, with every edge case

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

void reverseList(Node*& head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* nextNode = curr->next;   // 1. save the rest
        curr->next = prev;             // 2. reverse this link
        prev = curr;                   // 3. prev catches up
        curr = nextNode;               // 4. curr moves on
    }
    head = prev;                       // prev is the new head
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({10, 20, 30});
```

```
    reverseList(a);  printList(a);  freeList(a);

    Node* b = buildList({1, 2, 3, 4, 5});
    reverseList(b);  printList(b);  freeList(b);

    Node* c = buildList({42});          // one node
    reverseList(c);  printList(c);  freeList(c);

    Node* d = nullptr;                  // empty
    reverseList(d);  printList(d);
    return 0;
}
```

```
30 -> 20 -> 10
5 -> 4 -> 3 -> 2 -> 1
42
(empty)
```

Cost. $O(n)$ time — each node is visited once — and $O(1)$ space: three pointer variables, regardless of list length.

Both the empty list and the one-node list work with no special case. On the empty list, `curr` starts `nullptr`, the loop never runs, and `head = prev` sets `head` to `nullptr`, which it already was. That is the sign of a well-shaped loop.

Recursive reversal

The same job, expressed differently. Worth knowing because interviewers ask for it, and because the trade-off is instructive.

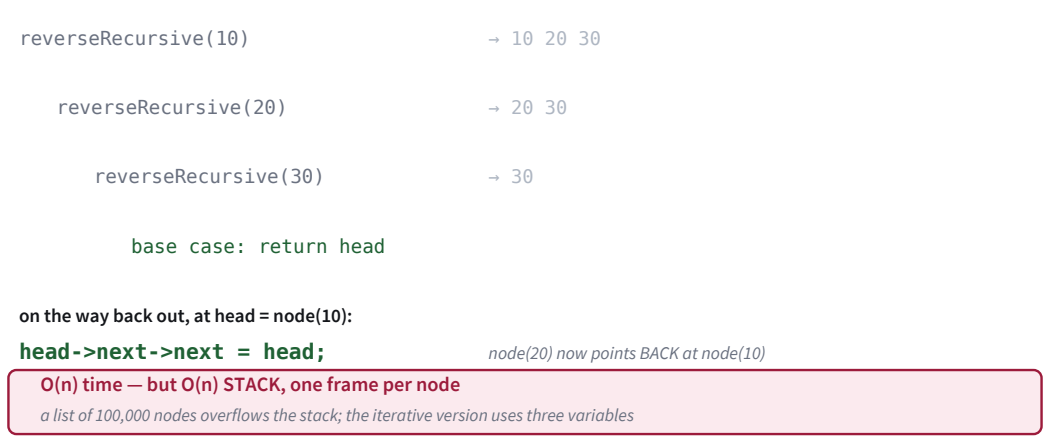


Figure 28.8 Recursive reversal. The recursion walks to the very end to find the new head, then reverses each link on the way back out — `head->next->next = head` turns the arrow around.

Recursive reversal

```
#include <iostream>

struct Node {
    int data;
```

```
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* reverseRecursive(Node* head) {
    // Base case: an empty list, or a single node, is already reversed.
    if (head == nullptr || head->next == nullptr) return head;

    // Reverse everything AFTER head. newHead is the last node of the original.
    Node* newHead = reverseRecursive(head->next);

    head->next->next = head;    // the node after head now points BACK at head
    head->next = nullptr;       // head becomes the new last node

    return newHead;             // the same new head is passed all the way up
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head = head->next; delete d; } }

int main() {
    Node* a = buildList({10, 20, 30});
    a = reverseRecursive(a);  printList(a);  freeList(a);

    Node* b = buildList({1});
    b = reverseRecursive(b);  printList(b);  freeList(b);

    printList(reverseRecursive(nullptr));
    return 0;
}
```

```
30 -> 20 -> 10
1
(empty)
```

KEY INSIGHT | The two lines in the middle, understood
Suppose the list is `10 -> 20 -> 30` and we are at `head = node(10)`.
The recursive call reverses the tail, leaving `30 -> 20` and returning `newHead = node(30)`. But node(10) still points at node(20), and node(20) now points at `nullptr` — so the picture is:

$$30 \to 20 \to \varnothing \text{ and } 10 \to 20$$

Now: `head->next` is node(20). So `head->next->next = head` sets node(20)'s `next` to node(10) — attaching 10 onto the end of the reversed part, giving 30→20→10→20. Almost right, but node(10) still points at node(20), which is a cycle.
`head->next = nullptr` cuts that, leaving 30→20→10→∅.
Both lines are needed, and in that order: the second destroys the `head->next` that the first depends on.

	Iterative	Recursive
Time	$O(n)$	$O(n)$
Auxiliary space	$O(1)$	$O(n)$ — one stack frame per node
Fails on long lists	no	yes — stack overflow around 10^5 nodes
Readability	explicit, slightly fiddly	shorter, but the two middle lines are subtle
Which to give first	**this one**	mention it as an alternative

Table 28.2 Iterative versus recursive reversal. The complexity is the same; the space is not, and on long lists the difference is a crash.

Interviewers often ask for both. Give the iterative version, state that it is $O(1)$ space, then offer the recursive one and **immediately note that it is $O(n)$ stack**. Volunteering that limitation is the part they are listening for.

Reversing part of a list

A common follow-up: reverse only the nodes from position m to position n.

KEY INSIGHT | How to attack it
It is the same four-line loop, run only over the sub-range, plus bookkeeping to stitch the reversed middle back in.
You need to remember two nodes before you start: the node **before** position m (call it `beforeStart`) and the node that is **at** position m (which will end up as the tail of the reversed section).
After reversing the middle:

- `beforeStart->next` must point at the new head of the reversed section;
- the old start's `next` must point at whatever followed position n.

The awkward case is m=1, where there is no `beforeStart` — which is exactly the problem a **dummy node** solves. Chapter 29 introduces it, and then this problem becomes straightforward. Try it now, notice the awkwardness, and read Chapter 29 with the motivation that discomfort gives you.

CHAPTER RECAP | **Chapter 28 in seven lines**

- Reversal turns every `next` around; no node moves and none is created.
- Three pointers: `prev` (reversed so far), `curr` (being turned now), `nextNode` (not yet touched).
- Save `nextNode` **first**, because line 2 destroys the only route to the rest of the list.
- `prev` starts at `nullptr` so the first node correctly becomes the last.
- Finish with `head = prev;` — forgetting it leaves a one-element list.
- Each wrong ordering has a distinct symptom: one element, or an infinite loop.
- Iterative is $O(1)$ space; recursive is $O(n)$ stack and overflows on long lists.

EXERCISES | **Practise**

1. Trace the loop on `1 -> 2 -> 3 -> 4` yourself, writing all three variables after every line. Do not look at the tables.
2. What is the value of `prev` after the loop ends, for a list of length n? For an empty list?
3. Run the reversal on an empty list and a one-node list by hand. Confirm no special case is needed.
4. Implement Mistake A deliberately and run it. Does the output match the symptom described?
5. Write `reverseList` so it **returns** the new head instead of taking `Node*&`. Which signature do you prefer, and which does LeetCode use?
6. Reverse only the first k nodes of a list, leaving the rest alone. What do you need to remember before you start?
7. Write a function that checks whether a list is a palindrome by reversing its second half. What does it cost, and what should it do to the list afterwards?
8. In `reverseRecursive`, what happens if you swap the two middle lines? Predict first, then test.

PART VII

Linked List Patterns and Problems

Dummy nodes, two speeds, and the courage to reverse things. Sixteen interview problems that between them cover almost every linked list question in circulation.

29 The Linked List Patterns

30 Essential Linked List Problems I

31 Essential Linked List Problems II

32 Advanced Linked List Problems

CHAPTER

29

The Linked List Patterns

Six techniques that between them solve almost every list problem

CORE PREREQS: Part VI

Linked list problems have far fewer patterns than array problems, and they repeat relentlessly. Learn the dummy node, the two speeds, pointer reversal, the gap, Floyd's cycle algorithm and merge-and-split, and you have the tools for every problem in Chapters 30 to 32 — including the ones you have not seen.

WHAT YOU WILL LEARN

- Use a dummy node to remove every head-is-special branch
- Use fast and slow pointers to find the middle in one pass
- Use a fixed gap between two pointers to find the k-th node from the end
- Apply Floyd's cycle detection, and **prove** why phase 2 finds the entrance
- Merge two sorted lists, and split one list into two
- Choose recursion knowingly, given its $O(n)$ stack cost

Pattern 1: The dummy node

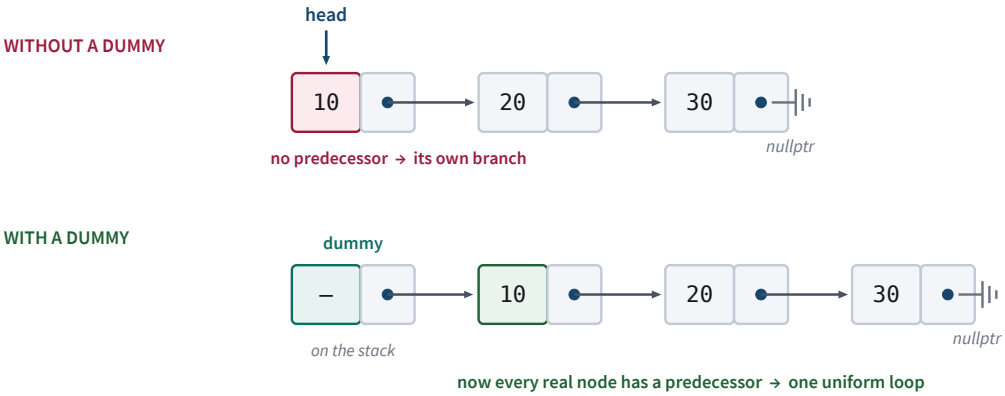
This is the single most useful trick in linked list programming, and Part VI has been building the case for it.

DEFINITION | The problem it solves
Almost every list operation needs a node's **predecessor**. Every node has one except the head, so every function ends up with a separate branch for the head — doubling the code and hiding most of the bugs.

KEY INSIGHT | Give the head a predecessor
Create one extra node, on the stack, whose `next` is the real head:

```
Node dummy(0);
dummy.next = head;
```

Now **every** real node has a predecessor, including the first. The special case disappears, because there is no longer a special case to handle.
At the end, the answer is `dummy.next` — which is automatically correct even if the original head was deleted, and even if the list is now empty.



`return dummy.next — correct even if every node was deleted`

Figure 29.1 The dummy node. One fake node in front means the real head is no longer exceptional; every node can be reached as `prev->next` for some `prev`, so one uniform loop handles everything.

The same function, with and without

Remove every node equal to val — the two versions

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// WITHOUT a dummy: two loops, one of which exists only for the head.
Node* removeAllPlain(Node* head, int val) {
    while (head != nullptr && head->data == val) {   // branch 1: the head
        Node* doomed = head;
        head = head->next;
        delete doomed;
    }
    if (head == nullptr) return nullptr;
    Node* prev = head;                               // branch 2:
    ↪ everything else
    while (prev->next != nullptr) {
        if (prev->next->data == val) {
            Node* doomed = prev->next;
            prev->next = doomed->next;
            delete doomed;
        } else {
            prev = prev->next;
        }
    }
    return head;
}

// WITH a dummy: one loop, no special case at all.
Node* removeAllDummy(Node* head, int val) {
    Node dummy(0);
```

```
    dummy.next = head;
    Node* prev = &dummy;
    while (prev->next != nullptr) {
        if (prev->next->data == val) {
            Node* doomed = prev->next;
            prev->next = doomed->next;
            delete doomed;
        } else {
            prev = prev->next;
        }
    }
    return dummy.next;          // correct even if every node was removed
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({7, 7, 1, 7, 2, 7});
    a = removeAllPlain(a, 7);  printList(a);  freeList(a);

    Node* b = buildList({7, 7, 1, 7, 2, 7});
    b = removeAllDummy(b, 7);  printList(b);  freeList(b);

    Node* c = buildList({7, 7, 7});          // everything removed
    c = removeAllDummy(c, 7);  printList(c);  freeList(c);
    return 0;
}
```

```
1 -> 2
1 -> 2
(empty)
```

Same answers, and the second function is half the length with one loop instead of two. Count the `delete`s: the dummy version has one, the plain version has two, which is one fewer place to get wrong.

COMMON PITFALL | Three details about dummy nodes

Put it on the stack. `Node dummy(0);` is a local object; it is destroyed automatically. `new Node(0)` would need deleting, and forgetting is a leak.

Return `dummy.next`, never `head`. After the loop, `head` may name a node that has been deleted. `dummy.next` is always the current first node.

Never delete the dummy itself. It is a stack object; `delete &dummy` is undefined behaviour. The loop condition `prev->next != nullptr` naturally never offers it for deletion, because nothing points *to* it.

INTERVIEW ANGLE | Reach for it immediately

Whenever a problem might change, delete or insert before the **first** node — remove nth from end, remove duplicates, merge two lists, partition, add two numbers — start by writing the dummy. It costs one line and removes a whole category of bug. Interviewers recognise it instantly as a sign that you have written list code before.

Pattern 2: Fast and slow pointers

DEFINITION | The two-speed walk

Move one pointer one node at a time and another two nodes at a time. When the fast pointer reaches the end, the slow one is at the **middle**.

Why: the fast pointer covers twice the distance, so when it has travelled n the slow one has travelled $n/2$.

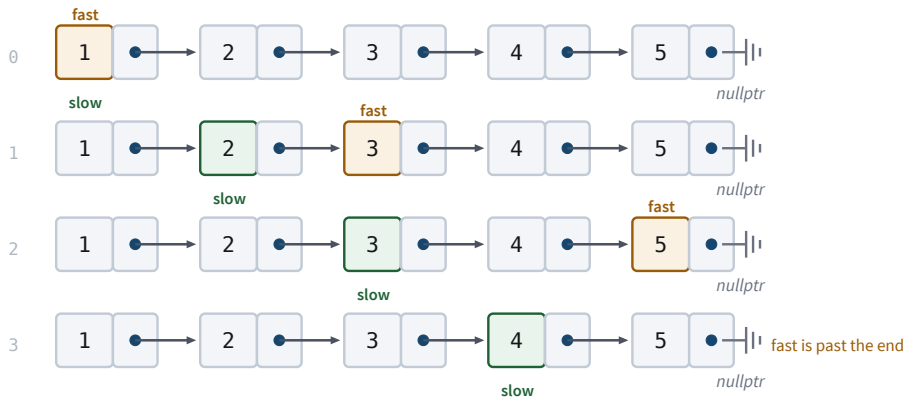


Figure 29.2 Fast and slow. After three steps the fast pointer has run off the end and the slow one is exactly halfway. One pass, no length count, $O(1)$ space.

Find the middle in one pass

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
```

```
};

// For an even length, returns the SECOND middle (1,2,3,4 -> 3).
Node* middleNode(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// For an even length, returns the FIRST middle (1,2,3,4 -> 2).
Node* firstMiddle(Node* head) {
    if (head == nullptr) return nullptr;
    Node* slow = head;
    Node* fast = head->next;             // the one-step head start does
        it
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* odd = buildList({1, 2, 3, 4, 5});
    std::cout << middleNode(odd)->data << " " << firstMiddle(odd)->data
        << "\n";
    Node* even = buildList({1, 2, 3, 4});
    std::cout << middleNode(even)->data << " " << firstMiddle(even)->data
        << "\n";
    Node* one = buildList({9});
    std::cout << middleNode(one)->data << "\n";
    freeList(odd); freeList(even); freeList(one);
    return 0;
}
```

```
3 3
3 2
9
```

COMMON PITFALL | The loop condition, and the order of its two tests

```
while (fast != nullptr && fast->next != nullptr)
```

Both tests are needed, and **in that order**:

- `fast != nullptr` guards the even-length case, where `fast` lands exactly on `nullptr`.
- `fast->next != nullptr` guards the odd-length case, where `fast` lands on the last node and `fast->next->next` would dereference null.

Short-circuit evaluation means the second test only runs if the first passed, so writing them the other way round crashes on an even-length list.
Which middle you get for even lengths depends on where `fast` starts: `head` gives the second middle, `head->next` gives the first. Problems care — "split the list in half" usually wants the **first** middle so the two halves come out right.

Pattern 3: The fixed gap

DEFINITION | Two pointers k apart
Advance one pointer k nodes. Then move both one step at a time. When the leading pointer reaches the end, the trailing one is exactly k nodes from the end.
This is how you find the k-th node from the end **in one pass**, without knowing the length.

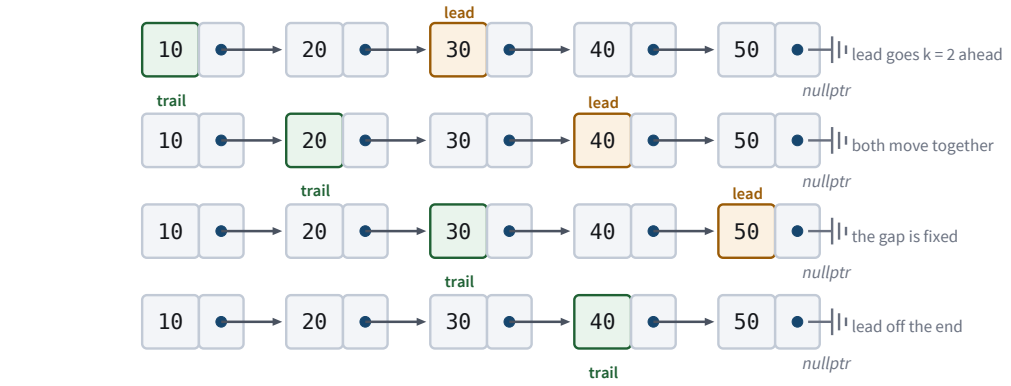


Figure 29.3 The fixed gap. The lead pointer is sent k nodes ahead; the two then move in lockstep, so the gap is preserved. When the lead falls off the end, the trailing pointer is k from the end.

The k-th node from the end, one pass

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Returns the k-th node from the end (k = 1 is the last), or nullptr.
```

```cpp
Node* kthFromEnd(Node* head, int k) {
    Node* lead = head;
    for (int i = 0; i < k; ++i) {
        if (lead == nullptr) return nullptr;   // the list is shorter than k
        lead = lead->next;
    }
    Node* trail = head;
    while (lead != nullptr) {                  // keep the gap, move together
        lead = lead->next;
        trail = trail->next;
    }
    return trail;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void freeList(Node* head) { while (head) { Node* d = head; head = head->next; delete d; } }

int main() {
    Node* h = buildList({10, 20, 30, 40, 50});
    std::cout << kthFromEnd(h, 1)->data << "\n";    // the last
    std::cout << kthFromEnd(h, 2)->data << "\n";
    std::cout << kthFromEnd(h, 5)->data << "\n";    // the first
    std::cout << (kthFromEnd(h, 6) ? "found" : "too short") << "\n";
    freeList(h);
    return 0;
}
```

```
50
40
10
too short
```

$O(n)$ time, $O(1)$ space, **one pass**. The two-pass alternative — count the length, then walk $n - k$ nodes — is equally $O(n)$ and is a perfectly good answer; the one-pass version is what the question is usually fishing for.

Pattern 4: Floyd's cycle detection

> **DEFINITION | The problem**
> A list may have a cycle: some node's `next` points back to an earlier node. Detect it, and find where the cycle begins, in $O(1)$ space.

A hash set of visited nodes solves it in $O(n)$ space. Floyd's algorithm — the "tortoise and hare" — does it in $O(1)$, and the proof is one of the most satisfying short arguments in the

subject.

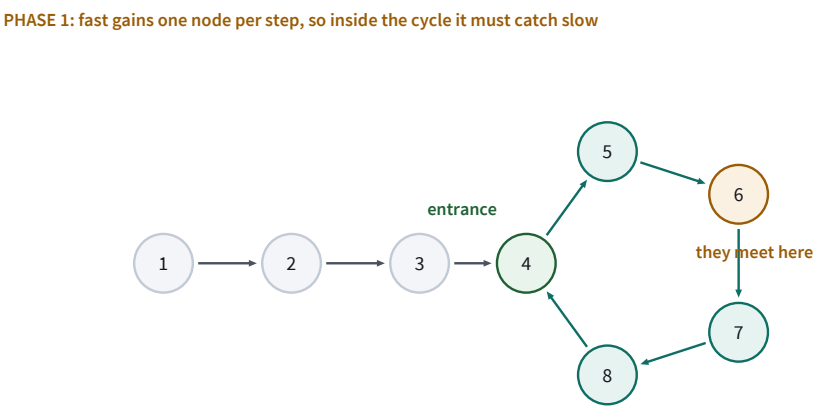


Figure 29.4 Floyd's algorithm. Phase 1: the fast pointer gains one node per step on the slow one, so inside the cycle it must eventually catch it. Phase 2: restarting one pointer at the head and moving both one step at a time makes them meet exactly at the cycle's entrance.

Phase 1: is there a cycle?

Move `slow` by one and `fast` by two. If there is no cycle, `fast` falls off the end. If there is, `fast` enters the cycle, and then **gains exactly one node per step** on `slow` — so the distance between them shrinks by one each step and must reach zero. They meet.

> **KEY INSIGHT | Why a two-speed chase always catches**
> Once both are inside the cycle, consider the gap from `slow` to `fast` measured forwards around the loop. Each step, `slow` advances 1 and `fast` advances 2, so the gap shrinks by exactly 1.
> A gap that decreases by exactly 1 each step cannot skip over zero. So they coincide within at most L steps, where L is the cycle length.
> This is also why speed 3 would be a worse choice: the gap would shrink by 2 and *could* skip zero, so it would need a divisibility argument to be correct. Speeds 1 and 2 are the simple case, which is why the algorithm uses them.

Phase 2: where does the cycle start?

This is the part that looks like magic. It is not.

DEFINITION | The proof

Let F be the distance from the head to the cycle entrance, a the distance from the entrance to the meeting point (measured forwards round the cycle), and L the cycle length.

When they meet, `slow` has travelled $F+a$ and `fast` has travelled $F+a+nL$ for some whole number $n \geq 1$ — the extra laps it made.

`fast` moved twice as far:

$$2(F+a) = F+a+nL$$
$$F+a = nL$$
$$F = nL-a$$

Now read that last line. $nL-a$ is *"go round the cycle n times and come back a short"* — which, starting from the meeting point, lands exactly on the entrance.

So: a pointer starting at the head and walking F steps reaches the entrance. A pointer starting at the meeting point and walking the same F steps also reaches the entrance. **Move both one step at a time, and they meet there.**

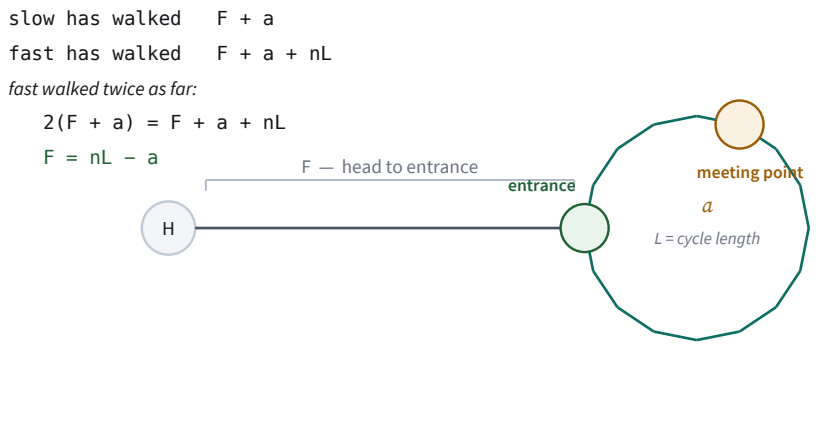


Figure 29.5 The distances in Floyd's proof. $F=nL-a$ is the whole argument: from the meeting point, F steps is some whole number of laps minus a, which lands on the entrance.

Detect a cycle, and find its entrance

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;      // compare NODES, not values
    }
```

```
    return false;
}

// Returns the first node of the cycle, or nullptr if there is none.
Node* cycleStart(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {                // phase 1 found a meeting
            point
            slow = head;                   // phase 2: restart one at
                the head
            while (slow != fast) {         // both move ONE step now
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }
    return nullptr;
}

int main() {
    Node* a = new Node(1);
    a->next = new Node(2);
    a->next->next = new Node(3);
    a->next->next->next = new Node(4);
    a->next->next->next->next = a->next;      // 4 points back at 2

    std::cout << hasCycle(a) << "\n";
    std::cout << cycleStart(a)->data << "\n";

    Node* b = new Node(1);
    b->next = new Node(2);
    std::cout << hasCycle(b) << "\n";
    std::cout << (cycleStart(b) == nullptr ? "no cycle" : "?") << "\n";

    a->next->next->next->next = nullptr;      // break it before freeing
    while (a) { Node* d = a; a = a->next; delete d; }
    while (b) { Node* d = b; b = b->next; delete d; }
    return 0;
}
```

```
1
2
0
no cycle
```

COMMON PITFALL | Three things about cycle code

Compare pointers, not values. `slow == fast` compares addresses. `slow->data == fast->data` would be true for any two nodes holding the same number.

In phase 2, both pointers move one step. Keeping `fast` at double speed is the most common error, and it finds the wrong node.

You cannot free a cyclic list with the normal loop. It never terminates, and it frees nodes twice. Break the cycle first, as the program above does, or the AddressSanitizer report will be dramatic.

Pattern 5: Merge and split

Merging two sorted lists is the engine of merge sort on lists, and is worth writing once properly.

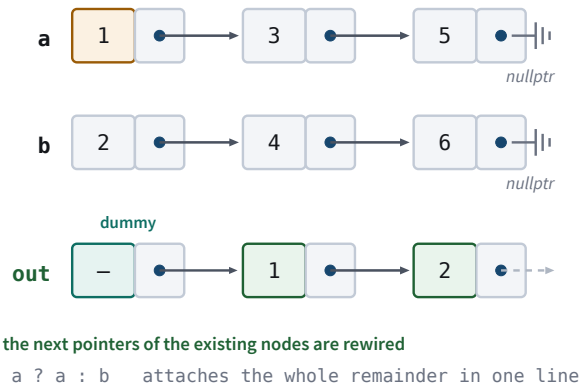


Figure 29.6 Merging two sorted lists. A dummy node means there is no "is this the first output node?" case; `tail` always has somewhere to attach. Each step takes the smaller front node and advances that list.

Merge two sorted lists, and split one in half

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

// Merges two sorted lists by RELINKING - no new nodes are created.
Node* mergeSorted(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else                    { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = (a != nullptr) ? a : b;   // attach whatever is left, in
        one go
```

```
    return dummy.next;
}

// Splits the list in half; returns the head of the second half.
Node* splitHalf(Node* head) {
    if (head == nullptr || head->next == nullptr) return nullptr;
    Node* slow = head;
    Node* fast = head->next;               // the FIRST middle, so halves
        are even
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* second = slow->next;
    slow->next = nullptr;                  // cut the first half loose
    return second;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 3, 5});
    Node* b = buildList({2, 4, 6, 8});
    Node* m = mergeSorted(a, b);
    printList(m);

    Node* second = splitHalf(m);
    printList(m);
    printList(second);

    freeList(m);
    freeList(second);

    Node* e = buildList({1, 2});
    printList(mergeSorted(e, nullptr));     // one side empty
    freeList(e);
    return 0;
}
```

```
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8
1 -> 2 -> 3 -> 4
5 -> 6 -> 8
1 -> 2
```

KEY INSIGHT | Relinking, not copying
`mergeSorted` creates **no new nodes**. It rearranges the `next` pointers of the nodes that already exist, which is why it is $O(1)$ space.
This is the structural advantage a linked list genuinely has over an array: merging two sorted arrays needs an output array of size $n+m$, but merging two sorted lists needs nothing. The same property makes `std::list::splice` $O(1)$ — moving a whole range between lists is four pointer updates, regardless of its length.
Note also the single line `tail->next = (a != nullptr) ? a : b;`. With arrays you would need two tail loops to copy the remainder. Here the remainder is already a correctly-linked list, so one pointer attaches all of it.

Pattern 6: Recursion on lists

A list is a recursive structure: a node, followed by a smaller list. So every list algorithm has a recursive form.

Three operations, recursively

```
int length(Node* head) {
    if (head == nullptr) return 0;
    return 1 + length(head->next);
}

void printReverse(Node* head) {
    if (head == nullptr) return;
    printReverse(head->next);        // go all the way down first...
    std::cout << head->data << " ";  // ...then print on the way back out
}

Node* mergeRec(Node* a, Node* b) {
    if (a == nullptr) return b;
    if (b == nullptr) return a;
    if (a->data <= b->data) { a->next = mergeRec(a->next, b); return a; }
    else                    { b->next = mergeRec(a, b->next); return b; }
}
```

`printReverse` is the one worth studying: it prints a list backwards without reversing it and without any extra data structure. The call stack *is* the storage — which is precisely why it is $O(n)$ space, not $O(1)$.

WATCH OUT | Recursion depth is real
Every recursive list function uses one stack frame per node. With a typical 64-byte frame and a 1 MB stack, that is roughly **16,000 nodes** before a stack overflow.
For an interview on a 5-node example, recursion is elegant and fine. For production code on real data, it is a crash waiting for a big enough input.
Say this out loud when you give a recursive answer. "This is $O(n)$ stack, so on a very long list I would use the iterative version" turns a liability into evidence of judgement.

The pattern map

Pattern	Use it for	Cost
Dummy node	anything that may change the head	$O(1)$ extra
Fast and slow	middle, cycle detection, splitting in half	$O(1)$
Fixed gap	k-th from the end, remove nth from end	$O(1)$
Floyd's algorithm	cycle detection and cycle entrance	$O(1)$
Merge / split	merge two sorted, sort a list, reorder	$O(1)$
Recursion	reverse, merge, add two numbers	$O(n)$ stack
Pointer reversal	reverse, palindrome, reverse in k-groups	$O(1)$

Table 29.1 The six patterns and the problems they unlock. Chapters 30 to 32 work through all of these.

CHAPTER RECAP | Chapter 29 in six lines

- A **dummy node** gives the head a predecessor and deletes an entire class of special case. Return `dummy.next`.
- **Fast and slow** finds the middle in one pass; the loop condition needs both tests, in order.
- Starting `fast` at `head` gives the second middle; at `head->next`, the first.
- A **fixed gap** of k finds the k-th from the end in one pass.
- **Floyd's** phase 2 works because $F = nL - a$; both pointers move one step in it.
- **Merging relinks** rather than copying, which is why it is $O(1)$ space — and why `splice` is $O(1)$.

EXERCISES | Practise

1. Rewrite `deleteValue` from Chapter 26 using a dummy node. How many lines disappear?
2. Why must a dummy node live on the stack rather than the heap? What happens if you `new` it and forget to delete it?
3. Modify `middleNode` to return the first middle on even lengths without the `head->next` trick. Which is clearer?
4. In `kthFromEnd`, what should happen when $k = 0$? When k is negative? Add the checks.
5. Write out Floyd's proof in your own words, defining F, a and L in a drawing.
6. Why is speed 2 used in Floyd's rather than speed 3? Give the gap argument.
7. Write `mergeSorted` so it produces a **descending** list. What changes?
8. Write `length` iteratively and recursively. At what list length does the recursive one crash on your machine? Measure it.

CHAPTER

30

Essential Linked List Problems I

The six that are asked most often

PRACTICE PREREQS: Chapter 29

Six problems that between them account for a large share of all linked list interview questions. Each is solved with one of the patterns from Chapter 29, and each is presented the way you should present it: statement, approach, code, edge cases, cost, and the mistake it invites.

WHAT YOU WILL LEARN

- Solve reverse, middle, cycle, cycle II, merge two sorted and remove nth from end
- Pick the right pattern for each from the problem statement alone
- Handle the empty list and the single-node list in every one
- Give the complexity and the space without being asked
- Recognise which of these appear as sub-steps in the harder problems later

1. Reverse Linked List

Statement. Reverse a singly linked list and return the new head.

Pattern. Pointer reversal — three pointers, four lines. Chapter 28 covers every detail; this is the version with the LeetCode signature.

Reverse Linked List

```
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;          // prev is the new head
}
```

Edge cases. Empty list returns `nullptr`; one node returns itself. Both work with no special case.

Cost. $O(n)$ time, $O(1)$ space. The recursive version is $O(n)$ stack.

Watch for. Returning `head` instead of `prev`. Forgetting to save `nextNode`.

Appears inside. Palindrome check, reorder list, reverse in k-groups, add two numbers (one formulation). Learn it until it is automatic.

2. Middle of the Linked List

Statement. Return the middle node. For even lengths, return the second middle.

Pattern. Fast and slow pointers.

Middle of the Linked List

```
Node* middleNode(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Edge cases. Empty returns `nullptr`. One node returns itself.

Cost. $O(n)$ time, $O(1)$ space, one pass.

Watch for. The two-pass version (count, then walk $n/2$) is correct and costs the same asymptotically — say so, then give the one-pass version as the improvement.

Appears inside. Palindrome check, reorder list, sort list.

3. Linked List Cycle

Statement. Does the list contain a cycle?

Pattern. Floyd's algorithm, phase 1.

Linked List Cycle

```
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Cost. $O(n)$ time, $O(1)$ space. The hash-set version is $O(n)$ space — mention it, then improve on it.

Watch for. Comparing `slow->data == fast->data` instead of the pointers.

4. Linked List Cycle II

Statement. Return the node where the cycle begins, or `nullptr`.

Pattern. Floyd's algorithm, both phases. The proof ($F = nL - a$) is in Chapter 29 and is worth being able to state.

Linked List Cycle II

```
Node* detectCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            slow = head;                // phase 2
            while (slow != fast) {      // both move ONE step
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }
    return nullptr;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Watch for. Leaving `fast` at double speed in phase 2.

Related. "Find the duplicate number" in Chapter 19 is this exact algorithm applied to an array read as a linked list.

Removing a cycle

A natural follow-up, and it needs one extra idea: to break the cycle you must reach the node **just before** the entrance.

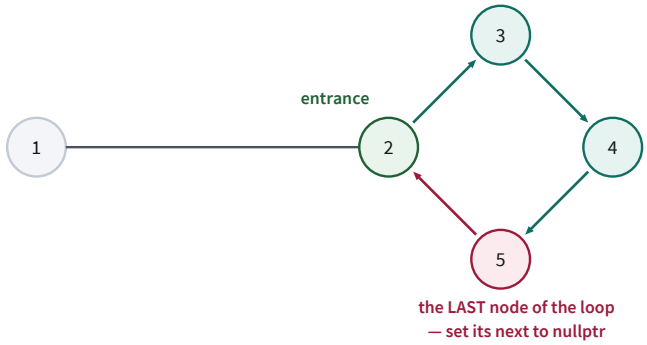


Figure 30.1 Removing a cycle. Finding the entrance is not enough; you need the last node of the loop — the one pointing back at the entrance — so that you can set its `next` to `nullptr`.

Detect, then remove, a cycle

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* detectCycle(Node* head) {
    Node* slow = head; Node* fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}

// Breaks the cycle if there is one. Returns whether one was removed.
bool removeCycle(Node* head) {
    Node* start = detectCycle(head);
    if (start == nullptr) return false;
    Node* last = start;
    while (last->next != start)      // walk round to the node before the
        entrance
        last = last->next;
    last->next = nullptr;            // break it
    return true;
}

int main() {
    Node* a = new Node(1);
    a->next = new Node(2);
```

```
    a->next->next = new Node(3);
    a->next->next->next = new Node(4);
    a->next->next->next->next = a->next;     // 4 -> 2

    std::cout << "entrance: " << detectCycle(a)->data << "\n";
    std::cout << "removed:  " << removeCycle(a) << "\n";
    std::cout << "again:    " << removeCycle(a) << "\n";
    for (Node* c = a; c; c = c->next) std::cout << c->data << " ";
    std::cout << "\n";
    while (a) { Node* d = a; a = a->next; delete d; }
    return 0;
}
```

```
entrance: 2
removed:  1
again:    0
1 2 3 4
```

Three things to notice. `removeCycle` is **idempotent**: calling it again returns `0` because `detectCycle` now finds nothing, and the list is left alone. The walk `while (last->next != start)` goes all the way round the loop, so it is $O(L)$ where L is the cycle length. And once the cycle is broken, the ordinary free loop works again — which it did not before.

5. Merge Two Sorted Lists

Statement. Merge two sorted lists into one sorted list, by relinking.

Pattern. Dummy node plus merge.

Merge Two Sorted Lists

```
Node* mergeTwoLists(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else                    { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = (a != nullptr) ? a : b;
    return dummy.next;
}
```

Edge cases. Either list empty — handled by the final line, which attaches the non-empty remainder (or `nullptr` if both are exhausted).

Cost. $O(n+m)$ time, $O(1)$ space. No nodes are created.

Watch for. `<=` rather than `<` keeps the merge **stable** — equal elements from `a` come before equal elements from `b`. It matters for merge sort.

Appears inside. Sort list (Chapter 32), merge k sorted lists.

6. Remove Nth Node From End

Statement. Remove the n-th node from the end and return the head.

Pattern. Fixed gap, plus a dummy node — because the node to remove might be the head.

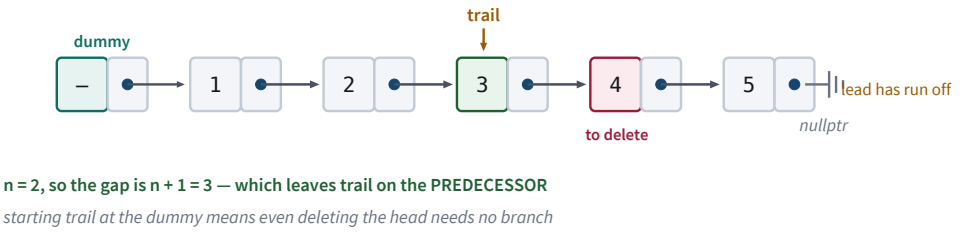


Figure 30.2 Remove nth from end. The gap is set to n+1 *from the dummy*, so that the trailing pointer stops on the node **before** the one to delete — which is what you need in order to unlink it.

> **KEY INSIGHT | Why the gap is n+1, and why the dummy matters**
> To delete a node you need its predecessor. So the trailing pointer must stop one node *earlier* than the target, which means a gap of n+1 rather than n.
> Starting the trailing pointer at the dummy rather than the head means that even when the target *is* the head, there is still a predecessor to work with. Without the dummy you would need a separate branch for "remove the first node", and that branch is where the bug would be.

Remove the nth node from the end, one pass

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* removeNthFromEnd(Node* head, int n) {
    Node dummy(0);
    dummy.next = head;
    Node* lead = &dummy;
    Node* trail = &dummy;

    for (int i = 0; i <= n; ++i) {        // n+1 steps: note the <=
        if (lead == nullptr) return head; // n is larger than the list
        lead = lead->next;
    }
    while (lead != nullptr) {             // move together, keeping the
```

```
        gap
        lead = lead->next;
        trail = trail->next;
    }
    Node* doomed = trail->next;          // trail is the predecessor
    trail->next = doomed->next;
    delete doomed;
    return dummy.next;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 2, 3, 4, 5});
    a = removeNthFromEnd(a, 2);          // remove 4
    printList(a); freeList(a);

    Node* b = buildList({1, 2, 3});
    b = removeNthFromEnd(b, 3);          // remove the head
    printList(b); freeList(b);

    Node* c = buildList({1});
    c = removeNthFromEnd(c, 1);          // the list becomes empty
    printList(c); freeList(c);
    return 0;
}
```

```
1 -> 2 -> 3 -> 5
2 -> 3
(empty)
```

Cost. $O(n)$ time, $O(1)$ space, one pass.

Watch for. `i <= n` in the first loop, not `i < n`. Returning `dummy.next`, not `head` — the third test case shows why: `head` has been deleted.

The six, at a glance

#	Problem	Pattern	Time	Space
1	Reverse Linked List	pointer reversal	$O(n)$	$O(1)$

#	Problem	Pattern	Time	Space
2	Middle of the List	fast and slow	$O(n)$	$O(1)$
3	Linked List Cycle	Floyd phase 1	$O(n)$	$O(1)$
4	Linked List Cycle II	Floyd both phases	$O(n)$	$O(1)$
5	Merge Two Sorted Lists	dummy + merge	$O(n+m)$	$O(1)$
6	Remove Nth From End	gap + dummy	$O(n)$	$O(1)$

Table 30.1 The six core problems. Four patterns cover all of them, and each appears again as a step inside the harder problems of Chapters 31 and 32.

INTERVIEW ANGLE | Every one of these is $O(1)$ space

That is not a coincidence — it is what linked list questions are *for*. An interviewer asking a list question is almost always checking whether you can manipulate pointers without allocating anything.

So if your solution builds a vector of nodes, or uses a hash set, expect the follow-up *"now without the extra space"*. Give the $O(n)$-space version if it comes to you first, say that it is $O(n)$ space, and then improve it. That sequence — working solution, honest cost, improvement — is the whole performance.

EXERCISES | Practise

1. Write all six from memory. Test each on: empty, one node, two nodes.
2. For problem 2, write the two-pass version and compare the code length. Which would you give first in an interview, and why?
3. For problem 6, what should happen if n is larger than the list? What does the code above do? Is that the right choice?
4. Solve problem 3 with a hash set instead. What is the space, and when would you actually prefer it?
5. Merge **three** sorted lists. Compare merging them one at a time with a divide-and-conquer merge. What are the two complexities?
6. For problem 4, construct a list where the cycle is the entire list (the last node points at the head). What does `detectCycle` return?
7. Write `removeCycle` without first calling `detectCycle` — can you break the cycle using only the meeting point from phase 1? Why or why not?

CHAPTER

31

Essential Linked List Problems II

Six problems that combine two patterns each

PRACTICE PREREQS: Chapter 30

These six are built from the pieces of Chapter 30. Palindrome is find-middle plus reverse; reorder is find-middle plus reverse plus merge. The skill being tested here is decomposition — seeing that an unfamiliar problem is two familiar ones stacked — and that is exactly the skill Chapter 34 turns into a method.

WHAT YOU WILL LEARN

- Solve palindrome, intersection, add two numbers, remove duplicates, delete node and reorder
- Decompose a problem into steps you have already solved
- Explain the two-pointer trick for intersection, and why the lengths cancel
- Handle carries correctly when adding numbers stored as lists
- Explain the delete-without-the-head trick, and when it is illegal

1. Palindrome Linked List

Statement. Is the list the same read forwards and backwards?

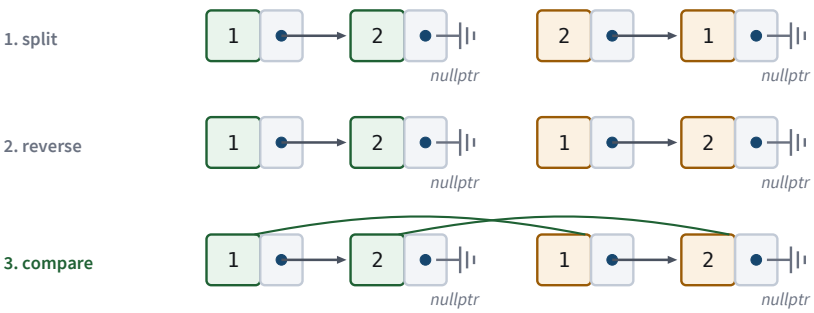
Think. With an array this is trivial — two pointers from the ends. A singly linked list cannot walk backwards, so the obvious approaches are: copy the values into a vector ($O(n)$ space), or use recursion ($O(n)$ stack).

The $O(1)$-space answer is a decomposition:

KEY INSIGHT | Three steps you have already written

1. Find the **middle** (fast and slow) — Chapter 29.
2. **Reverse** the second half — Chapter 28.
3. Compare the two halves node by node.

No new technique is involved. That is the point: hard list problems are usually two easy ones in sequence.



4. reverse the second half again — leave the list exactly as you found it

nothing new is needed: find-middle and reverse are both functions you already have

Figure 31.1 Palindrome by decomposition. Find the middle, reverse the second half, then walk the two halves in step. The comparison stops when the shorter half runs out, which handles odd lengths with no special case.

Palindrome Linked List, O(1) space

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* reverseList(Node* head) {
    Node* prev = nullptr;
    while (head != nullptr) {
        Node* nxt = head->next;
        head->next = prev;
        prev = head;
        head = nxt;
    }
    return prev;
}

bool isPalindrome(Node* head) {
    if (head == nullptr || head->next == nullptr) return true;

    Node* slow = head;                  // 1. find the first middle
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* second = reverseList(slow->next);   // 2. reverse the second
        half

    bool ok = true;                     // 3. compare
    Node* a = head;
    Node* b = second;
    while (b != nullptr) {              // the second half is never longer
        if (a->data != b->data) { ok = false; break; }
```

```
        a = a->next;
        b = b->next;
    }
    slow->next = reverseList(second);   // 4. put the list back as we
        found it
    return ok;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 2, 2, 1});
    std::cout << isPalindrome(a) << " ";  printList(a);   // restored
    Node* b = buildList({1, 2, 3, 2, 1});
    std::cout << isPalindrome(b) << " ";  printList(b);
    Node* c = buildList({1, 2, 3});
    std::cout << isPalindrome(c) << " ";  printList(c);
    Node* d = buildList({7});
    std::cout << isPalindrome(d) << "\n";
    freeList(a); freeList(b); freeList(c); freeList(d);
    return 0;
}
```

```
1 1 -> 2 -> 2 -> 1
1 1 -> 2 -> 3 -> 2 -> 1
0 1 -> 2 -> 3
1
```

Cost. $O(n)$ time, $O(1)$ space.

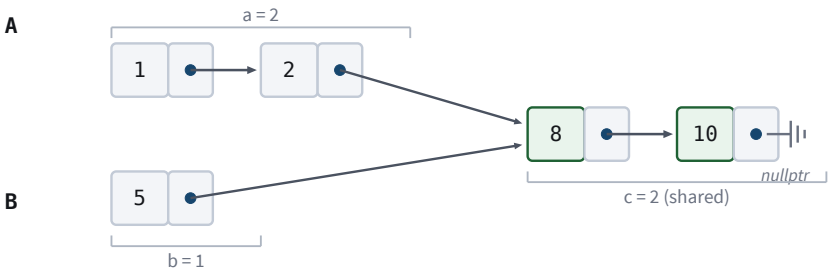
> **KEY INSIGHT | Step 4 is the one that marks you out**
> Most candidates stop after step 3 with the list left reversed in the middle — the function answered the question and silently corrupted its input.
> Restoring it costs three lines and is the difference between a function and a booby trap. If an interviewer does not ask about it, raise it yourself: *"this mutates the list, so I'll restore it before returning — unless you would rather I used O(n) space and left the input alone."*

2. Intersection of Two Linked Lists

Statement. Two lists may share a common tail. Return the first shared node, or `nullptr`. They intersect by **identity**, not by value.

Think. Brute force is every pair: $O(n \cdot m)$. A hash set of one list's nodes is $O(n+m)$ time and $O(n)$ space. The $O(1)$-space answer is lovely.

> **KEY INSIGHT | Why swapping to the other list equalises the lengths**
> Let list A have length a before the shared part and list B have length b, with a shared tail of length c.
> Walk pointer `pa` along A; when it reaches the end, send it to the head of B. Walk `pb` along B; when it reaches the end, send it to the head of A.
> `pa` travels $a+c$ then b — a total of $a+b+c$ before reaching the intersection. `pb` travels $b+c$ then a — also $a+b+c$. **They arrive at the same node at the same step**, because each has walked the length of both lists.
> If there is no intersection, both reach `nullptr` after $a+b$ steps, and the loop ends with the correct answer of `nullptr`.




```
pa walks  a + c  then  b     =  a + b + c
pb walks  b + c  then  a     =  a + b + c   →  they arrive together
```

Figure 31.2 The length-equalising walk. Each pointer walks its own list and then the other, so both cover exactly $a+b+c$ nodes and meet at the intersection — or at `nullptr` together if there is none.

Intersection of two lists, O(1) space

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* getIntersection(Node* headA, Node* headB) {
    if (headA == nullptr || headB == nullptr) return nullptr;
    Node* pa = headA;
    Node* pb = headB;
    while (pa != pb) {                        // compare NODES
        pa = (pa == nullptr) ? headB : pa->next;   // swap lists at the
```

```
            end
        pb = (pb == nullptr) ? headA : pb->next;
    }
    return pa;                                  // the node, or nullptr
}

int main() {
    Node* shared = new Node(8);
    shared->next = new Node(10);

    Node* a = new Node(1);                      // 1 -> 2 -> 8 -> 10
    a->next = new Node(2);
    a->next->next = shared;

    Node* b = new Node(5);                      // 5 -> 8 -> 10
    b->next = shared;

    Node* hit = getIntersection(a, b);
    std::cout << (hit ? hit->data : -1) << "\n";

    Node* c = new Node(99);                     // no intersection
    std::cout << (getIntersection(a, c) == nullptr ? "none" : "?") <<
        "\n";

    delete a->next; delete a;                   // free without
        double-deleting
    delete b;
    delete shared->next; delete shared;
    delete c;
    return 0;
}
```

```
8
none
```

COMMON PITFALL | Two traps in this problem
Identity, not value. Two nodes both holding 8 are not an intersection. The loop compares `pa != pb`, which compares addresses.
The `nullptr` swap must happen on the pointer itself, not on its `next`. Writing `pa = pa->next ? pa->next : headB;` skips the `nullptr` step, so the two pointers travel different distances and the no-intersection case loops forever. The version above lets each pointer *become* `nullptr` for one step, which is what makes the counts match and the loop terminate.

3. Add Two Numbers

Statement. Two numbers are stored as lists, **least significant digit first**. Return their sum in the same form. `2->4->3` is 342; `5->6->4` is 465; the answer is `7->0->8`, which is 807.

Think. This is primary-school addition. Walk both lists together, add the digits plus the carry, write the units digit and keep the tens as the new carry.

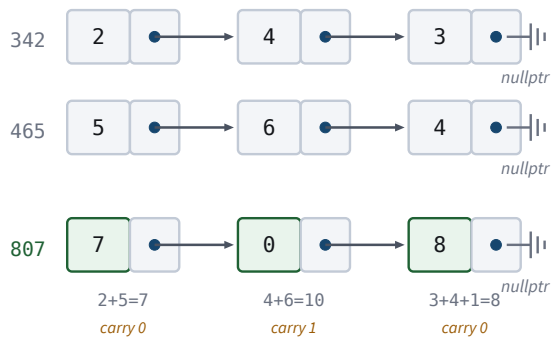


Figure 31.3 Adding two numbers digit by digit. Least-significant-first storage is exactly the order you add in, which is why the problem is posed that way. The carry survives past the end of both lists.

Add Two Numbers

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* addTwoNumbers(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    int carry = 0;

    // One loop handles both lists and the final carry.
    while (a != nullptr || b != nullptr || carry != 0) {
        int sum = carry;
        if (a != nullptr) { sum += a->data; a = a->next; }
        if (b != nullptr) { sum += b->data; b = b->next; }
        carry = sum / 10;
        tail->next = new Node(sum % 10);
        tail = tail->next;
    }
    return dummy.next;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
```

```
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({2, 4, 3});          // 342
    Node* b = buildList({5, 6, 4});          // 465
    Node* s = addTwoNumbers(a, b);
    printList(s);                            // 807
    freeList(a); freeList(b); freeList(s);

    Node* c = buildList({9, 9, 9});          // 999
    Node* d = buildList({1});                // 1
    Node* t = addTwoNumbers(c, d);
    printList(t);                            // 1000
    freeList(c); freeList(d); freeList(t);

    Node* e = buildList({0});
    Node* f = buildList({0});
    Node* u = addTwoNumbers(e, f);
    printList(u);
    freeList(e); freeList(f); freeList(u);
    return 0;
}
```

```
7 -> 0 -> 8
0 -> 0 -> 0 -> 1
0
```

KEY INSIGHT | The loop condition does three jobs

```
while (a != nullptr || b != nullptr || carry != 0)
```

`||`, not `&&`. The lists may be different lengths, so the loop must continue while **either** has digits left. And `carry != 0` keeps it going one more round after both are exhausted — which is what turns 999 + 1 into 1000 rather than 000.

Writing this as three separate loops (both lists, then the rest of A, then the rest of B, then the carry) is correct and four times as long. One condition covering every case is worth reaching for.

Cost. $O(\max(n, m))$ time; $O(\max(n, m))$ for the output list.

Variation. If the digits are stored **most** significant first, you cannot add left to right. Either reverse both lists, add, and reverse the answer; or push the digits onto two stacks and pop. Say which you would do and why.

4. Remove Duplicates from a Sorted List

Statement. Each value should appear once. The list is sorted.

Think. Sorted means duplicates are adjacent, so compare each node with the next. No `prev` is needed, because you delete `curr->next` rather than `curr`.

Remove duplicates from a sorted list

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* deleteDuplicates(Node* head) {
    Node* curr = head;
    while (curr != nullptr && curr->next != nullptr) {
        if (curr->data == curr->next->data) {
            Node* doomed = curr->next;
            curr->next = doomed->next;     // unlink
            delete doomed;                 // then free
            // do NOT advance: the new curr->next is unexamined
        } else {
            curr = curr->next;
        }
    }
    return head;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 1, 2, 3, 3});
    printList(deleteDuplicates(a)); freeList(a);
    Node* b = buildList({5, 5, 5});
    printList(deleteDuplicates(b)); freeList(b);
    Node* c = buildList({1, 2, 3});
    printList(deleteDuplicates(c)); freeList(c);
    return 0;
}
```

```
1 -> 2 -> 3
5
1 -> 2 -> 3
```

Cost. $O(n)$ time, $O(1)$ space.

Watch for. Not advancing `curr` after a deletion — the same rule as Chapter 26. `{5,5,5}` is the test that catches it.

Variation. "Remove *all* nodes that have duplicates" (so `1->1->2` becomes just `2`) is genuinely harder: the head may be deleted, so you need a dummy node and a `prev`. Try it.

5. Delete a Node, given only that node

Statement. You are given a pointer to a node to delete — **not** the head. The node is guaranteed not to be the last.

Think. You cannot reach the predecessor, so you cannot unlink this node.

> **KEY INSIGHT | Delete the next node instead, and steal its value**
> Copy the *next* node's data into this node, then delete the *next* node.
> The node you were asked to remove physically survives, but its value is gone and the list reads exactly as if it had been deleted. From outside, the result is indistinguishable.

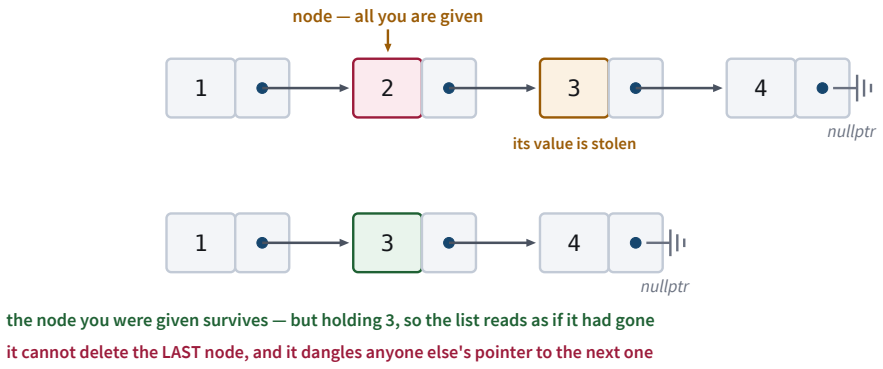


Figure 31.4 Deleting a node without its predecessor. The value from the following node is copied in, and that following node is unlinked and freed instead. The list ends up correct even though a different object was destroyed.

Delete a node given only that node

```
void deleteNode(Node* node) {
    Node* doomed = node->next;     // guaranteed non-null by the problem
    node->data = doomed->data;     // steal its value
    node->next = doomed->next;     // unlink it
    delete doomed;                 // and free it
}
```

Cost. $O(1)$ time and space.

WATCH OUT | When this trick is illegal

It cannot delete the last node. There is no next node to steal from, which is why the problem states the guarantee.

It invalidates other pointers. Anyone else holding a pointer to the following node now holds a dangling pointer — the node they expected has been freed, and its value has moved into a different object.

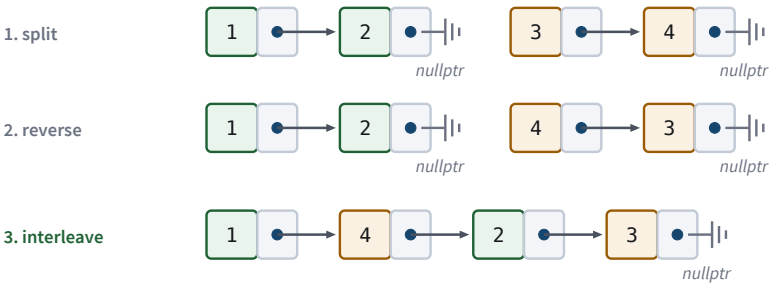
It copies the data. For an `int` that is free. For a large object it may be expensive, and for a non-copyable one it is impossible.

In real code this is a smell: it is a workaround for an API that should have given you the predecessor, or for a singly linked list that should have been doubly linked. As an interview answer it is exactly right; as a design, it is a sign something upstream is wrong.

6. Reorder List

Statement. Turn $L_0 \to L_1 \to \cdots \to L_{n-1}$ into $L_0 \to L_{n-1} \to L_1 \to L_{n-2} \to \cdots$

Think. Three familiar steps again, in sequence.



three steps, each one a function from an earlier chapter

split at the FIRST middle, so the first half is never the shorter one

Figure 31.5 Reorder, in three steps. Split at the middle, reverse the second half, then interleave the two halves one node at a time. Every step is a function you have already written.

Reorder List

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

void reorderList(Node* head) {
    if (head == nullptr || head->next == nullptr) return;

    Node* slow = head;                    // 1. split at the first middle
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
```

```
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* second = slow->next;
    slow->next = nullptr;

    Node* prev = nullptr;                // 2. reverse the second half
    while (second != nullptr) {
        Node* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }
    second = prev;

    Node* first = head;                  // 3. interleave
    while (second != nullptr) {
        Node* f = first->next;
        Node* s = second->next;
        first->next = second;
        second->next = f;
        first = f;
        second = s;
    }
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 2, 3, 4});
    reorderList(a); printList(a); freeList(a);
    Node* b = buildList({1, 2, 3, 4, 5});
    reorderList(b); printList(b); freeList(b);
    Node* c = buildList({1, 2});
    reorderList(c); printList(c); freeList(c);
    return 0;
}
```

```
1 -> 4 -> 2 -> 3
1 -> 5 -> 2 -> 4 -> 3
1 -> 2
```

Cost. $O(n)$ time, $O(1)$ space.

Watch for. Splitting at the **first** middle (`fast = head->next`) so the first half is never shorter than the second. The interleave loop then terminates naturally when `second` runs out.

The six, at a glance

#	Problem	Built from	Time	Space
1	Palindrome	middle + reverse	$O(n)$	$O(1)$
2	Intersection	the two-walk trick	$O(n+m)$	$O(1)$
3	Add Two Numbers	dummy + carry	$O(\max(n,m))$	output only
4	Remove duplicates (sorted)	adjacency	$O(n)$	$O(1)$
5	Delete a given node	the value-steal trick	$O(1)$	$O(1)$
6	Reorder List	middle + reverse + merge	$O(n)$	$O(1)$

Table 31.1 The six intermediate problems. Note the right-hand column: four of the six are built entirely from Chapter 30's problems.

CHAPTER RECAP | Chapter 31 in five lines

- Hard list problems are usually **two or three easy ones in sequence**. Look for the decomposition before looking for a new idea.
- Palindrome and reorder are both middle + reverse (+ merge).
- The intersection walk works because each pointer covers $a+b+c$ nodes.
- One loop with `||` and `carry != 0` handles every case of list addition.
- Restore any list you temporarily mutate, and say that you are doing it.

EXERCISES | Practise

1. Solve palindrome with a vector of values instead. What is the space, and when is that the better answer?
2. For intersection, construct two lists that share a tail and verify by hand that both pointers travel $a+b+c$ steps.
3. Add two numbers stored **most** significant digit first. Give both approaches, and say which you prefer.
4. Write "remove all nodes that have duplicates" — `1->1->2->3->3` becomes `2` . Which pattern do you need, and why does the plain version not work?
5. Why can `deleteNode` not remove the last node? Design a list structure where it could.
6. For reorder, what happens if you split at the *second* middle instead? Trace `{1,2,3,4}` .
7. Rotate a list right by k: `1->2->3->4->5` with k=2 becomes `4->5->1->2->3` . (Hint: make it circular, then break it in the right place.)

CHAPTER

32

Advanced Linked List Problems

Four problems that combine everything

ADVANCED PREREQS: Chapter 31

The four hardest standard linked list questions. Reverse in k-groups needs the reversal loop under a counter; rotate needs a circular trick; sort list is merge sort without an array; and copy list with random pointer has a solution so neat it is worth the chapter on its own. None introduces a new primitive — they combine the ones you have.

WHAT YOU WILL LEARN

- Reverse a list in groups of k, leaving a short final group alone
- Rotate a list right by k, using the make-it-circular trick
- Sort a list in $O(n \log n)$ time and $O(\log n)$ space with merge sort
- Explain why quicksort is a poor fit for linked lists and merge sort is ideal
- Deep-copy a list with random pointers in $O(1)$ extra space by interleaving

1. Reverse Nodes in k-Group

DEFINITION | The problem
Reverse the nodes of a list k at a time. If the number of remaining nodes is fewer than k, leave them as they are.
`1->2->3->4->5` with k=2 gives `2->1->4->3->5` — the final lone 5 is untouched.

Think. You already have the four-line reversal loop. Two new requirements: run it over exactly k nodes rather than the whole list, and check *before* reversing that k nodes remain.

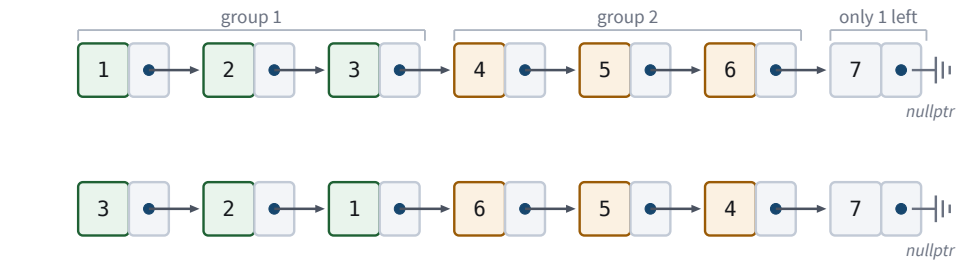


Figure 32.1 Reversing in groups of three. Each group is reversed by the ordinary loop, bounded by a counter; the group's old head becomes its tail and must be stitched onto the next group. The short final group is detected in advance and left alone.

KEY INSIGHT | The check must come first

After you reverse a group, you cannot tell how many nodes *were* in it — the information is gone. So count k nodes ahead **before** touching anything, and if you cannot find k, return immediately with the remainder intact.

That is also what makes the function naturally recursive: reverse the first group, then let the recursive call deal with everything after it, and attach the result to the group's new tail — which is the node that used to be its head.

Reverse Nodes in k-Group

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* reverseKGroup(Node* head, int k) {
    if (k <= 1 || head == nullptr) return head;

    Node* check = head;                        // are there k nodes left?
    for (int i = 0; i < k; ++i) {
        if (check == nullptr) return head;     // fewer than k - leave
            them alone
        check = check->next;
    }
    // `check` now points just past this group.
    Node* prev = nullptr;
    Node* curr = head;
    for (int i = 0; i < k; ++i) {              // the ordinary loop, k
        times
        Node* nxt = curr->next;
        curr->next = prev;
```

```
        prev = curr;
        curr = nxt;
    }
    // head is now this group's TAIL; attach the rest to it.
    head->next = reverseKGroup(curr, k);
    return prev;                              // prev is this group's new
        head
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 2, 3, 4, 5});
    a = reverseKGroup(a, 2); printList(a); freeList(a);

    Node* b = buildList({1, 2, 3, 4, 5});
    b = reverseKGroup(b, 3); printList(b); freeList(b);

    Node* c = buildList({1, 2, 3});
    c = reverseKGroup(c, 5); printList(c); freeList(c);   // k > length

    Node* d = buildList({1, 2, 3, 4});
    d = reverseKGroup(d, 1); printList(d); freeList(d);   // k = 1: no
        change
    return 0;
}
```

```
2 -> 1 -> 4 -> 3 -> 5
3 -> 2 -> 1 -> 4 -> 5
1 -> 2 -> 3
1 -> 2 -> 3 -> 4
```

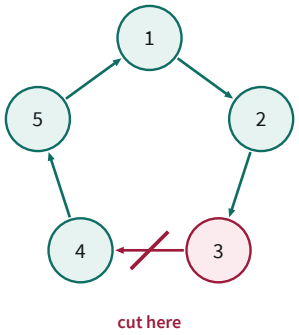
Cost. $O(n)$ time — each node is reversed once — and $O(n/k)$ stack from the recursion. An iterative version with a dummy node gets it to $O(1)$ space; it is longer, and worth writing as an exercise.

Watch for. `head->next = reverseKGroup(curr, k);` must come **after** the reversal loop, because `head` has become the group's tail by then. Writing it before gives a list that loses everything after the first group.

2. Rotate List

DEFINITION | The problem
Rotate right by k: `1->2->3->4->5` with k=2 becomes `4->5->1->2->3`. k may be larger than the list.

Think. The naive approach moves the last node to the front, k times: $O(n \cdot k)$. Better: find the new head directly.




```
1. walk to the tail, counting n
2. tail->next = head   (close the ring)
3. k %= n
4. walk n - k - 1 steps to the new tail
5. newHead = newTail->next; newTail->next = nullptr
```

once it is a circle, rotating is just choosing where to cut

Figure 32.2 Rotating by making the list circular. Join the tail to the head, walk to the node that should become the new tail, and cut there. Three steps, one pass each, and no nodes are created or destroyed.

KEY INSIGHT | Close the ring, then cut it in a different place

1. Walk to the tail, counting the length n along the way.
2. Join `tail->next = head`, making it circular.
3. Reduce k to $k \bmod n$ — a rotation by n is the identity.
4. The new tail is $n-k-1$ steps from the old head. Walk there.
5. The new head is `newTail->next`. Set `newTail->next = nullptr`.

Closing the ring removes all the awkward wrap-around arithmetic: once it is a circle, rotating is just choosing where to cut.

Rotate List right by k

```
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

Node* rotateRight(Node* head, int k) {
    if (head == nullptr || head->next == nullptr || k == 0) return head;

    int n = 1;                              // 1. find the tail and the
```

```
        length
    Node* tail = head;
    while (tail->next != nullptr) { tail = tail->next; ++n; }

    tail->next = head;                    // 2. close the ring
    k %= n;                               // 3. only k mod n matters
    if (k == 0) { tail->next = nullptr; return head; }   // a whole turn

    Node* newTail = head;                 // 4. walk n-k-1 steps
    for (int i = 0; i < n - k - 1; ++i) newTail = newTail->next;

    Node* newHead = newTail->next;        // 5. cut
    newTail->next = nullptr;
    return newHead;
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({1, 2, 3, 4, 5});
    a = rotateRight(a, 2); printList(a); freeList(a);

    Node* b = buildList({1, 2, 3});
    b = rotateRight(b, 4); printList(b); freeList(b);     // 4 % 3 == 1

    Node* c = buildList({1, 2, 3});
    c = rotateRight(c, 3); printList(c); freeList(c);     // a whole turn

    Node* d = buildList({1});
    d = rotateRight(d, 99); printList(d); freeList(d);
    return 0;
}
```

```
4 -> 5 -> 1 -> 2 -> 3
3 -> 1 -> 2
1 -> 2 -> 3
1
```

Cost. $O(n)$ time, $O(1)$ space.

Watch for. `k %= n` — without it the walk runs off the end of the ring… except that it cannot, because the ring has no end; instead it loops and gives the wrong answer silently, which is worse. And the `k == 0` branch after the modulo must restore

`tail->next = nullptr`, or you return a circular list.

3. Sort List

DEFINITION | The problem
Sort a linked list in $O(n \log n)$ time. The expected follow-up is $O(1)$ extra space.

Think. Which sorting algorithm suits a linked list?

KEY INSIGHT | Why merge sort, and not quicksort
Quicksort needs random access to pick a pivot well and to partition efficiently. On a list, reaching the middle is $O(n)$, so a median-of-three pivot is expensive and a first-element pivot degenerates to $O(n^2)$ on sorted input.
Merge sort needs only: split in half, and merge two sorted sequences. Both are natural on a list — splitting is fast-and-slow, and merging is relinking with no extra array at all.
And here is the pleasing part: merge sort on an **array** needs $O(n)$ auxiliary space for the merge buffer. On a **list** it needs none, because merging rearranges pointers in place. The linked list is the one data structure on which merge sort is strictly better behaved than on an array.
(The recursion still costs $O(\log n)$ stack. A bottom-up iterative merge sort reaches true $O(1)$, and that is what `std::list::sort` does.)

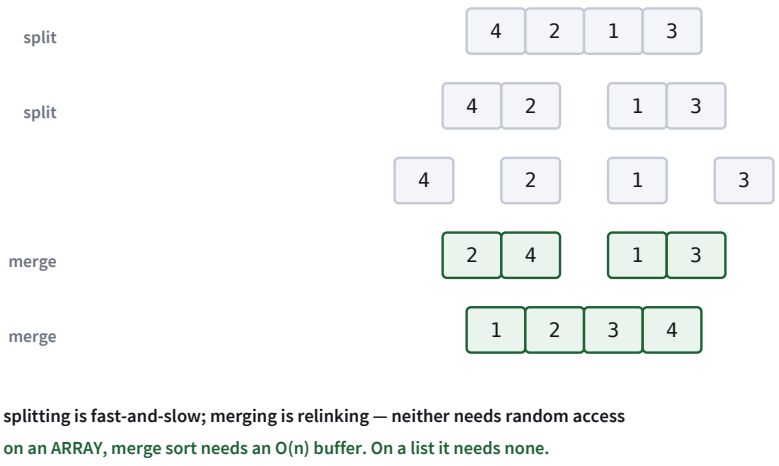


Figure 32.3 Merge sort on a list. Split with fast and slow, sort each half recursively, then merge by relinking. No auxiliary array exists at any point — the merge step only rewires `next` pointers.

Sort List — merge sort, O(n log n)

```cpp
#include <iostream>

struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};

```

```
Node* mergeSorted(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else                    { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = (a != nullptr) ? a : b;
    return dummy.next;
}

Node* sortList(Node* head) {
    if (head == nullptr || head->next == nullptr) return head;   // 0 or
        1 node

    Node* slow = head;                      // split at the FIRST middle
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* second = slow->next;
    slow->next = nullptr;                   // cut

    return mergeSorted(sortList(head), sortList(second));
}

Node* buildList(std::initializer_list<int> vs) {
    Node* head = nullptr; Node* tail = nullptr;
    for (int v : vs) { Node* n = new Node(v);
        if (!head) head = tail = n; else { tail->next = n; tail = n; } }
    return head;
}
void printList(Node* head) {
    for (Node* c = head; c; c = c->next) std::cout << c->data << (c->next
        ? " -> " : "\n");
    if (!head) std::cout << "(empty)\n";
}
void freeList(Node* head) { while (head) { Node* d = head; head =
    head->next; delete d; } }

int main() {
    Node* a = buildList({4, 2, 1, 3});
    a = sortList(a); printList(a); freeList(a);

    Node* b = buildList({-1, 5, 3, 4, 0});
    b = sortList(b); printList(b); freeList(b);

    Node* c = buildList({2, 2, 1});
    c = sortList(c); printList(c); freeList(c);

    printList(sortList(nullptr));
    return 0;
}
```

```
1 -> 2 -> 3 -> 4
-1 -> 0 -> 3 -> 4 -> 5
1 -> 2 -> 2
(empty)
```

Cost. $O(n \log n)$ time, $O(\log n)$ stack.

Watch for. Splitting with `fast = head->next` (the first middle). Using `fast = head` makes a two-node list split into two and zero nodes, so the recursion never shrinks and you get infinite recursion. Test `{2, 1}` specifically.

4. Copy List with Random Pointer

> **DEFINITION | The problem**
> Each node has `next` **and** a `random` pointer that may point to any node in the list, or to `nullptr`. Produce a deep copy: a completely separate list whose structure matches exactly.

Think. The difficulty: when you copy node X and want to set its `random`, the node it should point at may not have been created yet.

The $O(n)$-space answer. Build a hash map from each original node to its copy in one pass, then fill in the `next` and `random` pointers in a second pass using the map. Clear, correct, and the right first answer to give.

The $O(1)$-space answer is one of the neatest tricks in the subject.

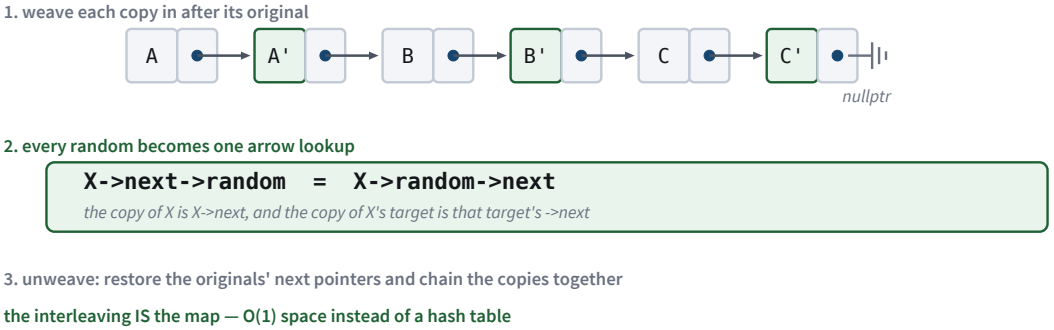


Figure 32.4 Copying with interleaving. Each copy is spliced in directly after its original, so `original->next` *is* the copy. That makes every random pointer computable locally: the copy of `x->random` is simply `x->random->next`.

KEY INSIGHT | Interleaving makes the map unnecessary

Step 1 — weave. After every original node, insert its copy: $A \to A' \to B \to B' \to C \to C'$.

Step 2 — set the randoms. For each original X, the copy of X is `X->next`, and the node X's random points at — call it Y — has its copy at `Y->next`. So:

```
X->next->random = (X->random != nullptr) ? X->random->next : nullptr;
```

The interleaving *is* the map. Every lookup that needed a hash table is now a single `->next`.

Step 3 — unweave. Separate the two lists, restoring the original's `next` pointers and linking the copies to each other.

$O(n)$ time and $O(1)$ extra space, where the output list does not count.

Copy List with Random Pointer, O(1) extra space

```
#include <iostream>

struct RNode {
    int data;
    RNode* next;
    RNode* random;
    RNode(int value) { data = value; next = nullptr; random = nullptr; }
};

RNode* copyRandomList(RNode* head) {
    if (head == nullptr) return nullptr;

    for (RNode* c = head; c != nullptr; ) {        // 1. weave copies in
        RNode* copy = new RNode(c->data);
        copy->next = c->next;
        c->next = copy;
        c = copy->next;                            // skip over the copy
    }
    for (RNode* c = head; c != nullptr; c = c->next->next)   // 2. set
    randoms
        c->next->random = (c->random != nullptr) ? c->random->next :
            nullptr;

    RNode dummy(0);                                // 3. unweave
    RNode* tail = &dummy;
    for (RNode* c = head; c != nullptr; ) {
        RNode* copy = c->next;
        c->next = copy->next;                      // restore the original
        tail->next = copy;                         // collect the copy
        tail = copy;
        c = c->next;
    }
    return dummy.next;
}

int main() {
    RNode* a = new RNode(7);
    RNode* b = new RNode(13);
    RNode* c = new RNode(11);
    a->next = b; b->next = c;
```

```
    a->random = nullptr;
    b->random = a;          // 13 -> 7
    c->random = a;          // 11 -> 7

    RNode* copy = copyRandomList(a);

    for (RNode* p = copy; p != nullptr; p = p->next) {
        std::cout << p->data << "(random ";
        std::cout << (p->random ? std::to_string(p->random->data) :
            "null");
        std::cout << ") ";
    }
    std::cout << "\n";

    // The copy must be a genuinely separate list.
    std::cout << "separate nodes: " << (copy != a) << "\n";
    std::cout << "original intact: " << (a->next == b && b->next == c) <<
        "\n";
    std::cout << "copy's random points into the COPY: "
              << (copy->next->random != a) << "\n";

    while (copy) { RNode* d = copy; copy = copy->next; delete d; }
    delete a; delete b; delete c;
    return 0;
}
```

```
7(random null) 13(random 7) 11(random 7)
separate nodes: 1
original intact: 1
copy's random points into the COPY: 1
```

Cost. $O(n)$ time, $O(1)$ extra space.

Watch for. Step 3 must restore the original list, not just extract the copy — the problem requires the input to be unchanged. The three checks in `main` test exactly the three things that can go wrong: shared nodes, a corrupted original, and random pointers that still point into the original.

The four, at a glance

#	Problem	Key idea	Time	Space
1	Reverse in k-groups	count first, then the standard loop	$O(n)$	$O(n/k)$ stack
2	Rotate List	close the ring, cut elsewhere	$O(n)$	$O(1)$
3	Sort List	merge sort — splitting and merging are both natural	$O(n \log n)$	$O(\log n)$
4	Copy with random	interleave, so the copy is always `->next`	$O(n)$	$O(1)$

Table 32.1 The four advanced problems and the ideas they rest on. Every technique here appeared earlier in the book; what is new is the combination.

INTERVIEW ANGLE | **What these four are really testing**

Not whether you have seen them. Whether, when you have not, you can:

- **decompose** — "this is reversal under a counter";
- **transform** — "if I make it circular, rotation becomes a cut";
- **choose** — "merge sort, because splitting and merging are cheap here";
- **substitute** — "interleaving gives me the map without the memory".

Each of those is a move you can practise. A candidate who says *"I don't know this one, but it looks like reversal with a bound — let me try that"* is doing better than one who recites a memorised solution to a different problem.

CHAPTER RECAP | **Chapter 32 in five lines**

- Check that k nodes remain **before** reversing a group; afterwards the count is unrecoverable.
- Making a list circular turns rotation into choosing a cut point.
- Merge sort suits lists because splitting and merging need no random access and no auxiliary array.
- Splitting must use the **first** middle, or a two-node list recurses forever.
- Interleaving copies with originals replaces a hash map with a `->next`.

EXERCISES | **Practise**

1. Write `reverseKGroup` iteratively with a dummy node, in $O(1)$ space.
2. Reverse in k-groups but leave the final short group reversed too. Which line changes?
3. For rotate, what goes wrong without `k %= n`? Trace `{1,2,3}` with k=7.
4. Rotate **left** by k instead. What is the relationship to rotating right?
5. Implement bottom-up iterative merge sort on a list, for true $O(1)$ space. Why is it harder than the recursive version?
6. Why does quicksort degrade on a linked list? Which specific step is expensive?
7. Solve "copy list with random pointer" with a hash map, then compare the two solutions on clarity and on space.
8. In the interleaving solution, what happens if you forget step 3's restoration of the original list? Write the test that would catch it.

PART VIII

Thinking Like a Problem Solver

The complete cost comparison of the two structures, a seventeen-step method that works on problems you have never seen, and the C++ habits that keep your code correct.

33 Arrays versus Linked Lists: The Complete Picture

34 How to Solve Any DSA Problem

35 C++ Implementation Standards

CHAPTER

33

Arrays versus Linked Lists: The Complete Picture

Every operation, every condition, stated precisely

REFERENCE PREREQS: Parts II and VI

This is the chapter to re-read before an interview. It states the cost of every operation on both structures with the conditions attached — because "linked list insertion is $O(1)$" is false as usually quoted, and knowing exactly when it is true is what the question is testing. It ends with the part the complexity tables hide: what actually happens on real hardware.

WHAT YOU WILL LEARN

- State the cost of every operation on both structures, with its precondition
- Explain *why* array indexing is $O(1)$ and list indexing is $O(n)$
- Give the precise conditions under which linked list insertion is $O(1)$
- Separate the cost of **navigation** from the cost of **modification**
- Explain why arrays usually win in practice despite the tables
- Choose the right structure for a stated workload, and defend it

The master table

Operation	Array / vector	Singly linked	Doubly linked	Condition
Access by index i	$O(1)$	$O(i)$	$O(i)$	—
Access the first element	$O(1)$	$O(1)$	$O(1)$	—
Access the last element	$O(1)$	$O(n)$	$O(1)$	list needs a tail pointer
Search (unsorted)	$O(n)$	$O(n)$	$O(n)$	—
Search (sorted)	$O(\log n)$	$O(n)$	$O(n)$	binary search needs random access
Insert at the front	$O(n)$	$O(1)$	$O(1)$	—
Insert at the back	$O(1)$ amortised	$O(n)$ / $O(1)$	$O(1)$	$O(1)$ only with a tail pointer

Operation	Array / vector	Singly linked	Doubly linked	Condition
Insert at index i	$O(n-i)$	$O(i)$	$O(i)$	both are $O(n)$ in the worst case
Insert given a pointer/iterator	$O(n)$	$O(1)$	$O(1)$	**the list's real advantage**
Delete the first element	$O(n)$	$O(1)$	$O(1)$	—
Delete the last element	$O(1)$	$O(n)$	$O(1)$	doubly needs a tail pointer
Delete at index i	$O(n-i)$	$O(i)$	$O(i)$	—
Delete given a pointer	$O(n)$	$O(n)$	$O(1)$	singly must find the predecessor
Traverse everything	$O(n)$	$O(n)$	$O(n)$	very different constants
Splice a whole range	$O(n)$	$O(1)$	$O(1)$	given pointers to both ends
Memory per `int` element	4 bytes	~16–32 bytes	~24–40 bytes	including allocator overhead
Needs contiguous memory	yes	no	no	—
Iterators survive insertion	no	yes	yes	vectors reallocate

Table 33.1 Every operation, both structures, with the condition each cost depends on. The condition column is the one most people omit, and it is where the real answer lives.

Read the table twice. The first reading says "they trade off". The second reading, with the condition column, says something sharper — which is the next section.

The single idea behind all of it

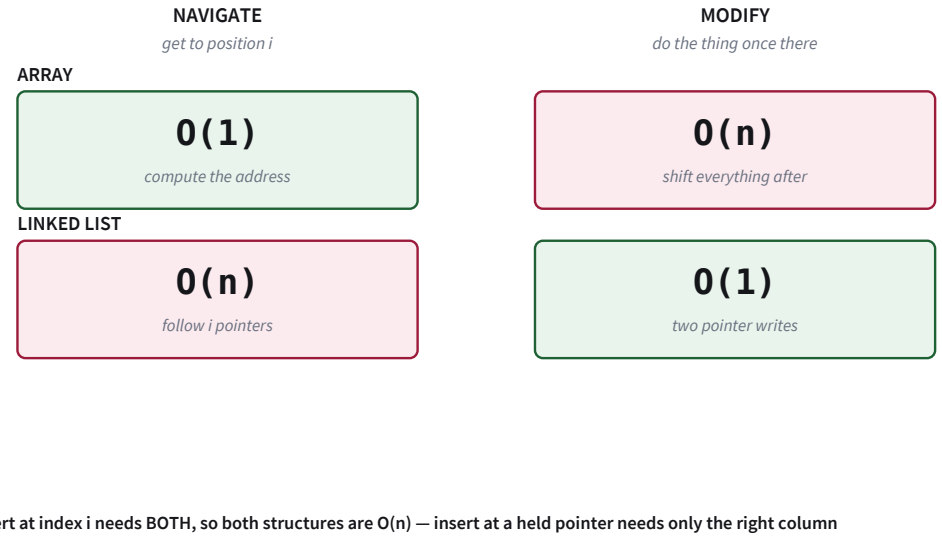


Figure 33.1 Navigation versus modification. An array pays nothing to navigate and everything to modify; a linked list pays everything to navigate and nothing to modify. Each operation's true cost is the sum of the two.

KEY INSIGHT | Separate navigation from modification
Every operation is really **two** operations: *get to the right place*, then *do the thing*.

	Navigate to position i	Modify once there
Array	$O(1)$ — compute the address	$O(n)$ — shift everything after
Linked list	$O(n)$ — follow i pointers	$O(1)$ — two pointer writes

Now every row of the master table falls out of this one table:

- *Insert at index i* needs both, so it is $O(n)$ for **both** structures. The array walks no distance and shifts $n-i$ elements; the list walks i nodes and shifts nothing. Same complexity, opposite work.
- *Insert given a pointer* needs only the modification, so the list is $O(1)$ and the array is still $O(n)$. **This is the list's genuine advantage, and it is the only row where it wins outright.**
- *Access by index* needs only the navigation, so the array is $O(1)$ and the list is $O(n)$.

If you remember one thing from this chapter, remember this table rather than the big one.

Why array indexing is O(1): the exact reason

Three facts, and the conclusion follows:

1. **Elements are the same type**, so each occupies the same number of bytes S.
2. **They are contiguous**, with no gaps.
3. Therefore element i begins at $B+i\times S$, where B is the base.

That expression is one multiplication and one addition, regardless of i and regardless of n. The processor performs it in a single instruction — on x86, the address mode `[base + index*4]` does it for free as part of the load.

KEY INSIGHT | Remove any of the three facts and it breaks

- **Different sizes?** Then element i's offset depends on the sizes of elements 0 through $i-1$, and you must examine them: $O(i)$.
- **Gaps?** Then the formula gives a wrong address.
- **Scattered?** Then there is no formula at all — which is exactly the linked list case.

A linked list fails all three. Its "elements" are nodes of equal size, but they are not contiguous and not in order, so no arithmetic can find them. The only route to node i is through nodes $0\ldots i-1$, and each step is one dereference: $O(i)$, and $O(n)$ in the worst case.

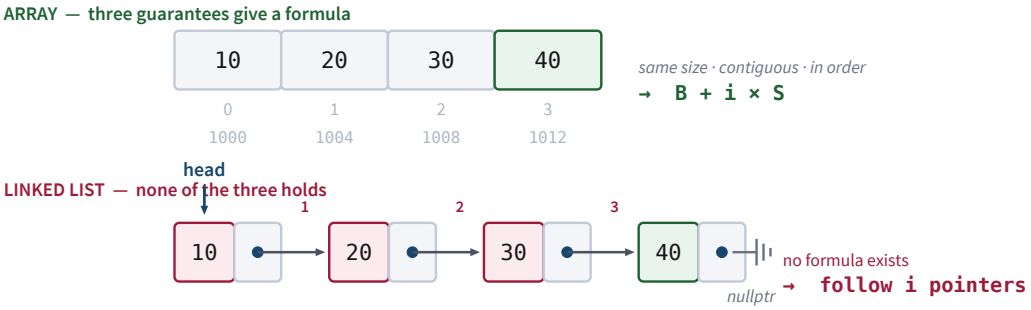


Figure 33.2 Why one is a calculation and the other is a walk. The array's three guarantees produce an address formula; the linked list offers no guarantee about location, so it offers no formula.

The claim people get wrong

WATCH OUT | **"Linked lists have O(1) insertion" — as usually said, this is false**

What is true: **given a pointer to the node you are inserting after**, insertion is $O(1)$.

What is false: inserting at an arbitrary position is $O(1)$. Finding that position is $O(i)$, and $O(i)+O(1)=O(i)$.

So for "insert at index 500 of a 1000-element structure":

- **Array:** 0 steps to navigate, 500 elements shifted. $O(n)$.
- **List:** 500 steps to navigate, 2 pointer writes. $O(n)$.

Both are $O(n)$. The list is not faster here. In fact it is usually *slower*, because 500 pointer dereferences across scattered memory cost far more than 500 contiguous `memmove`d integers — a point the next section makes concrete.

INTERVIEW ANGLE | **The precise sentence**

When asked to compare the two, say this:

> *"A linked list gives O(1) insertion and deletion **when you already hold a pointer to the position**. Getting that pointer is O(n) unless you obtained it some other way — from an iterator you were keeping, or from a hash map. An array gives O(1) access by index but O(n) modification. So the real question is whether the workload supplies you with positions, or asks you to find them."*

Then give the examples where a list genuinely wins:

- **LRU cache** — a hash map hands you the node pointer; you unlink it in $O(1)$. An array could not.
- `std::list::splice` — moving a range of any length between lists is four pointer writes, because nothing is copied.
- **Stable iterators** — pointers into a list survive insertions; pointers into a vector do not.

What the tables hide: real hardware

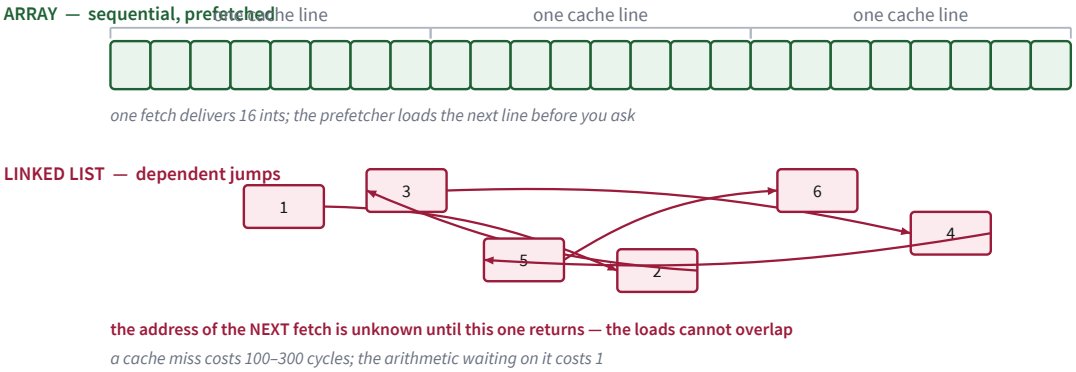


Figure 33.3 Why the same complexity is not the same speed. Traversing an array reads consecutive memory, which the processor prefetches in 64-byte lines. Traversing a list is a chain of dependent jumps to unpredictable addresses, and each one may stall for hundreds of cycles.

PERFORMANCE NOTE | The constant factor is roughly 10×, and it is not a detail

Both traversals are $O(n)$. On real hardware they are not close.

Array traversal. Elements are 4 bytes and a cache line is 64, so one memory fetch delivers 16 elements. The prefetcher sees the sequential pattern and loads the next line before you ask. Effective cost: a fraction of a cycle per element.

List traversal. Each node is somewhere else, so each step is a fresh fetch, and — worse — the address of the *next* fetch is not known until the current one completes. The loads cannot be overlapped. This is a **dependent load chain**, and it is the slowest pattern in computing. A cache miss costs roughly 100–300 cycles; the arithmetic it is waiting for costs 1.

Add the memory: a node holding one `int` typically occupies 32 bytes once allocator bookkeeping is counted — **eight times** the payload. Eight times the memory means one-eighth of your data fits in cache.

The practical consequence: for small elements, `std::vector` beats `std::list` at almost everything, *including* operations where the complexity table says the list should win. Inserting at the front of a 1,000-element vector means shifting 4 KB, which `memmove` does in a few hundred nanoseconds. Walking 1,000 list nodes to find a position costs more than that on its own.

NOTE | When should you actually use `std::list`?
Rarely, and for specific reasons:

- Elements are **large or expensive to move**, so shifting them is genuinely costly.
- You **already hold iterators** to the positions you modify — the LRU-cache pattern.
- You need **iterator stability** across insertions and deletions.
- You **splice** whole ranges between containers frequently.

Otherwise: `std::vector`, even when the complexity table suggests otherwise. Measure if it matters.
None of this diminishes the value of learning linked lists. They are the foundation of trees, graphs, hash table buckets and allocators; they are the standard vehicle for testing pointer skill; and they are the clearest possible illustration of the trade-off this chapter is about.

Choosing, for a stated workload

Workload	Choose	Why
Random access by index, data mostly read	**array**	$O(1)$ access; cache-friendly
Append and iterate, nothing else	**array**	amortised $O(1)$ append, fast scans
Binary search needed	**array**	random access is a precondition
Insert and delete at the front only	either	list $O(1)$; a deque is better still
Insert and delete at **held** positions	**list**	$O(1)$ modification, no shifting
LRU cache	**hash map + doubly linked list**	map finds the node, list unlinks in $O(1)$
Stable references across edits	**list**	vectors reallocate and invalidate
Huge elements, frequent reordering	**list**	relinking beats moving
Memory is tight	**array**	no per-element pointer overhead
Splicing ranges between containers	**list**	$O(1)$ regardless of range length

Table 33.2 Workload-driven choice. In an interview, ask which of these the problem is before choosing — the question is often precisely that.

The questions to ask before choosing

CHECKLIST | **A decision procedure**

1. **Do I need access by index?** Yes → array. Binary search also means yes.
2. **Where do insertions and deletions happen?** Only at the end → array. Only at the front → either, and a deque is better than both. Anywhere → ask question 3.
3. **Do I already hold a pointer to the position?** Yes → list genuinely wins. No → both are $O(n)$, so pick the array for its constant factor.
4. **Do references have to stay valid?** Yes → list.
5. **How big is an element?** Small → array, because overhead and cache dominate. Large → the list's no-copy advantage starts to matter.
6. **How large is n?** Below a few hundred, the constant factor decides everything and the array almost always wins.

CHAPTER RECAP | **Chapter 33 in six lines**

- Every operation is **navigate** then **modify**. Arrays are $O(1)/O(n)$; lists are $O(n)/O(1)$.
- Array indexing is $O(1)$ because equal sizes plus contiguity give an address formula. Lists have none of the three preconditions.
- "Lists have $O(1)$ insertion" is only true **given a pointer to the position**. At an arbitrary index, both structures are $O(n)$.
- Binary search needs random access, so it is an array algorithm — full stop.
- The constant factor is about 10×, from cache behaviour and per-node overhead. Equal complexity is not equal speed.
- In production, prefer `std::vector` unless you can name the specific reason not to.

EXERCISES | **Practise**

1. Fill in the master table from memory, including the condition column. Check it against the chapter.
2. For each of these, say which structure wins and why: appending a million integers; deleting the front element a million times; finding the median; maintaining a playlist where tracks are dragged around by the user.
3. Why can a linked list not support binary search, even when sorted? Answer in terms of navigation cost, not in terms of "it has no indices".
4. Describe an LRU cache's two data structures and state the cost of `get` and `put`. Which part does each structure contribute?
5. A colleague says "I used a list because I insert in the middle a lot". What is the one question you should ask them?
6. Estimate the memory used by a `std::vector<int>` and a `std::list<int>`, each holding a million elements. What is the ratio?
7. Both traversals are $O(n)$. Explain, in terms of the processor, why one may be ten times slower.
8. Give a workload where the list genuinely wins on wall-clock time, and one where the complexity table says it should win but it does not.

CHAPTER

34

How to Solve Any DSA Problem

A seventeen-step procedure for problems you have never seen

CORE PREREQS: Parts III and VII

The goal of this book is not that you can solve the problems in it — it is that you can solve the ones that are not. This chapter gives a repeatable procedure, applies it end to end to a problem you have not been shown, and explains what each step is actually protecting you from. Follow it mechanically at first; it becomes instinct faster than you expect.

WHAT YOU WILL LEARN

- Apply a seventeen-step method to an unfamiliar problem
- Restate a problem precisely, and ask the questions worth asking
- Reach a working brute force before attempting to be clever
- Find the bottleneck, and match it to a pattern
- Trace, test and cost your solution before declaring it finished
- Know what each step prevents, so you can skip steps knowingly rather than by accident

The method

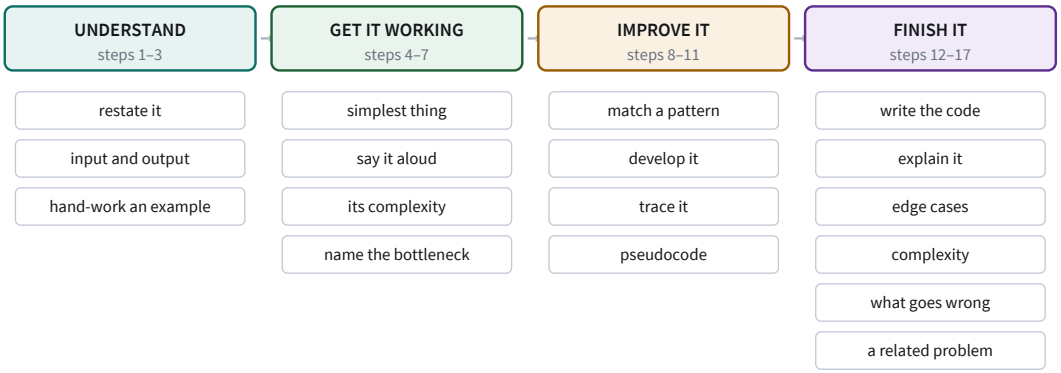


Figure 34.1 The seventeen steps in four phases. Most candidates begin at step 12 and fail at step 14. The first four steps take two minutes and prevent most of the ways an interview goes wrong.

CHECKLIST | The seventeen steps

Understand (steps 1–3)

1. Restate the problem in your own words, in one plain sentence.
2. Say exactly what the input is and what the output is, including types and the awkward values each may take.
3. Work one tiny example by hand, on paper. Three to five elements, never more.

Reach something that works (steps 4–7)

1. Identify the simplest thing that could possibly work, however slow.
2. Describe that brute force out loud, in one sentence.
3. State its time and space complexity.
4. Name the **bottleneck** — the one operation making it slow.

Improve it (steps 8–11)

1. Match the bottleneck to a pattern. (The table below does this for you.)
2. Develop the improved approach step by step, saying what each step buys.
3. Trace the new approach on your tiny example, as a table.
4. Write pseudocode — structure only, no syntax.

Finish properly (steps 12–17)

1. Write clean C++.
2. Explain the lines that are not obvious, as you write them.
3. Test the edge cases: empty, one element, all equal, negatives, duplicates, the largest allowed input.
4. State the final time and space complexity.
5. Say what commonly goes wrong in this solution.
6. Name a related problem that uses the same idea.

KEY INSIGHT | What each phase protects you from
Understand prevents the worst outcome: a correct solution to the wrong problem. It costs two minutes and it is the step candidates skip most often.
Reach something that works prevents the second-worst: twenty minutes of silence followed by nothing. A slow solution is worth far more than an elegant idea you did not finish, and it gives the interviewer something to discuss.
Improve it is the part being assessed. Note that it comes *after* a working solution, not instead of one.
Finish properly is where most of the remaining marks are. Untested code with an unknown complexity is not a finished answer.

The bottleneck-to-pattern table

Step 8 is the one that feels like magic. It is not — it is a lookup.

The brute force is slow because…	Reach for	Chapter
it searches the array again for each element	a hash map or set	14
it compares every pair	two pointers (if sorted)	9
it recomputes the sum of overlapping ranges	sliding window or prefix sums	10, 11
it scans a sorted array linearly	binary search	12
it re-derives "the best so far" each time	a running variable — Kadane's shape	8, 16
it uses a second array	a write pointer, or index marking	13
it needs order that the input lacks	sort first	15
it walks the list repeatedly to find a position	fast/slow, or a fixed gap	29
it needs to know what came before in a list	a dummy node, or `prev` / `curr`	26, 29
it needs to detect repetition with no memory	Floyd's cycle detection	29
it asks "the minimum X such that…"	binary search on the answer	12

Table 34.1 Step 8 made mechanical. Find the bottleneck in the left column, read the pattern in the right.

INTERVIEW ANGLE | The two sentences that most improve an answer
At step 2: *"Before I start — can the input be empty? Are there duplicates? Can the values be negative? Should I return the index or the value?"*
At step 6: *"This brute force is $O(n^2)$ time and $O(1)$ space. Shall I code it first, or go straight to optimising?"*
The first shows you know where bugs come from. The second shows you know what you have, and hands the interviewer a decision — which is always welcome.

The method applied, end to end

Here is a problem that has not appeared in this book.

> ***Problem.*** *Given an array of integers, return the length of the longest sub-array in which the difference between the largest and smallest element is at most k.*

Work through it with me, and notice that the solution is never guessed.

1. Restate it

"Find the longest run of neighbouring elements that are all within k of each other."

2. Input and output

- **Input:** `std::vector<int> v`, and an integer `k`.
- **Output:** one integer — a length, not the sub-array itself.
- **Awkward values:** empty array → 0. k=0 → the longest run of equal values. Negative values are fine; negative k makes no sub-array valid except possibly none, so I will ask, and assume $k \geq 0$.

3. A tiny example

`v = {1, 5, 6, 7, 2}`, k=2.

By hand: `{5, 6, 7}` has max −min =2≤2, length 3. Can I do better? `{1, 5, 6, 7}` spans 6 — no. `{6, 7, 2}` spans 5 — no. **Answer: 3.**

4–5. The brute force

For every start i, extend to every end j, tracking the min and max of the window as I go; whenever max −min ≤k, record the length.

6. Its complexity

$O(n^2)$ time — n starts, each extending up to n — and $O(1)$ space, because the min and max are maintained incrementally.

7. The bottleneck

Every start re-examines elements that previous starts already examined. The work on overlapping ranges is repeated.

8. Match it to a pattern

The table says: *recomputes over overlapping ranges* and *contiguous* → **sliding window**.

A quick check against Chapter 10's preconditions:

- Must the answer be contiguous? **Yes** — it says sub-array.
- Is the property monotonic in window size? **Yes** — if a window is valid, every window inside it is also valid, because removing elements cannot increase the spread. That is exactly what the shrink rule needs.

Both hold, so a sliding window applies.

9. Develop it

Grow the window to the right. While `max - min > k`, shrink from the left. The window is then valid; record its length.

One difficulty: **I need the max and min of the current window, and shrinking from the left can remove the element that was the max.** A plain running maximum cannot be undone.

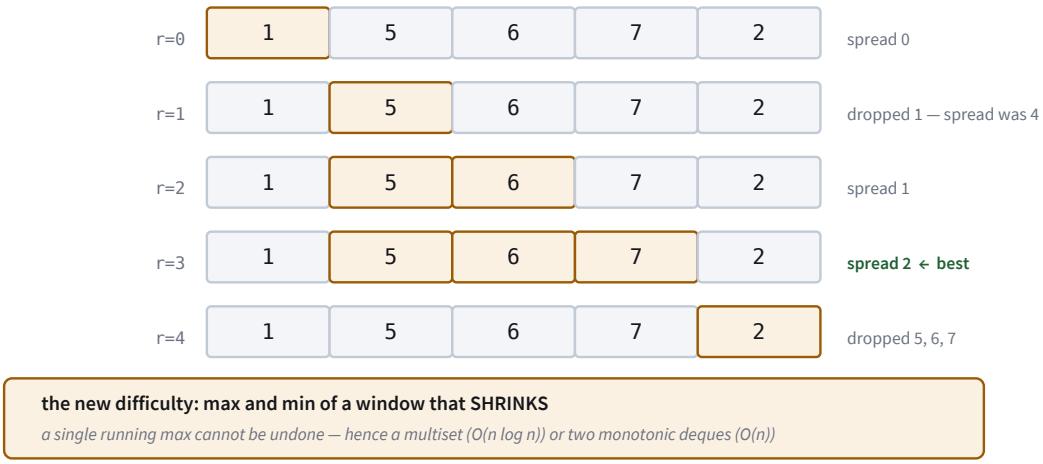


Figure 34.2 The worked problem. The window grows right and shrinks left as usual; the new part is needing the maximum and minimum of a window that shrinks, which a single running variable cannot provide.

Three ways to get the window's max and min:

- A **multiset**: $O(\log n)$ per insertion and removal → $O(n \log n)$ overall. Simple and correct.
- Two **monotonic deques**, one for the max and one for the min: $O(1)$ amortised each → $O(n)$ overall. More machinery.
- Recompute on each shrink: back to $O(n^2)$. No.

I will state both of the first two, code the multiset version because it is clearly correct, and mention the deque version as the optimum.

10. Trace it

`{1, 5, 6, 7, 2}`, k=2.

right	window	min, max	spread	shrink?	left	length	best
0	`1`	1, 1	0	no	0	1	1
1	`1 5`	1, 5	4	yes → drop 1	1	1	1
2	`5 6`	5, 6	1	no	1	2	2
3	`5 6 7`	5, 7	2	no	1	3	**3**
4	`5 6 7 2`	2, 7	5	yes → drop 5, 6, 7	4	1	3

Table 34.2 The trace. The window never shrinks further than it must, and `left` only moves forwards.

Answer 3, matching the hand-worked example in step 3. That agreement is the check — a trace that disagrees with your hand example means one of them is wrong, and you have found it before writing any code.

11. Pseudocode

```
best = 0
left = 0
window = empty multiset
for right in 0 .. n-1:
    insert v[right] into window
    while (max(window) - min(window) > k):
        remove v[left] from window
        left = left + 1
    best = max(best, right - left + 1)
return best
```

12–13. The code

Longest sub-array with spread at most k

```cpp
#include <algorithm>
#include <iostream>
#include <set>
#include <vector>

int longestWithinK(const std::vector<int>& v, int k) {
    if (k < 0) return 0;                    // no sub-array can qualify
    std::multiset<int> window;              // multiset: duplicates
        allowed
    int left = 0, best = 0;

    for (int right = 0; right < (int)v.size(); ++right) {
        window.insert(v[right]);                        // grow

        while (!window.empty() &&
               *window.rbegin() - *window.begin() > k) {   // max - min
            window.erase(window.find(v[left]));         // erase ONE copy
            ++left;
        }
        best = std::max(best, right - left + 1);        // record
    }
    return best;
}

int main() {
    std::cout << longestWithinK({1, 5, 6, 7, 2}, 2) << "\n";
    std::cout << longestWithinK({1, 1, 1}, 0)       << "\n";
    std::cout << longestWithinK({8, 2, 4, 7}, 4)    << "\n";
    std::cout << longestWithinK({}, 5)              << "\n";
    std::cout << longestWithinK({4}, 0)             << "\n";
    std::cout << longestWithinK({1, 2, 3}, -1)      << "\n";
    return 0;
}
```

```
3
3
2
0
1
0
```

The two lines worth explaining as you write them:

`window.erase(window.find(v[left]))` . `erase(value)` on a multiset removes **every** copy of that value. `erase(iterator)` removes exactly one. On `{5, 5, 6}` the first form would delete both 5s and corrupt the window. This is a real bug that compiles, runs, and gives wrong answers on inputs with duplicates.

`*window.rbegin()` **and** `*window.begin()` . A `std::multiset` is sorted, so the first element is the minimum and the last is the maximum, both in $O(1)$.

14. Edge cases

Tested above: empty array → 0; one element → 1; all equal with k=0 → 3; negative k → 0; no valid pair → the single-element answer. Also worth checking: large values, where `max - min` could overflow `int` — with values near $\pm 2^{31}$ the subtraction overflows, so a production version would use `long long` .

15. Complexity

$O(n \log n)$ time — each element is inserted once and erased at most once, each $O(\log n)$ — and $O(n)$ space for the multiset. The monotonic-deque version is $O(n)$ time and $O(n)$ space.

16. What goes wrong

- `erase(value)` instead of `erase(iterator)` — removes all duplicates.
- Forgetting that shrinking may need to happen several times, so a `while` , not an `if` .
- Assuming a running max can be maintained with a single variable; it cannot survive a shrink.
- Overflow in `max - min` on extreme values.

17. A related problem

“Longest sub-array with at most k distinct elements” — same window shape, with a frequency map instead of a multiset. And “sliding window maximum” is exactly the monotonic deque this problem’s optimal version needs, in isolation.

KEY INSIGHT | What just happened
The solution was never guessed. It was **derived**:

- the word "sub-array" said contiguous;
- "contiguous plus repeated overlapping work" said sliding window;
- checking the monotonicity precondition confirmed it;
- the only genuine difficulty — max and min of a shrinking window — was isolated and then solved on its own;
- the trace agreed with the hand-worked example before any code was written.

That is the whole method. You will not always have a pattern for the bottleneck, and then you think harder. But most of the time, you will.

Using the method under time pressure

You will not always have forty minutes.

CHECKLIST | The compressed version — five minutes

1. Restate, and ask about empty, duplicates, negatives, return type.
2. Hand-work one small example.
3. State the brute force and its complexity out loud. Do not code it yet.
4. Name the bottleneck; look it up in the pattern table.
5. State the improved approach and its complexity, then code that.
6. Trace one example. Test empty and one element.
7. State the complexity and one thing that could go wrong.

Steps 1, 3 and 7 are the ones never to skip. They take under a minute between them and they are where most of the impression is formed.

When the method does not produce an answer

Sometimes no pattern fits. Then:

1. **Solve a smaller version.** Can you do it for n=2? For n=3? A pattern in the small cases often reveals the general one.
2. **Solve a restricted version.** What if the values were sorted? All positive? Distinct? Solving the easier problem frequently shows the way, and it is a much better answer than nothing.
3. **Ask what you are not using.** Every stated constraint is a hint. "Values between 1 and n", "the array is sorted", "$O(1)$ space", "no division" — each rules out an easy answer and points at the intended one.
4. **Work backwards from the answer's shape.** If the answer must be $O(n)$, you get one pass; what could one pass possibly carry?
5. **Say what you are thinking.** Silence is the only genuinely bad outcome. An interviewer who hears "I'm looking for a way to avoid re-scanning the past, which usually means hashing" can help you; one who hears nothing cannot.

CHAPTER RECAP | **Chapter 34 in six lines**

- Understand, then get something working, then improve it, then finish it. **In that order.**
- A slow working solution beats an elegant unfinished one, every time.
- Find the **bottleneck**, then look the bottleneck up in the pattern table.
- Trace before you code, and check the trace against your hand-worked example.
- Test empty, one element, all-equal, negatives and duplicates without being prompted.
- Every unused constraint in the problem statement is a hint about the intended solution.

EXERCISES | **Practise**

1. Apply all seventeen steps, in writing, to: “find the two numbers in a sorted array that are closest to a target sum”. Do not look anything up.
2. Apply them to: “given a linked list, remove every node whose value appears more than once”. Which part is hardest, and why?
3. For each row of the bottleneck table, write down a problem from this book that it matches.
4. Take any problem from Chapter 20 and write out steps 1 to 7 only. Can you reach the bottleneck without recalling the solution?
5. Set a timer for five minutes and apply the compressed version to “longest sub-array with at most two distinct values”. What did you have to leave out?
6. Rewrite the worked problem of this chapter using two monotonic deques for $O(n)$. Which steps of the method change?
7. Find a problem you cannot solve. Apply the five “when it does not work” tactics in order, writing down what each one yields.

CHAPTER

35

C++ Implementation Standards

The habits that make DSA code correct, readable and interview-ready

REFERENCE PREREQS: Chapters 21 and 22

Every piece of code in this book follows the same small set of rules. They are not style preferences — each one prevents a specific bug or answers a specific question an interviewer will ask. This chapter collects them, with the reason for each, and ends with a checklist to run over any solution before you call it finished.

WHAT YOU WILL LEARN

- Write function signatures that say whether the input will be modified
- Choose between by-value, by-reference and by-pointer parameters correctly
- Use `std::vector` idiomatically, and know which operations are expensive
- Write memory-safe code with `new` and `delete`, and verify it
- Name things so that the code needs fewer comments
- Run a pre-submission checklist over any solution

1. Signatures say what the function does

The parameter list is documentation that the compiler enforces. Use it.

Four signatures, four promises

```
int  sum(const std::vector<int>& v);          // reads it, cannot modify, no copy
void sortInPlace(std::vector<int>& v);        // modifies the caller's vector
std::vector<int> sorted(std::vector<int> v);  // takes a copy, leaves the original
int  maxOf(int a, int b);                     // small types by value: cheapest
```

DEFINITION | **The rule**

Parameter	Use when
`const T&`	you read a container or large object and do not modify it
`T&`	you modify the caller's object
`T` (by value)	the type is small (`int`, `double`, a pointer) **or** you genuinely want a copy
`T*`	the argument may be absent (`nullptr`), or you must reseat it

Passing a `std::vector` **by value** copies every element: $O(n)$ time and $O(n)$ memory on every call. It is sometimes exactly what you want — Chapter 15 takes a vector by value precisely in order to sort a copy — but it should be a decision, never an accident.

INTERVIEW ANGLE | **State it out loud**

"I'll take this as `const std::vector<int>&` so we don't copy it, and because the function doesn't modify the input."

One sentence, and you have demonstrated that you know the difference. If the function *does* modify its input, say that too — an interviewer will often ask "does this change the caller's data?" and you want to have answered already.

2. Use std::vector, and know what is expensive

Vector habits worth having

```
std::vector<int> v;
v.reserve(n);                       // one allocation when you know the size

for (int x : v) { }                 // read-only traversal
for (int& x : v) { x *= 2; }        // modifying traversal: note the &
for (const auto& s : words) { }     // non-trivial elements: avoid the copy

v.push_back(x);                     // amortised O(1)
v.pop_back();                       // O(1)
v.insert(v.begin(), x);             // O(n) - usually a mistake inside a
    loop
v.erase(v.begin());                 // O(n) - likewise
```

Looks cheap	Really costs	Instead
`v.insert(v.begin(), x)` in a loop	$O(n^2)$	build forwards, then `std::reverse`
`v.erase(it)` in a loop	$O(n^2)$	the erase–remove idiom
`f(std::vector<int> v)`	$O(n)$ per call	`const std::vector<int>&`
`s = s + c` in a loop	$O(n^2)$	`s += c`, or `s.reserve` first

Looks cheap	Really costs	Instead
`v[i]` with `i` from `size() - 1` on an empty vector	wraps to a huge number	`(int)v.size() - 1`

Table 35.1 The operations whose cost surprises people. Each of these looks like one statement and contains a loop.

COMMON PITFALL | The unsigned comparison, once more
`v.size()` is unsigned. These two lines behave differently on an empty vector:

```
for (int i = 0; i < (int)v.size() - 1; ++i) { }   // OK  -1, loop does
    not run
for (size_t i = 0; i < v.size() - 1; ++i) { }     // BAD huge number,
    runs forever
```

This book casts to `int` at every comparison. It is slightly verbose and it removes the problem. The alternative — `i + 1 < v.size()` — never subtracts at all and is arguably better; use either, consistently.

3. One consistent Node definition

Every linked list chapter in this book uses exactly this, unchanged:

The Node struct used throughout

```
struct Node {
    int data;
    Node* next;
    Node(int value) { data = value; next = nullptr; }
};
```

KEY INSIGHT | Why consistency matters more than elegance
A reader who has seen this struct once never has to read it again. Changing the field names between chapters — `data` to `val`, `next` to `link` — would cost the reader attention on every single example for no benefit.
The same applies to your own code and to an interview: pick the names the problem gives you (LeetCode uses `ListNode` with `val` and `next`) and keep them. Renaming the interviewer's types is a small, needless friction.

4. Memory rules

CHECKLIST | The six memory rules this book follows

1. Every `new` has exactly one matching `delete`, and `new[]` matches `delete[]`.
2. **Unlink before you free.** Reroute the pointers, then destroy the node.
3. **Save `next` before deleting.** `temp = curr; curr = curr->next; delete temp;`
4. After freeing a list, set `head = nullptr`.
5. After `delete p`, set `p = nullptr` — `delete nullptr` is a no-op, so double frees become harmless.
6. Verify with `-fsanitize=address,undefined`, every time.

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined -g prog.cpp -o prog
./prog
```

`-Wall -Wextra` catches unused variables, signed/unsigned comparisons and uninitialised reads at compile time. The sanitizers catch out-of-bounds access, leaks and use-after-free at run time, with the exact line. Between them they find most of what this book warns about, and they cost nothing to turn on.

NOTE | In an interview you will not run anything
Which is precisely why the rules above are worth making automatic. Nobody will tell you that you leaked a node; they will simply notice.
Saying *"and I'd free the removed node here"* while writing the deletion is the cheapest credibility in the whole process.

5. Naming

Names that remove the need for comments

```
// Vague
int n, m, t;
Node* p;
for (int i = 0; i < x; ++i) { }

// Clear
int rows, cols, target;
Node* curr;
for (int row = 0; row < rows; ++row) { }
```

The conventions this book uses, which are close to universal:

Name	Means
`i`, `j`, `k`	loop indices — these are fine, and clearer than alternatives
`n`, `m`	sizes of the first and second input
`left`, `right`	converging two pointers
`read`, `write`	same-direction two pointers
`slow`, `fast`	two-speed pointers

Name	Means
`prev`, `curr`, `nextNode`	list traversal and reversal
`head`, `tail`, `dummy`	list anchors
`best`, `current`	a running answer and a running candidate
`target`	the value being searched for

Note `nextNode` rather than `next`: `next` is also a standard library function (`std::next`), and shadowing it is legal but confusing.

6. Say whether the function modifies its input

Make this explicit in three places: the signature, a one-line comment, and out loud.

Three ways to express the same operation

```cpp
// Modifies v in place. Returns nothing.
void reverseInPlace(std::vector<int>& v);

// Returns a reversed copy. v is untouched.
std::vector<int> reversed(const std::vector<int>& v);

// Modifies the list and returns the new head. The old head is now the
    tail.
Node* reverseList(Node* head);
```

That third comment is worth the line. “Returns the new head” is the single most common thing people forget about list reversal, and writing it down makes it impossible to forget.

7. Handle edge cases explicitly, or show that they fall out

Two acceptable approaches, and one unacceptable one.

Explicit, implicit, and absent

```cpp
// A. Explicit: state the case and handle it.
int largest(const std::vector<int>& v) {
    if (v.empty()) throw std::invalid_argument("empty");
    // ...
}

// B. Implicit, with a comment saying why: the structure handles it.
void printList(Node* head) {
    // An empty list prints nothing: the loop simply never runs.
    for (Node* curr = head; curr != nullptr; curr = curr->next) { }
}

// C. Absent: the case is simply not considered.
int largestBad(const std::vector<int>& v) {
    int best = v[0];                // undefined behaviour on an empty
```

```
        vector
    // ...
}
```

B is the best when it is available — an edge case that falls out of the structure cannot be forgotten. A is correct when it is not. **C is never acceptable**, even when the problem statement promises a non-empty input; say *"the problem guarantees non-empty, so I'll rely on that"* rather than silently relying on it.

> **CHECKLIST | The edge cases to check, every time**
> **Arrays:** empty · one element · two elements · all equal · all negative · already sorted · reverse sorted · duplicates · the maximum allowed size.
> **Linked lists:** empty (`nullptr`) · one node · two nodes · the operation targets the head · the operation targets the tail · the value is absent · duplicates · a cycle, if cycles are possible.

8. Avoid cleverness before clarity

The same swap, three ways

```
std::swap(a, b);                        // OK  clear, and optimal

int t = a; a = b; b = t;                // OK  fine, and obvious

a ^= b; b ^= a; a ^= b;                 // BAD "clever": no faster, and
                                        //   silently wrong when a and b
                                        //   are the same object
```

The XOR swap is a classic example of a trick that is slower on modern hardware (it creates a dependency chain the compiler cannot break), harder to read, and broken in a case the obvious version handles. It exists in folklore and nowhere else.

The same judgement applies to the genuinely useful tricks in this book — index marking, the $a + b \times n$ encoding, the interleaved copy. Each of those buys a real asymptotic improvement, which is what justifies the obscurity. **Earn the cleverness**: if it does not change the complexity, write the plain version.

9. Make every example compilable

Every program in this book compiles with `g++ -std=c++17` and produces exactly the output shown — that is checked mechanically, by a script that compiles each block and diffs its real output against the book's text.

What a complete example needs

```
#include <iostream>      // for std::cout
#include <vector>        // for std::vector
#include <algorithm>     // for std::sort, std::max, std::min, std::swap
#include <unordered_map> // for std::unordered_map
#include <string>        // for std::string
```

Two habits worth keeping:

Include what you use. Relying on one header to pull in another works until it does not, and the failure appears on a different compiler.

Do not write `using namespace std;`. It is fine in a five-line exercise and a genuine problem in anything larger — it drags in hundreds of names, and `std::count`, `std::distance` and `std::size` collide with ordinary variable names. Writing `std::` costs five characters and never surprises anyone. (In an interview, do whatever the interviewer's template does.)

10. The pre-submission checklist

CHECKLIST | Before you say "done"

1. Does it compile? Mentally walk the syntax: semicolons, braces, return statements on every path.
2. Does the empty input work?
3. Does a single-element input work?
4. Are all loop bounds right? Check every `<` versus `<=` against the last valid index.
5. Is every pointer checked before it is dereferenced?
6. Is every `new` matched by a `delete`?
7. Could any `int` overflow? Sums, products, `mid`, and `max - min`.
8. Is `size()` compared against a signed value anywhere?
9. If you modified the input, did you say so?
10. Can you state the time and space complexity without hesitating?

Run it in your head in under a minute. Items 2, 4 and 10 catch most real problems; item 6 is the one that matters in Part VI.

CHAPTER RECAP | Chapter 35 in six lines

- The signature is documentation: `const&` to read, `&` to modify, by value for small types, `*` for optional.
- `insert` / `erase` at the front, and vectors passed by value, are the hidden $O(n)$s.
- Keep one `Node` definition and one set of names throughout.
- Unlink before freeing; save `next` before deleting; null the pointer afterwards; check with the sanitizers.
- Prefer edge cases that fall out of the structure over edge cases you special-case — and never over ones you ignore.
- Earn your cleverness: if it does not improve the complexity, write the plain version.

EXERCISES | **Practise**

1. For each function you have written while reading this book, check the parameter against the rule in section 1. How many should change?
2. Compile three of your earlier solutions with `-Wall -Wextra -fsanitize=address,undefined`. What does it report?
3. Rewrite a function you wrote with vague names using the conventions in section 5. Does it need fewer comments?
4. Find a place in your own code where an edge case is handled implicitly. Add the one-line comment that says why it works.
5. Write the XOR swap and test it with `swap(a, a)`. What happens, and why?
6. Take any solution of yours and run the ten-item checklist. Time yourself.
7. Why is `using namespace std;` acceptable in a small exercise and not in a library header? Name a specific collision it could cause.

PART IX

Practice, Revision, Mastery

A five-level roadmap, quizzes and bug hunts after every chapter, a full answer key that explains rather than asserts, a mock interview, and a thirty-day plan to tie it together.

36 The Five-Level Practice Roadmap

37 Quizzes and Exercises

38 The Complete Answer Key

39 The Mock Interview

40 Reference: Plan, Checklist, Glossary

CHAPTER

36

The Five-Level Practice Roadmap

What to solve, in what order, and how to know you are ready

PRACTICE PREREQS: none — use this from day one

Knowing which problems to solve, in which order, is worth more than any individual solution. This roadmap orders every problem in the book into five levels, states the prerequisite skills for each, gives a hint before each solution, and tells you the signal that you are ready to move on. Work it in order; the levels are built so that nothing in one depends on anything in a later one.

WHAT YOU WILL LEARN

- Follow a problem order where nothing depends on material you have not met
- Know the prerequisite skills for each level before starting it
- Use the hints to get unstuck without reading a solution
- Identify the problems worth revisiting, and when
- Recognise the signal that you are ready for the next level

How to use this

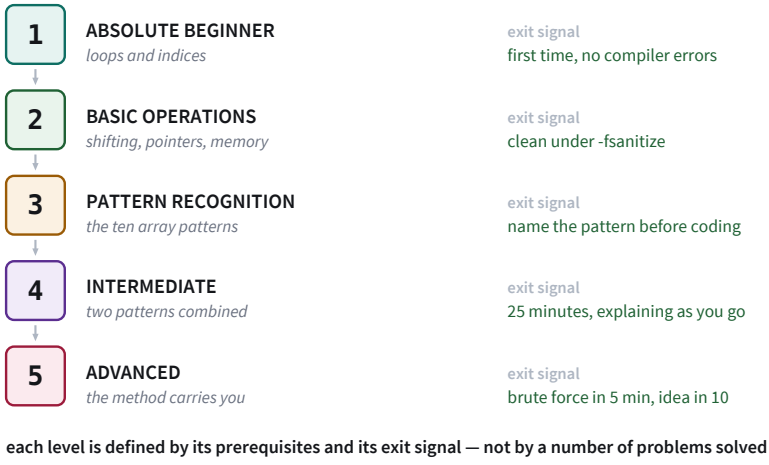


Figure 36.1 The five levels. Each is defined by what you must already be able to do, and each has an exit signal — a thing you can check — rather than a number of problems solved.

CHECKLIST | The rules of the roadmap

1. **Do them in order.** Nothing at a level needs material from a later one.
2. **Attempt before reading.** Give every problem 20 minutes of honest effort before looking at the hint, and 40 before the solution.
3. **Use the hint, not the solution.** The hint is one sentence; it restarts your thinking without finishing it for you.
4. **Re-solve, do not re-read.** A problem you read a solution for is not solved. Mark it and come back in three days with a blank page.
5. **Move on at the signal, not at the count.** The exit signal for each level is stated. Solving thirty problems badly is worse than solving ten properly.

Level 1 — Absolute beginner

Prerequisites: you can write a `for` loop and a function in C++.

Read first: Chapters 1–4.

Goal: be comfortable walking an array and reasoning about indices.

#	Problem	Chapter	Hint
1	Print an array forwards and backwards	4	the backwards loop starts at `n-1` and ends at `>= 0`
2	Sum and average	6	accumulate in `long long`; cast before dividing
3	Largest element	8	start `best` from `v[0]`, never from 0
4	Count occurrences of a value	6	one pass, one counter
5	Check whether an array is sorted	8	start the loop at `i = 1`, compare with `i-1`
6	Reverse an array in place	6	two pointers, `left < right`
7	Find the index of a value, or −1	4	−1 works as "absent" because 0 is a valid index
8	Second-largest distinct value	8	demote before you promote

Table 36.1 Level 1. None of these needs a pattern — they need correct loop bounds and attention to edge cases.

Exit signal. You can write any of these, first time, without a compiler error, and you test the empty array without being told to.

Level 2 — Basic operations

Prerequisites: Level 1, plus Chapters 5–7 and 21–22.

Goal: operations that change a structure, and the memory discipline that goes with them.

#	Problem	Chapter	Hint
9	Insert into an array at a position	6	copy **backwards**, from the far end
10	Delete from an array at a position	6	copy **forwards**, towards the hole
11	Remove duplicates from a sorted array	6	a write pointer, comparing with the last kept
12	Move zeroes to the end	6	compact the non-zeroes, then pad
13	Build a linked list from a vector	24	keep a `tail` pointer or it is $O(n^2)$
14	Print and count a linked list	24	walk with `curr`; `head` never moves
15	Insert at the front and back of a list	25	front needs `Node*&`; back needs `curr->next != nullptr`
16	Delete the first and last node	26	the last needs the **second-to-last** node
17	Free a whole list with no leaks	22, 24	save `next`, advance, **then** delete
18	Transpose a square matrix	7	inner loop starts at `c = r + 1`

Table 36.2 Level 2. The array half is about shifting; the list half is about pointers and about not leaking.

Exit signal. Every program you write in this level passes `-fsanitize=address,undefined` with no report.

Level 3 — Pattern recognition

Prerequisites: Level 2, plus Chapters 8–17.

Goal: given a problem, name the pattern before writing any code.

#	Problem	Pattern	Hint
19	Two Sum (sorted)	two pointers	the sum is too small → raise the small end
20	Two Sum (unsorted)	hash map	look up `target - x` before inserting `x`
21	Maximum consecutive ones	linear traversal	update `best` inside the run
22	Max sum of k consecutive	fixed window	add the incoming, subtract `v[i-k]`
23	Longest sub-string, no repeats	variable window	shrink while the **new** character repeats
24	Range-sum queries	prefix sums	`prefix[j+1] - prefix[i]`; keep the leading zero
25	Binary search	binary search	`mid = lo + (hi-lo)/2`; test on n=2
26	First and last occurrence	binary search	on a match, record and keep going
27	Rotate an array by k	three reversals	`k %= n` first

#	Problem	Pattern	Hint
28	Contains duplicate	hash set	`insert().second` checks and inserts at once
29	Valid anagram	count array	add for one string, subtract for the other
30	Maximum sub-array sum	Kadane	extend or restart; initialise from `v[0]`
31	Buy and sell stock	running minimum	the only thing the past contributes is the cheapest price
32	Smallest absolute difference	sort first	after sorting, only neighbours can be closest
33	Spiral matrix	four boundaries	the two guards are for single rows and columns
34	Search a sorted matrix	staircase	start at the top-right corner

Table 36.3 Level 3. For each one, write down the pattern **before** you start. That habit is the whole point of this level.

Exit signal. Shown a problem you have not seen, you can state the pattern and the complexity before writing a line — and be right more often than not.

Level 4 — Intermediate interview problems

Prerequisites: Level 3, plus Chapters 23–31.

Goal: problems that combine two patterns, or need one specific insight.

#	Problem	Chapter	Hint
35	Missing number	18	sum, or XOR the indices with the values
36	Single number	18	`x ^ x == 0`, and XOR is commutative
37	Majority element	19	pair off differing votes; a majority cannot be cancelled away
38	Sort 0s, 1s and 2s	19	three pointers; after swapping with `high`, do **not** advance `mid`
39	Merge sorted arrays in place	19	fill from the **back**, where the spare slots are
40	Product except self	19	prefix product × suffix product, no division
41	Sub-array sum equals K	11	`prefix[i] = running - K` is a hash lookup; seed with `{0:1}`
42	Longest sub-array with sum K	19	store the **earliest** index of each prefix value
43	Find the duplicate number	19	read the array as a linked list and use Floyd
44	Reverse a linked list	28	save `next` first; finish with `head = prev`
45	Middle of a list	29	fast moves two, slow moves one

#	Problem	Chapter	Hint
46	Detect a cycle	29	the gap shrinks by exactly one per step
47	Cycle II — find the entrance	29	$F=nL-a$; both pointers move one step in phase 2
48	Merge two sorted lists	29	a dummy node removes the first-node case
49	Remove nth from end	30	gap of $n+1$ from the dummy, so `trail` lands on the predecessor
50	Palindrome linked list	31	middle + reverse + compare, then restore
51	Intersection of two lists	31	each pointer walks $a+b+c$
52	Add two numbers	31	one loop, running while either list has digits or a carry remains
53	Reorder list	31	split, reverse, interleave

Table 36.4 Level 4. The right-hand column is the insight, not the algorithm — read it only after twenty minutes.

Exit signal. You can solve any of these in 25 minutes, explaining as you go, and you volunteer the complexity without being asked.

Level 5 — Advanced interview problems

Prerequisites: Level 4, plus Chapters 20, 32 and 34.

Goal: problems where the insight is not guessable, so the method has to carry you.

#	Problem	Chapter	Hint
54	Search in a rotated sorted array	20	one half is always properly sorted — work out which
55	Minimum of a rotated sorted array	20	compare the middle with the right end
56	Trapping rain water	20	solve one column first; then two pointers, moving the shorter wall
57	Maximum product sub-array	16	carry both the max and the min; a negative swaps them
58	Three Sum	15	sort, fix one, two-pointer the rest; skip duplicates
59	Container with most water	9	move the shorter wall — it is the only move that can help
60	First missing positive	13	the answer is in $1..n+1$; put value x into slot $x-1$
61	Find all duplicates in $1..n$	13	use the sign at index $\lvert v\rvert-1$ as a visited mark
62	Longest consecutive sequence	14	only start counting from values with no predecessor in the set
63	Reverse nodes in k-groups	32	check that k remain **before** reversing
64	Rotate list	32	close the ring, then cut elsewhere

#	Problem	Chapter	Hint
65	Sort list	32	merge sort; split at the **first** middle
66	Copy list with random pointer	32	interleave each copy after its original
67	Binary search on the answer	12	“minimum X such that…” with a monotonic test
68	Largest rectangle in a histogram	20	for each bar, how far can it extend before a shorter one?

Table 36.5 Level 5. Apply the seventeen-step method of Chapter 34 to every one of these, in writing.

Exit signal. Given an unseen problem of this difficulty, you can reach a correct brute force in five minutes and name a plausible improvement in ten — even when you do not finish.

The problems worth revisiting

Not all problems repay repetition equally. These twelve do, because each contains an idea that reappears everywhere.

Problem	The idea it contains
Reverse a linked list	pointer manipulation, in its purest form
Binary search	the invariant, and the off-by-one
Two Sum (both versions)	hashing versus two pointers, and the trade-off
Kadane’s algorithm	“best ending here” — the seed of dynamic programming
Sliding window (variable)	grow, shrink, record — and the amortised argument
Prefix sums + hash map	precompute once, answer in $O(1)$
Merge two sorted lists	relinking, and the dummy node
Floyd’s cycle detection	a non-obvious $O(1)$-space argument you can prove
Remove nth from end	the gap, and why it is $n{+}1$
Trapping rain water	solve one position, then optimise
Dutch national flag	three-way partitioning, and an asymmetric invariant
Merge sort on a list	divide and conquer with no auxiliary array

Table 36.6 The twelve to re-solve from blank paper every few weeks. Between them they contain almost every idea in the book.

KEY INSIGHT | **The difference between solving and knowing**

Solving a problem once means you found the idea with the problem in front of you. Knowing it means you can reconstruct it from nothing.

The test is simple and uncomfortable: **a blank page and no notes.** Set a timer for fifteen minutes, write the full solution, then compile it. Anything you cannot do that way, you have read rather than learned.

Three days is roughly the right interval for the first repeat, then a week, then a month. That spacing is doing most of the work.

CHAPTER RECAP | **Chapter 36 in five lines**

- Five levels, each defined by its prerequisites and its exit signal rather than a problem count.
- Attempt for twenty minutes, then a hint, then forty minutes, then the solution.
- At Level 3 the habit to build is **naming the pattern before coding**.
- At Level 5 the habit is **applying the method in writing**.
- Twelve problems repay re-solving from blank paper; the rest repay solving once properly.

CHAPTER

37

Quizzes and Exercises

Multiple choice, trace the code, find the bug, write the code

PRACTICE PREREQS: the chapters each set covers

Five kinds of exercise, because they test different things. Multiple choice checks recall; tracing checks whether you can simulate code rather than recognise it; bug-hunting checks whether you know where bugs live; writing checks production; and complexity analysis checks the skill interviews ask about most directly. All answers are in Chapter 38.

WHAT YOU WILL LEARN

- Check your recall of every core fact in the book
- Simulate unfamiliar code by hand without running it
- Find bugs by reading, in code that compiles and runs
- Write correct code from a specification, including the edge cases
- Determine complexity from code you have not seen

How to use this chapter

CHECKLIST | Rules that make these worth doing

1. **No compiler** for sets A, B and C. The point is to simulate, not to observe.
2. **Write the trace down.** A trace held in your head is a guess.
3. **Predict before running** in sets D and E. Then run, and compare.
4. **Mark what you got wrong** and re-do only those, three days later.
5. The answers are in Chapter 38, with explanations. Read the explanation even when you were right — several of them say something the question did not.

Set A — Multiple choice

A1. An array of `double` starts at address 2000. At what address does element 4 begin? **A** 2004 **B** 2016 **C** 2032 **D** it cannot be determined

A2. `std::vector<int> v;` then ten `push_back` calls. On libstdc++, what is `v.capacity()`? **A** 10 **B** 12 **C** 16 **D** 20

A3. Which is **not** a reason array indexing is $O(1)$? **A** elements are the same size **B** elements are contiguous **C** the array knows its own length **D** the base address is known

A4. `v.insert(v.begin(), x)` inside a loop running n times is: **A** $O(n)$ **B** $O(n \log n)$ **C** $O(n^2)$ **D** amortised $O(n)$

A5. Binary search on a `std::list` performs $O(\log n)$ comparisons. Its total time is: **A** $O(\log n)$ **B** $O(n)$ **C** $O(n \log n)$ **D** $O(1)$

A6. In a singly linked list, deleting a node you hold a pointer to is: **A** $O(1)$ **B** $O(n)$ **C** $O(\log n)$ **D** impossible

A7. What does `delete p;` do to the variable `p`? **A** sets it to `nullptr` **B** sets it to 0 **C** leaves it unchanged **D** undefined

A8. A sliding window with a `while` loop inside a `for` loop is: **A** always $O(n^2)$ **B** $O(n)$, because `left` advances at most n times in total **C** $O(n \log n)$ **D** it depends on the data

A9. Kadane's algorithm initialised with `best = 0` fails on: **A** an empty array **B** an all-negative array **C** a sorted array **D** an array with one element

A10. `std::unordered_map` lookup is: **A** $O(1)$ always **B** $O(1)$ average, $O(n)$ worst **C** $O(\log n)$ **D** $O(n)$

A11. Reversing a linked list and forgetting `head = prev;` produces: **A** a crash **B** an infinite loop **C** a one-element list **D** the original list

A12. In Floyd's cycle detection, phase 2 moves: **A** both pointers one step **B** slow one, fast two **C** both two steps **D** only the slow pointer

A13. A dummy node is placed on the stack rather than the heap because: **A** it is faster **B** it cannot then be deleted by mistake, and needs no cleanup **C** heap nodes cannot be dummies **D** it uses less memory

A14. Which traversal of a 2-D array is faster, and why? **A** column-major, fewer cache lines **B** row-major, sequential memory **C** identical, same complexity **D** depends on the compiler

A15. The three-reversal rotation works because: **A** reversal is its own inverse **B** $(AB)^{\mathrm{R}} = B^{\mathrm{R}} A^{\mathrm{R}}$ **C** $k \bmod n = k$ **D** reversal is $O(n)$

Set B — Trace the code

Write down every variable after every iteration. No compiler.

B1. What is printed?

B1

```
std::vector<int> v = {4, 2, 7, 1};
int best = v[0], second = -1000;
for (int i = 1; i < 4; ++i) {
    if (v[i] > best) { second = best; best = v[i]; }
    else if (v[i] > second) second = v[i];
}
```

```
std::cout << best << " " << second;
```

B2. What is the final content of `v`?

B2

```
std::vector<int> v = {0, 1, 0, 3, 0};
int w = 0;
for (int r = 0; r < 5; ++r)
    if (v[r] != 0) v[w++] = v[r];
while (w < 5) v[w++] = 0;
```

B3. How many iterations, and what is returned?

B3

```
std::vector<int> v = {1, 3, 5, 7, 9, 11};
int lo = 0, hi = 5, target = 9, steps = 0;
while (lo <= hi) {
    ++steps;
    int mid = lo + (hi - lo) / 2;
    if (v[mid] == target) break;
    if (v[mid] < target) lo = mid + 1; else hi = mid - 1;
}
std::cout << steps;
```

B4. What is `best` at the end?

B4

```
std::vector<int> v = {-2, 3, -1, 4, -5};
long long endingHere = v[0], best = v[0];
for (int i = 1; i < 5; ++i) {
    endingHere = std::max((long long)v[i], endingHere + v[i]);
    best = std::max(best, endingHere);
}
```

B5. Trace `prev`, `curr` and `nextNode` for the list `1 -> 2 -> 3`. What does the list look like after each iteration?

B5

```
Node* prev = nullptr;
Node* curr = head;
while (curr) {
    Node* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}
```

B6. For `{2, 1, 5, 1, 3, 2}` with target 7, what is returned?

B6

```
long long sum = 0;
int left = 0, best = 0;
for (int right = 0; right < 6; ++right) {
    sum += v[right];
    while (sum > 7 && left <= right) { sum -= v[left]; ++left; }
    best = std::max(best, right - left + 1);
}
```

B7. What does this print for the list `1 -> 2 -> 3 -> 4 -> 5`?

B7

```
Node* slow = head;
Node* fast = head->next;
while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
std::cout << slow->data;
```

Set C — Find the bug

Every one of these compiles and runs. Each contains exactly one bug.

C1.

C1

```
int sumAll(const std::vector<int>& v) {
    int total = 0;
    for (int i = 0; i <= (int)v.size(); ++i) total += v[i];
    return total;
}
```

C2.

C2

```
int largest(const std::vector<int>& v) {
    int best = 0;
    for (int x : v) if (x > best) best = x;
    return best;
}
```

C3.

C3

```
void reverse(std::vector<int>& v) {
    for (int i = 0; i < (int)v.size(); ++i)
        std::swap(v[i], v[v.size() - 1 - i]);
}
```

C4.

C4

```cpp
int binarySearch(const std::vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == target) return mid;
        if (v[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
```

C5.

C5

```cpp
void deleteFront(Node*& head) {
    if (!head) return;
    delete head;
    head = head->next;
}
```

C6.

C6

```cpp
void insertAfter(Node* node, int value) {
    Node* fresh = new Node(value);
    node->next = fresh;
    fresh->next = node->next;
}
```

C7.

C7

```cpp
int maxConsecutiveOnes(const std::vector<int>& v) {
    int current = 0, best = 0;
    for (int x : v) {
        if (x == 1) ++current;
        else { best = std::max(best, current); current = 0; }
    }
    return best;
}
```

C8.

C8

```cpp
void transpose(std::vector<std::vector<int>>& g) {
    int n = (int)g.size();
    for (int r = 0; r < n; ++r)
        for (int c = 0; c < n; ++c)
            std::swap(g[r][c], g[c][r]);
}
```

C9.

C9

```
std::pair<int,int> twoSum(const std::vector<int>& v, int target) {
    std::unordered_map<int,int> seen;
    for (int i = 0; i < (int)v.size(); ++i) {
        seen[v[i]] = i;
        auto it = seen.find(target - v[i]);
        if (it != seen.end()) return {it->second, i};
    }
    return {-1, -1};
}
```

C10.

C10

```
void rotateRight(std::vector<int>& v, int k) {
    int n = (int)v.size();
    std::reverse(v.begin(), v.end());
    std::reverse(v.begin(), v.begin() + k);
    std::reverse(v.begin() + k, v.end());
}
```

Set D — Write the code

Specifications only. Write the function, handle the edge cases, state the complexity.

D1. `int countPairs(const std::vector<int>& v, int target)` — the number of index pairs (i, j) with $i<j$ and `v[i] + v[j] == target`. $O(n)$ expected.

D2. `bool isRotatedSorted(const std::vector<int>& v)` — whether the array is a sorted array rotated some number of positions. `{3,4,5,1,2}` is; `{3,1,4}` is not. $O(n)$.

D3. `int longestEqualRun(const std::vector<int>& v)` — the length of the longest run of equal neighbouring values. $O(n)$, $O(1)$ space.

D4. `std::vector<int> mergeIntervals(std::vector<std::pair<int,int>> in)` — merge overlapping intervals. State what you sort by, and why.

D5. `Node* removeDuplicatesUnsorted(Node* head)` — remove duplicate values from an **unsorted** list, keeping the first occurrence. State the two approaches and their costs.

D6. `bool isCircular(Node* head)` — whether a list is circular (the last node points at the head). Not merely whether it has a cycle. $O(n)$, $O(1)$ space.

D7. `Node* swapPairs(Node* head)` — swap every two adjacent nodes: `1->2->3->4` becomes `2->1->4->3`. Swap the **nodes**, not the values.

D8. `int maxSubarraySumCircular(const std::vector<int>& v)` — maximum sub-array sum where the sub-array may wrap around. State the edge case that catches people.

Set E — Complexity analysis

For each, state the time complexity, the auxiliary space complexity, and what n is.

E1.

E1

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; j += 2)
        work();
```

E2.

E2

```
for (int i = 1; i < n; i *= 3)
    for (int j = 0; j < i; ++j)
        work();
```

E3.

E3

```
int f(std::vector<int> v) {          // note: by value
    std::sort(v.begin(), v.end());
    return v.empty() ? 0 : v[0];
}
```

E4.

E4

```
int g(const std::vector<int>& v) {
    int c = 0;
    for (int x : v)
        if (std::binary_search(v.begin(), v.end(), x * 2)) ++c;
    return c;
}
```

E5.

E5

```
void h(Node* head) {
    for (Node* a = head; a; a = a->next)
        for (Node* b = head; b; b = b->next)
            if (a->data == b->data) std::cout << a->data;
}
```

E6.

E6

```
int rec(int n) {
```

```
    if (n <= 1) return 1;
    return rec(n - 1) + rec(n - 1);
}
```

E7.

E7

```
std::vector<int> k(const std::vector<int>& v) {
    std::vector<int> out;
    for (int x : v) out.insert(out.begin(), x);
    return out;
}
```

E8.

E8

```
std::unordered_map<int,int> freq;
for (int x : v) ++freq[x];
for (const auto& [key, count] : freq)
    std::cout << key << count;
```

KEY INSIGHT | What each set is really testing
A — whether the facts are available instantly. In an interview you will not have time to derive that `erase` at the front is $O(n)$.
B — whether you can *simulate* code. Most debugging is simulation, and most people try to pattern-match instead.
C — whether you know where bugs live. Every bug in set C is one this book warned about, which is the point: the warnings are a map.
D — whether you can produce, not just recognise. This is the only set that measures what an interview measures.
E — the single skill asked about most directly, and the easiest to practise.

Answers, with explanations, are in Chapter 38.

CHAPTER

38

The Complete Answer Key

Every exercise, answered and explained

REFERENCE PREREQS: attempt the exercise first

Answers to Chapter 37's five sets and to the exercises at the end of every chapter. Each answer explains why, not merely what — several say something the question did not ask, because the interesting part of an exercise is usually just past its edge. Read the explanation even when you were right.

WHAT YOU WILL LEARN

- Check your work against a worked answer
- Understand the reasoning behind each answer, not just the result
- Find the chapter to revisit when an answer surprises you

WATCH OUT | Read this only after attempting
An answer you read is worth perhaps a tenth of an answer you found. If you have not spent twenty minutes on the exercise, go back; the hint in Chapter 36 will usually restart you without finishing the job.

Chapter 37, Set A — Multiple choice

A1 — C. 2032. A `double` is 8 bytes, so element 4 is at $2000+4\times8=2032$. The formula $B+i\times S$ needs only the base and the element size, which is why **D** is wrong.

A2 — C. 16. Capacity doubles at 1, 2, 4, 8, 16. After ten pushes the last doubling (from 8) has already happened, so the capacity is 16 with `size() == 10`.

A3 — C. Knowing the length is useful for bounds checking but plays no part in the address calculation. The other three are exactly the preconditions of $B+i\times S$.

A4 — C. $O(n^2)$. Each front insertion shifts every existing element, so the total is $1+2+\cdots+n=n(n+1)/2$.

A5 — B. $O(n)$. The comparisons are logarithmic but reaching the middle of a list requires walking there. This is the cleanest illustration that binary search is an *array* algorithm.

A6 — B. $O(n)$. You need the node's predecessor to unlink it, and finding it means walking from the head. In a **doubly** linked list it would be $O(1)$, which is the main reason that variant exists.

A7 — C. leaves it unchanged. `delete` destroys the object and frees the memory; the pointer still holds the old address. That is why `p = nullptr;` afterwards is a habit worth having.

A8 — B. `left` only ever increases and never exceeds n, so the inner loop's body executes at most n times **across the whole run**, not per iteration. The same accounting as amortised analysis.

A9 — B. an all-negative array. With `best = 0` the answer for `{-3, -1, -7}` comes out as 0 — the sum of the empty sub-array. Correct only if the empty sub-array is allowed, which is why you should ask.

A10 — B. Average $O(1)$; worst case $O(n)$ when every key collides. The constant factor also matters: a hash plus an indirection is far slower than a count array's single address calculation.

A11 — C. a one-element list. Every `next` is correctly reversed, but `head` still names the original first node — which is now the tail, with `next == nullptr`. Printing from it gives one element.

A12 — A. both pointers one step. Keeping `fast` at double speed is the classic error and finds the wrong node. The proof $F = nL - a$ requires equal speeds.

A13 — B. A stack dummy is destroyed automatically, so it cannot leak, and `delete &dummy` would be undefined behaviour — which the loop's structure naturally never attempts.

A14 — B. row-major. Row-major reads consecutive addresses, so one 64-byte cache line supplies 16 `int`s and the prefetcher keeps ahead. Column-major jumps a whole row each step. Same complexity, commonly 3–6× the time.

A15 — B. $(AB)^{\mathrm{R}} = B^{\mathrm{R}} A^{\mathrm{R}}$. Reversing a concatenation reverses the order of the parts as well as each part, which puts the two blocks in the right places (backwards); reversing each block individually then repairs them.

Chapter 37, Set B — Trace the code

B1 — prints `7 4`.

`i`	`v[i]`	branch	`best`	`second`
start	—	—	4	−1000
1	2	middle	4	2
2	7	new best	7	4
3	1	neither	7	4

Note row 2: the old champion 4 is demoted into `second` **before** `best` changes. Reversing those two lines would give `7 7`.

B2 — `{1, 3, 0, 0, 0}`. The first pass compacts the two non-zeroes to indices 0 and 1, leaving `w = 2`; the second pass fills indices 2, 3 and 4 with zeroes. The relative order of 1

and 3 is preserved, which is the requirement.

B3 — 2 iterations. Iteration 1: `mid = 2`, `v[2] = 5 < 9`, so `lo = 3`. Iteration 2: `mid = 4`, `v[4] = 9`, found. Linear search would have taken five comparisons.

B4 — `best = 6`.

`i`	`v[i]`	`endingHere`	`best`
0	−2	−2	−2
1	3	3 (restart)	3
2	−1	2 (extend)	3
3	4	6 (extend)	6
4	−5	1 (extend)	6

The best sub-array is `{3, -1, 4}`. Row 1 restarts because the running sum −2 was negative.

B5 — the list becomes `3 -> 2 -> 1`.

iteration	`prev`	`curr`	`nextNode`	links after
start	`nullptr`	1	—	`1->2->3->∅`
1	1	2	2	`1->∅`, `2->3->∅`
2	2	3	3	`2->1->∅`, `3->∅`
3	3	`nullptr`	`nullptr`	`3->2->1->∅`

The caller must then set `head = prev`.

B6 — returns 3. The longest window with sum at most 7 is `5 1 1` … no: trace carefully. At `right = 3` the window is `1 5 1` with sum 7, length 3. At `right = 4` the sum reaches 10 and two elements are dropped, leaving length 2. At `right = 5` the window is `1 3 2` with sum 6, length 3. The answer is 3.

B7 — prints 3. With `fast` starting at `head->next`, `slow` lands on the **first** middle. For an odd length both middles coincide, so the answer is the same as starting `fast` at `head` — the difference only shows on even lengths.

Chapter 37, Set C — Find the bug

C1 — `i <= v.size()` reads one past the end. The last valid index is `size() - 1`. It usually adds a garbage value rather than crashing, which is worse. Fix: `i < (int)v.size()`.

C2 — `best` starts at 0. On `{-5, -2, -9}` the function returns 0, a value not in the array. Fix: start from `v[0]` after checking the array is non-empty.

C3 — every pair is swapped twice. The loop runs to `size()`, so after swapping $(0, 3)$ it later swaps $(3, 0)$ back. The array is returned unchanged. Fix: loop to `size() / 2`, or use

converging pointers with `left < right`.

C4 — `while (lo < hi)` misses the one-element region. When `lo == hi` there is still one unexamined element. Searching a one-element array for its own value returns −1. Fix: `lo <= hi`.

C5 — use after free. `head->next` is read after `head` has been deleted. Fix: save it first — `Node* d = head; head = head->next; delete d;`.

C6 — the order is reversed, so `fresh->next` points at `fresh`. After `node->next = fresh`, the expression `node->next` *is* `fresh`. The result is a self-referential node, and every later traversal hangs. Fix: set `fresh->next` first.

C7 — `best` is only updated when a run ends. On `{1, 1, 1}` the run never ends, so the function returns 0. Fix: update `best` inside the `if (x == 1)` branch, which also removes the need for a correction after the loop.

C8 — the whole matrix is transposed twice. Swapping (r, c) and then later (c, r) restores the original. Fix: start the inner loop at `c = r + 1` so each pair is visited once.

C9 — it inserts before looking up. With `v = {3, 2, 4}` and target 6, at `i = 0` the map already holds `3 → 0`; the complement of 3 is 3, and it returns $(0, 0)$ — one element used twice. Fix: look up first, then insert.

C10 — `k` is not reduced modulo `n`. If `k >= n`, `v.begin() + k` is past the end and the behaviour is undefined. Fix: `k %= n;` first, and handle `k == 0`.

Chapter 37, Set D — Write the code

D1 — count pairs summing to a target. Walk once with a frequency map; for each `x`, add `freq[target - x]` to the count *before* inserting `x`. That counts each pair exactly once and handles duplicates correctly. $O(n)$ expected time, $O(n)$ space. The sorted two-pointer alternative is $O(n \log n)$ with $O(1)$ space, and needs care to count duplicate pairs.

D2 — is it a rotated sorted array? Count the "descents" — positions where `v[i] > v[i+1]` — and also compare the last element with the first. A rotated sorted array has **at most one** descent, and if it has exactly one then `v.back() <= v.front()` must hold. Zero descents means it is already sorted, which counts as a rotation by 0. $O(n)$, $O(1)$.

D3 — longest run of equal values. The current-run/best-run shape from Chapter 8: reset `current` to 1 when `v[i] != v[i-1]`, otherwise increment, and update `best` every step. An empty array returns 0; one element returns 1. $O(n)$, $O(1)$.

D4 — merge intervals. Sort by **start**. Then walk, keeping the current merged interval; if the next interval's start is at most the current end, extend the end to the maximum of the two, otherwise emit and begin a new one. Sorting by start is what guarantees that anything overlapping the current interval comes next. $O(n \log n)$.

D5 — remove duplicates from an unsorted list. Either a hash set of seen values in one pass ($O(n)$ time, $O(n)$ space), or a nested loop removing later duplicates of each node ($O(n^2)$ time, $O(1)$ space). State both and the trade-off; which is right depends on whether extra space is allowed.

D6 — is the list circular? Floyd's detection finds *a* cycle; this question is narrower. Walk from `head->next` and check whether you return to `head`; if you reach `nullptr` first, it is not circular. A list with a cycle that does not include the head is **not** circular by this definition, and your code should reflect that. $O(n)$, $O(1)$.

D7 — swap pairs. Use a dummy node, since the head changes. Each step rewires three pointers: `prev->next = second`, `first->next = second->next`, `second->next = first`; then advance `prev` to `first`. Stop when fewer than two nodes remain. $O(n)$, $O(1)$.

D8 — maximum circular sub-array sum. The answer is either a normal Kadane result, or `total - minSubarraySum` (the wrapping case, where the elements left out form a contiguous minimum). **The edge case:** if every element is negative, `total - min` is 0, which corresponds to taking nothing — so return the plain Kadane answer in that case. Detect it by checking whether the maximum is negative.

Chapter 37, Set E — Complexity analysis

E1 — $O(n^2)$ time, $O(1)$ space. The inner loop runs $n/2$ times, so the total is $n^2/2$ — and the constant $1/2$ is dropped. A common mistake is to answer $O(n)$ because of the `j += 2`.

E2 — $O(n)$ time, $O(1)$ space. The outer loop takes $i = 1, 3, 9, \ldots$, and the inner runs i times, so the total is $1+3+9+\cdots+n$. That geometric series sums to about $3n/2$. Deeply nested is not the same as quadratic.

E3 — $O(n \log n)$ time, $O(n)$ space. The parameter is taken **by value**, so the whole vector is copied on entry: that is the $O(n)$ space, and it is easy to miss.

E4 — $O(n \log n)$ time, $O(1)$ space. Also note a correctness problem: `binary_search` requires a sorted range, and nothing here guarantees one. The complexity is unaffected; the answer is meaningless. Spotting that is worth more than the complexity.

E5 — $O(n^2)$ time, $O(1)$ space, where n is the number of nodes. Both loops traverse the whole list.

E6 — $O(2^n)$ time, $O(n)$ space. Each call makes two calls on $n-1$, so the call count doubles with each increment of n. The space is the recursion depth, which is n, not 2^n — only one root-to-leaf path exists at a time.

E7 — $O(n^2)$ time, $O(n)$ space. `insert` at the beginning shifts everything already placed. This whole function is `std::reverse` written quadratically; building forwards and reversing once is $O(n)$.

E8 — $O(n)$ time, $O(n)$ space. Building the map is n insertions at average $O(1)$; iterating it visits at most n distinct keys.

Part I — Chapters 1 to 3

Chapter 1 — What a data structure really is

1. **Array.** "Skip to track 47" is access by index, which is $O(1)$ for an array and $O(n)$ for a list; reordering is rare, so the list's advantage never gets used.

2. **Linked list**, in principle — insertion in the middle is cheap. In practice real editors use a *gap buffer* or a *rope*, because a list of individual characters wastes enormous memory and destroys cache performance. Both halves of that answer are worth giving.
3. Because every value takes the same amount of room and they are packed with no gaps, you can work out where the 7th one begins by multiplying — you never have to look at the first six.
4. **Insertion at the front.** Everything already there must physically move one place along, because the values are required to stay packed together with no gaps.
5. The array gives instant access to a *bucket*; the lists let each bucket hold several items that happen to collide, growing and shrinking without disturbing the other buckets. Neither property alone is enough.

Chapter 2 — Memory and addresses

1. 2^{16}=65,536. Each of the 16 bits doubles the number of patterns.
2. 2000+6×8=2048. S=8 because a `double` is 8 bytes.
3. The **name** is a label you write in your source code and the compiler throws away. The **address** is which numbered byte the value starts at. The **value** is what is stored in those bytes.
4. The addresses change between runs (address space layout randomisation) but the *difference* between two locals in the same frame usually does not.
5. It reports the exact line of the out-of-bounds access, the size of the object and how far past it you went — none of which the plain build mentions at all.
6. Because an array is one object with a guaranteed layout, while three variables are three separate objects the compiler may place, reorder, pad or keep entirely in registers.

Chapter 3 — Big O

A $O(n)$ time, $O(1)$ space — two sequential loops add, they do not multiply. **B** $O(n \log n)$ — the outer loop runs $\log_2 n$ times, the inner n times. **C** $O(n)$ time and $O(n)$ space — the by-value parameter is one copy and `copy` is a second. **D** $O(n^3)$ — the triangular version of a triple loop is $n^3/6$, and the constant is dropped. **E** $O(n^2)$ — `insert` at the beginning shifts everything, n times.

1. $O(n^2)$. The $50n$ term is larger while $n^2/1000 < 50n$, that is $n < 50{,}000$. So for every input below fifty thousand the "dominant" term is not dominant at all — which is why Big O is a statement about scaling, not a prediction for a particular input.
2. $O(1)$, $O(\log n)$, $O(n)$ and $O(n \log n)$ are comfortable (10^6 and 2×10^7 operations). $O(n^2)$ is 10^{12} — about three hours. Not acceptable.

Part II — Chapters 4 to 7

Chapter 4 — The array

1. Last index 5; `sizeof(arr)` is 24 bytes.

2. 3000+5×8=3040.
3. `for (int i = n - 1; i >= 0; --i)`. The condition must be `>= 0`, not `> 0`, or element 0 is never printed.
4. Because the index says how far to move from the beginning, and the first element is where the beginning already is — you move nothing.
5. The sanitizer names the line, the array, its size and the offset read. The plain build prints a plausible-looking number and exits successfully.
6. `i <= n` reads `arr[n]`, one past the end.
7. Because the bytes immediately after the block were never reserved for it and are probably in use by something else. Growing means finding a bigger free block elsewhere and copying — which is what `std::vector` does.

Chapter 5 — Vectors and capacity

1. Seven reallocations (at 1, 2, 4, 8, 16, 32, 64), ending with capacity 128.
2. 1+2+4+8=15 copies for 16 pushes, and 15<32. The series is always under $2n$.
3. Most pushes are free because there is spare room. Occasionally the vector runs out and has to move everything, which is expensive — but because the capacity doubles, those moves get rarer exactly as fast as they get bigger, so the *average* cost per push stays constant.
4. It prints `0`, then reads `v[0]` on an empty vector — undefined behaviour. `reserve` sets capacity, not size.
5. It is building the reverse of `in`. Write `std::vector<int> out(in.rbegin(), in.rend());`, or push forwards and call `std::reverse`. Either is $O(n)$.
6. Yes. The series becomes $1+1.5+1.5^2+\cdots$, which is still geometric and still sums to a constant multiple of n. Any growth **factor** greater than 1 works; a growth **increment** does not.
7. Because you will probably refill it, and keeping the block avoids reallocating. It wastes memory when you clear a huge vector and never reuse it — then use the swap idiom: `std::vector<int>().swap(v);`.

Chapter 6 — Core array operations

1. Inserting at index 0 of `{1,2,3}`: `v[3]=v[2]`, `v[2]=v[1]`, `v[1]=v[0]`, `v[0]=5` — three moves. At index 3: zero moves.
2. Copying forwards from the insertion point gives `{1,2,2,2}` — every slot becomes a copy of the first one moved, because each read happens after the write that destroyed it.
3. Scan for the value, then shift everything after it left and `pop_back`. With two occurrences it removes only the first, and the function should say so in its name or its documentation.
4. Compare `v[r]` with `v[w - 2]`, and start `w` at `min(2, n)` so the first two elements are always kept.
5. Walk from the **right** with a write pointer starting at `n - 1`, copying non-zeroes; then fill the remaining front slots with zeroes.

6. The loop runs the full length, so each pair is swapped twice and the array is unchanged. `{1,2,3,4}` comes back as `{1,2,3,4}`.
7. $1000 \times 100{,}000 = 10^8$ operations. Build one frequency map in $O(n)$ and answer each query in $O(1)$: $O(n+q)$ total, at the cost of $O(k)$ space for the distinct values.
8. Deduplicate **first**. Moving zeroes first would scatter the duplicates away from each other, and the deduplication step depends on duplicates being adjacent.

Chapter 7 — 2-D arrays

1. Flat index $3 \times 8 + 6 = 30$; address $4000 + 30 \times 4 = 4120$.
2. $r = 30 / 7 = 4$, $c = 30 \bmod 7 = 2$.
3. Sum the first and last rows entirely, then the first and last columns excluding their first and last elements — otherwise the four corners are counted twice.
4. `c = r + 1` visits each unordered pair once. `c = r` would swap the diagonal element with itself, which is harmless but pointless. `c = 0` swaps every pair twice and leaves the matrix unchanged.
5. Compare `g[r][c]` with `g[c][r]` for `c > r` only — $n(n-1)/2$ comparisons.
6. Both are $\Theta(R \cdot C)$. The difference is a **constant factor** of roughly 3–6×, caused by cache behaviour, which Big O deliberately ignores.
7. Row r has $r+1$ entries; `tri[r][0] = tri[r][r] = 1`, and for $0 < c < r$, `tri[r][c] = tri[r-1][c-1] + tri[r-1][c]`.

Part III — Chapters 8 to 17

Chapter 8 — Linear traversal

1. Three variables (`best`, `second`, `third`), updated in strict descending order: shift `second` into `third`, then `best` into `second`, then set `best`. The order is the whole difficulty, exactly as with two.
2. Compare `v[i]` with both neighbours for `0 < i < n-1`. The two ends have only one neighbour each and so can never qualify — state that explicitly rather than letting it happen by accident.
3. Carry two booleans, `couldBeAscending` and `couldBeDescending`, and clear whichever a violation rules out. `{3,3,3}` satisfies both under non-strict comparison, so it is sorted in either direction.
4. Running state: `currentValue`, `currentLength`, `bestValue`, `bestLength`. Reset the current pair when the value changes.
5. A sliding window where the window is invalid once it contains two zeroes — Chapter 10. One pass is possible; it is not a pure linear traversal, which is why the exercise says to come back.
6. You cannot compare an element with the average until you have seen every element, and the last one can change the average. A **running total** can be compared against as you go, because it depends only on the past.
7. It needs to know about the future as well as the past, so one pass with a fixed notebook is not enough. Prefix maxima and suffix minima (Chapter 11) make it two

passes and $O(n)$.

Chapter 9 — Two pointers

1. `{1,3,4,6,8,11}`, target 10: (0,5)→12 too big; (0,4)→9 too small; (1,4)→11 too big; (1,3)→9 too small; (2,3)→10 found. Five iterations, replacing 15 brute-force pairs.
2. On a match, record it and move **both** pointers, then skip past any duplicates of the values just used — `while (left < right && v[left] == v[left-1]) ++left;` and the mirror on the right.
3. Advance `left` past non-letters and retreat `right` past non-letters before each comparison, then compare lowercased. Both skips must happen before the comparison, not after.
4. Sort, then for each `i` run two pointers on `i+1 .. n-1` looking for `-v[i]`. The outer loop is n and each inner scan is $O(n)$, so $O(n^2)$ — not $O(n^3)$, because the inner search is linear rather than quadratic. Skip duplicates at every level.
5. No such input exists. Moving the taller wall keeps the same (or a smaller) minimum height while reducing the width, so the area cannot increase.
6. Start at both ends of the input and fill the output from the **back**, because the largest square is always at one end or the other — the most negative or the most positive value.
7. Converging pointers that each skip forward to the next vowel, then swap.
8. Moving a pointer inwards is justified only by knowing that every pair using it is now impossible, and that conclusion depends on the remaining elements being bounded by the current one — which is exactly what sortedness provides and what unsorted data does not.

Chapter 10 — Sliding window

1. With target 9, row 2's sum of 8 is unchanged, but at `right = 3` the sum of 9 is no longer over the target, so nothing shrinks and the window `2 1 5 1` has length 4. The answer becomes 4.
2. The shape is the same; the shrink condition becomes "while the window is still **valid**", and you record the length *inside* the shrink loop rather than after it — because for a minimum you want the smallest valid window, not the largest.
3. Track the running sum and divide only once at the end, or compare sums directly since all windows have the same size. Dividing inside the loop costs a division per step and introduces floating-point error.
4. The window state is the number of zeroes it contains; it is invalid when that count exceeds 1. Shrink from the left until it is 1 again.
5. Change the constant 1 to k. Nothing else.
6. The state is a map from character to count; the window is invalid when the map has more than two entries. Erase a key when its count reaches zero, or the size test is wrong.
7. `{5, -3, 4}` with target 5: the window `5 -3 4` sums to 6 and is invalid, but shrinking from the left gives `-3 4` = 1, which is valid — so the sum did not decrease monotonically as the window shrank, and the shrink rule's assumption fails.

8. A fixed window of `s1.size()`, with two 26-entry count arrays compared each step. Comparing two 26-entry arrays is $O(1)$, so the whole thing is $O(n)$; maintaining a single "number of matching letters" counter removes even that.

Chapter 11 — Prefix sums

1. `{0, 5, 3, 10, 11}`. Sum of `v[1..3]` is 11−5=6.
2. So that `sum(0, j)` is `prefix[j+1] - prefix[0]` with `prefix[0] = 0`. Without the extra entry, any range starting at index 0 needs a special case.
3. Store the **earliest** index at which each prefix value occurred, and never overwrite it — an earlier start gives a longer sub-array. Seed the map with `{0: -1}`.
4. Only the parity of each prefix matters, so there are two classes. Count how many prefixes are even and how many odd; the answer is $(e/2)+(o/2)$ where e includes the empty prefix.
5. Map 0 to −1, then the sub-arrays with equal counts are exactly those with sum 0 — the previous exercise's machinery with K=0.
6. With K=0 and input `{0}`: inserting first puts `0 → 1` in the map, then the lookup for `running - 0 = 0` finds it and counts the single element twice over — once legitimately and once against itself.
7. All ones: `P[r][c] = r × c`. The middle 2×2 is $P[3][3]-P[1][3]-P[3][1]+P[1][1]=9-3-3+1=4$. Correct.
8. Changing `v[i]` invalidates every prefix entry from i+1 onwards, so a rebuild is $O(n)$ per update. With frequent updates, a Fenwick tree or segment tree gives $O(\log n)$ for both update and query.

Chapter 12 — Binary search

1. `{1,3,5,7,9,11}` for 11: `mid=2` (5 < 11, lo=3); `mid=4` (9 < 11, lo=5); `mid=5` (11, found). Three comparisons.
2. Searching for 6: `mid=2` (5<6, lo=3); `mid=4` (9>6, hi=3); `mid=3` (7>6, hi=2). Now `lo=3 > hi=2`, the loop ends. Note that `lo` is the index where 6 *would* be inserted — which is exactly what `lower_bound` returns.
3. With `hi = size()` and `while (lo < hi)`: the right branch becomes `hi = mid` (not `mid - 1`), because `hi` is now exclusive. The left branch stays `lo = mid + 1`. Both conventions are correct; mixing them is not.
4. After a match at `mid`, any earlier occurrence must be strictly left of `mid`, so `hi = mid - 1` is right. `hi = mid` would leave `mid` in the region and, on a two-element array of equal values, loop forever.
5. Find which half is sorted by comparing `v[lo]` with `v[mid]`, then test whether the target lies within that sorted half's range; search there if so, the other half if not.
6. The test is "is `v[mid] < v[mid+1]`?" — if yes the peak is to the right, if no it is at `mid` or to the left. Binary search needs only a **monotonic predicate**, not a sorted array, and this is the cleanest demonstration of that.
7. Candidates are speeds 1 to `max(piles)`. The test is "can all piles be eaten in H hours at speed k?", computed by summing $\lceil p_i/k \rceil$. It is monotonic because a larger speed never takes more time.

8. The comparisons are logarithmic, but a list cannot jump to its middle — each "jump" is a walk of up to $n/2$ nodes, so the walking dominates.

Chapter 13 — In-place modification

1. Rotating left by 2 is rotating right by $n-2=3$: reverse all, reverse the first 3, reverse the last 2. Equivalently, reverse the first 2, reverse the last 3, then reverse all.
2. `v.begin() + 5` on a 3-element vector is past the end; `std::reverse` on that range is undefined behaviour.
3. Use the sign-marking trick: the slot that is already negative when you reach it identifies the duplicate, and the slot that is still positive after the pass identifies the missing value.
4. The answer must be in $1..n+1$, because n slots cannot contain more than n distinct positive integers — so if $1..n$ are all present the answer is $n+1$. Place each value x in $1..n$ into slot $x-1$ by swapping, then scan for the first slot whose value is wrong.
5. Dutch National Flag — `low`, `mid`, `high`. See Chapter 19.
6. You must remember every value seen so far, and the number of distinct values can be n. Any structure holding that is $O(n)$. Sorting would allow $O(1)$ space but destroys the required order.
7. `abs(0) - 1` is -1, which is not a valid index. Shift the mapping: use index $|v|$ rather than $|v|-1$ with an array one larger, or mark with a sentinel value that cannot occur.
8. Filling from the front overwrites elements of `a` that have not been placed yet. Filling from the back writes only into the spare slots, which by definition hold nothing that matters.

Chapter 14 — Frequency counting

1. Two passes with a 256-entry count array: one to count, one to find the first character whose count is 1. $O(n)$ time, $O(1)$ space.
2. Put the **smaller** array into the set, then scan the larger one. The set's size is then $O(\min(n, m))$ rather than $O(n)$.
3. XOR works because $x \oplus x = 0$ and XOR is commutative and associative, so the pairs cancel in any order and only the lone value survives.
4. Either the sorted characters of the word, or a 26-entry count signature. The first is simpler; the second is faster for long words.
5. Boyer–Moore pairs off each majority element with a non-majority one; since the majority has more than half the votes, it cannot be fully cancelled. See Chapter 19.
6. `{3, 2, 4}` with target 6: inserting first means 3 is already in the map when you look for its complement 3, giving $(0, 0)$.
7. A hash map — a count array of 10^9 entries is 4 GB, while the map holds at most 10^5 entries.
8. Put every value in a set, then for each value `x` **only start counting if** `x - 1` **is absent**. That guarantee means each run is walked exactly once in total, so the whole thing is $O(n)$ rather than $O(n^2)$.

Chapter 15 — Sorting first

1. Sort, then check for three equal neighbours — `v[i] == v[i+2]`. $O(n \log n)$ time, $O(1)$ extra.
2. Remove the `out.back() != x` guards and always push. Equal elements from both arrays are then both kept.
3. Sort, fix `i`, two-pointer the remainder. The sort is $O(n \log n)$ and the loop is n scans of $O(n)$, so $O(n^2)$ overall. Skip duplicates for `i`, and after each match skip duplicates for both inner pointers.
4. Separating the starts from the ends works because you only need to know *how many* meetings are active at each moment, not which ones. Sweep both sorted arrays together, incrementing on a start and decrementing on an end, and take the maximum.
5. The standard requires a *strict weak ordering*, meaning `comp(a, a)` must be false. `<=` violates it, and `std::sort` may then read outside the range — undefined behaviour, not merely a wrong order.
6. `std::sort` is $O(n \log n)$ and orders all 10^7 elements; `nth_element` is $O(n)$ average and merely partitions around the median. The second does roughly a fifth of the work.
7. Sort with `a + b > b + a` as strings — `"9" + "5" = "95"` beats `"5" + "9" = "59"`, so 9 comes first. It looks wrong because it compares concatenations rather than values, and it is right for exactly that reason.
8. Two Sum returning indices. Sorting destroys the positions; use a hash map, or sort `(value, index)` pairs if $O(n)$ space is acceptable anyway.

Chapter 16 — Kadane

1. `{5,-2,3}`: 5→3→6, best 6, no restarts. `{-1,-2,-3}`: −1→−2→−3 — each step restarts, because the running sum is always negative; best is −1.
2. Initialise `endingHere` and `best` to 0 and use `endingHere = max(0, endingHere + v[i])`. Two lines.
3. Either the sub-array does not wrap (plain Kadane) or it does, in which case the excluded elements form a contiguous minimum — so the answer is `total - minSubarraySum`. **Edge case:** if every element is negative, `total - min` is 0, meaning "take nothing"; return the plain Kadane answer instead.
4. `endingHere` is the length of the increasing run ending at i; it restarts at 1 whenever `v[i] <= v[i-1]`.
5. Two variables: the best including house i and the best excluding it. The recurrence is `incl = excl + v[i]` and `excl = max(incl_prev, excl_prev)`.
6. The running product at the middle is −6, which a max-only Kadane would discard; multiplied by the final −4 it becomes 24, the answer.
7. Four running values: the best after buying once, after selling once, after buying twice, after selling twice — each updated from the previous.
8. The sub-problem is "the maximum sum of a sub-array ending exactly at i", and the recurrence is $\text{end}(i) = \max(v[i], \text{end}(i-1) + v[i])$. That is a dynamic program with one state variable and $O(1)$ memory.

Chapter 17 — Matrix patterns

1. $(0,0)\to(2,0)$, $(0,2)\to(0,0)$, $(2,2)\to(0,2)$, giving $(r,c)\to(n-1-c,r)$. As two steps: transpose, then reverse each **column** — or reverse each row first, then transpose.
2. Rotate 90° twice, or equivalently reverse the order of the rows and then reverse each row.
3. On a 3×5 grid, after the top row and the right column are consumed, `top` becomes 1 and `right` becomes 3; the first guard matters on the final inner ring, when `top > bottom` and the bottom row would otherwise be re-traversed.
4. Use row 0 and column 0 as the marker arrays. The cell `g[0][0]` is shared by both, so it cannot record two independent facts — keep a separate boolean for whether column 0 itself must be cleared, and process row 0 and column 0 last.
5. At the top-left, a value smaller than the target could be anywhere to the right **or** below, and a larger value rules out nothing useful — so no comparison eliminates a whole row or column.
6. Depth-first search from each unvisited 1, marking cells as visited (or overwriting them with 0) as you go. Each cell is visited once, so it is $O(R\cdot C)$ in total however many islands there are.
7. A diagonal is a set of cells with constant $r-c$ (or constant $r+c$ for the anti-diagonal direction) — Chapter 7 gave both.
8. On 1000×1000: staircase is 2,000 steps; per-row binary search is 1000×10=10,000. Staircase wins. On 10×100,000: staircase is 100,010 steps; per-row binary search is 10 ×17=170. Binary search wins by a wide margin — the shape of the matrix decides.

Part IV — Chapters 18 to 20

Chapter 18 — Foundational problems

1. Write them, then test each on: `{}`, `{5}`, `{3,3,3}`, `{-1,-9,-4}`. Those four inputs catch most of the standard bugs.
2. 1–3 and 10 are linear traversal; 4 and 8 are two pointers; 5–7 are the write pointer and reversal; 9 and 11 are arithmetic identities.
3. Problems 5 (already requires sorted), 8 (already sorted) and 11 (sorting would put the pairs adjacent). For 11, sorting costs $O(n\log n)$ and the XOR answer is $O(n)$ with $O(1)$ space, so it is never worth it.
4. 9 (sum or XOR) and 11 (XOR).
5. One pass with a write pointer: keep an element if it differs from the last kept value **and** is non-zero; then pad with the zeroes you counted.
6. The sum $n(n+1)/2$ overflows a signed 32-bit `int` when $n(n+1)/2>2^{31}-1$, that is around $n=65{,}535$. XOR has no equivalent limit — it never produces a value outside the range of its inputs.

Chapter 19 — Intermediate problems

1. Work through each; the single observation is listed in Table 19.1's right-hand column.
2. **Boyer–Moore:** each decrement pairs one majority element with one non-majority element and removes both; the non-majority elements run out first because there are fewer than $n/2$ of them. **Floyd on an array:** the array defines a function $i \mapsto v[i]$ that never escapes $1..n$, so iterating it must cycle; two positions holding the same value point to the same place, which makes the duplicate the cycle's entrance.
3. `a = {1,2,3,_,_,_}`, `b = {2,5,6}`. Filling forwards, the first comparison writes 1 into `a[0]` — which already held 1, so no harm yet; but at the second step `b[0] = 2` would be written into `a[1]`, destroying the 2 that has not been placed. Filling from the back writes only into slots that are empty.
4. `total = product of all; out[i] = total / v[i]`. It breaks on any input containing a zero (division by zero for that element, and every *other* element's answer is 0 rather than `total/v[i]`), and on two or more zeros, where every answer is 0. Handling it needs a zero count and three branches.
5. Problem 9 counts **every** occurrence of each prefix value; problem 10 keeps only the **earliest**. One `if` guards the insertion.
6. Sorting plus two pointers. $O(n \log n)$ for the sort plus $O(n^2)$ for the scans, so $O(n^2)$.
7. The three-pointer method handles exactly three values. For k values you would need k boundaries, which becomes unmanageable around k=4; use counting sort ($O(n+k)$) instead, which is the natural generalisation.

Chapter 20 — Advanced problems

1. `{4,5,6,7,0,1,2}` for 0: `mid=3` (7), `v[0]=4 <= 7` so the left half is sorted; is $4 \le 0 < 7$? No, so go right: `lo=4`. `mid=5` (1), `v[4]=0 <= 1` so the left half is sorted; is $0 \le 0 < 1$? Yes, so `hi=4`. `mid=4` (0) — found.
2. Compare `v[mid]` with `v[hi]`: if `v[mid] > v[hi]` the minimum is strictly right of `mid`, so `lo = mid + 1`; otherwise it is at `mid` or left, so `hi = mid`. Comparing with `v[lo]` instead fails on an unrotated array.
3. Writing all three and cross-checking on random inputs is the exercise; the agreement between them is the proof that you understood each.
4. `{4,2,0,3,2,5}`: maxLeft is `{4,4,4,4,4,5}` and maxRight is `{5,5,5,5,5,5}`, so the water is `{0,2,4,1,2,0}` — total 9.
5. The stack fills water in **horizontal layers** between a bar and the previous taller bar, rather than computing a vertical column at a time. Both reach 9; the intermediate quantities are completely different.
6. With all values equal, `v[lo] == v[mid] == v[hi]` gives no information about which half is sorted, so the only safe step is to shrink the range by one at each end — $O(n)$. The worst case is an array of identical values with the target absent.
7. It most resembles approach B: for each bar you need the nearest shorter bar on each side, which is a pair of precomputed arrays — classically built with a monotonic stack rather than prefix maxima.
8. Even negatives: `{-2, -3}` gives 6. Stopping before a zero: `{-2, 3, 0, 4}` gives 4, because the zero resets both running values and the best before it was 3.

Part V — Chapters 21 and 22

Chapter 21 — Pointers

1. `x` is the value 5; `&x` is where it lives; `p` holds that same address; `*p` follows it back to 5; `&p` is where the pointer variable itself lives.
2. Prints `10 20`. `*p = 10` writes through `p` into `a`; then `p` is reseated to `&b` and `*p = 20` writes into `b`.
3. The pointer version is called as `swap(&a, &b)` and can be passed `nullptr`; the reference version is called as `swap(a, b)` and cannot. The reference version is easier to call and impossible to misuse, so prefer it unless absence is meaningful.
4. `p` is a pointer, and `.` is for objects. Change it to `p->x`, which is `(*p).x`.
5. `p + 1` moves `sizeof(int)` = 4 bytes, because pointer arithmetic is scaled by the pointed-to type. `(char*)p + 1` moves exactly 1 byte.
6. A `nullptr` is explicitly, checkably invalid, and dereferencing it crashes immediately. A dangling pointer looks valid, often still reads the old value, and fails later and elsewhere — which makes it far more dangerous.
7. `void zeroIfNotNull(int* p) { if (p) *p = 0; }`. A reference cannot express "there may be nothing here" — a reference must always bind to an object.
8. `pp` holds the address of `p`, which holds the address of `x`; `**pp` is 1. You need a pointer to a pointer when a function must change the caller's pointer itself — which is exactly what `Node**` does in C-style linked list code, and what `Node*&` does more readably in C++.

Chapter 22 — Stack, heap, new and delete

1. Stack: `int x`, `int* p` (the pointer), `std::vector<int> v` (the object), `Node* head` (the pointer). Heap: the `int` that `p` points at, `v`'s element block, and the node `head` points at.
2. `new int[10];` with no `delete[]` leaks 40 bytes; the sanitizer reports the size and the allocating line.
3. `leak2`: `delete p;` before reassigning. `leak3`: either `delete p;` before the early return, or restructure so there is a single exit — the second is better, because it cannot be forgotten when a third return is added later. Best of all: use `std::unique_ptr` and neither problem exists.
4. Nothing at all; it is explicitly defined as a no-op. That is what makes `delete p; p = nullptr;` safe against a second delete.
5. The sanitizer reports zero leaks for the correct version, and a heap-use-after-free with both stack traces for the broken one.
6. `delete head` frees only the first node. The second node is still allocated, and the only pointer to it was `head->next`, which has just been freed — so it is unreachable. Leaking 1 of 2 nodes.
7. Each recursive call pushes a frame; an iterative loop reuses one. 1 MB ÷ 64 bytes ≈ 16,000 frames before overflow.

8. `std::unique_ptr<Node> head` with `std::unique_ptr<Node> next` inside each node. Easier: no `delete` anywhere, and no possible leak. Harder: moving nodes between lists requires `std::move`, reversal becomes noticeably more awkward, and destroying a long list recurses through the destructors and can overflow the stack — which is itself a good lesson.

Part VI — Chapters 23 to 28

Chapter 23 — The linked list idea

1. Four boxes, each split into data and next, with arrows between them, a `head` arrow into the first, and `nullptr` after the last.
2. `head->next->next->data` is 15. `head->next->next->next->next` is `nullptr` — the fourth node's `next`.
3. Binary search requires reaching the middle element in $O(1)$; a list requires $O(n)$ to reach it, so each "halving" costs a linear walk and the whole thing degenerates to $O(n)$.
4. 4 bytes of data plus 8 bytes of pointer is 12, but allocators round up and add bookkeeping — typically 32 bytes per node. So roughly 32 MB against the array's 4 MB: about **8×**.
5. Right: an LRU cache, where a hash map hands you node pointers and you unlink in $O(1)$. Wrong: anything you index into, or any large collection of small values you mostly scan.
6. `head = head->next;` in a loop without saving anything, or simply `head = nullptr;` while nodes are still allocated.
7. There is no `nullptr`, so the loop never terminates. Use `do { ... } while (curr != head);` guarded by a check that the list is non-empty.

Chapter 24 — Building a list

1. `struct Node {` opens a new type; `int data;` is the stored value; `Node* next;` is a pointer to the following node; `Node(int value) {` is the constructor; the two assignments initialise both fields; `};` closes the type.
2. A `Node` containing a `Node` would contain another, forever, so its size could not be computed and the compiler rejects it. A pointer is a fixed 8 bytes whatever it points at, so the type's size is finite.
3. `head->next->next->data`.
4. Both should walk with `curr`. `sum` returns 0 for an empty list, which is the natural identity. `maxValue` has no answer for an empty list — throw, or return a flag, and say which you chose.
5. Two passes: count the length n, then walk $n-k$ nodes. The one-pass version is the fixed-gap pattern of Chapter 29.
6. `delete head;` runs before `head = head->next;`, so the second line reads through a freed pointer. It also leaks everything after the first node once the read returns rubbish.

7. `void freeList(Node* h) { if (!h) return; freeList(h->next); delete h; }`. It uses one frame per node, so it overflows at roughly 16,000 nodes with a 1 MB stack — which is why list code is written iteratively.

Chapter 25 — Insertion

1. `node->next = head->next->next` (the node holding 30), then `head->next->next = node`. Two changes, in that order.
2. `insertBack` becomes $O(1)$; `deleteBack`, `deleteValue` and any `clear` must now update `tail`, or it will dangle.
3. Walk with `prev` until `prev->next->data >= value`. The head case — inserting before the current first node — is the one that changes `head`, and a dummy node removes it.
4. It should succeed: `pos == length` means "append". Check that your loop's range tests allow it; many implementations reject it by accident.
5. `Node** head` requires `*head = node;` and calls like `insertFront(&head, 5)`. C has no references, so `**` is the only option there; C++ has both, and `Node*&` reads better.
6. Return without doing anything. Without the check it dereferences `nullptr` and crashes.
7. Insert a new node after `node`, then swap the two nodes' `data`. It is unacceptable when copying the data is expensive, when the data cannot be copied, or when another part of the program holds a pointer to the original node and expects its value to stay put.
8. Roughly a factor of $n/2$ — several seconds versus a few milliseconds at 100,000 elements. Without the tail pointer the total work is $1+2+\cdots+n = n^2/2$ pointer hops.

Chapter 26 — Deletion

1. `doomed = head` · `head = head->next` · `delete doomed`. Moving the delete before the advance reads a freed pointer; removing the save leaks the node.
2. Because you must set the *second-to-last* node's `next` to `nullptr`. `while (prev->next != nullptr)` leaves `prev` on the **last** node, where setting `prev->next = nullptr` does nothing useful.
3. Walk the whole list keeping track of the last matching node's predecessor, then unlink after the walk. One pass is enough — you simply cannot act until you know there is nothing later.
4. On `{7, 7}`, removing the first 7 leaves `prev->next` pointing at the second 7; advancing `prev` skips it. Output keeps one 7.
5. It is "remove duplicates from a sorted array" with pointers — compare each node with the next and unlink when equal.
6. The sanitizer reports a leak naming the `new` that created the node you stopped freeing.
7. A tail pointer tells you where the last node *is*, but after deleting it you need the new last node — which is the second-to-last, and finding that requires walking. A doubly linked list gives it as `tail->prev`, in $O(1)$.
8. Three designs: return a `bool` success flag and take the value by reference; throw on an empty list; or return `std::optional<int>`. The third is clearest in modern C++

and impossible to misuse.

Chapter 27 — Doubly and circular

1. Six pointers: three `next` (the last being `nullptr`), three `prev` (the first being `nullptr`).
2. `insertBefore` needs the node's predecessor, which `prev` supplies directly. A singly linked list would have to walk from the head to find it: $O(n)$.
3. You cannot unlink it without the predecessor, so you must walk from the head: $O(n)$. Chapter 31's trick copies the next node's data instead and deletes *that* node, in $O(1)$ — but it cannot handle the last node.
4. For each node, check `node->next == nullptr || node->next->prev == node`, and the mirror with `prev`.
5. Walk to the tail and set `tail->next = head`; to undo, walk until `curr->next == head` and set it to `nullptr`. To detect circularity, walk from `head->next` and see whether you return to `head` before reaching `nullptr`.
6. Build a circular list of n nodes; repeatedly advance $k-1$ nodes and delete the next, until one remains. $O(nk)$ with a list; $O(n)$ with the mathematical recurrence.
7. `std::list::erase` holds the node and both its neighbours, so it is four pointer writes. `std::vector::erase` must shift every later element. `operator[]` is not provided because it would be $O(n)$ and would invite people to write accidentally quadratic loops.
8. A judgement call: the overhead is 400% for a 4-byte payload, 25% for a 64-byte one, and under 5% for a 512-byte one. Somewhere above a few hundred bytes it stops being a consideration.

Chapter 28 — Reversal

1. Four iterations; after each, `prev` is the head of the growing reversed chain.
2. `prev` is the last node processed, which is the original tail — the new head. For an empty list the loop never runs, so `prev` stays `nullptr`, which is correct.
3. Empty: the loop body never executes, `head = prev` sets `nullptr` to `nullptr`. One node: one iteration sets its `next` to `nullptr` (already true), and `head = prev` names the same node.
4. Yes: it returns a one-element list, because `nextNode` reads the value that line 2 just wrote, which is `prev` — `nullptr` on the first iteration.
5. `Node* reverseList(Node* head)` returning `prev` is LeetCode's signature and is the more common convention; `Node*&` is more convenient when you own the list variable. Know both.
6. The node that was k-th (it becomes the tail of the reversed part) and the node that was $(k+1)$-th (the rest), so that you can join them afterwards.
7. Find the middle, reverse the second half, compare, then reverse it back. $O(n)$ time, $O(1)$ space — and restoring the list is the part that marks out a careful answer.
8. Swapping them sets `head->next = nullptr` first, so `head->next->next` then dereferences `nullptr` and crashes.

Part VII — Chapters 29 to 32

Chapter 29 — The patterns

1. The entire head-handling branch — roughly six lines — disappears, leaving one loop.
2. A stack dummy is freed automatically when the function returns. A heap dummy must be deleted, and the natural `return dummy->next` path makes it very easy to forget, leaking one node per call.
3. Count the length first, then walk $\lfloor (n-1)/2 \rfloor$ nodes. It is clearer to read and takes two passes; the `head->next` trick is one pass and slightly cryptic. Both are defensible.
4. k=0 has no meaningful answer — reject it, or define it as "one past the last", and say which. A negative k should be rejected outright.
5. F is head-to-entrance, a is entrance-to-meeting-point measured forward round the cycle, L is the cycle length.
6. With speed 2 the gap shrinks by exactly 1 per step, so it cannot skip zero. With speed 3 it shrinks by 2 and could step straight over zero, so correctness would depend on L being even or on a divisibility argument.
7. Change `a->data <= b->data` to `a->data >= b->data`, assuming both inputs are themselves descending.
8. Typically around 10,000 to 100,000 depending on the frame size and the stack limit. Measuring it yourself is the point of the exercise.

Chapter 30 — Core problems

1. Writing them from memory, with those three tests each, is the exercise.
2. The two-pass version counts then walks; it is marginally clearer. Give the one-pass version, because the question is usually asking for it, but mention that two passes is equally $O(n)$.
3. With n larger than the list, the function above returns the head unchanged. That is a defensible choice; throwing or returning an error flag is also defensible. What is not defensible is dereferencing `nullptr`.
4. A hash set of visited node pointers: $O(n)$ time, $O(n)$ space. You would prefer it only if you also need to know *which* nodes were visited, or if the list is being modified concurrently and Floyd's invariant could break.
5. One at a time: merging lists of sizes $n, 2n, 3n, \ldots$ costs $O(k^2 n)$. Divide and conquer pairs them up, giving $O(kn \log k)$.
6. `detectCycle` returns the head itself, because the entrance is the head and F=0.
7. No. The meeting point is somewhere inside the cycle but not necessarily just before the entrance, so you still need phase 2 to locate the entrance and then a walk round to its predecessor.

Chapter 31 — Intermediate problems

1. Copy the values into a vector and use converging pointers: $O(n)$ time, $O(n)$ space. Prefer it when the list must not be modified at all — for example if another thread is reading it.

2. Construct two lists sharing a tail and count the steps; both pointers reach the shared node after exactly $a+b+c$ moves.
3. Either reverse both lists, add, and reverse the result; or push both onto stacks and pop. The stack version does not modify the inputs, which is usually preferable.
4. You need a dummy node (the head itself may be removed) and a `prev` pointer. The plain version keeps one copy of each duplicated value; this one must remove all of them, so you must skip the whole run and link `prev` past it.
5. There is no next node to take a value from. A doubly linked list would allow it, because the node knows its predecessor.
6. Splitting at the second middle on `{1,2,3,4}` gives halves of 3 and 1, so the interleave runs out of second-half nodes early and leaves the list wrong.
7. Make it circular, walk to the node $n-(k \bmod n)-1$ steps from the head, and cut there — Chapter 32.

Chapter 32 — Advanced problems

1. Use a dummy node and a `groupPrev` pointer; for each group, find the k-th node, remember what follows it, reverse the group, then re-link. The pointer bookkeeping is the whole difficulty, which is why the recursive version is usually given first.
2. Remove the early `return head` when fewer than k nodes remain, and reverse whatever is left.
3. `{1,2,3}` with $k=7$: `n - k - 1` is $3-7-1=-5$, so the walk loop does not run and the "new tail" is the head — producing a rotation by 2 rather than by 1. Silently wrong rather than crashing, because the list is circular at that moment.
4. Rotating left by k is rotating right by $n-(k \bmod n)$.
5. Bottom-up merges runs of size 1, then 2, then 4, splicing them back together with a dummy node. It is harder because you must split and re-join explicitly at every pass rather than letting the recursion do it.
6. Choosing a good pivot and partitioning both require random access. Reaching the middle of a list is $O(n)$, so median-of-three is expensive, and a first-element pivot degenerates to $O(n^2)$ on sorted input.
7. The map version is two clear passes and $O(n)$ space; the interleaving version is three subtler passes and $O(1)$ space. Give the map version first.
8. The original list would be left interleaved with the copy — so the caller's list is corrupted. The test is `original intact`, comparing `a->next == b` after the call.

Part VIII and IX — Chapters 33 to 36

Chapter 33 — The complexity showdown

1. The condition column is the point; check it row by row.
2. Appending a million integers: **array** (amortised $O(1)$, cache-friendly). Deleting the front a million times: **list** $O(1)$ each, though a deque is better than both. Finding the median: **array**, because `nth_element` needs random access. A drag-and-drop playlist: **list**, if you hold iterators to the dragged items; otherwise an array is fine and

simpler.

3. Each halving step requires reaching the middle element of the current range, which on a list means walking there — $O(n)$ per step rather than $O(1)$.
4. A hash map from key to list node, plus a doubly linked list of nodes in recency order. `get` is $O(1)$: the map finds the node, the list moves it to the front. `put` is $O(1)$: insert at the front and, if full, remove the tail. Neither structure can do it alone.
5. *"Do you already hold an iterator to the insertion point, or do you search for it each time?"* If they search, both structures are $O(n)$ and the vector is faster.
6. Vector: about 4 MB. List: about 32 MB with typical allocator overhead. Roughly **8×**.
7. The array's accesses are sequential, so one cache line supplies 16 elements and the prefetcher stays ahead. The list's accesses form a dependent chain — the address of the next load is unknown until the current one returns — so they cannot be overlapped, and each may stall for hundreds of cycles.
8. **Wins:** an LRU cache's eviction, or splicing a 10,000-element range between two lists in $O(1)$. **Does not win despite the table:** inserting at the front of a 1,000-element container of `int`, where `memmove` beats the allocation.

Chapter 34 — The method

1–7. These are process exercises; what matters is that you wrote the steps down rather than did them in your head. For step 8, the bottleneck table is the lookup; for step 14, the edge-case checklist in Chapter 35 is the list to run.

For exercise 1 specifically: the bottleneck is "it compares every pair of a sorted array", so the pattern is converging two pointers, and the answer is $O(n)$ after the array is already sorted.

For exercise 2: the bottleneck is "it rescans the list for each node", so the pattern is a hash set of seen values — $O(n)$ time and $O(n)$ space — with a dummy node because the head may be removed. The hard part is that you must delete nodes while walking, which means keeping `prev`.

Chapter 35 — C++ standards

1. Any function taking a container by value that does not need a copy should be `const&`. In most people's early code that is most of them.
2. Typically: signed/unsigned comparison warnings, unused parameters, and — if you wrote any linked list code — at least one leak.
3. It almost always does: `for (int row = 0; row < rows; ++row)` needs no comment where `for (int i = 0; i < n; ++i)` might.
4. For example, above a list traversal: *"an empty list needs no special case — the loop simply never runs."*
5. `a ^= a` sets `a` to 0, and the rest of the sequence keeps it at 0. The value is destroyed. `std::swap(a, a)` is harmless.
6. Under a minute with practice. Items 2, 4 and 10 catch most real problems.
7. In a header it injects every name in `std` into every file that includes it. A concrete collision: a variable or function called `count`, `distance`, `size`, `data` or `begin` — all of

which are both common identifiers and names in `std`.

Chapter 36 — The roadmap

The roadmap has no exercises of its own; its "answers" are the exit signals. If you cannot yet meet a level's exit signal, the problems to repeat are the ones in that level you needed a hint for.

CHAPTER

39

The Mock Interview

Four rounds, with what the interviewer is actually listening for

PRACTICE PREREQS: Parts I to VIII

Four complete interview rounds, each with the problem, a realistic transcript of a strong answer, and a commentary on what earned the marks. The commentary matters more than the solutions — in a coding interview, roughly half the assessment is about how you communicate while solving, and that half is almost never practised.

WHAT YOU WILL LEARN

- Know what an interviewer is assessing, beyond whether the code works
- Structure an answer out loud, from clarification to complexity
- Handle a hint, a follow-up and a bug without losing composure
- Recognise the five behaviours that cost candidates offers
- Run a realistic mock session on your own

What is actually being assessed

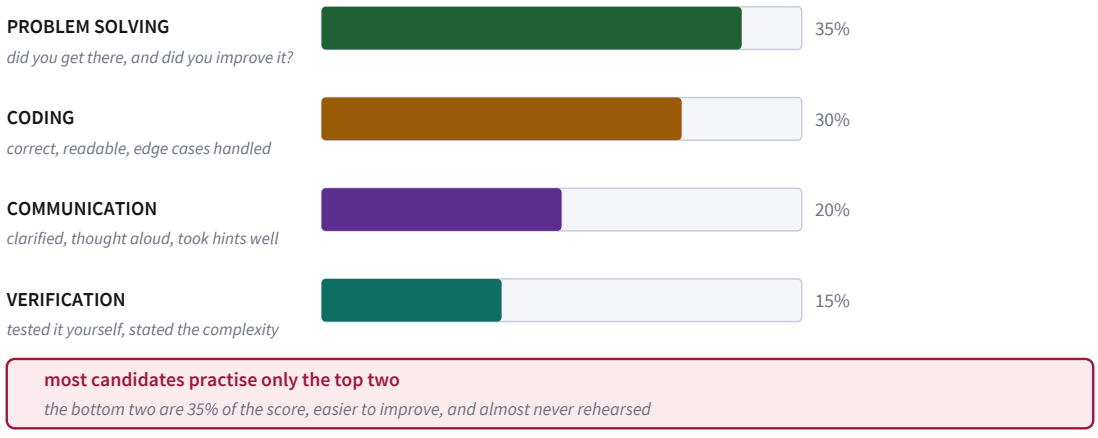


Figure 39.1 A typical rubric. Correct code is necessary and is roughly a third of the score. The other two thirds are things you can practise independently of the problems.

KEY INSIGHT | The four categories, and what each looks like

Problem solving (≈35%). Did you reach a working solution? Did you improve it? Did you find the optimisation yourself, or need the hint?

Coding (≈30%). Is the code correct, readable and complete? Are the edge cases handled? Would it compile?

Communication (≈20%). Did you clarify before coding? Did you think out loud? Could the interviewer follow your reasoning? Did you take hints gracefully?

Verification (≈15%). Did you test it yourself? Did you state the complexity unprompted? Did you find your own bug, or did the interviewer have to?

Most candidates practise only the first two. The last two are easier to improve and are where offers are most often lost.

Round 1 — Warm-up (15 minutes)

> ***Interviewer.*** *Given an array of integers, move all the zeroes to the end while keeping the order of the non-zero elements. Do it in place.*

NOTE | A strong answer, as a transcript

Candidate. *Let me make sure I have it. Input is a vector of ints, I rearrange it in place so every zero is at the end, and the non-zero elements keep their relative order. Output is nothing — the input is modified. Is that right?*

Interviewer. Yes.

Candidate. *A couple of questions. Can the array be empty? And does the order of the zeroes among themselves matter?*

Interviewer. It can be empty. The zeroes are all identical, so no.

Candidate. *Then let me try a small example.* `{0, 1, 0, 3, 12}` *should become* `{1, 3, 12, 0, 0}`.

The brute force would be: for each zero, shift everything after it left by one. That is O(n) per zero, so $O(n^2)$ in the worst case — an array of all zeroes.

The waste is that I move the same elements repeatedly. Instead I can use a write pointer: walk through with a read pointer, and every time I see a non-zero, write it at the write position and advance it. At the end, everything from the write position onwards becomes zero. That is one pass to compact and one to pad, so O(n) time and O(1) space.

Interviewer. Go ahead.

Candidate. *(writes the function, saying "the write pointer only advances when we keep something, so it never overtakes read — that's why this is safe")*

Let me test it. `{0,1,0,3,12}` *: read 0, skip. Read 1, write at 0. Read 0, skip. Read 3, write at 1. Read 12, write at 2. So w is 3, and positions 3 and 4 become zero:* `{1,3,12,0,0}` *. Correct.*

Edge cases: empty array — the loops never run, which is right. All zeroes — nothing is written in the first pass, and the second pass fills everything with zero, which it already was. No zeroes at all — every element is copied onto itself, which is wasteful but correct.

O(n) time, O(1) space. One variation: I could swap instead of copying, which does it in a single pass — it's the same complexity, with more writes when zeroes are rare and fewer when they're common.

KEY INSIGHT | What earned the marks

- **Restated and confirmed** before touching anything.
- **Asked two questions**, both of which genuinely affect the answer.
- **Gave the brute force and its complexity** before optimising.
- **Named the waste** rather than jumping to the answer.
- **Explained the invariant** (`write <= read`) while coding — the detail that shows the solution is understood rather than recalled.
- **Tested without being asked**, including the all-zeroes case.
- **Volunteered a variation** and compared it honestly.

That is about four minutes of talking and it is worth more than the code.

Round 2 — The standard question (25 minutes)

Interviewer. *Given a linked list, remove the nth node from the end and return the head.*

NOTE | The transcript, with a hint

Candidate. *So the list is singly linked, n counts from the end with n=1 meaning the last node, and I return the head — which might change if n equals the length. Can n be larger than the list?*
Interviewer. Assume n is valid.
Candidate. *The obvious approach is two passes: count the length L, then walk to node L−n−1 and unlink the one after it. That is O(n) time and O(1) space, and it is correct. Shall I write that, or try for one pass?*
Interviewer. Try one pass.
Candidate. *One pass means I cannot know the length in advance, so I need something that reaches the end and tells me where I was n nodes ago. Two pointers with a fixed gap: send one pointer n nodes ahead, then move both until the leader falls off the end — the trailing one is then n from the end.*
*But I need the node **before** the one I'm deleting, so the gap should be n+1, not n.*
And if the node to delete is the head, there is no predecessor. Rather than special-case that, I'll use a dummy node in front of the head, start both pointers there, and return `dummy.next` *at the end.*
Interviewer. Go ahead.
Candidate. *(writes it) Let me trace* `1→2→3→4→5` *with n=2. The lead moves 3 steps from the dummy, landing on node 3. Then both move: lead on 4, trail on 1; lead on 5, trail on 2; lead is null, trail is on 3. So I delete trail->next, which is node 4.* `1→2→3→5` *. Correct.*
Now `1→2→3` *with n=3 — deleting the head. Lead moves 4 steps:* `dummy→1→2→3→null` *. So the loop does not run, and trail is still the dummy;* `trail->next` *is the head, which is exactly what we delete. Returning* `dummy.next` *gives* `2→3` *. The dummy earned its place.*
O(n) time, O(1) space, one pass. And I free the removed node — if this were a language with garbage collection I would not, but in C++ it leaks otherwise.

KEY INSIGHT | What earned the marks

- **Offered the two-pass solution first and asked** whether to proceed. That makes the interviewer a collaborator and guarantees something on the board.
- **Derived n+1** out loud from "I need the predecessor", rather than stating it as a remembered fact.
- **Introduced the dummy node with its reason**, not as a trick.
- **Traced the hard case** — deleting the head — not just the easy one.
- **Mentioned the `delete`**, and explained why it matters in C++ specifically.

Round 3 — The harder question (35 minutes)

> ***Interviewer.*** *Given an array of heights, compute how much rain water is trapped between them.*

NOTE | The transcript, including getting stuck

Candidate. *Each element is a bar of width 1, and I want the total water above the bars. Let me take `{0,1,0,2,1,0,1,3,2,1,2,1}` — I believe the answer is 6, but let me work out part of it. Above index 2, which has height 0: to its left the tallest is 1, to its right the tallest is 3. The water level is the smaller of those, so 1, minus the bar's own height 0 — so 1 unit sits there.*
That generalises. For any position i, the water is `min(maxLeft, maxRight) - height[i]`, floored at zero.
Interviewer. Good. How fast is that?
Candidate. *If I scan left and right for each position, it's $O(n^2)$. But those scans repeat work — maxLeft at position i is just maxLeft at i−1 combined with height[i]. So I can precompute both arrays in two passes and then sum in a third. O(n) time, O(n) space.*
Interviewer. Can you do it in $O(1)$ space?
Candidate. *(pause) Let me think about that out loud. I'd want two pointers from the ends, carrying running maxima — but the problem is that from the left I only know the maximum of what I have **seen**, not the true maximum to the right.*
Interviewer. Do you always need the true maximum?
Candidate. *…No. If my running maxLeft is smaller than my running maxRight, then the true maximum on the right is **at least** maxRight, which is bigger than maxLeft — so the minimum of the two is maxLeft regardless of what else is out there on the right. The shorter side is the one I know exactly.*
So: move whichever pointer is at the shorter bar, and compute that position's water using that side's running maximum. O(n) time, O(1) space.
(writes it, then traces the first four steps)
Edge cases: fewer than three bars traps nothing, since you need a wall on each side. All equal heights traps nothing. A strictly increasing array traps nothing.

KEY INSIGHT | What earned the marks

- **Solved one position first.** That single move turns this problem from intimidating into routine, and it is the transferable lesson.
- **Did not panic at the $O(1)$ follow-up.** The pause was filled with a statement of the difficulty, which is what let the interviewer give a useful hint.
- **Took the hint and ran with it.** "Do you always need the true maximum?" is a large hint; the candidate supplied the actual argument, which is what is assessed.
- **Gave three degenerate cases** that trap nothing, showing the solution was reasoned about rather than memorised.

Needing a hint is not a failure. How you use it is the measurement.

Round 4 — The follow-up round (20 minutes)

Many interviews end with variations on what you have already built. These test whether your solution was understood or recalled.

Follow-up	What it is testing
"Now do it in $O(1)$ space."	whether you know the in-place techniques
"What if the input doesn't fit in memory?"	whether you can think about streaming and external data
"What if it's called a million times?"	whether you spot that precomputation now pays
"What if the array is sorted?"	whether you notice binary search and two pointers become available
"What if there are duplicates?"	whether your algorithm's correctness depended on distinctness
"What if it must be thread-safe?"	whether you know what shared mutable state costs
"How would you test this?"	whether you think about edge cases systematically
"What breaks first if n grows 1000×?"	whether you can identify the real bottleneck

Table 39.1 Typical follow-ups, and what each is probing. Prepare for these as seriously as for the main problems.

INTERVIEW ANGLE | Answering "how would you test this?"
Give a structure, not a list of examples:

> *"Three groups. **Degenerate inputs**: empty, one element, two elements. **Structural cases**: all equal, already sorted, reverse sorted, all negative, with duplicates. **Boundary cases**: the target at the first position, at the last, and absent; the maximum allowed n; values at `INT_MAX` and `INT_MIN` to check for overflow. Then a randomised test against the brute force — generate random arrays, run both, and compare. That catches the cases I did not think of."*

That last sentence is the one that lands. It is how real bugs are found, and very few candidates mention it.

The five behaviours that cost offers

WATCH OUT | **In roughly descending order of damage**

Silence. Thinking without speaking for three minutes reads as being stuck, and the interviewer cannot help. Narrate, even when the narration is *"I'm trying to find a way to avoid re-scanning, which usually means hashing."*

Coding before clarifying. A correct solution to a misunderstood problem scores almost nothing, and a single question would have prevented it.

Optimising before anything works. Twenty minutes chasing the elegant answer and finishing with nothing on the board is the most common way strong candidates fail.

Defending a wrong answer. When an interviewer says *"what about an empty array?"*, they have found a bug. Check it; do not explain why it is fine.

Claiming a complexity you have not verified. Saying $O(n)$ for something quadratic is worse than saying "I think this is $O(n)$ but let me count the nested work". The second is honest and shows method.

How to run a mock on your own

CHECKLIST | **A realistic solo session**

1. Pick a problem you have not solved, at Level 4 or 5 of Chapter 36.
2. Set a timer for 35 minutes.
3. **Talk out loud the whole time**, and record it. This is the part that feels silly and is the part that works.
4. Write the code on paper or in a plain text editor — no compiler, no autocomplete, no internet.
5. When the timer stops, stop.
6. Only then, type it in and compile. Count the compile errors; each one is a detail you did not have fully in mind.
7. Play back the recording. Note every silence longer than fifteen seconds, and whether you stated the complexity without prompting.
8. Write down one thing to do differently next time. Only one.

Three of these a week for a month will change your interviews more than thirty extra solved problems.

CHAPTER RECAP | **Chapter 39 in five lines**

- Correct code is about a third of the assessment; communication and verification are another third and are rarely practised.
- Clarify, give the brute force, name the bottleneck, then optimise — in that order.
- Trace the **hard** case, not the easy one, and test before being asked.
- A hint is not a failure; what you do with it is the measurement.
- Record yourself. The silences are what you need to hear.

CHAPTER

40

Reference: Plan, Checklist, Glossary

A thirty-day plan, the formula sheets, and every term defined

REFERENCE PREREQS: none — use this throughout

The part of the book to keep open. A thirty-day study plan that assumes about ninety minutes a day, the pattern and complexity sheets condensed to a page each, a revision checklist to run before an interview, and a glossary of every technical term this book uses.

WHAT YOU WILL LEARN

- Follow a day-by-day plan through the whole book
- Recall any pattern or complexity from a one-page summary
- Run a final revision check before an interview
- Look up any term used anywhere in this book

The thirty-day plan

Each day is about ninety minutes: roughly forty of reading and fifty of problems. If you have less time, take sixty days and halve each day — do **not** skip the problem half, which is where the learning is.

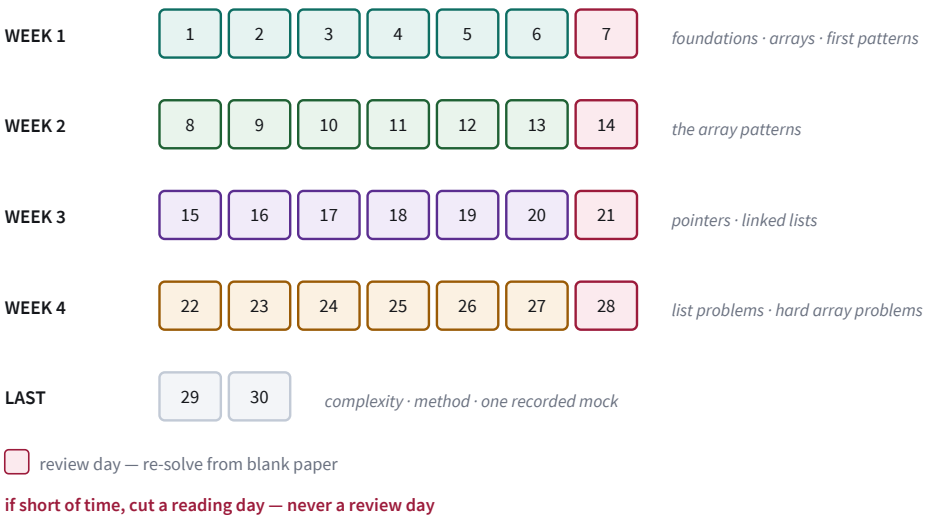


Figure 40.1 The thirty days by theme. Note that the two review days and the four buffer days are not optional padding — spaced repetition is doing a large part of the work.

Day	Read	Do
1	Chapters 1–2	Ch 1 and 2 exercises; run the `sizeof` / `&` programs yourself
2	Chapter 3	Ch 3 complexity drills; Exercise set E in Chapter 37
3	Chapters 4–5	Roadmap Level 1, problems 1–8
4	Chapter 6	Roadmap Level 2, problems 9–12; trace each by hand first
5	Chapter 7	Problem 18; write spiral traversal for a 3×5 grid
6	Chapters 8–9	Problems 19–21; set B of Chapter 37
7	**Review**	Re-solve problems 1–12 from blank paper. No notes.

Table 40.1 Week 1 — foundations and arrays. By the end you can write any basic array operation without thinking.

Day	Read	Do
8	Chapter 10	Problems 22–23; write the fixed and variable window from scratch
9	Chapter 11	Problem 24; sub-array sum equals K
10	Chapter 12	Problems 25–26; binary search, then first/last occurrence
11	Chapter 13	Problem 27; rotate by three reversals
12	Chapter 14	Problems 20, 28, 29
13	Chapters 15–16	Problems 30–32
14	**Review**	Chapter 37 sets A and C. Re-solve anything you got wrong.

Table 40.2 Week 2 — the array patterns. The goal is naming the pattern before coding, every time.

Day	Read	Do
15	Chapters 17, 21	Problems 33–34; the Chapter 21 pointer exercises
16	Chapter 22	Build and free a list; verify with `-fsanitize=address`
17	Chapters 23–24	Problems 13–14; build, print, count, search
18	Chapter 25	Problem 15; all four insertions
19	Chapter 26	Problems 16–17; all four deletions, no leaks
20	Chapter 27	Doubly linked insert and delete; the circular traversal
21	**Review**	Re-solve every list operation from blank paper.

Table 40.3 Week 3 — pointers and linked lists. The slowest week for most people; do not rush it.

Day	Read	Do
22	Chapter 28	Problem 44; reverse iteratively, then recursively, then trace both
23	Chapter 29	Problems 45–48
24	Chapter 30	Problem 49; the six core list problems
25	Chapter 31	Problems 50–53

Day	Read	Do
26	Chapters 18–19	Problems 35–43
27	Chapter 20	Problems 54–57
28	Chapter 32	Problems 63–66

Table 40.4 Week 4 — list problems and the harder array problems.

Day	Read	Do
29	Chapters 33–35	The complexity showdown; run the Chapter 35 checklist over old code
30	Chapters 34, 39	One full mock interview, recorded, from Chapter 39
Buffer	—	Four spare days. Use them on whatever you needed a hint for.

Table 40.5 The last two days, plus how to use the buffer.

KEY INSIGHT | Why the review days matter more than the reading days
Days 7, 14 and 21 ask you to re-solve from a blank page. That feels like repetition and is not — it is **retrieval practice**, and the evidence that it beats re-reading is overwhelming. If you are short of time, cut a reading day. Never cut a review day.

Pattern summary sheet

Pattern	Trigger in the problem	Time	Space	Chapter
Linear traversal	a summary of everything seen	$O(n)$	$O(1)$	8
Two pointers, converging	sorted array, a **pair** wanted	$O(n)$	$O(1)$	9
Two pointers, same direction	filter or compact in place	$O(n)$	$O(1)$	9
Sliding window, fixed	"every window of size k"	$O(n)$	$O(1)$	10
Sliding window, variable	"longest/shortest contiguous…"	$O(n)$	$O(1)$	10
Prefix sums	many range queries; sums with negatives	$O(n)$ build	$O(n)$	11
Binary search	sorted array, or a monotonic test	$O(\log n)$	$O(1)$	12
Binary search on the answer	"minimum X such that…"	$O(\log R \cdot C)$	$O(1)$	12
In-place / index marking	"$O(1)$ space", values in $1..n$	$O(n)$	$O(1)$	13
Frequency counting	"have I seen this?"	$O(n)$	$O(n)$	14
Sort first	closest pair, duplicates, grouping	$O(n \log n)$	varies	15

Pattern	Trigger in the problem	Time	Space	Chapter
Kadane	"maximum sub-array..."	$O(n)$	$O(1)$	16
Matrix boundaries	spiral, rotate, layer traversal	$O(RC)$	$O(1)$	17
Dummy node	the head may change	$O(1)$ extra	$O(1)$	29
Fast and slow	middle, cycle, split in half	$O(n)$	$O(1)$	29
Fixed gap	"k-th from the end"	$O(n)$	$O(1)$	29
Floyd's algorithm	cycle detection or its entrance	$O(n)$	$O(1)$	29
Merge / split	sorted lists, sorting a list	$O(n)$	$O(1)$	29
Pointer reversal	reverse all or part of a list	$O(n)$	$O(1)$	28

Table 40.6 Every pattern in the book, with its trigger and its cost. If you remember one page, make it this one.

Complexity summary sheet

Complexity	Name	Practical up to	Typical source
$O(1)$	constant	any n	indexing, hash lookup, arithmetic
$O(\log n)$	logarithmic	any n	binary search, balanced tree
$O(n)$	linear	$\sim 10^8$	one pass
$O(n \log n)$	linearithmic	$\sim 10^6$	sorting, divide and conquer
$O(n^2)$	quadratic	$\sim 10^4$	nested loops over the input
$O(n^3)$	cubic	~ 500	triple loops, naive matrix multiply
$O(2^n)$	exponential	~ 25	all subsets
$O(n!)$	factorial	~ 11	all permutations

Table 40.7 The growth hierarchy, with the input size each is still practical for at roughly 10^8 operations per second.

Operation	Array	Singly linked	Doubly linked
Access index i	$O(1)$	$O(n)$	$O(n)$
Search unsorted	$O(n)$	$O(n)$	$O(n)$
Search sorted	$O(\log n)$	$O(n)$	$O(n)$
Insert front	$O(n)$	$O(1)$	$O(1)$
Insert back	$O(1)$ am.	$O(n)$ / $O(1)$ with tail	$O(1)$ with tail
Insert at a held position	$O(n)$	$O(1)$	$O(1)$
Delete front	$O(n)$	$O(1)$	$O(1)$
Delete back	$O(1)$	$O(n)$	$O(1)$ with tail
Delete a held node	$O(n)$	$O(n)$	$O(1)$

Table 40.8 Array and list operations at a glance — the condensed version of Chapter 33.

The revision checklist

> **CHECKLIST | Run this the day before an interview**
> **Can you write these from memory, correctly, first time?** Binary search · reverse a linked list · Kadane · the variable sliding window · the two-pointer Two Sum · merge two sorted lists · Floyd's cycle detection · the write-pointer filter.
> **Can you state these without hesitating?** Why `mid = lo + (hi-lo)/2` · why `push_back` is amortised $O(1)$ · why linked list insertion is only $O(1)$ given a pointer · why binary search needs random access · what `delete` does to the pointer · why the dummy node exists.
> **Do you do these automatically?** Ask about empty input and duplicates · state the brute force first · name the complexity unprompted · test the empty and single-element cases · free what you allocate · say whether you modified the input.
> **Have you, in the last week?** Recorded one mock · re-solved three problems from blank paper · run something under `-fsanitize=address`.

Glossary

Address. The number identifying one byte of memory. Chapter 2.

Amortised complexity. The average cost per operation over a long sequence, where occasional expensive operations are paid for by many cheap ones. Not the same as average case. Chapter 5.

Array. A fixed number of same-typed elements in one contiguous block. Chapter 4.

Auxiliary space. Memory an algorithm uses in addition to its input. Recursion depth counts. Chapter 3.

Base address. The address of an array's first element; the B in $B+i\times S$. Chapter 2.

Big O. Notation for an upper bound on growth: $f(n)=O(g(n))$ if beyond some point f is at most a constant multiple of g. Chapter 3.

Bit / byte. A bit is one binary digit; a byte is eight of them, and is the unit memory is addressed in. Chapter 2.

Cache line. The block of memory (typically 64 bytes) the processor fetches at once. Why sequential access is far faster than scattered access. Chapters 7, 33.

Capacity. How many elements a vector's current block could hold, as opposed to how many it does hold. Chapter 5.

Circular linked list. One whose last node points back to the first, so there is no `nullptr` terminator. Chapter 27.

Contiguous. Occupying consecutive addresses with no gaps. The property that makes array indexing $O(1)$. Chapter 4.

Constructor. A function run automatically when an object is created; it has the type's name and no return type. Chapter 24.

Dangling pointer. A pointer to memory that has been freed or to an object that no longer exists. Chapters 21, 22.

Dereference. To follow a pointer and reach the thing it points at: `*p` or `p->field`. Chapter 21.

Doubly linked list. One whose nodes have both `next` and `prev`, so it can be walked in either direction and a held node deleted in $O(1)$. Chapter 27.

Dummy node. A placeholder node placed before the real head so that every real node has a predecessor, removing head special cases. Chapter 29.

Dynamic array. An array that can grow by allocating a larger block and copying. `std::vector`. Chapter 5.

Floyd's cycle detection. The tortoise-and-hare algorithm, which detects a cycle and finds its entrance in $O(1)$ space. Chapter 29.

Head. The pointer to a linked list's first node. Not itself a node. Chapter 23.

Heap. The memory region you allocate from with `new` and must return with `delete`. Chapter 22.

In place. Using $O(1)$ auxiliary space; the input may be modified and used as workspace. Chapter 13.

Invariant. A property that is true before and after every iteration of a loop, used to prove correctness. Chapter 12.

Iterator invalidation. When an operation (such as a vector reallocating) makes existing pointers, references and iterators unusable. Chapter 5.

Linked list. A sequence of nodes, each holding a value and a pointer to the next. Chapter 23.

Memory leak. Heap memory that is unreachable and has not been freed. Produces no error of any kind. Chapter 22.

Node. One element of a linked structure: a value plus one or more pointers. Chapter 23.

`nullptr`**.** The pointer value meaning "points at nothing". Chapter 21.

Ownership. Responsibility for deleting a heap object. Every object needs exactly one owner. Chapter 22.

Pointer. A variable whose value is a memory address. Chapter 21.

Prefix sum. An array where entry i is the sum of everything before index i, allowing any range sum in $O(1)$. Chapter 11.

RAII. Tying a resource's lifetime to an object's lifetime so that the destructor releases it. The idea behind `std::vector` and `std::unique_ptr`. Chapter 22.

Random access. The ability to reach any element in the same constant time. Arrays have it; linked lists do not. Chapter 4.

Reference. An alias for an existing object: `int& r = x;` . Cannot be null and cannot be reseated. Chapter 21.

Row-major order. Storing a 2-D array one complete row after another, which is what C++ does. Chapter 7.

Sentinel. A value or node used to mark a boundary, such as -1 for "not found" or a dummy node for "before the head". Chapters 4, 29.

Sliding window. A sub-array whose two edges move forwards, so that overlapping ranges are updated rather than recomputed. Chapter 10.

Stack (memory). The region holding local variables, managed automatically, one frame per active call. Chapter 22.

Stable sort. One that preserves the relative order of equal elements. `std::stable_sort` ; `std::sort` is not stable. Chapter 15.

Tail. The last node of a linked list — the one whose `next` is `nullptr` . Chapter 23.

Two pointers. Two indices moving with purpose, either converging from the ends or chasing each other from the same end. Chapter 9.

Undefined behaviour. Something the standard places no requirements on, such as reading past the end of an array. Anything at all may happen. Chapter 4.

Use after free. Reading or writing through a pointer whose object has been deleted. Chapter 22.

Write pointer. The trailing index in a same-direction two-pointer scan, marking where the next kept element goes. Chapters 6, 9, 13.

Where to go next

This book covered two structures completely. The natural order after it:

Next	Built from	Why now
Stacks and queues	an array or a list, with a rule about which end	trivial after Part VI
Hash tables	an array of buckets, each a linked list	you have both halves already
Trees	a node with two `next` pointers	Chapter 24's struct, extended
Heaps	a tree stored in an array	needs the index arithmetic of Chapter 7
Graphs	an array of linked lists	the adjacency list is literally that
Recursion and backtracking	the call stack of Chapter 22	you have met the cost already

Next	Built from	Why now
Dynamic programming	Kadane, generalised	Chapter 16 was your first DP

Table 40.9 What each later structure borrows from this book. None of them is a fresh start.

CHAPTER RECAP | The last word

You began this book not knowing what a data structure was. You can now explain why an array indexes in constant time and a linked list does not, reverse a linked list on paper, recognise ten patterns on sight, and derive a solution to a problem you have never seen. The remaining work is repetition, and repetition is a schedule rather than a talent. Pick a problem, set a timer, talk out loud, and compile it afterwards.

Good luck.